Learn Being Forward

COURSES · CAMPUS PARTNERSHIPS · PLACEMENTS

STUDY NOTES · BASIC TO ADVANCED

Raspberry Pi Linux, Python and Hardware: Build Real Systems

Complete notes for the Learn Being Forward Raspberry Pi syllabus: hardware, Linux, Python, GPIO, sensors, buses, displays, motors, camera, web dashboards, databases, IoT, home automation, servers, security, computer vision, edge AI, Pico, ROS 2 and product design, explained with diagrams and tested commands and programs, followed by basic, intermediate and advanced questions with fully worked solutions.

27	3	59	675
modules	levels	minimum days	questions and programs

How to use these notes

These notes follow the **Learn Being Forward Raspberry Pi syllabus** exactly: the same 27 modules, in the same order, with every topic and subtopic explained step by step. Each subtopic number (for example **7.4.1**) matches the syllabus, so you can always find where you are.

Level	Modules	Minimum	Outcome
1. Beginner	1 to 9	19 days	Understand Raspberry Pi hardware, install and configure the OS, use the Linux command line, work remotely over SSH, program in Python and control GPIO, sensors and basic projects
2. Intermediate	10 to 19	19 days	Use I2C, SPI and UART devices, displays, motors and relays, the camera and audio, build web dashboards, log data to databases, connect devices with MQTT, automate a home and administer Linux reliably
3. Advanced	20 to 27	21 days	Run the Pi as a secure server with Docker, build computer vision and edge AI systems, use the Pico with MicroPython, build robots with ROS 2, design for power and production, and complete a capstone project
Total	1 to 27	59 days	About 10 weeks at 6 study days per week

1 · Understand
Read the concept and study the diagram until you can explain every connection and every line of code.

2 · Build
Wire the circuit, type the commands and programs yourself on the Pi, and change one thing at a time.

3 · Practise
Solve the basic, intermediate and advanced questions, commands and programs before looking at the solutions.

4 · Check and extend
Compare with the solutions, complete the practice checkpoint and record a short demo of what you built.

Every topic follows the same pattern: the concept in plain words, diagrams, annotated commands and programs, **shortcut tricks** and safety notes. Each module then gives about 25 practice questions in three levels, **basic**, **intermediate** and **advanced**: concept and calculation questions plus terminal and programming tasks. Terminal transcripts marked TERMINAL are real runs of the commands in a Linux shell as user **pi**; Python programs were run with Python 3, with GPIO, sensors and buses replaced by a simulated hardware layer and a simulated clock, so the output shown beside each program is its real output. Panels marked TYPICAL OUTPUT show representative output of commands that need a real Pi, network or server. All calculations were checked by program.

HOW TO READ THE BOXES
Gold-bordered boxes hold **shortcut tricks**, worked examples and notes, red boxes warn about common mistakes and hardware dangers, and navy boxes hold the key rule to remember. Grey panels under a program show its output, a terminal transcript or a simulated run with timed hardware events.

Course plan at a glance

Module	Topic	Level	Min days
01	Introduction to Raspberry Pi	Beginner	1
02	Raspberry Pi hardware and models	Beginner	2

Module	Topic	Level	Min days
03	Setting up the operating system	Beginner	2
04	Linux command line essentials	Beginner	3
05	Remote access and networking basics	Beginner	2
06	Python programming on Raspberry Pi	Beginner	3
07	GPIO fundamentals	Beginner	2
08	Sensors and basic input devices	Beginner	2
09	Beginner projects	Beginner	2
10	Communication protocols: I2C, SPI and UART	Intermediate	2
11	Displays and user interfaces	Intermediate	2
12	Motors, relays and actuators	Intermediate	2
13	Camera and image capture	Intermediate	2
14	Audio and multimedia	Intermediate	1
15	Web servers and web dashboards	Intermediate	2
16	Databases and data logging	Intermediate	2
17	IoT with MQTT and cloud platforms	Intermediate	2
18	Home automation	Intermediate	2
19	Linux system administration for Raspberry Pi	Intermediate	2
20	Raspberry Pi as a server	Advanced	2
21	Security and hardening	Advanced	2
22	Computer vision with OpenCV	Advanced	3
23	AI and machine learning at the edge	Advanced	3
24	Raspberry Pi Pico and MicroPython	Advanced	2
25	Robotics with Raspberry Pi and ROS 2	Advanced	3
26	Performance, power and product design	Advanced	2
27	Capstone project and career path	Advanced	4

Table of contents

LEVEL 1 · BEGINNER LEVEL · MODULES 1 TO 9

Module 01 · Introduction to Raspberry Pi 10

1.1 What is Raspberry Pi? 10
1.2 History of Raspberry Pi 10
1.3 Why do we need Raspberry Pi? 10
1.4 Where is Raspberry Pi used? 10
1.5 Microprocessor vs microcontroller 10
1.6 What you can build 10
1.7 Practice questions 10
1.8 Solutions 10

Module 02 · Raspberry Pi hardware and models 22

2.1 Board anatomy (Raspberry Pi 5 and Pi 4) 22
2.2 Model comparison 22
2.3 Essential accessories 22
2.4 HATs and add-on boards 22
2.5 Handling and safety 22
2.6 Practice questions 22
2.7 Solutions 22

Module 03 · Setting up the operating system 35

3.1 Operating system choices 35
3.2 Raspberry Pi Imager 35
3.3 First boot 35
3.4 raspi-config and the Control Centre 35
3.5 Headless setup 35
3.6 Booting from USB or NVMe SSD 35
3.7 Backing up and cloning the SD card 35
3.8 Practice questions 35
3.9 Solutions 35

Module 04 · Linux command line essentials 46

4.1 Terminal basics 46
4.2 The Linux file system 46
4.3 File and directory commands 46
4.4 Text editors 46
4.5 Users and permissions 46
4.6 Package management 46
4.7 Processes and system monitoring 46
4.8 Pipes, redirection and wildcards 46
4.9 Introduction to shell scripts 46
4.10 Practice questions 46
4.11 Solutions 46

Module 05 · Remote access and networking basics 62

5.1 Networking fundamentals 62
5.2 SSH 62

5.3 Remote desktop 62
5.4 Network configuration 62
5.5 Remote development 62
5.6 Practice questions 62
5.7 Solutions 62

Module 06 · Python programming on Raspberry Pi 74

6.1 Python setup 74
6.2 Virtual environments and packages 74
6.3 Python fundamentals 74
6.4 Data structures 74
6.5 Functions and modules 74
6.6 Files and exceptions 74
6.7 Time and dates 74
6.8 Introduction to object-oriented Python 74
6.9 Practice questions 74
6.10 Solutions 74

Module 07 · GPIO fundamentals 91

7.1 The 40-pin GPIO header 91
7.2 Electrical rules 91
7.3 Electronics basics recap 91
7.4 GPIO libraries 91
7.5 Digital output 91
7.6 Digital input 91
7.7 PWM 91
7.8 Practice questions 91
7.9 Solutions 91

Module 08 · Sensors and basic input devices 110

8.1 Digital sensors 110
8.2 Distance measurement 110
8.3 Temperature and humidity 110
8.4 Analog input 110
8.5 Sense HAT 110
8.6 Logging and plotting data 110
8.7 Practice questions 110
8.8 Solutions 110

Module 09 · Beginner projects 126

9.1 Project planning 126
9.2 Project ideas 126
9.3 Running programs at boot 126
9.4 Documenting a project 126
9.5 Practice questions 126
9.6 Solutions 126

LEVEL 2 · INTERMEDIATE LEVEL · MODULES 10 TO 19

Module 10 · Communication protocols: I2C, SPI and UART 141

10.1 I2C 141
10.2 SPI 141

10.3 UART (serial) 141
10.4 1-Wire 141
10.5 Logic level shifting 141
10.6 Device tree overlays 141
10.7 Adafruit Blinka and CircuitPython libraries 141
10.8 Practice questions 141
10.9 Solutions 141

Module 11 · Displays and user interfaces 156

11.1 Small displays 156
11.2 Touchscreens 156
11.3 Addressable LEDs 156
11.4 Desktop GUIs in Python 156
11.5 Input devices 156
11.6 Practice questions 156
11.7 Solutions 156

Module 12 · Motors, relays and actuators 169

12.1 Driving loads safely 169
12.2 DC motors 169
12.3 Servo motors 169
12.4 Stepper motors 169
12.5 Relays 169
12.6 Motor HATs 169
12.7 Practice questions 169
12.8 Solutions 169

Module 13 · Camera and image capture 184

13.1 Camera hardware 184
13.2 Command-line tools 184
13.3 Picamera2 library 184
13.4 Camera projects 184
13.5 Practice questions 184
13.6 Solutions 184

Module 14 · Audio and multimedia 197

14.1 Audio output 197
14.2 Audio input 197
14.3 Speech 197
14.4 Media centre 197
14.5 Retro gaming 197
14.6 Practice questions 197
14.7 Solutions 197

Module 15 · Web servers and web dashboards 208

15.1 Web basics 208
15.2 Flask 208
15.3 FastAPI as an alternative 208
15.4 Web servers 208
15.5 Dashboards 208
15.6 Accessing the Pi from outside the home network 208
15.7 Practice questions 208

15.8 Solutions 208

Module 16 · Databases and data logging 224

16.1 File-based logging 224
16.2 SQLite 224
16.3 Server databases 224
16.4 Time-series databases 224
16.5 Cloud data storage 224
16.6 Protecting the SD card 224
16.7 Practice questions 224
16.8 Solutions 224

Module 17 · IoT with MQTT and cloud platforms 238

17.1 IoT fundamentals 238
17.2 MQTT 238
17.3 Node-RED 238
17.4 Cloud IoT platforms 238
17.5 Notifications 238
17.6 Practice questions 238
17.7 Solutions 238

Module 18 · Home automation 253

18.1 Home automation concepts 253
18.2 Home Assistant 253
18.3 Smart home protocols 253
18.4 DIY smart devices 253
18.5 Voice control 253
18.6 Electrical safety in home projects 253
18.7 Practice questions 253
18.8 Solutions 253

Module 19 · Linux system administration for Raspberry Pi 266

19.1 systemd 266
19.2 Scheduling 266
19.3 Storage 266
19.4 Users and groups management 266
19.5 Boot configuration 266
19.6 Backups and recovery 266
19.7 Advanced Bash scripting 266
19.8 Practice questions 266
19.9 Solutions 266

LEVEL 3 · ADVANCED LEVEL · MODULES 20 TO 27

Module 20 · Raspberry Pi as a server 282

20.1 Network services 282
20.2 File servers and NAS 282
20.3 Docker on Raspberry Pi 282
20.4 Web hosting 282
20.5 Monitoring 282
20.6 Clusters 282
20.7 Practice questions 282

20.8 Solutions 282

Module 21 · Security and hardening 295

21.1 Threats to IoT devices 295
21.2 Account security 295
21.3 Network security 295
21.4 Updates 295
21.5 Securing applications 295
21.6 Physical and data security 295
21.7 Practice questions 295
21.8 Solutions 295

Module 22 · Computer vision with OpenCV 309

22.1 OpenCV setup 309
22.2 Image basics 309
22.3 Image processing 309
22.4 Colour tracking 309
22.5 Detection 309
22.6 Performance on the Pi 309
22.7 Practice questions 309
22.8 Solutions 309

Module 23 · AI and machine learning at the edge 326

23.1 Edge AI concepts 326
23.2 Machine learning basics on the Pi 326
23.3 TensorFlow Lite / LiteRT 326
23.4 Training custom models 326
23.5 AI accelerators 326
23.6 Generative AI on the Pi 326
23.7 Responsible AI 326
23.8 Practice questions 326
23.9 Solutions 326

Module 24 · Raspberry Pi Pico and MicroPython 342

24.1 Pico family 342
24.2 MicroPython 342
24.3 Pico W networking 342
24.4 PIO (programmable I/O) 342
24.5 C/C++ SDK overview 342
24.6 Pi + Pico architecture 342
24.7 Low-power operation 342
24.8 Practice questions 342
24.9 Solutions 342

Module 25 · Robotics with Raspberry Pi and ROS 2 357

25.1 Robot architecture 357
25.2 Building a mobile robot 357
25.3 Autonomous behaviours 357
25.4 ROS 2 fundamentals 357
25.5 Mapping and navigation 357
25.6 Simulation with Gazebo 357
25.7 Practice questions 357

25.8 Solutions 357

Module 26 · Performance, power and product design 374

26.1 Performance 374
26.2 Thermal management 374
26.3 Power 374
26.4 Storage reliability 374
26.5 From prototype to product 374
26.6 Practice questions 374
26.7 Solutions 374

Module 27 · Capstone project and career path 390

27.1 Capstone project ideas 390
27.2 Project process 390
27.3 Portfolio 390
27.4 Careers 390
27.5 Further learning 390
27.6 Practice questions 390
27.7 Solutions 390

LEVEL 1

Beginner level

Understand Raspberry Pi hardware, install and configure the OS, use the Linux command line, work remotely over SSH, program in Python and control GPIO, sensors and basic projects

9 MODULES · 19 DAYS MINIMUM

Module 01 · Introduction to Raspberry Pi	1 day
Module 02 · Raspberry Pi hardware and models	2 days
Module 03 · Setting up the operating system	2 days
Module 04 · Linux command line essentials	3 days
Module 05 · Remote access and networking basics	2 days
Module 06 · Python programming on Raspberry Pi	3 days
Module 07 · GPIO fundamentals	2 days
Module 08 · Sensors and basic input devices	2 days
Module 09 · Beginner projects	2 days

MODULE 01 · BEGINNER

Introduction to Raspberry Pi

Minimum duration **1 day** | Level **Beginner**

This module explains what a Raspberry Pi is, where it came from, why it is used everywhere from classrooms to factories, how a microprocessor board differs from a microcontroller board such as Arduino or ESP32, and what you will be able to build by the end of this course, followed by 25 solved questions.

IN THIS MODULE

1.1	What is Raspberry Pi?	1.2	History of Raspberry Pi
1.3	Why do we need Raspberry Pi?	1.4	Where is Raspberry Pi used?
1.5	Microprocessor vs microcontroller	1.6	What you can build
1.7	Practice questions	1.8	Solutions

1.1 What is Raspberry Pi?

1.1.1 Definition: a low-cost, credit-card-sized single-board computer (SBC)

A **Raspberry Pi** is a small, low-cost computer built on a single printed circuit board about the size of a credit card (85 mm × 56 mm for the flagship models). Everything a computer needs, processor, memory, input/output and networking, is on that one board, so it is called a **single-board computer (SBC)**.

1.1.2 A full computer: CPU, RAM, storage, USB, networking, display and GPIO pins

THE PARTS OF A RASPBERRY PI: ONE SYSTEM-ON-CHIP CONNECTED TO MEMORY, STORAGE AND INPUT/OUTPUT

Part	What it does	On a Raspberry Pi 5
CPU (processor)	runs the operating system and your programs	4 × Arm Cortex-A76 cores at 2.4 GHz
GPU (graphics)	draws the desktop, decodes video	VideoCore VII

Part	What it does	On a Raspberry Pi 5
RAM	holds running programs and data	2 GB to 16 GB (LPDDR4X)
Storage	holds the OS and files (no built-in disk)	microSD card, or NVMe SSD through PCIe
USB	connects keyboards, mice, drives, cameras	2 × USB 3.0, 2 × USB 2.0
Networking	wired and wireless connections	Gigabit Ethernet, dual-band Wi-Fi, Bluetooth 5.0
Display	connects monitors	2 × micro-HDMI (4K at 60 Hz)
GPIO	connects your own electronics	40-pin header, 3.3 V logic

1.1.3 Runs a full operating system (Linux)

Unlike a microcontroller, a Raspberry Pi boots a complete **operating system**, normally **Raspberry Pi OS**, a version of Debian Linux. That means a desktop, a web browser, a terminal, multiple users, many programs running at once, and the same tools used on servers in data centres: Python, Git, Docker, databases and web servers.

1.1.4 The Raspberry Pi Foundation and Raspberry Pi Ltd

Organisation	Role
Raspberry Pi Foundation	UK educational charity; promotes computing education worldwide (free courses, Code Club, CoderDojo, teacher training)
Raspberry Pi Ltd	the company that designs and sells the computers; its profits support the Foundation. Listed on the London Stock Exchange in 2024

1.2 History of Raspberry Pi

1.2.1 Created in the UK to teach computing in schools (first board 2012)

In 2006, a team at the University of Cambridge noticed that fewer students applying for computer science had hands-on programming experience: home computers had become closed appliances. Their answer was a computer cheap enough for every child to own and experiment with. The first **Raspberry Pi Model B** went on sale on **29 February 2012** for **US $35**.

1.2.2 Founder Eben Upton and team

Eben Upton led the design, together with colleagues including Rob Mullins, Jack Lang and Alan Mycroft. The Raspberry Pi Foundation was registered as a charity in 2009.

1.2.3 Generations: Pi 1, Pi 2, Pi 3, Pi 4 and Pi 5

Year	Model	Big change
2012	Raspberry Pi 1	first board; single-core 700 MHz, 256–512 MB RAM
2015	Raspberry Pi 2	quad-core processor, 1 GB RAM

Year	Model	Big change
2016	Raspberry Pi 3	64-bit CPU, built-in Wi-Fi and Bluetooth
2019	Raspberry Pi 4	up to 8 GB RAM, USB 3.0, dual 4K displays, USB-C power
2023	Raspberry Pi 5	about 2–3× faster, RP1 I/O chip, PCIe, power button, RTC

FLAGSHIP GENERATIONS

1.2.4 Keyboard computers: Pi 400 and Pi 500

The **Pi 400** (2020, based on the Pi 4) and **Pi 500** (2024, based on the Pi 5) put the computer inside a compact keyboard: plug in a monitor and power and you have a complete desktop PC. They keep a GPIO header on the back.

1.2.5 Microcontroller line: Raspberry Pi Pico and Pico 2

In 2021 Raspberry Pi released the **Pico**, built on its own **RP2040** microcontroller chip, followed by **Pico 2** (RP2350) in 2024. A Pico does not run Linux: it runs one program directly, in C/C++ or MicroPython, like an Arduino. Module 24 covers it.

1.3 Why do we need Raspberry Pi?

1.3.1 Low cost and low power

A Raspberry Pi costs a fraction of a laptop and uses about **3–8 W** in normal use (a desktop PC uses 50–200 W), so it can run 24 hours a day as a server for very little electricity.

WORKED EXAMPLE

Yearly energy of an always-on Pi

Average power 5 W × 24 h × 365 days = 43 800 Wh = **43.8 kWh** per year. At ₹8 per kWh that is about **₹350 a year**. A 100 W PC doing the same job uses 876 kWh, 20 times more.

1.3.2 Learning Linux, programming and electronics on one device

One board lets you learn three industry skills together: the **Linux** operating system used on most servers, **programming** (Python, C, JavaScript and more), and **electronics** through the GPIO pins. If you break the software, you just re-flash the microSD card.

1.3.3 GPIO pins for hardware projects

The 40-pin **GPIO** (General-Purpose Input/Output) header lets programs switch LEDs, relays and motors and read buttons and sensors. Normal laptops have no such pins.

1.3.4 Huge community, documentation and accessories

Tens of millions of boards have been sold, so almost every problem has been solved and documented somewhere: official documentation, forums, magazines (The MagPi), thousands of tutorials and

hundreds of add-on boards (HATs).

1.4 Where is Raspberry Pi used?

Area	Typical uses
1.4.1 Education and coding labs	school computer labs, coding clubs, university electronics labs
1.4.2 Home automation and IoT gateways	Home Assistant hubs, MQTT brokers, sensor gateways
1.4.3 Media centres and retro gaming	Kodi/LibreELEC media players, RetroPie consoles
1.4.4 Home servers, network tools and NAS	Pi-hole ad blocking, VPN servers, file servers, Nextcloud
1.4.5 Robotics and computer vision	mobile robots, ROS 2, OpenCV object tracking
1.4.6 Industrial control, kiosks and signage	machine monitoring, factory dashboards, information screens
1.4.7 Edge AI and research	AI cameras, wildlife monitoring, weather stations, lab instruments

DID YOU KNOW

Many industrial products contain a **Compute Module**, the Raspberry Pi electronics on a small board designed to be plugged into a custom carrier board. Industry is now the largest market for Raspberry Pi Ltd.

1.5 Microprocessor vs microcontroller

A **microprocessor** (MPU) is only the processing core: memory, storage and peripherals are separate chips, and it runs a full operating system. A **microcontroller** (MCU) puts the CPU, a small amount of RAM, flash memory and I/O peripherals on one chip and runs a single program directly, with no operating system.

1.5.1 Raspberry Pi vs Arduino vs ESP32

Feature	Raspberry Pi 5	Arduino Uno R3	ESP32
Type	single-board computer (MPU)	microcontroller board	microcontroller with Wi-Fi
Processor	4 cores, 2.4 GHz, 64-bit	1 core, 16 MHz, 8-bit	2 cores, 240 MHz, 32-bit
Memory	2–16 GB RAM	2 KB RAM, 32 KB flash	520 KB RAM, 4 MB flash
Operating system	Linux	none	none (or FreeRTOS)
Start-up time	10–30 seconds	under 1 second	under 1 second
Logic level	3.3 V	5 V	3.3 V
Analog inputs	none (needs an ADC chip)	6 × 10-bit	about 18 × 12-bit
Power	3–12 W	about 0.25 W	0.1–1 W

Feature	Raspberry Pi 5	Arduino Uno R3	ESP32
Must be shut down safely?	yes	no	no
Best at	networking, video, AI, databases, web	simple, exact real-time control	Wi-Fi sensors, battery IoT

1.5.2 When to use each, and when to combine them

Choose a Pi when
the project needs a camera, a screen, a web server, a database, AI or many programs at once.

Choose a microcontroller when
it needs exact timing, instant start-up, battery life of months, or simple sensor reading.

Combine them when
a robot needs both: the Pi thinks (vision, navigation) and a Pico or Arduino drives motors in real time.

THE COMMON PATTERN: THE PI FOR HIGH-LEVEL THINKING, A MICROCONTROLLER FOR REAL-TIME CONTROL

SHORTCUT TRICK
Ask two questions. **Does it need Linux-level features** (camera, network services, screen, AI)? Then a Pi. **Does it need to start instantly, run on a small battery or time pulses exactly?** Then a microcontroller. If both answers are yes, use both.

1.6 What you can build

1.6.1 Showcase of beginner, intermediate and advanced projects

Level	Example projects in this course
Beginner (modules 1–9)	traffic lights, reaction game, motion alarm, room monitor, weather station, smart night light
Intermediate (modules 10–19)	touchscreen control panel, pan-tilt camera, voice assistant, web dashboard, MQTT sensor network, Home Assistant hub
Advanced (modules 20–27)	Docker home server with VPN, object-tracking camera, people counter with AI, Pico sensor nodes, ROS 2 robot, capstone product

Here is a first taste of what Python on a Pi looks like. Six lines blink an LED connected to GPIO 17. The panel under the program is a simulated run that shows each change of the pin with its time.

BLINK.PY

```
from gpiozero import LED
from time import sleep

led = LED(17)
for i in range(3):
    led.on()
    sleep(0.5)
    led.off()
    sleep(0.5)
print("Done")
```

SIMULATED RUN (hardware events and printed output)

```
[  0.00 s] LED(GPIO17) on
[  0.50 s] LED(GPIO17) off
[  1.00 s] LED(GPIO17) on
[  1.50 s] LED(GPIO17) off
[  2.00 s] LED(GPIO17) on
[  2.50 s] LED(GPIO17) off
Done
```

PRACTICE CHECKPOINT

Explain in your own words what a single-board computer is, compare Raspberry Pi with Arduino, and list ten project ideas you would like to build.

1.7 Practice questions

1.7.1 Basic level

Q1 A Raspberry Pi is best described as a

(a) microcontroller
(b) graphics card
(c) single-board computer
(d) smartphone

Q2 Which operating system does a Raspberry Pi normally run?

(a) Raspberry Pi OS (Debian Linux)
(b) Windows 11
(c) macOS
(d) no operating system

Q3 In which year was the first Raspberry Pi released?

(a) 2006 (b) 2009 (c) 2016 (d) 2012

Q4 Who led the design of the Raspberry Pi?

(a) Massimo Banzi (b) Eben Upton (c) Linus Torvalds (d) Steve Wozniak

Q5 What was the original purpose of the Raspberry Pi?

(a) to replace data-centre servers
(b) to teach computing to school students
(c) to compete with smartphones
(d) to mine cryptocurrency

Q6 Which model first included built-in Wi-Fi and Bluetooth?

(a) Raspberry Pi 1 (b) Raspberry Pi 2 (c) Raspberry Pi 5 (d) Raspberry Pi 3

Q7 Raspberry Pi Pico is

(a) a microcontroller board that does not run Linux
(b) a smaller Pi that runs Raspberry Pi OS
(c) a keyboard computer
(d) a camera module

Q8 What does GPIO stand for?

(a) Graphics Processing Input/Output
(b) General Power In/Out
(c) General-Purpose Input/Output
(d) Gigabit Port I/O

1.7.2 Intermediate level

Q9 Where does a Raspberry Pi store its operating system?

(a) on a built-in hard disk
(b) in the GPU
(c) in the GPIO header
(d) on a microSD card (or a USB/NVMe drive)

Q10 Which board would you choose for a battery-powered sensor that must run for months?

(a) Raspberry Pi 5
(b) a microcontroller such as ESP32 or Pico
(c) Raspberry Pi 400

(d) a desktop PC

Q11 Which project most clearly needs a Raspberry Pi rather than an Arduino?

(a) a camera that recognises faces and serves a web page
(b) blinking an LED
(c) reading one button
(d) a simple kitchen timer

Q12 What is a Compute Module?

(a) a cloud computing service
(b) a Python library
(c) Raspberry Pi electronics on a small board designed to be built into products
(d) a type of microSD card

Q13 Why must a Raspberry Pi be shut down properly rather than just unplugged?

(a) the OS may be writing to the SD card, and cutting power can corrupt files
(b) the GPU will be damaged
(c) the CPU loses its serial number
(d) the GPIO pins will short

Q14 Which charity promotes computing education and is supported by Raspberry Pi Ltd?

(a) the Arduino Foundation
(b) the Raspberry Pi Foundation
(c) the Linux Foundation
(d) the Python Software Foundation

Q15 A Raspberry Pi 5 has how many CPU cores?

(a) 1 (b) 2 (c) 8 (d) 4

Q16 A Pi server uses 4 W on average and runs all year. How much energy does it use in a year?

(a) 3.504 kWh (b) 350.4 kWh (c) 35.04 kWh (d) 96 kWh

1.7.3 Advanced level

Q17 At ₹8 per kWh, what does a 6 W always-on Pi cost per year (to the nearest rupee)?

(a) ₹48 (b) ₹1 051 (c) ₹420 (d) ₹4 205

Q18 A Raspberry Pi 5 official supply gives 5 V at 5 A. What is its maximum power?

(a) 10 W (b) 15 W (c) 27 W (d) 25 W

Q19 An Arduino Uno runs at 16 MHz and a Pi 5 core at 2.4 GHz. How many times faster is the Pi clock?

(a) 15 (b) 150 (c) 1 500 (d) 24

Q20 Which statement about Raspberry Pi and Arduino logic levels is correct?

(a) Pi GPIO is 3.3 V; Uno pins are 5 V
(b) both use 5 V
(c) both use 3.3 V
(d) Pi uses 12 V

Q21 A robot must run vision at 15 frames per second and drive four motors with precise PWM. The best design is

(a) an Arduino Uno alone
(b) a Pi for vision and planning plus a microcontroller for motor control

(c) a Pi alone with Python time.sleep() for PWM
(d) two Arduinos

Q22 Which of these is NOT something a Raspberry Pi has on the board?

(a) a built-in analog-to-digital converter for sensors
(b) USB ports
(c) Wi-Fi
(d) HDMI output

Q23 A school wants 30 student computers for coding and electronics on a small budget. Why is the Pi a good choice?

(a) it is waterproof
(b) it needs no monitor ever
(c) it is cheap, runs Linux and Python, has GPIO and a broken OS is fixed by re-flashing the card
(d) it runs Windows games

Q24 Which pair is correctly matched?

(a) Pico: runs Raspberry Pi OS
(b) Pi 5: first model with USB
(c) Pi 2: first model with Wi-Fi
(d) Pi 400: Pi 4 inside a keyboard

Q25 Using gpiozero, write a program that blinks an LED on GPIO 27 five times (0.2 s on, 0.2 s off) and then prints "Finished".

1.8 Solutions

Q1 **Answer: (c) single-board computer**

It is a complete computer on one board: a **single-board computer (SBC)**.

Q2 **Answer: (a) Raspberry Pi OS (Debian Linux)**

The official OS is **Raspberry Pi OS**, based on Debian Linux.

Q3 **Answer: (d) 2012**

It went on sale on 29 February **2012**. (2006 is when the idea started; 2009 when the charity was registered.)

Q4 **Answer: (b) Eben Upton**

Eben Upton. Massimo Banzi co-founded Arduino; Linus Torvalds created Linux.

Q5 **Answer: (b) to teach computing to school students**

It was created to get young people **programming and learning computing**.

Q6 **Answer: (d) Raspberry Pi 3**

The **Pi 3** (2016) was the first flagship with on-board Wi-Fi and Bluetooth.

Q7 **Answer: (a) a microcontroller board that does not run Linux**

Pico uses the RP2040 **microcontroller** and runs one program directly.

Q8 **Answer: (c) General-Purpose Input/Output**

General-Purpose Input/Output: pins your program can use as inputs or outputs.

Q9 **Answer: (d) on a microSD card (or a USB/NVMe drive)**

There is no built-in disk: the OS lives on a **microSD card** or an external SSD.

Q10 **Answer: (b) a microcontroller such as ESP32 or Pico**

Microcontrollers use milliwatts and can sleep deeply; a Pi uses watts.

Q11 **Answer: (a) a camera that recognises faces and serves a web page**

Vision plus a web server needs **Linux-level resources**.

Q12 **Answer: (c) Raspberry Pi electronics on a small board designed to be built into products**

Compute Modules (CM4, CM5) plug into a **custom carrier board**.

Q13 **Answer: (a) the OS may be writing to the SD card, and cutting power can corrupt files**

A running OS writes files in the background; pulling power mid-write can **corrupt the file system**.

Q14 **Answer: (b) the Raspberry Pi Foundation**

Profits from Raspberry Pi Ltd support the **Raspberry Pi Foundation**.

Q15 **Answer: (d) 4**

Four Arm Cortex-A76 cores.

Q16 **Answer: (c) 35.04 kWh**

4 W × 24 × 365 = 35 040 Wh = **35.04 kWh**.

Q17 **Answer: (c) ₹420**

6 × 8760 = 52 560 Wh = 52.56 kWh; × ₹8 = ₹420.48 ≈ **₹420**.

Q18 **Answer: (d) 25 W**

P = V × I = 5 × 5 = **25 W**. (The supply is rated 27 W because it can also deliver higher voltages to other devices.)

Q19 **Answer: (b) 150**

2400 MHz ÷ 16 MHz = **150** (and the Pi has four 64-bit cores, so the real speed gap is far larger).

Q20 **Answer: (a) Pi GPIO is 3.3 V; Uno pins are 5 V**

Pi is **3.3 V** and not 5 V tolerant, so 5 V signals from an Uno need level shifting.

Q21 **Answer: (b) a Pi for vision and planning plus a microcontroller for motor control**

Split the work: **Pi for heavy computing**, **microcontroller for real-time control**, linked by serial.

Q22 **Answer: (a) a built-in analog-to-digital converter for sensors**

The Pi has **no ADC**: analog sensors need a chip such as the MCP3008 (module 8).

Q23 **Answer: (c) it is cheap, runs Linux and Python, has GPIO and a broken OS is fixed by re-flashing the card**

Low cost, a full learning OS, hardware pins and **easy recovery**.

Q24 **Answer: (d) Pi 400: Pi 4 inside a keyboard**

The **Pi 400** is a Pi 4 built into a keyboard; the Pi 500 does the same with a Pi 5.

Q25 **Answer: Solution program**

LED(27) controls GPIO 27; on()/off() switch it and sleep() waits. The simulated run shows five on/off pairs, 0.4 s apart.

```
from gpiozero import LED
from time import sleep

led = LED(27)
for _ in range(5):
    led.on()
    sleep(0.2)
    led.off()
    sleep(0.2)
print("Finished")
```

SIMULATED RUN

```
[  0.00 s] LED(GPIO27) on
[  0.20 s] LED(GPIO27) off
[  0.40 s] LED(GPIO27) on
[  0.60 s] LED(GPIO27) off
[  0.80 s] LED(GPIO27) on
[  1.00 s] LED(GPIO27) off
[  1.20 s] LED(GPIO27) on
[  1.40 s] LED(GPIO27) off
[  1.60 s] LED(GPIO27) on
[  1.80 s] LED(GPIO27) off
Finished
```

A Raspberry Pi is a low-cost single-board computer that runs Linux and has GPIO pins. It was created in 2012 to teach computing. Use a Pi for networking, video, AI and many programs at once; use a microcontroller for instant, exact, low-power control; combine them for robots and products.

MODULE 02 · BEGINNER

Raspberry Pi hardware and models

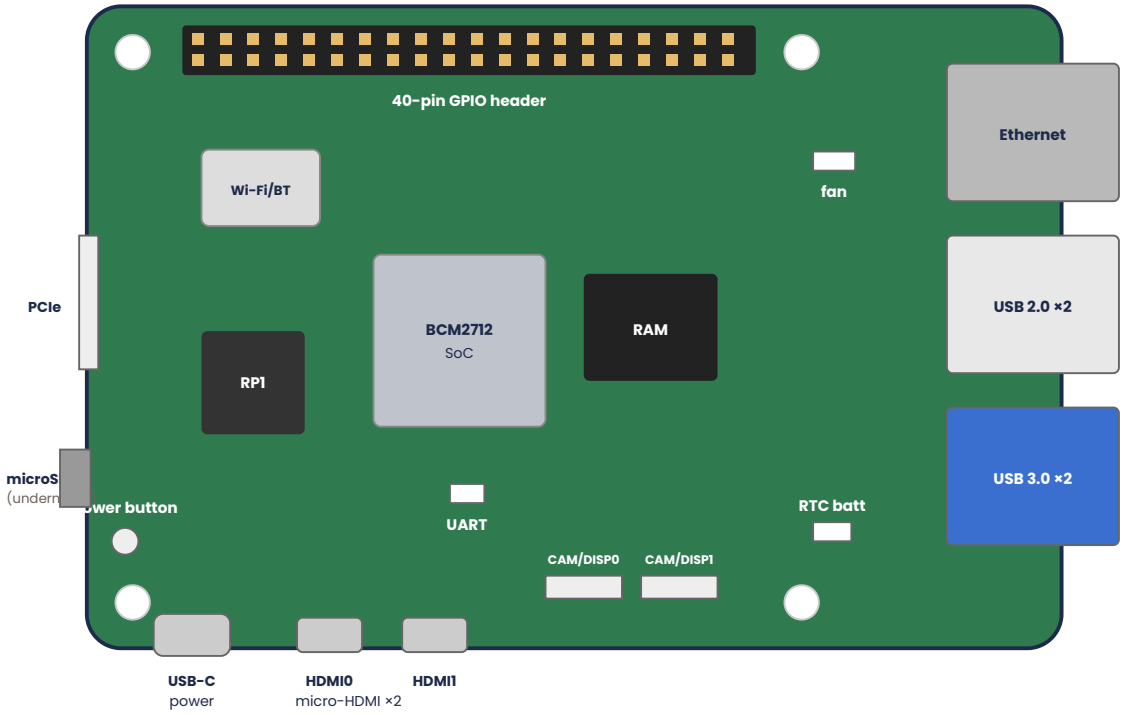
Minimum duration **2 days** | Level **Beginner**

Choosing the right board and accessories saves money and avoids power and compatibility problems later. This module walks round the Raspberry Pi 5 and Pi 4 boards connector by connector, compares every current model including the Zero, keyboard computers, Compute Modules and Pico, explains which power supply, microSD card, cooling and storage to buy, introduces HATs, and finishes with handling rules, undervoltage and throttling, followed by 25 solved questions.

IN THIS MODULE

2.1 Board anatomy (Pi 5 and Pi 4)
2.2 Model comparison
2.3 Essential accessories
2.4 HATs and add-on boards
2.5 Handling and safety
2.6 Practice questions
2.7 Solutions

2.1 Board anatomy (Raspberry Pi 5 and Pi 4)



RASPBERRY PI 5, TOP VIEW (SIMPLIFIED, NOT TO SCALE)

2.1.1 SoC: CPU, GPU and memory

The big silver chip is the **system-on-chip (SoC)**: CPU cores, GPU, memory controller and video units in one package. The Pi 5 uses the Broadcom **BCM2712** (4 × Cortex-A76, 2.4 GHz, VideoCore VII GPU); the Pi 4

uses the **BCM2711** (4 × Cortex-A72, 1.5–1.8 GHz, VideoCore VI).

2.1.2 RAM options

RAM is the black chip next to the SoC. It cannot be upgraded later, so choose when you buy: 2 GB is enough for headless sensor projects, 4 GB for the desktop and light servers, 8 GB or 16 GB for Docker stacks, computer vision and local AI models.

2.1.3 microSD card slot

The slot on the underside holds the card with the operating system. The Pi 5 supports faster **SDR104** cards (about twice the Pi 4 speed). Never remove the card while the Pi is running.

2.1.4 USB 2.0 and USB 3.0 ports

The **blue** ports are USB 3.0 (up to 5 Gbit/s): use them for SSDs and fast drives. The black ports are USB 2.0 (480 Mbit/s): fine for keyboards, mice and microcontrollers.

2.1.5 Gigabit Ethernet, Wi-Fi and Bluetooth

A wired **Gigabit Ethernet** port (1000 Mbit/s), dual-band **Wi-Fi** (2.4 and 5 GHz, 802.11ac) and **Bluetooth 5.0 / BLE** are built in on the Pi 4 and Pi 5.

2.1.6 Micro-HDMI ports

Two **micro-HDMI** ports each drive a 4K monitor at 60 Hz. Use port **HDMI0** (next to the power socket) for a single monitor. You need a micro-HDMI to HDMI cable or adapter.

2.1.7 Camera and display (MIPI CSI/DSI) connectors

Flat flexible cables connect cameras (**CSI**, Camera Serial Interface) and displays (**DSI**, Display Serial Interface). The Pi 4 has one 15-pin camera and one 15-pin display connector; the Pi 5 has **two 22-pin connectors**, each usable for a camera or a display, so it needs a different (narrower) cable.

2.1.8 40-pin GPIO header

The double row of pins along one edge carries 3.3 V and 5 V power, ground and 26 general-purpose input/output pins, several with special functions (I2C, SPI, UART, PWM). The layout is the same on every model since 2014, so HATs fit all of them. Module 7 covers it in detail.

2.1.9 USB-C power input

Power enters through the USB-C socket. The Pi 4 needs **5.1 V, 3 A** (15 W); the Pi 5 needs **5.1 V, 5 A** (27 W supply) to give full current to USB devices.

2.1.10 Pi 5 extras: power button, RTC battery connector, fan connector, PCIe connector, RP1 I/O chip

Feature	What it gives you
Power button	press to boot after shutdown; press once for a clean shutdown; hold to force off
RTC + battery connector	a real-time clock that keeps time while unplugged when a small rechargeable battery is fitted

Feature	What it gives you
Fan connector	4-pin socket for the official Active Cooler; the fan speed follows the temperature
PCIe connector	one PCIe 2.0 ×1 lane on a flat cable for NVMe SSDs and AI accelerators via an M.2 HAT
RP1 chip	Raspberry Pi's own I/O controller: GPIO, USB, Ethernet, camera and display interfaces (this is why older GPIO libraries that wrote to Broadcom registers do not work on a Pi 5)
UART connector	3-pin debug serial port, so the GPIO serial pins stay free

2.2 Model comparison

Model	Processor	RAM	Highlights	Power
2.2.1 Raspberry Pi 5	4 × A76, 2.4 GHz	2–16 GB	fastest; PCIe, RTC, power button, 2 × 4K	5 V 5 A USB-C
2.2.2 Raspberry Pi 4 Model B	4 × A72, 1.5–1.8 GHz	1–8 GB	USB 3, Gigabit, 2 × 4K; still widely used	5 V 3 A USB-C
2.2.3 Raspberry Pi 3 Model B+	4 × A53, 1.4 GHz	1 GB	older; full-size HDMI, micro-USB power	5 V 2.5 A micro-USB
2.2.4 Raspberry Pi Zero 2 W	4 × A53, 1 GHz	512 MB	65 × 30 mm, Wi-Fi, very cheap, low power	5 V 2.5 A micro-USB
2.2.5 Pi 400 / Pi 500	as Pi 4 / Pi 5	4 GB / 8 GB	computer inside a keyboard	USB-C
2.2.6 Compute Module 4 / 5	as Pi 4 / Pi 5	1–16 GB	optional on-board eMMC storage; plugs into a carrier board for products	from carrier
2.2.7 Pico / Pico 2	RP2040: 2 × M0+, 133 MHz / RP2350: 2 × M33, 150 MHz	264 KB / 520 KB	microcontroller; no Linux; MicroPython or C	micro-USB, about 0.1 W

SHORTCUT TRICK

Pick by the job, not the price: **Zero 2 W** for small, low-power headless gadgets; **Pi 4** (4 GB) for most learning and home-server work; **Pi 5** (8 GB) for vision, AI, NVMe and the desktop; **CM5** for a product; **Pico** for real-time control and batteries.

2.3 Essential accessories

2.3.1 Official power supply and why it matters

Cheap phone chargers often sag below 5 V under load. The Pi then shows an **undervoltage warning**, slows down, USB drives disconnect and the SD card can be corrupted. The official supplies output 5.1 V (a

little extra to cover cable losses). On a Pi 5 with a 3 A supply, the total USB current is limited to 600 mA; with the 5 A supply it is 1.6 A.

WORKED EXAMPLE

Why cable resistance matters

A thin cable with 0.2 Ω total resistance carrying 3 A drops V = I × R = 3 × 0.2 = **0.6 V**. From a 5.0 V charger only 4.4 V reaches the Pi, below the 4.63 V undervoltage threshold. A 5.1 V supply with a good 0.05 Ω cable drops just 0.15 V.

2.3.2 microSD card class and size

Marking	Meaning	Recommendation
A1 / A2	application performance (random reads and writes)	A2 for best OS performance
Class 10, U1, U3, V30	minimum sequential write speed: 10, 10, 30, 30 MB/s	at least Class 10 / U1
Size	8 GB works for Lite; desktop and projects fill space fast	32 GB (64 GB for camera or data projects)

2.3.3 Case and cooling (heatsink, active cooler)

The Pi 5 SoC can reach 85 °C under full load without cooling and then slows down. A case with a fan or the official **Active Cooler** (heatsink + fan on the fan connector) keeps it near 50–60 °C. Passive heatsink cases suit lighter loads.

2.3.4 Micro-HDMI cable, keyboard and mouse

For desktop use: a micro-HDMI to HDMI cable, a USB keyboard and mouse. For headless use (module 3) you need none of them, only the network.

2.3.5 NVMe SSD with M.2 HAT (Pi 5)

The **M.2 HAT+** connects an NVMe SSD to the PCIe connector. Even at PCIe 2.0 ×1 (about 450 MB/s) it is 4–5 times faster than a good microSD card and much more durable for databases and logging. The Pi 5 can boot from it (module 3.6).

TYPICAL SEQUENTIAL READ SPEED ON A PI 5

2.4 HATs and add-on boards

2.4.1 What a HAT is

A **HAT** (Hardware Attached on Top) is an add-on board that plugs onto the 40-pin header and matches the Pi's size and mounting holes. A HAT carries a small **ID EEPROM** on pins 27 and 28 (ID_SD, ID_SC) that tells the Pi what it is, so the right drivers load automatically. The newer **HAT+** specification adds power

and PCIe features for the Pi 5. Boards without the EEPROM are usually called "pHATs", "bonnets" or simply add-on boards.

2.4.2 Common HATs: Sense HAT, motor HAT, PoE HAT, AI HAT

HAT	What it adds
Sense HAT	8 × 8 RGB LED matrix, 5-way joystick, accelerometer, gyroscope, magnetometer, temperature, humidity and pressure sensors (used on the ISS!)
Motor HAT / driver HAT	H-bridge chips to run DC motors and steppers, often a PCA9685 for servos
PoE+ HAT	powers the Pi through the Ethernet cable from a PoE switch: one cable for data and power
AI HAT+ / AI Kit	a Hailo neural-network accelerator (13 or 26 TOPS) on PCIe for fast object detection
M.2 HAT+	NVMe SSD on PCIe

2.5 Handling and safety

2.5.1 Static electricity

A static spark you cannot even feel can damage the chips. Touch a grounded metal object before handling the board, hold it by the edges, keep it on a non-conductive surface (never on metal or carpet), and never handle it while powered.

2.5.2 Undervoltage warnings

If the supply voltage falls below about **4.63 V**, the Pi logs an undervoltage warning (a lightning-bolt icon on the desktop) and may throttle. Check from the terminal with `vcgencmd get_throttled`: `0x0` means everything has been fine since boot.

Bit	Hex value	Meaning
0	0x1	undervoltage **now**
1	0x2	ARM frequency capped now
2	0x4	throttled now
3	0x8	soft temperature limit active now
16	0x10000	undervoltage has occurred since boot
17	0x20000	frequency capping has occurred
18	0x40000	throttling has occurred
19	0x80000	soft temperature limit has occurred

Bits are combined: `0x50005` = 0x40000 + 0x10000 + 0x4 + 0x1 means undervoltage and throttling right now, and both have happened before. A program can decode any value:

THROTTLED.PY

```
FLAGS = {0: "under-voltage now", 1: "frequency capped now", 2: "throttled now",
         3: "soft temperature limit now", 16: "under-voltage has occurred",
         17: "frequency capping has occurred", 18: "throttling has occurred",
         19: "soft temperature limit has occurred"}

def decode(value):
    found = [text for bit, text in FLAGS.items() if value & (1 << bit)]
    return found or ["no problems"]

for v in (0x0, 0x50005, 0x80008):
    print(hex(v), "->", "; ".join(decode(v)))
```

OUTPUT

```
0x0 -> no problems
0x50005 -> under-voltage now; throttled now; under-voltage has occurred; throttling has occurred
0x80008 -> soft temperature limit now; soft temperature limit has occurred
```

2.5.3 Overheating and throttling

When the SoC reaches about **80 °C** the firmware starts lowering the CPU clock (soft throttling), and at **85 °C** it throttles harder to protect the chip. Performance can halve. Check with `vcgencmd measure_temp` and add cooling if you see temperatures above 75 °C under load.

CHECKING TEMPERATURE, THROTTLING AND CLOCK SPEED

```
pi@raspberrypi:~ $ vcgencmd measure_temp
temp=47.2'C
pi@raspberrypi:~ $ vcgencmd get_throttled
throttled=0x0
pi@raspberrypi:~ $ vcgencmd measure_clock arm
frequency(0)=2400000000
```

WATCH OUT

Never short GPIO pins with a screwdriver or loose wire while the Pi is on, never connect 5 V to a GPIO pin, and never power the Pi through the 5 V pin without a protected, regulated supply. These are the commonest ways boards are destroyed.

PRACTICE CHECKPOINT

Label every connector on a Raspberry Pi board diagram, and write a parts list with reasons for a home-server build and for a robot build.

Home-server build	Reason	Robot build	Reason
Pi 5, 8 GB	Docker containers need RAM	Pi 4 or Pi 5, 4 GB	vision and control
27 W official supply	stable power for SSD	battery + 5 V 5 A regulator	mobile power
M.2 HAT+ and NVMe SSD	fast, durable storage	microSD 32 GB A2	light storage

Home-server build	Reason	Robot build	Reason
Active Cooler, case	24/7 load	camera module + motor driver HAT	see and move
Ethernet cable	reliable network	Pico for motor timing (optional)	real-time control

2.6 Practice questions

2.6.1 Basic level

Q1 Which chip is the system-on-chip of the Raspberry Pi 5?

(a) BCM2711 **(b)** RP2040 **(c)** ATmega328P **(d)** BCM2712

Q2 Which USB ports should you use for an external SSD?

(a) the black USB 2.0 ports **(b)** the USB-C power port
(c) the blue USB 3.0 ports **(d)** any port; they are identical

Q3 What kind of cable connects a monitor to a Pi 4 or Pi 5?

(a) micro-HDMI to HDMI **(b)** full-size HDMI
(c) VGA **(d)** mini-HDMI

Q4 What does CSI stand for on the camera connector?

(a) Central Signal Input **(b)** Colour Sensor Input
(c) Camera Shutter Interface **(d)** Camera Serial Interface

Q5 The official Raspberry Pi 5 power supply provides

(a) 5 V at 1 A **(b)** 5.1 V at 5 A **(c)** 12 V at 2 A **(d)** 3.3 V at 5 A

Q6 What does HAT stand for?

(a) High Amperage Terminal **(b)** Hardware Attached on Top
(c) Header Adapter Type **(d)** Hardware Access Tool

Q7 Which model is about 65 mm × 30 mm and costs the least among Linux boards?

(a) Raspberry Pi 5 **(b)** Raspberry Pi 400
(c) Compute Module 5 **(d)** Raspberry Pi Zero 2 W

Q8 Can you upgrade the RAM on a Raspberry Pi later?

(a) no, it is soldered; choose the size when buying
(b) yes, it uses laptop RAM sticks
(c) yes, through the GPIO header
(d) yes, with a USB stick

2.6.2 Intermediate level

Q9 Which Pi 5 feature lets it keep the time while unplugged?

(a) the PCIe connector
(b) the fan connector
(c) the RTC with a battery on the RTC connector
(d) the UART connector

Q10 What is the job of the RP1 chip on the Pi 5?

(a) it is the main CPU
(b) it stores the operating system

(c) it is the Wi-Fi antenna
(d) it is the I/O controller for GPIO, USB, Ethernet and camera/display

Q11 Why do HATs have an ID EEPROM on pins 27 and 28?

(a) to store user programs
(b) so the Pi can identify the board and load the right drivers automatically
(c) to supply extra power
(d) to cool the board

Q12 Which microSD marking best indicates good random-access performance for an OS?

(a) A2 (b) Class 4 (c) V6 (d) UHS-I only

Q13 What does vcgencmd get_throttled = 0x0 mean?

(a) the Pi is throttled now
(b) the Pi has no power
(c) no undervoltage or throttling has happened since boot
(d) the command failed

Q14 At about what temperature does a Pi 4 or Pi 5 start to throttle the CPU?

(a) 80 °C (b) 40 °C (c) 60 °C (d) 120 °C

Q15 What lets a Pi 5 boot from a fast NVMe SSD?

(a) a USB 2.0 hub
(b) an M.2 HAT+ on the PCIe connector
(c) the DSI connector
(d) the GPIO header alone

Q16 Which board is the right choice for a commercial product that needs Pi 5 performance on a custom PCB?

(a) Raspberry Pi 500
(b) Pico 2
(c) Raspberry Pi 3B+
(d) Compute Module 5

2.6.3 Advanced level

Q17 A 5.0 V charger feeds a Pi through a cable with 0.25 Ω total resistance at 2.4 A. What voltage reaches the Pi?

(a) 4.75 V (b) 4.9 V (c) 4.4 V (d) 5.0 V

Q18 vcgencmd get_throttled returns 0x50000. What does it mean?

(a) undervoltage is happening now
(b) the temperature limit is active now
(c) undervoltage and throttling occurred earlier, but neither is happening now
(d) no problems at all

Q19 vcgencmd get_throttled returns 0x80008. What should you do?

(a) buy a stronger power supply
(b) replace the SD card
(c) nothing; it is normal
(d) improve cooling: the soft temperature limit is active and has occurred

Q20 A Pi 5 has a 3 A supply. How much current can all USB devices draw together?

(a) 1.6 A **(b)** 600 mA **(c)** 3 A **(d)** 5 A

Q21 An NVMe SSD reads at 450 MB/s and a microSD at 90 MB/s. How long does each take to read a 2.7 GB file (1 GB = 1000 MB)?

(a) 6 s and 30 s **(b)** 60 s and 300 s **(c)** 0.6 s and 3 s **(d)** 6 s and 3 s

Q22 Your Pi camera project works on a Pi 4 but the same ribbon cable does not fit the Pi 5. Why?

(a) the Pi 5 has no camera support

(b) the Pi 5 uses smaller 22-pin camera/display connectors

(c) the cable must be USB

(d) the Pi 5 needs a HAT for cameras

Q23 Write a program that reads the throttled value as a hexadecimal string such as "0x50005" and prints "UNDERVOLTAGE NOW" if bit 0 is set and "HAS BEEN THROTTLED" if bit 18 is set. Test it with "0x50005" and "0x40000".

Q24 The fan connector speed follows temperature: off below 50 °C, 30 % at 50–59 °C, 60 % at 60–69 °C, 100 % at 70 °C and above. Write a function fan_speed(t) and print the speed for 45, 55, 67 and 82 °C.

Q25 Which parts list suits an always-on home server best?

(a) Pi 5 8 GB, official 27 W supply, NVMe SSD on M.2 HAT+, Active Cooler, Ethernet

(b) Pi Zero 2 W, phone charger, 8 GB SD card, Wi-Fi

(c) Pico 2, battery, no storage

(d) Pi 400 with Wi-Fi only and no cooling

2.7 Solutions

Q1 **Answer: (d) BCM2712**

Pi 5: **BCM2712**. BCM2711 is the Pi 4; RP2040 is the Pico; ATmega328P is the Arduino Uno.

Q2 **Answer: (c) the blue USB 3.0 ports**

Blue = **USB 3.0**, up to 5 Gbit/s.

Q3 **Answer: (a) micro-HDMI to HDMI**

Both have two **micro-HDMI** ports.

Q4 **Answer: (d) Camera Serial Interface**

MIPI **Camera Serial Interface**; DSI is the Display Serial Interface.

Q5 **Answer: (b) 5.1 V at 5 A**

5.1 V, 5 A (27 W USB-C PD supply).

Q6 **Answer: (b) Hardware Attached on Top**

Hardware Attached on Top.

Q7 **Answer: (d) Raspberry Pi Zero 2 W**

The **Zero 2 W**.

Q8 **Answer: (a) no, it is soldered; choose the size when buying**

RAM is **soldered** next to the SoC.

Q9 **Answer: (c) the RTC with a battery on the RTC connector**

The Pi 5 has a **real-time clock**; fit the battery to keep time without power.

Q10 **Answer: (d) it is the I/O controller for GPIO, USB, Ethernet and camera/display**

RP1 is Raspberry Pi's own southbridge I/O chip.

Q11 **Answer: (b) so the Pi can identify the board and load the right drivers automatically**

The EEPROM describes the HAT for **automatic configuration**.

Q12 **Answer: (a) A2**

A2 (application performance class 2).

Q13 **Answer: (c) no undervoltage or throttling has happened since boot**

0x0: all clear since boot.

Q14 **Answer: (a) 80 °C**

Soft throttling starts at about **80 °C**, harder at 85 °C.

Q15 **Answer: (b) an M.2 HAT+ on the PCIe connector**

NVMe connects through the **PCIe** connector and an M.2 HAT.

Q16 **Answer: (d) Compute Module 5**

The **CM5** plugs into a custom carrier board.

Q17 **Answer: (c) 4.4 V**

Drop = 2.4 × 0.25 = 0.6 V; 5.0 - 0.6 = **4.4 V**, below the 4.63 V threshold: undervoltage.

Q18 **Answer: (c) undervoltage and throttling occurred earlier, but neither is happening now**

0x50000 = 0x40000 (throttling has occurred) + 0x10000 (undervoltage has occurred); the low bits are 0, so nothing is happening **now**.

Q19 **Answer: (d) improve cooling: the soft temperature limit is active and has occurred**

0x8 = soft temperature limit now; 0x80000 = it has occurred. It is a **heat** problem.

Q20 **Answer: (b) 600 mA**

With a 3 A supply the Pi 5 limits USB to **600 mA** total (1.6 A with the 5 A supply).

Q21 **Answer: (a) 6 s and 30 s**

2700/450 = **6 s**; 2700/90 = **30 s**.

Q22 **Answer: (b) the Pi 5 uses smaller 22-pin camera/display connectors**

Pi 5 connectors are **22-pin, 0.5 mm pitch**; use a 22-to-15-pin cable.

Q23 **Answer: Solution program**

int(text, 16) converts the hex string; v & mask tests one bit. 1 << 18 = 0x40000.

```
def check(text):
    v = int(text, 16)
    print(text)
    if v & 0x1:
        print("  UNDERVOLTAGE NOW")
    if v & (1 << 18):
        print("  HAS BEEN THROTTLED")

check("0x50005")
check("0x40000")
```

OUTPUT

```
0x50005
  UNDERVOLTAGE NOW
  HAS BEEN THROTTLED
0x40000
  HAS BEEN THROTTLED
```

Q24 **Answer: Solution program**

An if/elif chain tests the ranges from lowest to highest, so each temperature matches exactly one band.

```
def fan_speed(t):
    if t < 50:
        return 0
    elif t < 60:
        return 30
    elif t < 70:
        return 60
    return 100

for t in (45, 55, 67, 82):
    print(f"{t} °C -> fan {fan_speed(t)} %")
```

OUTPUT

```
45 °C -> fan 0 %
55 °C -> fan 30 %
67 °C -> fan 60 %
82 °C -> fan 100 %
```

Q25 **Answer: (a) Pi 5 8 GB, official 27 W supply, NVMe SSD on M.2 HAT+, Active Cooler, Ethernet**

RAM for containers, stable power, durable fast storage, cooling and a wired network.

Pi 5: BCM2712, PCIe, RTC, power button, 22-pin camera connectors, 5.1 V 5 A supply. Pi 4: BCM2711, 5.1 V 3 A. Use blue USB 3 ports for drives, an A2 microSD or an NVMe SSD, the official supply and cooling. Check vcgencmd get_throttled (0x0 = fine) and measure_temp (throttling from 80 °C).

MODULE 03 · BEGINNER

Setting up the operating system

Minimum duration **2 days** | Level **Beginner**

A correctly installed and configured OS is the foundation for every project in this course. This module compares operating systems, shows how to write a card with Raspberry Pi Imager and its customisation settings, walks through first boot and updates, configures interfaces with raspi-config, sets up a Pi headless and finds its IP address, boots from USB or NVMe, and backs up the card, followed by 25 solved questions.

IN THIS MODULE

3.1	Operating system choices	3.2	Raspberry Pi Imager
3.3	First boot	3.4	raspi-config and the Control Centre
3.5	Headless setup	3.6	Booting from USB or NVMe SSD
3.7	Backing up and cloning the SD card	3.8	Practice questions
3.9	Solutions		

3.1 Operating system choices

3.1.1 Raspberry Pi OS (with desktop, full, Lite)

Edition	Contents	Use it for
Raspberry Pi OS with desktop	desktop, browser, Thonny, core tools	learning, desktop projects, touchscreens
Raspberry Pi OS Full	desktop + LibreOffice, extra programming tools	a classroom or home PC
Raspberry Pi OS Lite	no desktop: command line only (about 0.5 GB)	headless servers, IoT, robots: faster boot, less RAM, smaller attack surface

3.1.2 32-bit vs 64-bit

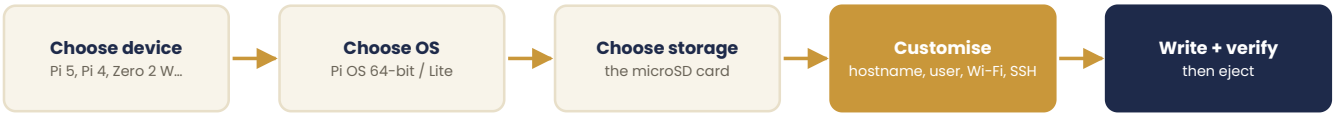
The Pi 3, 4, 5 and Zero 2 W have 64-bit CPUs. Choose **64-bit** (arm64) for them: it is faster for many tasks, lets one program use more than 4 GB RAM, and most modern software (Docker images, AI frameworks, OpenCV wheels) is published for arm64 first. Use 32-bit (armhf) only for the Pi 1, Pi 2, original Zero or special legacy software.

3.1.3 Other OSes: Ubuntu, LibreELEC, RetroPie, Home Assistant OS

OS	Purpose
Ubuntu (Desktop/Server)	standard Ubuntu; needed for ROS 2 packages (module 25)
LibreELEC	minimal Linux that boots straight into Kodi (media centre)
RetroPie / Recalbox	retro-gaming consoles
Home Assistant OS	dedicated smart-home hub (module 18)
DietPi, Alpine	very light distributions for servers

3.2 Raspberry Pi Imager

Raspberry Pi Imager is the official, free tool (Windows, macOS, Linux) that downloads an OS image and writes it to a microSD card or USB drive in one step.



THE IMAGER WORKFLOW

3.2.1 Choosing the device, OS and storage

Choosing the device first filters the OS list to images that support that board. For storage, double-check the drive: Imager erases everything on it.

3.2.2 OS customisation: hostname, username and password, Wi-Fi, locale, SSH

Setting	Example	Why
Hostname	labpi	reach it as `labpi.local` on the network
Username and password	lohit / a strong password	there is no default "pi/raspberry" account any more; you must create one
Wireless LAN	SSID, password, country IN	connects to Wi-Fi on first boot; the country sets legal radio channels
Locale	Asia/Kolkata, keyboard us	correct time and key layout
Services: SSH	enabled, password or public key	log in remotely with no screen (headless)

3.2.3 Writing and verifying the card

Imager writes the image, then reads it back and compares it (**verify**). A verify failure usually means a fake or failing card. When it finishes, eject the card safely and put it in the Pi.

3.3 First boot

3.3.1 Desktop setup wizard

The first boot expands the file system to fill the card and applies your settings; it may reboot once. With the desktop image, a wizard asks for country, user and Wi-Fi if you did not customise them in Imager, and offers to update software.

3.3.2 Connecting to Wi-Fi and Ethernet

Ethernet works automatically through DHCP. Wi-Fi is set in Imager, through the network icon on the desktop, or from the terminal with `nmcli` (module 5).

3.3.3 Updating the system

UPDATE THE SYSTEM

```
sudo apt update        # download the latest package lists
sudo apt full-upgrade  # install all available upgrades
sudo reboot            # restart if the kernel or firmware changed
```

> **TIP**
> Run these two commands on every new install and then regularly (at least monthly). `update` only refreshes the list; `full-upgrade` actually installs the new versions.

3.4 raspi-config and the Control Centre

`sudo raspi-config` opens a text menu for system settings; on the desktop the same options are under **Preferences → Control Centre** (previously "Raspberry Pi Configuration").

3.4.1 Enabling interfaces: SSH, VNC, I2C, SPI, serial

Interface	Needed for
SSH	remote terminal login
VNC	remote desktop
I2C	BME280, OLED, PCA9685 and other I2C devices
SPI	MCP3008 ADC, SPI displays
Serial port	GPS modules, talking to Arduino; also choose whether the login console uses it
1-Wire	DS18B20 temperature sensors

raspi-config also has a non-interactive mode, ideal for scripts (0 means enable):

ENABLING INTERFACES FROM A SCRIPT

```
sudo raspi-config nonint do_ssh 0
sudo raspi-config nonint do_i2c 0
sudo raspi-config nonint do_spi 0
sudo raspi-config nonint do_serial_hw 0   # enable the UART hardware
sudo raspi-config nonint do_serial_cons 1 # but no login console on it
```

Behind the scenes these options add lines to `/boot/firmware/config.txt`, for example `dtparam=i2c_arm=on` and `dtparam=spi=on`.

3.4.2 Display, locale, timezone and keyboard

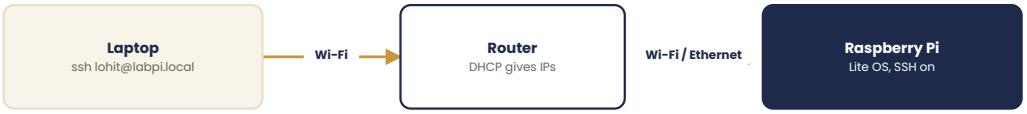
Set the timezone (Localisation → Timezone → Asia → Kolkata), locale (en_IN.UTF-8 or en_GB.UTF-8), keyboard layout and Wi-Fi country here. A wrong timezone makes every timestamp in your logs wrong.

3.4.3 Boot options

Choose boot to desktop or console, auto-login, wait for network at boot, and the splash screen. Headless servers should boot to the console.

3.5 Headless setup

3.5.1 Running without monitor, keyboard or mouse



A HEADLESS PI IS CONTROLLED OVER THE NETWORK

1. In Imager, set hostname, user and password, Wi-Fi and **enable SSH**.
2. Insert the card, power on, wait 1–2 minutes for the first boot.
3. From the laptop: `ssh lohit@labpi.local`, answer "yes" to the fingerprint question, type the password.
4. Update the system and enable the interfaces you need.

3.5.2 Finding the Pi's IP address

Method	How
mDNS name	`ping labpi.local` shows the address (works on most networks)
Router admin page	list of DHCP clients ("labpi")
Network scanner	Fing app on a phone, or `nmap -sn 192.168.1.0/24`
On the Pi itself	`hostname -I`

ON THE PI

```
pi@raspberrypi:~ $ hostname
raspberrypi
pi@raspberrypi:~ $ hostname -I
192.168.1.42
```

3.6 Booting from USB or NVMe SSD

3.6.1 Bootloader and EEPROM updates

The Pi 4 and Pi 5 keep their bootloader in an on-board **EEPROM** chip, not on the card. Keep it current so new boot devices work:

BOOTLOADER UPDATE

```
sudo rpi-eeprom-update        # show current and latest version
sudo rpi-eeprom-update -a     # install the latest, then reboot
```

3.6.2 Boot order

`BOOT_ORDER` in the EEPROM configuration lists the devices to try, read **right to left**, as hex digits: 1 = SD card, 4 = USB, 6 = NVMe, f = restart the list. Set it with raspi-config (Advanced Options → Boot Order) or `sudo rpi-eeprom-config --edit`.

BOOT_ORDER	Tries in order
0xf41	SD → USB → repeat
0xf14	USB → SD → repeat
0xf416	NVMe → SD → USB → repeat
0xf461	SD → NVMe → USB → repeat

SHORTCUT TRICK

Read BOOT_ORDER from the **right**: the last digit is tried first. 0xf416 → 6 (NVMe), then 1 (SD), then 4 (USB), then f (start again).

To move to an SSD: write the OS onto it with Imager (from a laptop or from the running Pi using the SD Card Copier tool), set the boot order, power off, remove the card, boot.

3.7 Backing up and cloning the SD card

Method	Details
SD Card Copier (desktop)	copies the running system to a USB card reader or SSD; also used to migrate to a bigger card
Image on a PC	Linux/macOS: `sudo dd if=/dev/sdX of=pi-backup.img bs=4M status=progress`; Windows: Win32 Disk Imager "Read"

Method	Details
rpi-clone / rsync	incremental file-level backups to another drive (module 19)

WATCH OUT

`dd` copies whatever device you name. Swapping `if` and `of`, or naming your laptop's own disk, destroys data. Check the device name with `lsblk` first.

PRACTICE CHECKPOINT

Install Raspberry Pi OS headless using Imager, connect over the network, update the system and enable I2C, SPI and SSH.

3.8 Practice questions

3.8.1 Basic level

Q1 Which edition of Raspberry Pi OS has no desktop?

(a) Full (b) with desktop (c) Lite (d) Server Plus

Q2 Which tool is the official way to write an OS to a microSD card?

(a) Thonny (b) raspi-config
(c) Kodi (d) Raspberry Pi Imager

Q3 Which command refreshes the list of available packages?

(a) sudo apt full-upgrade (b) sudo reboot
(c) sudo raspi-config (d) sudo apt update

Q4 Which command opens the text-based configuration tool?

(a) sudo config (b) sudo pi-setup
(c) sudo raspi-config (d) nano config

Q5 What does "headless" mean?

(a) running the Pi without monitor, keyboard and mouse, controlled over the network
(b) running without an SD card
(c) running without power
(d) running without an operating system

Q6 Which command shows the Pi's IP address on the Pi itself?

(a) ls -ip (b) ipconfig (c) show ip (d) hostname -I

Q7 Which architecture should you choose for a Pi 5?

(a) 32-bit (armhf) (b) 64-bit (arm64) (c) x86-64 (d) RISC-V

Q8 Which OS boots straight into the Kodi media centre?

(a) Home Assistant OS (b) LibreELEC
(c) Raspberry Pi OS Lite (d) Ubuntu Server

3.8.2 Intermediate level

Q9 Why should you set the Wi-Fi country in Imager?

(a) it sets the keyboard layout
(b) it sets the language of menus
(c) it is only for statistics
(d) it selects the legal radio channels; Wi-Fi may stay disabled without it

Q10 Which hostname lets you connect without knowing the IP address, if the hostname is "labpi"?

(a) labpi.local (b) labpi.com (c) www.labpi (d) 192.168.labpi

Q11 Which file receives lines such as dtparam=i2c_arm=on?

(a) /etc/hosts (b) /home/pi/.bashrc

(c) /boot/firmware/config.txt (d) /var/log/syslog

Q12 What does "verify" in Imager do?

(a) checks your Wi-Fi password
(b) tests the Pi hardware
(c) updates the bootloader
(d) reads the card back and compares it with the image

Q13 Which raspi-config command enables I2C from a script?

(a) sudo raspi-config nonint do_i2c 1 (b) sudo raspi-config nonint do_i2c 0
(c) sudo i2c enable (d) sudo apt install i2c

Q14 Where is the Pi 4/Pi 5 bootloader stored?

(a) in an EEPROM chip on the board (b) on the microSD card only
(c) in the GPU (d) on a USB stick

Q15 Why is Lite preferred for a headless server?

(a) it is the only edition with SSH
(b) it has a better desktop
(c) it uses less RAM and storage, boots faster and runs fewer services
(d) it includes LibreOffice

Q16 Which settings must be customised in Imager for a headless Pi on Wi-Fi?

(a) user and password, Wi-Fi SSID and password, country, and SSH enabled
(b) only the hostname
(c) only the keyboard layout
(d) nothing; defaults work

3.8.3 Advanced level

Q17 BOOT_ORDER is 0xf416. Which device is tried first?

(a) SD card (b) NVMe (c) USB (d) network

Q18 You want SD first, then USB, then repeat. Which BOOT_ORDER is correct?

(a) 0xf14 (b) 0x14f (c) 0xf16 (d) 0xf41

Q19 Which BOOT_ORDER tries USB first, then NVMe, then SD, then repeats?

(a) 0xf461 (b) 0xf416 (c) 0xf164 (d) 0x461f

Q20 A Lite image is 0.5 GB compressed and downloads at 2.5 MB/s. About how long does the download take (1 GB = 1000 MB)?

(a) 20 s (b) 2 000 s (c) 200 s (d) 1 250 s

Q21 After writing a card you cannot find labpi.local, but the router lists labpi at 192.168.1.57. What should you do?

(a) re-flash the card (b) buy a new Pi
(c) connect a monitor to enable SSH (d) connect with ssh user@192.168.1.57

Q22 A student swapped if and of in a dd backup command. What happened?

(a) nothing; dd detects mistakes
(b) the old image file was written over the SD card, destroying its contents
(c) the backup became faster
(d) dd made two copies

Q23 Which order is right for a new headless install?

(a) Imager with customisation → boot → ssh → apt update and full-upgrade → enable interfaces
(b) boot → Imager → ssh
(c) ssh → Imager → boot
(d) apt update → Imager → boot

Q24 Show the hostname and IP address of the Pi, then print the Raspberry Pi temperature and throttled state in one go.

Q25 Which command enables SSH and disables the serial login console but keeps the UART hardware on, in non-interactive mode?

(a) do_ssh 1, do_serial_hw 1, do_serial_cons 0
(b) do_ssh 0, do_serial_hw 0, do_serial_cons 1
(c) do_ssh 0, do_serial_hw 1, do_serial_cons 1
(d) do_ssh 0 only

3.9 Solutions

Q1 **Answer: (c) Lite**

Lite is command-line only.

Q2 **Answer: (d) Raspberry Pi Imager**

Raspberry Pi Imager.

Q3 **Answer: (d) sudo apt update**

`apt update` only refreshes the lists; `full-upgrade` installs.

Q4 **Answer: (c) sudo raspi-config**

`sudo raspi-config`.

Q5 **Answer: (a) running the Pi without monitor, keyboard and mouse, controlled over the network**

Headless = **no screen or keyboard**; use SSH.

Q6 **Answer: (d) hostname -I**

`hostname -I` prints all IP addresses.

Q7 **Answer: (b) 64-bit (arm64)**

Use **64-bit** on all 64-bit capable Pis.

Q8 **Answer: (b) LibreELEC**

LibreELEC.

Q9 **Answer: (d) it selects the legal radio channels; Wi-Fi may stay disabled without it**

The country controls **legal Wi-Fi channels**; Raspberry Pi OS blocks Wi-Fi until it is set.

Q10 **Answer: (a) labpi.local**

mDNS answers to **hostname.local**.

Q11 **Answer: (c) /boot/firmware/config.txt**

Hardware settings live in **/boot/firmware/config.txt** (on older systems /boot/config.txt).

Q12 **Answer: (d) reads the card back and compares it with the image**

Verify detects bad or fake cards.

Q13 **Answer: (b) sudo raspi-config nonint do_i2c 0**

In nonint mode **0 means enable**, 1 means disable.

Q14 **Answer: (a) in an EEPROM chip on the board**

It is in the on-board **EEPROM**, updated with rpi-eeprom-update.

Q15 **Answer: (c) it uses less RAM and storage, boots faster and runs fewer services**

Fewer packages: **lighter and more secure**.

Q16 **Answer: (a) user and password, Wi-Fi SSID and password, country, and SSH enabled**

Without Wi-Fi and SSH set, you cannot reach it.

Q17 **Answer: (b) NVMe**

Read from the right: **6 = NVMe**, then 1 (SD), then 4 (USB), then f (repeat).

Q18 **Answer: (d) 0xf41**

Right to left: 1 (SD), 4 (USB), f: **0xf41**.

Q19 **Answer: (c) 0xf164**

Rightmost first: 4 (USB), 6 (NVMe), 1 (SD), f → **0xf164**.

Q20 **Answer: (c) 200 s**

500 MB ÷ 2.5 MB/s = **200 s** (3 min 20 s).

Q21 **Answer: (d) connect with ssh user@192.168.1.57**

mDNS may be blocked on some networks; the **IP address** always works.

Q22 **Answer: (b) the old image file was written over the SD card, destroying its contents**

`if` = input, `of` = output; swapping them **overwrites the card**.

Q23 **Answer: (a) Imager with customisation → boot → ssh → apt update and full-upgrade → enable interfaces**

Write and customise, boot, connect, **update**, then configure.

Q24 **Answer: Commands**

`;` runs commands one after another on one line.

```
pi@raspberrypi:~ $ hostname
raspberrypi
pi@raspberrypi:~ $ hostname -I
192.168.1.42
pi@raspberrypi:~ $ vcgencmd measure_temp; vcgencmd get_throttled
temp=47.2'C
throttled=0x0
```

Q25 **Answer: (b) do_ssh 0, do_serial_hw 0, do_serial_cons 1**

0 enables, 1 disables: SSH on (0), UART hardware on (0), console off (1).

Use Raspberry Pi Imager: device → OS (64-bit; Lite for servers) → storage → customise (hostname, user, Wi-Fi, country, SSH) → write and verify. Then ssh user@host.local, sudo apt update && sudo apt full-upgrade, and enable interfaces with raspi-config. BOOT_ORDER is read right to left.

MODULE 04 · BEGINNER

Linux command line essentials

Minimum duration **3 days** | Level **Beginner**

The terminal is how professionals control a Raspberry Pi, so this module builds real command-line fluency: the shell and getting help, the Linux directory tree and paths, file commands, nano and vim, users, permissions and groups, apt package management, processes and monitoring including vcgencmd, pipes, redirection and wildcards, and a first look at Bash scripts. Every transcript is a real run, followed by 25 solved questions and tasks.

IN THIS MODULE

4.1 Terminal basics
4.2 The Linux file system
4.3 File and directory commands
4.4 Text editors
4.5 Users and permissions
4.6 Package management
4.7 Processes and system monitoring
4.8 Pipes, redirection and wildcards
4.9 Introduction to shell scripts
4.10 Practice questions
4.11 Solutions

4.1 Terminal basics

4.1.1 Shell, prompt and command structure

The **terminal** is a window; the **shell** (normally **Bash**) is the program inside it that reads your commands and runs them. The **prompt** tells you who and where you are:

READING THE PROMPT

A command has a name, then **options** (starting with - or --) that change how it behaves, then **arguments** (what to act on): `ls -l /etc` = command `ls`, option `-l` (long listing), argument `/etc`. Linux is **case-sensitive**: `File.txt` and `file.txt` are different files.

4.1.2 Getting help: man, --help

`man ls` opens the manual page (Space = next page, / = search, q = quit). `ls --help` prints a short summary. `apropos copy` searches manual titles.

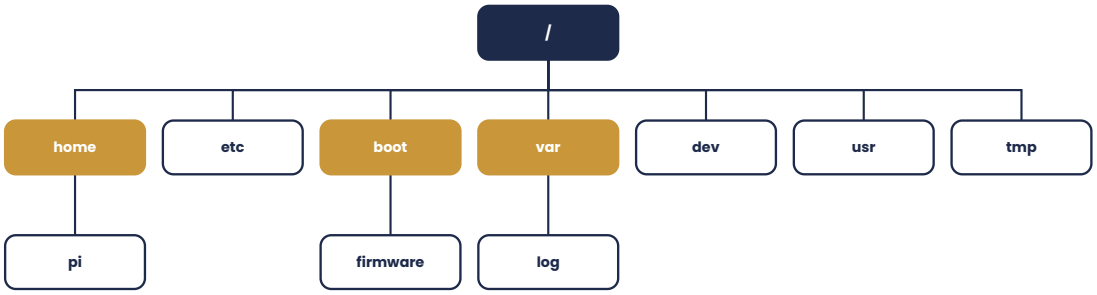
4.1.3 Tab completion and history

Key	Effect
Tab	complete a command or file name; press twice to list choices
↑ / ↓	previous / next command from history
Ctrl+R	search history as you type
Ctrl+C	stop the running program
Ctrl+L or clear	clear the screen
!!	repeat the last command (sudo !! repeats it with sudo)
history	list previous commands with numbers

SHORTCUT TRICK
Type two or three letters and press **Tab**. It saves time and, more importantly, prevents typing mistakes in long file paths.

4.2 The Linux file system

4.2.1 Directory tree: /, /home, /etc, /boot/firmware, /var, /dev



EVERYTHING HANGS OFF ONE ROOT DIRECTORY /

Directory	Contents
/	the root of the whole tree (there are no drive letters)
/home/pi	your files; shortcut ~
/etc	system configuration files (hostname, fstab, ssh)
/boot/firmware	boot files: config.txt, cmdline.txt, kernel
/var/log	log files
/dev	devices as files: /dev/i2c-1, /dev/spidev0.0, /dev/ttyAMA0, /dev/gpiochip0
/usr/bin	installed programs
/tmp	temporary files, cleared at reboot

Directory	Contents
/media, /mnt	mounted USB drives and disks

4.2.2 Absolute and relative paths

An **absolute path** starts at `/` and works from anywhere: `/home/pi/projects/led.py`. A **relative path** starts from the current folder: `projects/led.py`. Special names: `.` = this folder, `..` = parent folder, `~` = home folder, `-` (with cd) = previous folder.

4.3 File and directory commands

4.3.1 pwd, ls, cd

MOVING AROUND

```
pi@raspberrypi:~ $ pwd
/home/pi
pi@raspberrypi:~ $ mkdir -p projects/weather/data projects/robot
pi@raspberrypi:~ $ ls
projects
pi@raspberrypi:~ $ cd projects
pi@raspberrypi:~/projects $ ls -l
total 8
drwxr-xr-x 2 pi pi 4096 Nov  2 00:16 robot
drwxr-xr-x 3 pi pi 4096 Nov  2 00:16 weather
pi@raspberrypi:~/projects $ cd weather/data
pi@raspberrypi:~/projects/weather/data $ pwd
/home/pi/projects/weather/data
pi@raspberrypi:~/projects/weather/data $ cd ../..
pi@raspberrypi:~/projects $ pwd
/home/pi/projects
pi@raspberrypi:~/projects $ cd ~
pi@raspberrypi:~ $ pwd
/home/pi
```

Command	Meaning
ls -l	long listing: permissions, owner, size, date
ls -a	show hidden files (names starting with .)
ls -lh	sizes in K, M, G
ls -R	list subfolders recursively

4.3.2 mkdir, touch, cp, mv, rm

CREATING, COPYING, RENAMING AND DELETING

```
pi@raspberrypi:~ $ mkdir notes
pi@raspberrypi:~ $ touch notes/a.txt notes/b.txt
pi@raspberrypi:~ $ cp notes/a.txt notes/a_backup.txt
pi@raspberrypi:~ $ mv notes/b.txt notes/readme.txt
pi@raspberrypi:~ $ ls notes
a.txt
a_backup.txt
readme.txt
pi@raspberrypi:~ $ rm notes/a_backup.txt
pi@raspberrypi:~ $ cp -r notes notes2
pi@raspberrypi:~ $ rm -r notes2
pi@raspberrypi:~ $ ls
notes
```

WATCH OUT

`rm` does not use a recycle bin: deleted files are gone. `rm -r` deletes whole folders. Double-check before pressing Enter, especially with `sudo` and wildcards; `rm -i` asks before each file.

4.3.3 cat, less, head, tail

VIEWING FILES

```
pi@raspberrypi:~ $ seq 1 10 > numbers.txt
pi@raspberrypi:~ $ cat numbers.txt | tr "\n" " "; echo
1 2 3 4 5 6 7 8 9 10
pi@raspberrypi:~ $ head -n 3 numbers.txt
1
2
3
pi@raspberrypi:~ $ tail -n 2 numbers.txt
9
10
pi@raspberrypi:~ $ wc -l numbers.txt
10 numbers.txt
```

`less file` pages through long files (q quits). `tail -f /var/log/syslog` follows a log live as lines are added: very useful when debugging a service.

4.3.4 find and grep

SEARCHING FOR FILES AND FOR TEXT INSIDE FILES

```
pi@raspberrypi:~ $ find . -name "*.py"
./code/led.py
./code/test.py
pi@raspberrypi:~ $ grep -n gpiozero code/*.py
code/led.py:2:from gpiozero import LED
pi@raspberrypi:~ $ grep -rl "import time" .
./code/led.py
./code/test.py
pi@raspberrypi:~ $ grep -c import code/led.py
2
```

Option	Meaning
grep -i	ignore case
grep -n	show line numbers
grep -r	search folders recursively
grep -v	show lines that do NOT match
grep -c	count matching lines
find -name "*.log" -mtime -1	files modified in the last day

4.4 Text editors

4.4.1 nano

`nano file.txt` opens a simple editor; the shortcuts are listed at the bottom (^ means Ctrl). **Ctrl+O** then Enter saves, **Ctrl+X** exits, Ctrl+W searches, Ctrl+K cuts a line, Ctrl+U pastes. Edit system files with `sudo nano /boot/firmware/config.txt`.

4.4.2 Introduction to vim

vim is a powerful editor found on every Linux server. It has **modes**: press `i` to insert text, **Esc** to return to normal mode, then `:w` save, `:q` quit, `:wq` save and quit, `:q!` quit without saving, `dd` delete a line, `/word` search.

> **SHORTCUT TRICK**
> Stuck in vim? Press **Esc**, type **:q!** and press Enter. You leave without saving anything.

4.5 Users and permissions

4.5.1 sudo and the root user

root is the administrator account that can change anything. Normal users run one command as root by putting `sudo` in front (you type your own password). Use sudo only when needed: installing software,

editing files in /etc, rebooting.

4.5.2 Read, write and execute permissions

LS -L PERMISSION STRING FOR A FILE WITH MODE 754

Letter	On a file	On a folder	Value
r	read the contents	list the names	4
w	change the contents	create/delete files inside	2
x	run it as a program	enter it with cd	1

SHORTCUT TRICK
Add the values for each group: rwx = 4+2+1 = **7**, r-x = 4+1 = **5**, r-- = **4**, rw- = **6**. So rwxr-xr-- = **754**, rw-r--r-- = **644** (normal file), rwxr-xr-x = **755** (script or folder), rw------- = **600** (private key).

4.5.3 chmod and chown

CHANGING PERMISSIONS

```
pi@raspberrypi:~ $ echo "echo Hello from a script" > hello.sh
pi@raspberrypi:~ $ ls -l hello.sh
-rw-r--r-- 1 pi pi 25 Nov  2 00:16 hello.sh
pi@raspberrypi:~ $ ./hello.sh
-bash: ./hello.sh: Permission denied
pi@raspberrypi:~ $ chmod +x hello.sh
pi@raspberrypi:~ $ ls -l hello.sh
-rwxr-xr-x 1 pi pi 25 Nov  2 00:16 hello.sh
pi@raspberrypi:~ $ ./hello.sh
Hello from a script
pi@raspberrypi:~ $ chmod 600 hello.sh
pi@raspberrypi:~ $ ls -l hello.sh
-rw------- 1 pi pi 25 Nov  2 00:16 hello.sh
pi@raspberrypi:~ $ chmod u=rwx,g=rx,o= hello.sh
pi@raspberrypi:~ $ stat -c "%A %a %n" hello.sh
-rwxr-x--- 750 hello.sh
```

`sudo chown pi:pi file` changes the owner and group; `-R` applies it to a whole folder.

4.5.4 Groups: gpio, i2c, spi, dialout

Hardware devices in /dev belong to groups. A user in the **gpio**, **i2c**, **spi** or **dialout** (serial ports) group can use that hardware without sudo. The first user created by Imager is already in them. Add another user with `sudo usermod -aG gpio,i2c,spi,dialout username` (then log out and in). Check with `groups`.

WATCH OUT
Always use `-aG` (append). `usermod -G gpio user` without `-a` **removes** the user from all other groups, including sudo.

4.6 Package management

4.6.1 apt update, upgrade, install, remove

Command	Effect
sudo apt update	refresh package lists
sudo apt full-upgrade	upgrade everything installed
sudo apt install htop git	install packages (and their dependencies)
sudo apt remove htop	uninstall (keeps config files)
sudo apt purge htop	uninstall including config files
sudo apt autoremove	remove dependencies nothing needs any more
sudo apt clean	delete downloaded package files to free space

4.6.2 Searching for packages

`apt search mosquitto` searches names and descriptions; `apt show mosquitto` shows details; `apt list --installed | grep python3` lists what is installed.

4.7 Processes and system monitoring

4.7.1 ps, top, htop, kill

RUNNING A PROGRAM IN THE BACKGROUND AND STOPPING IT

```
pi@raspberrypi:~ $ python3 logger.py &
[1] 2417
pi@raspberrypi:~ $ ps aux | grep [l]ogger
pi        2417  0.3  0.2  21340  9876 pts/0    S    10:02   0:00 python3 logger.py
pi@raspberrypi:~ $ kill 2417
[1]+  Terminated              python3 logger.py
```

Command	Use
ps aux	every process with user, CPU %, memory %
top / htop	live view sorted by CPU (q quits); htop is colourful and easier
kill PID	ask a process to stop (signal TERM)
kill -9 PID	force it to stop (KILL) when it ignores TERM
pkill -f script.py	stop processes by name
command &	run in the background; jobs, fg, bg manage it

4.7.2 df, du, free

DISK AND MEMORY USAGE

```
pi@raspberrypi:~ $ df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/mmcblk0p2   29G  5.1G   23G  19% /
pi@raspberrypi:~ $ free -h
               total        used        free      shared  buff/cache   available
Mem:           7.9Gi       612Mi       6.4Gi        41Mi       1.0Gi       7.3Gi
Swap:          511Mi          0B       511Mi
pi@raspberrypi:~ $ du -sh ~/projects
148K    /home/pi/projects
```

`df -h` = free space on each file system (mmcblk0 is the SD card, nvme0n1 an NVMe SSD). `du -sh folder` = space used by a folder. `free -h` = RAM; look at "available".

4.7.3 vcgencmd: temperature, throttling and clocks

PI-SPECIFIC MEASUREMENTS

```
pi@raspberrypi:~ $ vcgencmd measure_temp
temp=47.2'C
pi@raspberrypi:~ $ vcgencmd measure_clock arm
frequency(0)=2400000000
pi@raspberrypi:~ $ vcgencmd get_throttled
throttled=0x0
pi@raspberrypi:~ $ vcgencmd measure_temp | cut -d= -f2 | cut -d"'" -f1
47.2
```

The last command extracts just the number, ready for logging. The kernel also exposes the temperature in millidegrees in `/sys/class/thermal/thermal_zone0/temp` (for example 47200 = 47.2 °C).

4.8 Pipes, redirection and wildcards

Symbol	Meaning	Example
>	send output to a file (overwrite)	ls > files.txt
>>	append output to a file	date >> log.txt
<	read input from a file	sort < names.txt
2>	redirect error messages	cmd 2> errors.txt
&> or >f 2>&1	output and errors together	cmd > all.txt 2>&1
|	pipe: output of one command becomes input of the next	ps aux | grep python
&&	run the next command only if the first succeeded	sudo apt update && sudo apt full-upgrade
*	any characters	ls *.py
?	exactly one character	ls log?.txt

Symbol	Meaning	Example
[ ]	one of the listed characters	ls data[12].csv

PIPES AND WILDCARDS IN ACTION

```
pi@raspberrypi:~ $ printf "banana\napple\ncherry\napple\n" > fruit.txt
pi@raspberrypi:~ $ sort fruit.txt | uniq -c
      2 apple
      1 banana
      1 cherry
pi@raspberrypi:~ $ sort fruit.txt | uniq | wc -l
3
pi@raspberrypi:~ $ touch log1.txt log2.txt log10.txt data1.csv
pi@raspberrypi:~ $ ls log?.txt
log1.txt
log2.txt
pi@raspberrypi:~ $ ls *.txt | wc -l
4
pi@raspberrypi:~ $ echo "first" > out.txt; echo "second" >> out.txt; cat out.txt
first
second
pi@raspberrypi:~ $ ls nofile 2> err.txt; cat err.txt
ls: cannot access 'nofile': No such file or directory
```

4.9 Introduction to shell scripts

4.9.1 Variables, conditions and loops in Bash

TEMPLOG.SH

```
#!/bin/bash
# templog.sh - log the CPU temperature 3 times with a warning above a limit
LIMIT=60
LOG=temps.log
for i in 1 2 3; do
    T=$(vcgencmd measure_temp | cut -d= -f2 | cut -d"'" -f1)
    echo "$(date +%H:%M:%S) $T" >> "$LOG"
    if (( $(echo "$T > $LIMIT" | bc) )); then
        echo "Reading $i: $T C  WARNING: hot"
    else
        echo "Reading $i: $T C  ok"
    fi
done
echo "Lines in $LOG: $(wc -l < "$LOG")"
```

OUTPUT

```
Reading 1: 47.2 C  ok
Reading 2: 47.2 C  ok
Reading 3: 47.2 C  ok
Lines in temps.log: 3
```

Bash feature	Syntax
variable	NAME=value (no spaces round =), use as "$NAME"
command output	X=$(command)
if	if ["$A" = "yes"]; then … elif …; else …; fi
number tests	[$N -gt 5] (-eq -ne -lt -le -gt -ge) or ((N > 5))
file tests	[-f file] exists, [-d dir] folder, [-x file] executable
for loop	for f in *.csv; do … done
while loop	while [$N -lt 10]; do N=$((N+1)); done
arguments	$1, $2 … ; $# = how many; $@ = all

Bash arithmetic works on whole numbers only, so the script uses `bc` to compare decimals such as 47.2.

4.9.2 Making a script executable

1. First line `#!/bin/bash` (the "shebang") says which interpreter runs it.
2. `chmod +x templog.sh` gives execute permission.
3. Run it with `./templog.sh` (the ./ means "in this folder"), or put it in ~/bin or /usr/local/bin to run it by name from anywhere.

PRACTICE CHECKPOINT

Using only the terminal, create a project folder structure, install a package, write a Bash script that logs CPU temperature to a file, and set correct permissions on it.

4.10 Practice questions

4.10.1 Basic level

Q1 Which command prints the current working directory?

(a) pwd (b) cwd (c) dir (d) where

Q2 What does ~ stand for in a path?

(a) the root folder
(b) the previous folder
(c) the current user's home folder
(d) a hidden file

Q3 Which command shows hidden files?

(a) ls -h (b) ls -r (c) ls -x (d) ls -a

Q4 Which directory holds system configuration files?

(a) /home (b) /dev (c) /tmp (d) /etc

Q5 Which key completes a file name in the terminal?

(a) Ctrl+C (b) Esc (c) Tab (d) F1

Q6 Which command copies a whole folder?

(a) cp -r src dest (b) cp src dest (c) mv -c src dest (d) copy src dest

Q7 Which command installs a package?

(a) apt get name
(b) sudo pip install-apt name
(c) install name
(d) sudo apt install name

Q8 How do you save in nano?

(a) Ctrl+S
(b) Ctrl+O then Enter
(c) :w
(d) Esc then s

4.10.2 Intermediate level

Q9 What is the numeric mode of rwxr-xr--?

(a) 744 (b) 754 (c) 755 (d) 654

Q10 Which mode gives the owner read/write and nobody else anything?

(a) 644 (b) 700 (c) 066 (d) 600

Q11 What does chmod +x script.sh do?

(a) adds execute permission so it can be run
(b) deletes the script
(c) makes it read-only
(d) changes its owner

Q12 Which redirection appends to a file instead of overwriting it?

(a) > (b) < (c) >> (d) |

Q13 What does the pipe in ps aux | grep python do?

(a) runs both commands at the same time on the same file
(b) saves the output to a file named grep
(c) stops python processes
(d) sends the output of ps aux into grep, which keeps lines containing python

Q14 Which group lets a user open serial ports such as /dev/ttyUSB0?

(a) serial (b) dialout (c) tty-users (d) usb

Q15 Which ls pattern matches log1.txt and log2.txt but not log10.txt?

(a) log?.txt (b) log*.txt (c) log[10].txt (d) *.txt

Q16 You are in /home/pi/projects/weather. Where does cd ../robot take you?

(a) /home/pi/robot
(b) /robot
(c) /home/pi/projects/robot
(d) /home/pi/projects/weather/robot

4.10.3 Advanced level

Q17 Why is usermod -G gpio alice dangerous?

(a) without -a it replaces all of alice's groups, removing sudo and others
(b) it deletes alice
(c) it gives alice root
(d) nothing; it is safe

Q18 What does sudo apt update && sudo apt full-upgrade do if apt update fails?

(a) full-upgrade runs anyway
(b) full-upgrade does not run
(c) both run in parallel
(d) the Pi reboots

Q19 A file reads 52800 in /sys/class/thermal/thermal_zone0/temp. What is the temperature?

(a) 528 °C (b) 5.28 °C (c) 52 800 °C (d) 52.8 °C

Q20 Create folders project/src, project/data and project/logs in one command, create an empty file project/src/main.py, then list the whole tree.

Q21 A file names.txt contains names with duplicates. Print the unique names in alphabetical order and how many unique names there are.

Q22 Make backup.sh executable by its owner only (rwx------), confirm the mode, then run it.

Q23 Write a Bash script count.sh that takes a folder name as an argument and prints how many .py files it contains, or an error message if the folder does not exist.

Q24 Write templimit.sh: read the temperature with vcgencmd, print it, and print "COOLING NEEDED" if it is above 45 °C (compare as an integer using the part before the decimal point).

Q25 Which command finds every .log file under /var/log changed in the last day?

(a) grep -r .log /var/log
(b) ls /var/log/*.log -1
(c) find /var/log -name "*.log" -mtime -1
(d) find /var/log -size -1

4.11 Solutions

Q1 **Answer: (a) pwd**

`pwd` = print working directory.

Q2 **Answer: (c) the current user's home folder**

~ = /home/username.

Q3 **Answer: (d) ls -a**

`-a` = all, including names starting with a dot.

Q4 **Answer: (d) /etc**

/etc.

Q5 **Answer: (c) Tab**

Tab completion.

Q6 **Answer: (a) cp -r src dest**

`-r` = recursive.

Q7 **Answer: (d) sudo apt install name**

`sudo apt install`.

Q8 **Answer: (b) Ctrl+O then Enter**

Ctrl+O (write Out); Ctrl+X exits.

Q9 **Answer: (b) 754**

rwx = 7, r-x = 5, r-- = 4 → **754**.

Q10 **Answer: (d) 600**

rw- = 6 for the owner, 0 for group and others: **600**.

Q11 **Answer: (a) adds execute permission so it can be run**

+x = add **execute**.

Q12 **Answer: (c) >>**

`>>` appends.

Q13 **Answer: (d) sends the output of ps aux into grep, which keeps lines containing python**

A pipe connects one command's **output** to the next one's **input**.

Q14 **Answer: (b) dialout**

Serial devices belong to **dialout**.

Q15 **Answer: (a) log?.txt**

`?` matches exactly one character.

Q16 **Answer: (c) /home/pi/projects/robot**

.. = /home/pi/projects, then robot.

Q17 **Answer: (a) without -a it replaces all of alice's groups, removing sudo and others**

Use **-aG** to append.

Q18 **Answer: (b) full-upgrade does not run**

&& runs the second command **only on success**.

Q19 **Answer: (d) 52.8 °C**

The value is in millidegrees: 52800 ÷ 1000 = **52.8 °C**.

Q20 **Answer: Commands**

`mkdir -p` creates parents as needed and several folders at once; `ls -R` lists recursively.

```
pi@raspberrypi:~ $ mkdir -p project/src project/data project/logs
pi@raspberrypi:~ $ touch project/src/main.py
pi@raspberrypi:~ $ ls -R project
project:
data
logs
src

project/data:

project/logs:

project/src:
main.py
```

Q21 **Answer: Commands**

`uniq` only removes **adjacent** duplicates, so sort first.

```
pi@raspberrypi:~ $ printf "Ravi\nAsha\nRavi\nZoya\nAsha\n" > names.txt
pi@raspberrypi:~ $ sort names.txt | uniq
Asha
Ravi
Zoya
pi@raspberrypi:~ $ sort names.txt | uniq | wc -l
3
```

Q22 **Answer: Commands**

7 for the owner, 0 for group and others.

```
pi@raspberrypi:~ $ printf "#!/bin/bash\necho Backup started\n" > backup.sh
pi@raspberrypi:~ $ chmod 700 backup.sh
pi@raspberrypi:~ $ stat -c "%A %a" backup.sh
-rwx------ 700
pi@raspberrypi:~ $ ./backup.sh
Backup started
```

Q23 **Answer: Solution script**

$# counts arguments, [-d] tests for a folder, and exit codes other than 0 signal errors. Run here with the argument `code`, a folder holding three .py files.

```
#!/bin/bash
if [ $# -ne 1 ]; then
    echo "Usage: $0 folder"; exit 1
fi
if [ ! -d "$1" ]; then
    echo "Error: $1 is not a folder"; exit 2
fi
N=$(find "$1" -name "*.py" | wc -l)
echo "$1 contains $N Python files"
```

OUTPUT

```
code contains 3 Python files
```

Q24 **Answer: Solution script**

${T%.*} removes the shortest ".something" from the end, leaving the whole number, which [-gt] can compare.

```
#!/bin/bash
T=$(vcgencmd measure_temp | cut -d= -f2 | cut -d"'" -f1)
WHOLE=${T%.*}
echo "Temperature: $T C"
if [ "$WHOLE" -gt 45 ]; then
    echo "COOLING NEEDED"
fi
```

OUTPUT

```
Temperature: 47.2 C
COOLING NEEDED
```

Q25 **Answer: (c) find /var/log -name "*.log" -mtime -1**

`-mtime -1` = modified less than 1 day ago.

Navigate with pwd, ls, cd (absolute paths start with /). Manage files with mkdir -p, cp -r, mv, rm (no undo). Permissions r=4 w=2 x=1 per owner/group/others (644 files, 755 scripts, 600 secrets). Install with sudo apt. Combine commands with | > >> && and wildcards. Scripts need #!/bin/bash and chmod +x.

MODULE 05 · BEGINNER

Remote access and networking basics

Minimum duration **2 days** | Level **Beginner**

Most Raspberry Pi projects run without a screen, so remote access is essential. This module explains IP addresses, subnets, gateways, DHCP, static addresses and mDNS, checks the network from the terminal, logs in with SSH and SSH keys, copies files with scp, rsync and SFTP, opens the desktop remotely with VNC and Raspberry Pi Connect, configures networks with NetworkManager and nmcli, and edits code on the Pi from a laptop with VS Code Remote-SSH, followed by 25 solved questions.

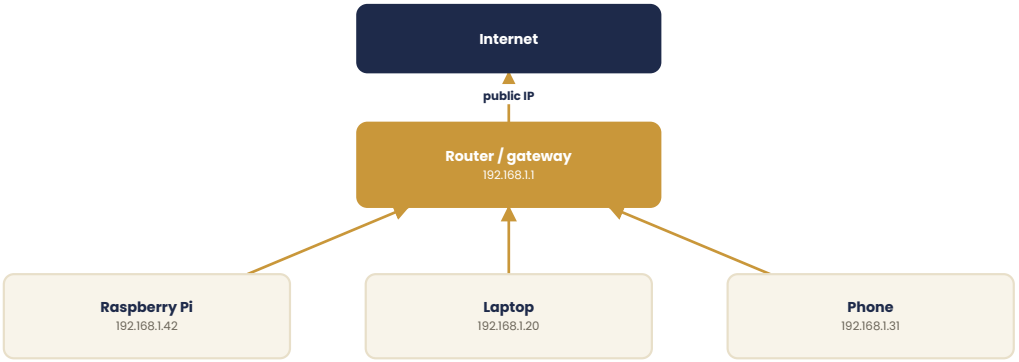
IN THIS MODULE

5.1 Networking fundamentals
5.2 SSH
5.3 Remote desktop
5.4 Network configuration
5.5 Remote development
5.6 Practice questions
5.7 Solutions

5.1 Networking fundamentals

5.1.1 IP addresses, subnets and gateways

Every device on a network has an **IP address**. An IPv4 address is four numbers from 0 to 255, for example `192.168.1.42` (32 bits). The **subnet mask** (or prefix such as `/24`) says how many leading bits identify the network; the rest identify the device. Devices in the same subnet talk directly; to reach anything else (the internet) they send packets to the **gateway**, normally the router.



A HOME NETWORK: SUBNET 192.168.1.0/24, GATEWAY 192.168.1.1

Prefix	Mask	Usable host addresses
/24	255.255.255.0	$2^8 - 2 = 254$
/16	255.255.0.0	$2^{16} - 2 = 65\,534$

Prefix	Mask	Usable host addresses
/28	255.255.255.240	$2^4 - 2 = 14$

SHORTCUT TRICK

Usable hosts in a /n subnet = $2^{(32 - n)} - 2$ (the first address names the network, the last is the broadcast address). Private ranges you will see at home: **192.168.x.x**, **10.x.x.x** and **172.16–31.x.x**.

SUBNET.PY

```
import ipaddress

net = ipaddress.ip_network("192.168.1.0/24")
print("Network:  ", net.network_address)
print("Broadcast:", net.broadcast_address)
print("Hosts:    ", net.num_addresses - 2)
for ip in ("192.168.1.42", "192.168.2.42"):
    print(ip, "in subnet?", ipaddress.ip_address(ip) in net)
```

OUTPUT

```
Network:   192.168.1.0
Broadcast: 192.168.1.255
Hosts:     254
192.168.1.42 in subnet? True
192.168.2.42 in subnet? False
```

5.1.2 DHCP and static IP addresses

DHCP (Dynamic Host Configuration Protocol): the router hands out an address, mask, gateway and DNS server when a device joins. Addresses can change after a reboot. A server you connect to often should have a **fixed address**: either a **DHCP reservation** on the router (best: the router always gives that device the same IP) or a **static IP** set on the Pi (5.4.2).

5.1.3 Hostnames and mDNS (raspberrypi.local)

DNS turns names like `raspberrypi.com` into addresses. On a local network the Pi announces its own name through **mDNS** (multicast DNS, the Avahi service), so `labpi.local` works without any DNS server. Windows 10/11, macOS, Linux, iOS and Android support it.

5.1.4 Checking the network: ip, ping, hostname -I

CHECKING ADDRESSES, ROUTES AND CONNECTIVITY

```
pi@raspberrypi:~ $ ip -brief address
lo       UNKNOWN  127.0.0.1/8 ::1/128
eth0     DOWN
wlan0    UP       192.168.1.42/24 fe80::5c1a:2b3c:4d5e:6f70/64
pi@raspberrypi:~ $ ip route
default via 192.168.1.1 dev wlan0 proto dhcp src 192.168.1.42 metric 600
192.168.1.0/24 dev wlan0 proto kernel scope link src 192.168.1.42 metric 600
pi@raspberrypi:~ $ ping -c 3 192.168.1.1
64 bytes from 192.168.1.1: icmp_seq=1 ttl=64 time=2.31 ms
64 bytes from 192.168.1.1: icmp_seq=2 ttl=64 time=1.87 ms
64 bytes from 192.168.1.1: icmp_seq=3 ttl=64 time=2.05 ms
--- 192.168.1.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
```

Command	Tells you
ip a / ip -brief a	addresses on each interface (eth0 = Ethernet, wlan0 = Wi-Fi, lo = loopback)
ip route	the default gateway
ping -c 3 host	whether a host answers and how fast (Ctrl+C to stop without -c)
hostname -I	just the IP addresses
nslookup / host name	DNS resolution
iwconfig / nmcli dev wifi	Wi-Fi signal and networks

SHORTCUT TRICK

Troubleshoot from the inside out: **ping 127.0.0.1** (network stack) → **ping the gateway** (local network) → **ping 8.8.8.8** (internet) → **ping google.com** (DNS). The first one that fails shows where the problem is.

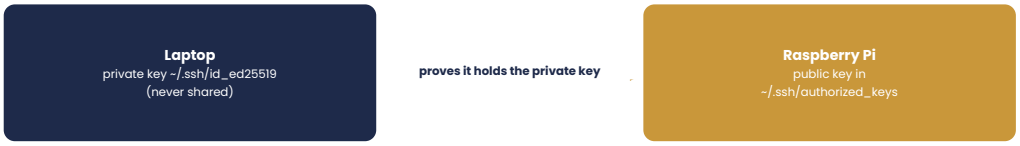
5.2 SSH

SSH (Secure Shell) gives you an encrypted terminal on the Pi from another computer, on TCP **port 22**.

5.2.1 Connecting from Windows, macOS and Linux

Windows 10/11 (PowerShell or Terminal), macOS and Linux all include the `ssh` client: `ssh lohit@labpi.local`. The first time, you are asked to confirm the host's **fingerprint**; it is then stored in `~/.ssh/known_hosts`. If it changes later (for example after re-flashing the card) you get a warning; remove the old entry with `ssh-keygen -R labpi.local`. PuTTY is a graphical alternative on Windows.

5.2.2 SSH keys and passwordless login



KEY-BASED LOGIN: THE PRIVATE KEY STAYS ON THE LAPTOP; THE PI ONLY KNOWS THE PUBLIC KEY

SETTING UP KEYS (RUN ON THE LAPTOP)

```
ssh-keygen -t ed25519 -C "lohit-laptop"     # on the laptop: create the key pair
ssh-copy-id lohit@labpi.local               # copy the public key to the Pi
ssh lohit@labpi.local                       # logs in with the key: no password
```

On Windows without ssh-copy-id, append the contents of `id_ed25519.pub` to `~/.ssh/authorized_keys` on the Pi. Protect the private key with a passphrase. Once keys work, password login can be switched off (module 21).

5.2.3 Copying files: scp, rsync, SFTP

Tool	Example (run on the laptop)	Best for
scp	scp led.py lohit@labpi.local:~/projects/	a quick copy of a few files
scp -r	scp -r lohit@labpi.local:~/data ./	a folder (here from the Pi to the laptop)
rsync	rsync -avz --delete project/ lohit@labpi.local:~/project/	syncing folders: copies only changes, resumes
SFTP	FileZilla or WinSCP, host sftp://labpi.local	drag-and-drop in a GUI

rsync options: `-a` archive (keep permissions and times, recursive), `-v` verbose, `-z` compress, `--delete` remove files at the destination that no longer exist at the source, `-n` dry run (show what would happen). A local example:

RSYNC COPIES EVERYTHING THE FIRST TIME, THEN ONLY WHAT CHANGED

```
pi@raspberrypi:~ $ rsync -av src/ backup/
sending incremental file list
./
a.txt
data/
data/b.txt

sent 236 bytes  received 65 bytes  602.00 bytes/sec
total size is 8  speedup is 0.03
pi@raspberrypi:~ $ echo three > src/c.txt
pi@raspberrypi:~ $ rsync -av src/ backup/
sending incremental file list
./
c.txt

sent 171 bytes  received 38 bytes  418.00 bytes/sec
total size is 14  speedup is 0.07
```

SHORTCUT TRICK

A trailing slash on the source matters: `rsync -a src/ dest/` copies the **contents** of src into dest; `rsync -a src dest/` creates **dest/src**.

5.3 Remote desktop

5.3.1 VNC

VNC shows the Pi's desktop on another computer. Enable it in raspi-config (Interface Options → VNC); the Pi runs the **WayVNC** server on port 5900 with the Wayland desktop. Connect with a VNC viewer (for example TigerVNC or RealVNC Viewer) to `labpi.local` and log in with your Pi username and password.

5.3.2 Raspberry Pi Connect

Raspberry Pi Connect is Raspberry Pi's free service for reaching your Pi's desktop or a remote shell from a web browser anywhere, without port forwarding: install with `sudo apt install rpi-connect` (or rpi-connect-lite for shell only), run `rpi-connect signin`, then open connect.raspberrypi.com.

5.4 Network configuration

5.4.1 NetworkManager and nmcli

Current Raspberry Pi OS (Bookworm and later) manages networks with **NetworkManager**. Use the desktop network icon, `nmtui` (a text menu) or the command `nmcli`.

EVERYDAY NMCLI COMMANDS

```
nmcli device status                       # interfaces and their state
nmcli connection show                     # saved connections
nmcli device wifi list                    # visible Wi-Fi networks and signal
sudo nmcli device wifi connect "HomeNet" password "s3cretpass"
```

5.4.2 Setting a static IP

STATIC ADDRESS ON THE WI-FI CONNECTION CREATED BY IMAGER (NAMED "PRECONFIGURED")

```
sudo nmcli connection modify "preconfigured" \
    ipv4.method manual \
    ipv4.addresses 192.168.1.50/24 \
    ipv4.gateway 192.168.1.1 \
    ipv4.dns "192.168.1.1 8.8.8.8"
sudo nmcli connection up "preconfigured"
```

WATCH OUT

Pick a static address **outside the router's DHCP pool** (for example the router hands out .100–.200, so use .50), or two devices may get the same address and both drop off the network. A DHCP reservation on the router avoids this completely.

5.4.3 Connecting to multiple Wi-Fi networks

Add one connection per network; NetworkManager joins whichever is available, preferring the higher `connection.autoconnect-priority`:

HOME PREFERRED (20) OVER COLLEGE (10)

```
sudo nmcli connection add type wifi con-name College ssid "CollegeWiFi" \
    wifi-sec.key-mgmt wpa-psk wifi-sec.psk "collegepass"
sudo nmcli connection modify College connection.autoconnect-priority 10
sudo nmcli connection modify preconfigured connection.autoconnect-priority 20
```

5.5 Remote development

5.5.1 VS Code Remote-SSH

With the **Remote - SSH** extension, VS Code on your laptop connects to the Pi over SSH and edits files, runs a terminal and debugs Python directly **on the Pi**. Command palette → "Remote-SSH: Connect to Host" → `lohit@labpi.local`. It needs a 64-bit OS and at least 1 GB RAM (it installs a small server on the Pi).

5.5.2 Editing code on the Pi from a laptop

Method	Pros	Cons
VS Code Remote-SSH	full IDE, terminal, debugger	uses RAM on the Pi
nano/vim over SSH	works everywhere	basic
Edit locally + rsync	laptop tools, versioned	extra step each change
Git push/pull	history, collaboration	commit for every test

PRACTICE CHECKPOINT

Connect to the Pi by SSH with key-based login, transfer a folder with rsync, set a static IP and open the desktop remotely.

5.6 Practice questions

5.6.1 Basic level

Q1 Which port does SSH use by default?

(a) 80 (b) 443 (c) 22 (d) 5900

Q2 What does DHCP do?

(a) encrypts web traffic
(b) translates names to addresses only
(c) copies files
(d) automatically gives devices an IP address, mask, gateway and DNS server

Q3 Which command logs in to the Pi as user lohit?

(a) ssh labpi.local@lohit (b) ssh lohit@labpi.local
(c) scp lohit labpi (d) login labpi lohit

Q4 What is the usual role of 192.168.1.1 on a home network?

(a) the router / default gateway (b) the Raspberry Pi
(c) a DNS server on the internet (d) the broadcast address

Q5 Which interface name is the Pi's Wi-Fi?

(a) eth0 (b) wlan0 (c) lo (d) usb0

Q6 Which tool copies only changed files and can resume?

(a) rsync (b) scp (c) cp (d) ping

Q7 Which protocol lets you see the Pi's desktop remotely?

(a) SSH only (b) DHCP (c) VNC (d) mDNS

Q8 What does mDNS let you do?

(a) encrypt SSH
(b) assign static IPs
(c) speed up Wi-Fi
(d) reach the Pi as hostname.local without a DNS server

5.6.2 Intermediate level

Q9 How many usable host addresses are in a /24 network?

(a) 256 (b) 255 (c) 24 (d) 254

Q10 Where must your public key be placed on the Pi for key login?

(a) ~/.ssh/known_hosts (b) /etc/hostname
(c) ~/.ssh/authorized_keys (d) ~/.bashrc

Q11 Which file must never leave your laptop?

(a) the private key id_ed25519 (b) id_ed25519.pub
(c) known_hosts (d) authorized_keys

Q12 Which tool manages networks on current Raspberry Pi OS?

(a) dhcpcd only **(b)** ifconfig only
(c) raspi-wifi **(d)** NetworkManager (nmcli, nmtui)

Q13 Why is a DHCP reservation often better than a static IP set on the Pi?

(a) it is faster
(b) the router guarantees the address and will not give it to another device
(c) it works without a router
(d) it encrypts traffic

Q14 After re-flashing the Pi, ssh warns that the host identification has changed. What is the safe fix if you know you re-flashed it?

(a) delete the whole .ssh folder on the Pi
(b) ssh-keygen -R labpi.local, then connect and accept the new fingerprint
(c) disable SSH encryption
(d) reinstall the laptop

Q15 Which command connects to a Wi-Fi network from the terminal?

(a) wifi join HomeNet
(b) ip wifi HomeNet
(c) sudo raspi-wifi HomeNet
(d) sudo nmcli device wifi connect "HomeNet" password "pass"

Q16 ping 192.168.1.1 works but ping google.com fails while ping 8.8.8.8 works. What is wrong?

(a) DNS name resolution **(b)** the Wi-Fi adapter
(c) the router's LAN **(d)** the Ethernet cable

5.6.3 Advanced level

Q17 How many usable hosts are in a /28 subnet?

(a) 16 **(b)** 28 **(c)** 14 **(d)** 30

Q18 Is 192.168.2.42 in the subnet 192.168.1.0/24?

(a) yes; all 192.168 addresses are one subnet
(b) yes, if the mask is 255.255.255.0
(c) only on Wi-Fi
(d) no; the third number differs, and /24 fixes the first three numbers

Q19 The router's DHCP pool is 192.168.1.100–199. Which static address is safe for the Pi?

(a) 192.168.1.150 **(b)** 192.168.1.50 **(c)** 192.168.1.255 **(d)** 192.168.1.0

Q20 What does rsync -a src dest/ (no slash after src) create?

(a) dest/src containing the files **(b)** the files directly in dest
(c) nothing; it is an error **(d)** a zip file

Q21 Which rsync option shows what would be copied without copying?

(a) -z **(b)** -v **(c)** -n (dry run) **(d)** --delete

Q22 Using Python's ipaddress module, print the network address, broadcast address and number of usable hosts of 10.0.5.0/26, and check whether 10.0.5.70 belongs to it.

Q23 Write a function ping_report(results) that takes a list of round-trip times in ms (None = lost packet) and prints packets sent, received, % loss and average time. Test with [2.3, 1.9, None, 2.1].

Q24 Back up a folder notes into ~/backup with rsync, then delete a file from notes and resync so the backup matches exactly.

Q25 You want to edit and debug Python on the Pi with a full IDE from your laptop. Which is the best option?

(a) VS Code with the Remote-SSH extension
(b) VNC with Notepad
(c) scp after every change
(d) ping

5.7 Solutions

Q1 **Answer: (c) 22**

SSH = TCP **22**; 80 is HTTP, 443 HTTPS, 5900 VNC.

Q2 **Answer: (d) automatically gives devices an IP address, mask, gateway and DNS server**

DHCP hands out **network settings**.

Q3 **Answer: (b) ssh lohit@labpi.local**

Format: `ssh user@host`.

Q4 **Answer: (a) the router / default gateway**

The router is usually **.1**.

Q5 **Answer: (b) wlan0**

wlan0; eth0 is Ethernet, lo is loopback.

Q6 **Answer: (a) rsync**

rsync.

Q7 **Answer: (c) VNC**

VNC (or Raspberry Pi Connect).

Q8 **Answer: (d) reach the Pi as hostname.local without a DNS server**

Multicast DNS answers **.local** names.

Q9 **Answer: (d) 254**

$2^8 - 2 = \mathbf{254}$.

Q10 **Answer: (c) ~/.ssh/authorized_keys**

authorized_keys lists keys allowed to log in. known_hosts is on the client, for host fingerprints.

Q11 **Answer: (a) the private key id_ed25519**

The **private key** proves your identity; the .pub file is safe to share.

Q12 **Answer: (d) NetworkManager (nmcli, nmtui)**

Bookworm and later use **NetworkManager**.

Q13 **Answer: (b) the router guarantees the address and will not give it to another device**

The router manages the pool, so there are **no conflicts**.

Q14 **Answer: (b) ssh-keygen -R labpi.local, then connect and accept the new fingerprint**

Remove the stale entry from known_hosts with **ssh-keygen -R**.

Q15 **Answer: (d) sudo nmcli device wifi connect "HomeNet" password "pass"**

`nmcli device wifi connect`.

Q16 **Answer: (a) DNS name resolution**

Internet by IP works, so names are not resolving: **DNS**.

Q17 **Answer: (c) 14**

$2^4 - 2 =$ **14**.

Q18 **Answer: (d) no; the third number differs, and /24 fixes the first three numbers**

/24 = first 24 bits (three numbers) identify the network: 192.168.1 ≠ 192.168.2.

Q19 **Answer: (b) 192.168.1.50**

.50 is outside the pool; .255 is broadcast and .0 the network address.

Q20 **Answer: (a) dest/src containing the files**

Without the trailing slash the folder itself is copied: **dest/src**.

Q21 **Answer: (c) -n (dry run)**

`-n` / `--dry-run`.

Q22 **Answer: Solution program**

/26 leaves 6 host bits: 64 addresses, 62 usable, from 10.0.5.0 to 10.0.5.63. So .70 is outside.

```
import ipaddress
net = ipaddress.ip_network("10.0.5.0/26")
print(net.network_address, net.broadcast_address, net.num_addresses - 2)
print(ipaddress.ip_address("10.0.5.70") in net)
```

OUTPUT

```
10.0.5.0 10.0.5.63 62
False
```

Q23 **Answer: Solution program**

Filter out the lost packets, then loss = lost ÷ sent × 100 and average = sum ÷ count of received.

```
def ping_report(results):
    sent = len(results)
    ok = [r for r in results if r is not None]
    loss = 100 * (sent - len(ok)) / sent
    print(f"{sent} sent, {len(ok)} received, {loss:.0f}% loss")
    if ok:
        print(f"average {sum(ok) / len(ok):.2f} ms")

ping_report([2.3, 1.9, None, 2.1])
```

OUTPUT

```
4 sent, 3 received, 25% loss
average 2.10 ms
```

Q24 **Answer: Commands**

`--delete` removes b.txt from the backup because it no longer exists in the source.

```
pi@raspberrypi:~ $ rsync -a notes/ backup/
pi@raspberrypi:~ $ rm notes/b.txt
pi@raspberrypi:~ $ rsync -av --delete notes/ backup/
sending incremental file list
deleting b.txt
./

sent 92 bytes  received 26 bytes  236.00 bytes/sec
total size is 2  speedup is 0.02
pi@raspberrypi:~ $ ls backup
a.txt
```

Q25 **Answer: (a) VS Code with the Remote-SSH extension**

Remote-SSH runs the editor UI locally and the code on the Pi.

Know the Pi's address (hostname -I, name.local) and gateway (ip route). Usable hosts in /n = 2^(32-n) - 2. Log in with ssh user@host; use ed25519 keys (private key stays on the laptop). Copy with scp, sync with rsync -avz (mind the trailing slash). Configure networks with nmcli; use DHCP reservations or static IPs outside the pool.

MODULE 06 · BEGINNER

Python programming on Raspberry Pi

Minimum duration **3 days** | Level **Beginner**

Python is the main language for Raspberry Pi projects. This module sets up Python, Thonny and virtual environments (including why system-wide pip is blocked), then covers the core language with a Pi focus: variables, operators, input and output, conditions, loops, lists, tuples, dictionaries and sets, functions and modules, CSV files and exceptions, time and dates, and classes for hardware devices. Every program shown was run, followed by 25 solved questions and programs.

IN THIS MODULE

6.1	Python setup	6.2	Virtual environments and packages
6.3	Python fundamentals	6.4	Data structures
6.5	Functions and modules	6.6	Files and exceptions
6.7	Time and dates	6.8	Introduction to object-oriented Python
6.9	Practice questions	6.10	Solutions

6.1 Python setup

6.1.1 Python 3 on Raspberry Pi OS

Raspberry Pi OS comes with **Python 3** and many hardware libraries (gpiozero, picamera2, smbus2…) already installed. The command is `python3`; `python3 --version` shows the version. Python 2 is obsolete and not installed.

6.1.2 Thonny IDE

Thonny (Menu → Programming) is a beginner-friendly editor: a Run button, a shell pane for output, a variables view and a step-by-step debugger. It can also program a Pico in MicroPython (module 24).

6.1.3 Running scripts from the terminal

WAYS TO RUN PYTHON

```
python3 hello.py            # run a script
python3                     # interactive shell (>>> prompt); exit() to leave
python3 -m venv env         # run a module as a program (here: venv)
chmod +x hello.py && ./hello.py   # if the first line is #!/usr/bin/env python3
```

6.2 Virtual environments and packages

6.2.1 Why system pip installs are blocked (externally managed environment)

On Raspberry Pi OS Bookworm and later, `pip install something` outside a virtual environment fails with **error: externally-managed-environment**. This protects the system: packages installed by pip could overwrite the versions apt installed and break OS tools. The fix is a **virtual environment** (venv): a private folder with its own Python packages for one project.

6.2.2 Creating a venv

CREATE AND ACTIVATE A VENV

```
pi@raspberrypi:~ $ python3 -m venv --system-site-packages ~/weather/env
pi@raspberrypi:~ $ source ~/weather/env/bin/activate; which python; python -c "import sys;
print(sys.prefix != sys.base_prefix)"
/home/pi/weather/env/bin/python
True
```

After `source env/bin/activate` the prompt shows `(env)` and `python` / `pip` refer to the venv. `deactivate` leaves it. Never put the venv inside a folder you sync to Git; list packages in `requirements.txt` instead (`pip freeze > requirements.txt`, then `pip install -r requirements.txt`).

6.2.3 pip install inside a venv

INSTALLING PACKAGES FOR ONE PROJECT

```
source ~/weather/env/bin/activate
pip install requests paho-mqtt
pip list
```

6.2.4 Using apt-provided Python packages (python3-xxx)

Many libraries are packaged by Raspberry Pi OS: `sudo apt install python3-numpy python3-opencv python3-flask`. They are tested with the system, fast to install (no compiling) and updated with apt. Prefer apt for big libraries when the packaged version is new enough.

6.2.5 --system-site-packages for hardware libraries

A venv created with `--system-site-packages` can still import the system's packages (gpiozero, lgpio, picamera2, and anything installed with apt) while pip installs go into the venv. This is the recommended setup for hardware projects.

> **SHORTCUT TRICK**
> Rule of thumb on a Pi: **python3 -m venv --system-site-packages env** for every project, apt for big system libraries, pip (inside the venv) for everything else.

6.3 Python fundamentals

6.3.1 Variables and data types

TYPES.PY

```
name = "labpi"          # str
pin = 17                # int
temp = 47.25            # float
led_on = False          # bool
readings = None         # NoneType: "no value yet"
for v in (name, pin, temp, led_on, readings):
    print(repr(v), type(v).__name__)
print(int("42") + 1, float("3.5") * 2, str(pin) + "A")
```

OUTPUT

```
'labpi' str
17 int
47.25 float
False bool
None NoneType
43 7.0 17A
```

6.3.2 Operators

Kind	Operators	Example → result
Arithmetic	+ - * / // % **	7 / 2 → 3.5, 7 // 2 → 3, 7 % 2 → 1, 2 ** 10 → 1024
Comparison	== != < > <= >=	temp > 60 → False
Logical	and or not	temp > 30 and humid > 70
Assignment	= += -= *=	count += 1
Bitwise	& | ^ ~ << >>	0b1010 & 0b0110 → 0b0010 (used with registers)
Membership	in, not in	17 in [17, 27] → True

6.3.3 Input and output

IO.PY

```
name = input("Your name: ")
age = int(input("Your age: "))
print(f"Hello {name}, in 5 years you will be {age + 5}.")
t = 47.256
print(f"Temperature: {t:.1f} C | padded: [{t:8.2f}] | percent: {0.375:.0%}")
```

OUTPUT

```
Your name: Asha
Your age: 19
Hello Asha, in 5 years you will be 24.
Temperature: 47.3 C | padded: [   47.26] | percent: 38%
```

f-strings (f"...{expression}...") format values; `:.1f` = 1 decimal place, `:8.2f` = width 8, `:.0%` = percentage. `input()` always returns a string, so convert with int() or float().

6.3.4 Conditions

CONDITIONS.PY

```
def status(temp):
    if temp >= 80:
        return "THROTTLING"
    elif temp >= 70:
        return "hot"
    elif temp >= 50:
        return "warm"
    else:
        return "cool"

for t in (42, 55.5, 71, 83):
    print(t, status(t))
```

OUTPUT

```
42 cool
55.5 warm
71 hot
83 THROTTLING
```

6.3.5 Loops

LOOPS.PY

```
for pin in [17, 27, 22]:            # for: over a sequence
    print("set up GPIO", pin)

for i in range(0, 10, 3):           # range(start, stop, step): stop not included
    print(i, end=" ")
print()

count = 0
while True:                         # while: until a condition, here with break
    count += 1
    if count % 2 == 0:
        continue                    # skip the rest for even numbers
    if count > 7:
        break
    print("odd:", count)
```

OUTPUT

```
set up GPIO 17
set up GPIO 27
set up GPIO 22
0 3 6 9
odd: 1
odd: 3
odd: 5
odd: 7
```

6.4 Data structures

6.4.1 Lists, tuples, dictionaries and sets

Type	Syntax	Ordered?	Changeable?	Use
list	[21.5, 22.0, 21.8]	yes	yes	a series of readings
tuple	(17, 27, 22)	yes	no	fixed pins, (x, y) pairs
dict	{"temp": 21.5, "hum": 60}	by insertion	yes	named values, JSON, config
set	{17, 27}	no	yes	unique items, fast "in" tests

STRUCTURES.PY

```
temps = [21.5, 22.0, 23.4, 21.8]
temps.append(22.6)
print("count", len(temps), "max", max(temps), "avg", round(sum(temps) / len(temps), 2))
print("first two", temps[:2], "last", temps[-1])
print("above 22:", [t for t in temps if t > 22])          # list comprehension

reading = {"sensor": "BME280", "temp": 22.6, "hum": 58}
reading["pressure"] = 1008.2
for key, value in reading.items():
    print(f"  {key}: {value}")
print(reading.get("wind", "no wind sensor"))

pins_used = {17, 27, 22}
print(18 in pins_used, sorted(pins_used | {18}))
```

OUTPUT

```
count 5 max 23.4 avg 22.26
first two [21.5, 22.0] last 22.6
above 22: [23.4, 22.6]
  sensor: BME280
  temp: 22.6
  hum: 58
  pressure: 1008.2
no wind sensor
False [17, 18, 22, 27]
```

6.5 Functions and modules

6.5.1 Defining functions

FUNCTIONS.PY

```
def c_to_f(c):
    """Convert Celsius to Fahrenheit."""
    return c * 9 / 5 + 32

def describe(value, unit="C", decimals=1):     # default arguments
    return f"{value:.{decimals}f} {unit}"

print(describe(c_to_f(25), "F"))
print(describe(21.456, decimals=2))             # keyword argument
```

OUTPUT

```
77.0 F
21.46 C
```

6.5.2 Importing modules

IMPORTS.PY

```
import math
from random import seed, randint
import statistics as st

seed(1)
print(math.sqrt(16), math.pi)
rolls = [randint(1, 6) for _ in range(6)]
print(rolls, "mean", st.mean(rolls), "median", st.median(rolls))
```

OUTPUT

```
4.0 3.141592653589793
[2, 5, 1, 3, 1, 4] mean 2.6666666666666665 median 2.5
```

6.5.3 Writing your own module

Any .py file is a module. Put reusable functions in `sensors_util.py` and import them from another file in the same folder. Code under `if __name__ == "__main__":` runs only when the file is run directly, not when imported.

MAIN.PY

```
from sensors_util import c_to_f, average

print(c_to_f(30))
print(average([20, 22, 24]))
```

OUTPUT

```
86.0
22.0
```

SENSORS_UTIL.PY (IN THE SAME FOLDER)

```
def c_to_f(c):
    return c * 9 / 5 + 32

def average(values):
    return sum(values) / len(values)

if __name__ == "__main__":
    print("self-test", c_to_f(100))
```

6.6 Files and exceptions

6.6.1 Reading and writing text and CSV files

CSV_DEMO.PY

```
import csv

rows = [("09:00", 21.5, 60), ("09:05", 21.9, 59), ("09:10", 22.4, 57)]
with open("log.csv", "w", newline="") as f:          # 'w' overwrites, 'a' appends
    w = csv.writer(f)
    w.writerow(["time", "temp", "hum"])
    w.writerows(rows)

with open("log.csv") as f:
    print(f.read())

with open("log.csv", newline="") as f:
    data = list(csv.DictReader(f))
temps = [float(r["temp"]) for r in data]
print("rows:", len(data), "max temp:", max(temps))
```

OUTPUT

```
time,temp,hum
09:00,21.5,60
09:05,21.9,59
09:10,22.4,57

rows: 3 max temp: 22.4
```

`with open(...)` closes the file automatically, even after an error. CSV values come back as strings, so convert them.

6.6.2 try, except and finally

ERRORS.PY

```
def read_temp(text):
    try:
        value = float(text)
        if not -40 <= value <= 125:
            raise ValueError(f"out of range: {value}")
        return value
    except ValueError as e:
        print("  bad reading:", e)
        return None
    finally:
        print("  (checked", repr(text) + ")")

for t in ("21.5", "abc", "300"):
    print(read_temp(t))

try:
    open("missing.txt")
except FileNotFoundError:
    print("missing.txt not found: create it first")
```

OUTPUT

```
  (checked '21.5')
21.5
  bad reading: could not convert string to float: 'abc'
  (checked 'abc')
None
  bad reading: out of range: 300.0
  (checked '300')
None
missing.txt not found: create it first
```

WATCH OUT

Never write a bare `except:` that silently passes: it hides real bugs and even swallows Ctrl+C. Catch the specific errors you expect (ValueError, OSError, FileNotFoundError) and print or log them.

6.7 Time and dates

6.7.1 time.sleep and timestamps

TIMING.PY

```
import time
start = time.monotonic()           # for measuring intervals (never jumps)
for i in range(3):
    time.sleep(2)
    print(f"tick {i}, elapsed {time.monotonic() - start:.1f} s")
print("Unix timestamp:", int(time.time()))
```

OUTPUT

```
tick 0, elapsed 2.0 s
tick 1, elapsed 4.0 s
tick 2, elapsed 6.0 s
Unix timestamp: 1773565206
```

`time.time()` = seconds since 1 January 1970 UTC (a Unix timestamp, good for storing). `time.monotonic()` never goes backwards when the clock is corrected, so use it for measuring durations.

6.7.2 datetime

DATES.PY

```
from datetime import datetime, timedelta

now = datetime.now()
print(now.strftime("%Y-%m-%d %H:%M:%S"))
print(now.isoformat(timespec="seconds"))
print("tomorrow:", (now + timedelta(days=1)).strftime("%A %d %B"))
t = datetime.strptime("2026-03-15 08:30", "%Y-%m-%d %H:%M")
print("minutes since then:", (now - t).total_seconds() / 60)
```

OUTPUT

```
2026-03-15 09:00:00
2026-03-15T09:00:00
tomorrow: Monday 16 March
minutes since then: 30.0
```

Code	Meaning	Code	Meaning
%Y	4-digit year	%H	hour 00–23
%m	month 01–12	%M	minute
%d	day 01–31	%S	second
%A / %B	weekday / month name	%I %p	12-hour clock with AM/PM

NOTE

The simulated clock used to run the programs in this book starts at 15 March 2026, 09:00:00, so printed dates and times are consistent from run to run.

6.8 Introduction to object-oriented Python

6.8.1 Classes and objects for hardware devices

A **class** is a blueprint that bundles data (attributes) and behaviour (methods); an **object** is one instance of it. gpiozero's LED and Button are classes. Writing your own class keeps each device's state and logic together:

CLASSES.PY

```
class TempSensor:
    def __init__(self, name, limit=60):
        self.name = name
        self.limit = limit
        self.history = []

    def add(self, value):
        self.history.append(value)
        if value > self.limit:
            print(f"{self.name}: ALERT {value} > {self.limit}")

    @property
    def average(self):
        return sum(self.history) / len(self.history)

    def __str__(self):
        return f"{self.name}: {len(self.history)} readings, avg {self.average:.1f}"

cpu = TempSensor("CPU", limit=70)
room = TempSensor("Room", limit=30)
for v in (48, 55, 72):
    cpu.add(v)
for v in (24, 31):
    room.add(v)
print(cpu)
print(room)
```

OUTPUT

```
CPU: ALERT 72 > 70
Room: ALERT 31 > 30
CPU: 3 readings, avg 58.3
Room: 2 readings, avg 27.5
```

PRACTICE CHECKPOINT

Create a venv, write a Python program that reads system temperature every few seconds, saves readings with timestamps to a CSV file, and handles errors cleanly.

TEMP_LOGGER.PY (CHECKPOINT SOLUTION)

```
import csv, time
from datetime import datetime

LOG = "cpu_temp.csv"

def read_cpu_temp():
    with open("/sys/class/thermal/thermal_zone0/temp") as f:
        return int(f.read()) / 1000

try:
    with open(LOG, "a", newline="") as f:
        w = csv.writer(f)
        for _ in range(4):
            try:
                t = read_cpu_temp()
            except (OSError, ValueError) as e:
                print("read failed:", e)
                continue
            stamp = datetime.now().isoformat(timespec="seconds")
            w.writerow([stamp, t])
            print(stamp, t)
            time.sleep(5)
except KeyboardInterrupt:
    print("stopped by user")
finally:
    print("log saved to", LOG)
```

OUTPUT

```
2026-03-15T09:00:00 48.2
2026-03-15T09:00:05 48.2
2026-03-15T09:00:10 49.0
2026-03-15T09:00:15 51.7
log saved to cpu_temp.csv
```

6.9 Practice questions

6.9.1 Basic level

Q1 What does input() return?

(a) an integer (b) a string (c) a float (d) a list

Q2 What is 17 // 5?

(a) 3.4 (b) 2 (c) 4 (d) 3

Q3 What is 17 % 5?

(a) 3 (b) 3.4 (c) 2 (d) 0

Q4 Which data type is (17, 27, 22)?

(a) list (b) set (c) tuple (d) dict

Q5 Which command creates a virtual environment called env?

(a) pip venv env (b) sudo apt venv env
(c) python3 env create (d) python3 -m venv env

Q6 What does range(2, 10, 3) produce?

(a) 2, 5, 8, 11 (b) 2, 5, 8 (c) 2, 3, 4 … 10 (d) 3, 6, 9

Q7 Which keyword handles an error?

(a) except (b) catch (c) error (d) rescue

Q8 Which IDE comes with Raspberry Pi OS and suits beginners?

(a) Visual Studio (b) Thonny (c) Xcode (d) Eclipse

6.9.2 Intermediate level

Q9 Why does pip install fail outside a venv on Raspberry Pi OS?

(a) the system Python is "externally managed" to protect apt-installed packages
(b) pip is not installed
(c) the Pi has no internet
(d) Python 3 cannot install packages

Q10 What does --system-site-packages add to a venv?

(a) root permissions
(b) Python 2 support
(c) access to system packages such as gpiozero and picamera2
(d) faster pip downloads

Q11 What does f"{3.14159:.2f}" give?

(a) 3.1 (b) 3.14159 (c) 3 (d) 3.14

Q12 Which structure stores named values like {"temp": 21.5}?

(a) list (b) tuple (c) set (d) dict

Q13 What does reading.get("wind", 0) return when "wind" is not a key?

(a) None **(b)** KeyError **(c)** 0 **(d)** "wind"

Q14 Why use with open(...) as f?

(a) the file is closed automatically, even if an error occurs

(b) it makes the file read-only

(c) it is faster

(d) it encrypts the file

Q15 Which is best for measuring how long a loop took?

(a) time.time() **(b)** datetime.now().day

(c) time.sleep() **(d)** time.monotonic()

Q16 What does if __name__ == "__main__": do?

(a) marks the main loop

(b) runs the code below it only when the file is run directly, not when imported

(c) makes the program run at boot

(d) imports __main__

6.9.3 Advanced level

Q17 What is printed? x = [1, 2, 3]; y = x; y.append(4); print(len(x))

(a) 3 **(b)** 4 **(c)** error **(d)** 1

Q18 What does [t for t in [20, 25, 31, 18] if t > 22] produce?

(a) [20, 18] **(b)** [31] **(c)** [25, 31, 18] **(d)** [25, 31]

Q19 Which strftime format gives "2026-03-15 09:05"?

(a) %Y-%m-%d %H:%M **(b)** %d-%m-%Y %H:%M **(c)** %Y-%M-%D %h:%m **(d)** %y/%m/%d %I:%M

Q20 Write a function stats(values) that returns the minimum, maximum and average (rounded to 1 decimal) of a list, and print it for [21.5, 23.0, 22.4, 24.1, 20.9].

Q21 Ask the user for a temperature in Celsius until they type q, printing it in Fahrenheit; print "not a number" for bad input.

Q22 A CSV file readings.csv has columns time,temp. Read it and print the time of the highest temperature.

Q23 Write a class Counter for a button with methods press() and reset() and an attribute count; simulate 3 presses, print the count, reset and print again.

Q24 Log the CPU temperature every 10 seconds for one minute into a list of (seconds, temp) tuples, then print the readings above 50 °C.

Q25 Which is the correct order to set up a new hardware project folder?

(a) sudo pip install … → python3 -m venv env

(b) pip freeze → pip install → venv

(c) python3 -m venv --system-site-packages env → source env/bin/activate → pip install … → pip freeze > requirements.txt

(d) apt remove python3 → pip install python

6.10 Solutions

Q1 **Answer: (b) a string**

Always a **str**; convert with int() or float().

Q2 **Answer: (d) 3**

// is floor division: **3**.

Q3 **Answer: (c) 2**

% is the remainder: 17 = 3 × 5 + **2**.

Q4 **Answer: (c) tuple**

Round brackets: **tuple** (cannot be changed).

Q5 **Answer: (d) python3 -m venv env**

`python3 -m venv env`.

Q6 **Answer: (b) 2, 5, 8**

Start 2, step 3, stop before 10: **2, 5, 8**.

Q7 **Answer: (a) except**

Python uses try / **except** / finally.

Q8 **Answer: (b) Thonny**

Thonny.

Q9 **Answer: (a) the system Python is "externally managed" to protect apt-installed packages**

PEP 668: **externally-managed-environment**.

Q10 **Answer: (c) access to system packages such as gpiozero and picamera2**

The venv can **import system packages** while keeping its own.

Q11 **Answer: (d) 3.14**

`.2f` = 2 decimal places.

Q12 **Answer: (d) dict**

A **dictionary** maps keys to values.

Q13 **Answer: (c) 0**

get() returns the **default** instead of raising KeyError.

Q14 **Answer: (a) the file is closed automatically, even if an error occurs**

A context manager **always closes** the file.

Q15 **Answer: (d) time.monotonic()**

monotonic() **never jumps** when the clock is corrected.

Q16 **Answer: (b) runs the code below it only when the file is run directly, not when imported**

It separates **self-tests or main code** from importable functions.

Q17 **Answer: (b) 4**

y and x name the **same list**.

Q18 **Answer: (d) [25, 31]**

Keeps values above 22.

Q19 **Answer: (a) %Y-%m-%d %H:%M**

%Y year, %m month, %d day, %H hour, %M minute.

Q20 **Answer: Solution program**

Returning a tuple lets the caller unpack three values at once. Average = 111.9 / 5 = 22.38 → 22.4.

```
def stats(values):
    return min(values), max(values), round(sum(values) / len(values), 1)

lo, hi, avg = stats([21.5, 23.0, 22.4, 24.1, 20.9])
print(f"min {lo}  max {hi}  avg {avg}")
```

OUTPUT

```
min 20.9  max 24.1  avg 22.4
```

Q21 **Answer: Solution program**

The loop runs until "q"; try/except catches non-numeric input. The run typed 25, hot, -40 and q.

```
while True:
    text = input("Celsius (q to quit): ")
    if text == "q":
        break
    try:
        c = float(text)
    except ValueError:
        print("not a number")
        continue
    print(f"{c} C = {c * 9 / 5 + 32:.1f} F")
print("bye")
```

OUTPUT

```
Celsius (q to quit): 25
25.0 C = 77.0 F
Celsius (q to quit): hot
not a number
Celsius (q to quit): -40
-40.0 C = -40.0 F
Celsius (q to quit): q
bye
```

Q22 **Answer: Solution program**

max() with key= compares rows by their temperature converted to float.

```
import csv
with open("readings.csv", newline="") as f:
    rows = list(csv.DictReader(f))
best = max(rows, key=lambda r: float(r["temp"]))
print("hottest at", best["time"], "with", best["temp"], "C")
```

OUTPUT

```
hottest at 11:00 with 26.1 C
```

Q23 **Answer: Solution program**

__init__ sets the starting state; each method changes self.count, which belongs to that object.

```
class Counter:
    def __init__(self):
        self.count = 0
    def press(self):
        self.count += 1
    def reset(self):
        self.count = 0

c = Counter()
for _ in range(3):
    c.press()
print("count:", c.count)
c.reset()
print("after reset:", c.count)
```

OUTPUT

```
count: 3
after reset: 0
```

Q24 **Answer: Solution program**

The loop runs while less than 60 s have passed, giving readings at 0, 10 … 50 s. The simulated CPU warmed from 47 °C to 53 °C.

```
import time
start = time.monotonic()
log = []
while time.monotonic() - start < 60:
    with open("/sys/class/thermal/thermal_zone0/temp") as f:
        log.append((round(time.monotonic() - start), int(f.read()) / 1000))
    time.sleep(10)
print("readings:", len(log))
print("above 50:", [r for r in log if r[1] > 50])
```

OUTPUT

```
readings: 6
above 50: [(30, 51.2), (40, 52.8), (50, 53.4)]
```

Q25 **Answer: (c) python3 -m venv --system-site-packages env → source env/bin/activate → pip install … → pip freeze > requirements.txt**

Create, activate, install, record.

Use a venv per project (python3 -m venv --system-site-packages env), apt for big system libraries and pip inside the venv for the rest. Know types, f-strings, if/elif, for/while, lists/dicts, functions, modules, with open + csv, specific try/except, time.monotonic for durations, datetime for timestamps, and classes to model devices.

MODULE 07 · BEGINNER

GPIO fundamentals

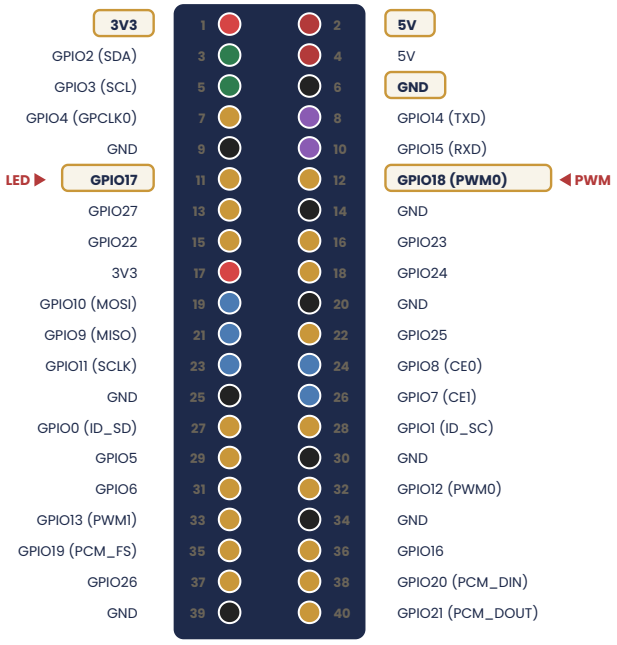
Minimum duration **2 days** | Level **Beginner**

GPIO pins let the Pi control and sense the physical world. This module maps the 40-pin header (power pins, special functions, BCM and physical numbering), sets out the electrical rules that keep the Pi safe (3.3 V logic, current limits, resistors, pull-ups), recaps Ohm's law and the breadboard, compares GPIO libraries (gpiozero, lgpio, why RPi.GPIO fails on a Pi 5, pinctrl and gpioset), and then drives LEDs, traffic lights and buzzers, reads buttons with pull-ups, debouncing and event callbacks, and uses PWM for brightness and colour, followed by 25 solved questions and programs.

IN THIS MODULE

7.1 The 40-pin GPIO header
7.2 Electrical rules
7.3 Electronics basics recap
7.4 GPIO libraries
7.5 Digital output
7.6 Digital input
7.7 PWM
7.8 Practice questions
7.9 Solutions

7.1 The 40-pin GPIO header



THE 40-PIN HEADER (PIN 1 IS THE SQUARE PAD NEAREST THE CORNER; ODD PINS ON THE LEFT COLUMN, EVEN ON THE RIGHT)

7.1.1 Power pins: 3.3 V, 5 V and GND

Pins **1 and 17** supply 3.3 V; pins **2 and 4** supply 5 V straight from the power input; **eight GND pins** (6, 9, 14, 20, 25, 30, 34, 39) are ground. Power pins are always on: they cannot be switched by software.

7.1.2 GPIO pins and special-function pins

Function	BCM pins (physical)	Used for
I2C1	GPIO2 SDA (3), GPIO3 SCL (5)	sensors, displays (module 10); have fixed 1.8 kΩ pull-ups
SPI0	GPIO10 MOSI (19), GPIO9 MISO (21), GPIO11 SCLK (23), GPIO8 CE0 (24), GPIO7 CE1 (26)	ADCs, fast displays
UART	GPIO14 TXD (8), GPIO15 RXD (10)	serial devices, GPS
Hardware PWM	GPIO12, 13, 18, 19	servos, precise PWM
ID EEPROM	GPIO0, GPIO1 (27, 28)	reserved for HAT identification: do not use

7.1.3 BCM numbering vs physical numbering

Each pin has two numbers: its **physical** position on the header (1–40) and its **BCM** (Broadcom GPIO) number. GPIO17 is physical pin 11. gpiozero uses BCM numbers by default (`LED(17)`); you can also write `LED("GPIO17")` or `LED("BOARD11")`.

> **SHORTCUT TRICK**
> Remember the landmarks: **pin 1 = 3.3 V** in the corner, **pin 2 = 5 V** next to it, **pin 6 = GND**, and **GPIO17 = pin 11**, **GPIO18 = pin 12**, **GPIO27 = pin 13**, **GPIO22 = pin 15**: the four most-used project pins in a row.

PIN_LOOKUP.PY

```
BOARD = {3: 2, 5: 3, 7: 4, 8: 14, 10: 15, 11: 17, 12: 18, 13: 27, 15: 22, 16: 23,
         18: 24, 19: 10, 21: 9, 22: 25, 23: 11, 24: 8, 26: 7, 29: 5, 31: 6,
         32: 12, 33: 13, 35: 19, 36: 16, 37: 26, 38: 20, 40: 21}
BCM = {bcm: phys for phys, bcm in BOARD.items()}
for g in (17, 18, 27, 22, 4):
    print(f"GPIO{g:<2} = physical pin {BCM[g]}")
```

OUTPUT

```
GPIO17 = physical pin 11
GPIO18 = physical pin 12
GPIO27 = physical pin 13
GPIO22 = physical pin 15
GPIO4  = physical pin 7
```

7.1.4 The pinout command and pinout.xyz

Typing `pinout` in the terminal draws your board's header with every pin labelled. The website **pinout.xyz** shows the same, plus which pins each HAT uses. Keep one of them open whenever you wire.

7.2 Electrical rules

7.2.1 3.3 V logic: GPIO is not 5 V tolerant

A GPIO output gives 3.3 V for HIGH and 0 V for LOW. Inputs read HIGH above about 2 V. **Never connect 5 V to a GPIO pin**: there is no protection, and it can destroy the pin or the whole SoC. 5 V signals (an HC-SR04 echo, an Arduino output) need a voltage divider or level shifter (modules 8 and 10).

7.2.2 Current limits per pin

GPIO pins are signals, not power supplies. Keep each pin below about **16 mA** and all pins together below about **50 mA**. An LED at 5–10 mA is fine; a motor, relay coil or long LED strip needs a transistor or driver (module 12).

7.2.3 Current-limiting resistors for LEDs

WORKED EXAMPLE

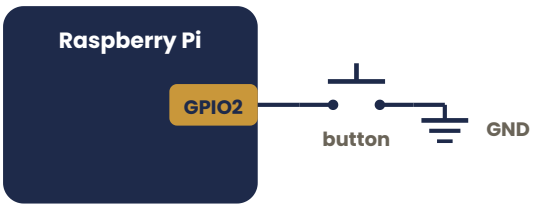
Choosing an LED resistor

$R = (V_{supply} - V_{LED}) \div I$. Red LED ($V_F \approx 2.0$ V) at 10 mA from 3.3 V: $R = (3.3 - 2.0) \div 0.010 =$ **130 Ω**. Use the next standard value up: 150 Ω, or the common **220 Ω**, which gives $1.3 \div 220 \approx 5.9$ mA: bright enough and safe.

LED ON GPIO17 (PIN 11) WITH A 220 Ω RESISTOR TO GND (PIN 9)

7.2.4 Pull-up and pull-down resistors

An input connected to nothing **floats** and reads random values. A **pull-up** resistor holds it HIGH until a button connects it to GND; a **pull-down** holds it LOW until a button connects it to 3.3 V. Every GPIO has internal pull-ups and pull-downs of about 50 kΩ that software can switch on. gpiozero's Button enables the pull-up by default, so wire the button between the GPIO and **GND**.



BUTTON WITH THE INTERNAL PULL-UP: RELEASED = HIGH, PRESSED = LOW (GPIOZERO REPORTS IS_PRESSED = TRUE)

7.3 Electronics basics recap

7.3.1 Ohm's law

V = I × R: voltage (volts) = current (amps) × resistance (ohms). Power **P = V × I**. A 220 Ω resistor with 1.3 V across it passes 5.9 mA and dissipates 1.3 × 0.0059 ≈ 7.7 mW, far below a ¼ W rating.

7.3.2 Breadboard and jumper wires

In a breadboard, the five holes of each short row are connected; the long **rails** along the sides carry power (red +) and ground (blue -). The centre gap separates the two halves so chips can straddle it. Use female-to-male jumper wires from the Pi header to the breadboard.

7.3.3 GPIO breakout boards

A **T-cobbler** and ribbon cable bring all 40 pins onto the breadboard with printed labels. It prevents wrong-pin mistakes and keeps wiring tidy.

> **WATCH OUT**
> Wire with the Pi **switched off**, check every connection against the pinout, then power up. One wire from 5 V (pin 2 or 4) to a GPIO pin or a short from 3.3 V to GND can kill the board.

7.4 GPIO libraries

7.4.1 gpiozero (recommended)

gpiozero is the official beginner-to-professional library: devices are objects (LED, Button, Motor, Servo, DistanceSensor…), with methods such as on(), blink() and properties such as is_pressed, plus event callbacks. It is pre-installed and picks the right low-level backend for each Pi model (lgpio on the Pi 5).

7.4.2 lgpio

lgpio is a lower-level C library with Python bindings that talks to the Linux GPIO character device (/dev/gpiochip*). It works on every model including the Pi 5 and gives direct control (claim a line, write a level, read a level, software PWM). gpiozero uses it underneath.

LGPIO DIRECTLY (FOR REFERENCE)

```
import lgpio
h = lgpio.gpiochip_open(0)          # the 40-pin header chip
lgpio.gpio_claim_output(h, 17)
lgpio.gpio_write(h, 17, 1)          # GPIO17 HIGH
lgpio.gpiochip_close(h)
```

7.4.3 Why RPi.GPIO does not work on Pi 5, and rpi-lgpio as a replacement

The classic **RPi.GPIO** library writes directly to the Broadcom SoC's GPIO registers. On the Pi 5 the header pins are controlled by the new **RP1** chip, so those registers do not exist and RPi.GPIO fails ("Cannot determine SOC peripheral base address"). The **rpi-lgpio** package (`sudo apt install python3-rpi-lgpio`) is a drop-in replacement: old code that says `import RPi.GPIO as GPIO` runs unchanged on top of lgpio. For new code, use gpiozero.

7.4.4 Command-line tools: pinctrl and gpioget/gpioset

TESTING PINS WITHOUT WRITING A PROGRAM

```
pinctrl get 17                # show mode and level of GPIO17
pinctrl set 17 op dh          # output, drive high  (dl = drive low)
pinctrl set 4 ip pu           # input with pull-up
gpioget gpiochip0 4           # read GPIO4 (libgpiod)
gpioset gpiochip0 17=1        # set GPIO17 high (libgpiod v1 syntax)
```

On current kernels the 40-pin header is `gpiochip0` on every model (very early Pi 5 kernels called it gpiochip4); `gpiodetect` lists the chips.

7.5 Digital output

7.5.1 LED on and off

LED_ON_OFF.PY

```
from gpiozero import LED
from time import sleep

led = LED(17)
led.on()
print("lit?", led.is_lit)
sleep(1)
led.off()
print("lit?", led.is_lit)
```

SIMULATED RUN (hardware events and printed output)

```
[  0.00 s] LED(GPIO17) on
lit? True
[  1.00 s] LED(GPIO17) off
lit? False
```

7.5.2 Blinking an LED

BLINK_BG.PY

```
from gpiozero import LED
from signal import pause

led = LED(17)
led.blink(on_time=0.5, off_time=1.5)   # runs in the background
pause()                                 # keep the program alive (Ctrl+C to stop)
```

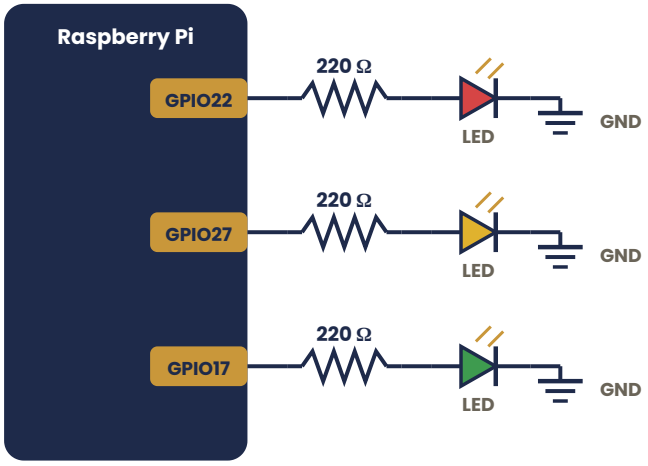
SIMULATED RUN (hardware events and printed output)

```
[  0.00 s] LED(GPIO17) on
[  0.50 s] LED(GPIO17) off
[  2.00 s] LED(GPIO17) on
[  2.50 s] LED(GPIO17) off
[  4.00 s] LED(GPIO17) on
```

```
[  4.50 s] LED(GPIO17) off
[  6.00 s] LED(GPIO17) on
```

`blink()` runs in a background thread, so the program can do other work. `pause()` waits forever. The simulated run was stopped after 6 s, as if Ctrl+C had been pressed.

7.5.3 Traffic light sequence



THREE LEDS, EACH WITH ITS OWN RESISTOR

TRAFFIC.PY

```
from gpiozero import TrafficLights
from time import sleep

lights = TrafficLights(red=22, amber=27, green=17)
for cycle in range(1):
    lights.green.on();  sleep(4)
    lights.green.off(); lights.amber.on(); sleep(1)
    lights.amber.off(); lights.red.on();   sleep(4)
    lights.amber.on();  sleep(1)          # red + amber: get ready
    lights.off()
print("cycle complete")
```

SIMULATED RUN (hardware events and printed output)

```
[  0.00 s] LED(GPIO17) on
[  4.00 s] LED(GPIO17) off
[  4.00 s] LED(GPIO27) on
[  5.00 s] LED(GPIO27) off
[  5.00 s] LED(GPIO22) on
[  9.00 s] LED(GPIO27) on
[ 10.00 s] LED(GPIO22) off
[ 10.00 s] LED(GPIO27) off
cycle complete
```

7.5.4 Buzzer

An **active buzzer** beeps at its own fixed tone when powered: control it like an LED with `Buzzer(pin)`. A **passive buzzer** needs a square wave at the note's frequency: use `TonalBuzzer`. Small buzzers draw

about 20–30 mA, so use a transistor or a buzzer module with one built in.

BUZZER.PY

```
from gpiozero import Buzzer
from time import sleep

bz = Buzzer(23)
bz.beep(on_time=0.1, off_time=0.1, n=3, background=False)   # three short beeps
sleep(0.5)
bz.on(); sleep(0.6); bz.off()                                 # one long beep
```

SIMULATED RUN (hardware events and printed output)

```
[  0.00 s] Buzzer(GPIO23) on
[  0.10 s] Buzzer(GPIO23) off
[  0.20 s] Buzzer(GPIO23) on
[  0.30 s] Buzzer(GPIO23) off
[  0.40 s] Buzzer(GPIO23) on
[  0.50 s] Buzzer(GPIO23) off
[  1.10 s] Buzzer(GPIO23) on
[  1.70 s] Buzzer(GPIO23) off
```

7.6 Digital input

7.6.1 Reading a push button

BUTTON_POLL.PY

```
from gpiozero import Button
from time import sleep

button = Button(2)                # pull-up on, button to GND
for _ in range(8):
    print("pressed" if button.is_pressed else "released")
    sleep(0.25)
```

OUTPUT

```
released
released
released
pressed
pressed
pressed
released
released
```

The program **polls** the button four times a second. The simulation pressed it from 0.6 s to 1.3 s.

7.6.2 Internal pull-ups

`Button(2)` = pull-up on (button to GND). `Button(2, pull_up=False)` = pull-down on (button to 3.3 V). For a sensor module that drives its own output, use `DigitalInputDevice(pin, pull_up=None, active_state=True)`.

7.6.3 Debouncing

A button's metal contacts bounce for a few milliseconds, so one press can look like several. gpiozero's `bounce_time` ignores further changes for that many seconds after an edge:

DEBOUNCE.PY

```
from gpiozero import Button
from signal import pause

presses = 0
def count():
    global presses
    presses += 1
    print("press", presses)

b = Button(2, bounce_time=0.05)
b.when_pressed = count
pause()
```

OUTPUT

```
press 1
press 2
```

The scripted input bounced on both presses (three edges within 5 ms), but only two presses were counted.

7.6.4 Event callbacks: when_pressed, when_released

BUTTON_LED.PY

```
from gpiozero import Button, LED
from signal import pause

led = LED(17)
button = Button(2)
button.when_pressed = led.on       # pass the function, do not call it
button.when_released = led.off
pause()
```

SIMULATED RUN (hardware events and printed output)

```
[  1.00 s] LED(GPIO17) on
[  2.50 s] LED(GPIO17) off
[  4.00 s] LED(GPIO17) on
[  4.40 s] LED(GPIO17) off
```

WATCH OUT

Assign the function itself: `button.when_pressed = led.on`. Writing `led.on()` calls it once immediately and assigns its result (None), so nothing happens on later presses.

SHORTCUT TRICK

Polling (checking is_pressed in a loop) is simple; **callbacks** are better when the program has other work, because gpiozero watches the pin in a background thread and calls your function at the moment of the edge.

7.7 PWM

7.7.1 Software vs hardware PWM

PWM (pulse-width modulation) switches a pin on and off quickly; the fraction of time it is on is the **duty cycle**. An LED at 25 % duty looks a quarter as bright. **Software PWM** is timed by the CPU and works on any pin but can jitter when the system is busy: fine for LEDs. **Hardware PWM** (GPIO12, 13, 18, 19) is generated by a dedicated peripheral with exact timing: use it for servos and motors (enable with `dtoverlay=pwm-2chan`).

PWM WAVEFORM

7.7.2 LED brightness control

BRIGHTNESS.PY

```
from gpiozero import PWMLED
from time import sleep

led = PWMLED(18)
for level in (0, 0.25, 0.5, 0.75, 1.0):
    led.value = level
    sleep(0.5)
led.off()
```

SIMULATED RUN (hardware events and printed output)

```
[  0.50 s] PWMLED(GPIO18) value 0.25
[  1.00 s] PWMLED(GPIO18) value 0.50
[  1.50 s] PWMLED(GPIO18) value 0.75
[  2.00 s] PWMLED(GPIO18) on
[  2.50 s] PWMLED(GPIO18) off
```

7.7.3 RGB LEDs

An RGB LED has red, green and blue LEDs in one package with a common cathode (to GND) or common anode (to 3.3 V; use `active_high=False`). Each colour needs its own GPIO and resistor; mixing PWM levels makes any colour.

RGB.PY

```
from gpiozero import RGBLED
from time import sleep

rgb = RGBLED(red=9, green=10, blue=11)
for name, colour in [("red", (1, 0, 0)), ("yellow", (1, 1, 0)), ("purple", (0.5, 0, 0.5)), ("white",
(1, 1, 1))]:
    rgb.color = colour
    print(name)
    sleep(1)
rgb.off()
```

SIMULATED RUN (hardware events and printed output)

```
[  0.00 s] RGBLED(GPIO9,10,11) color=(1.00, 0.00, 0.00)
red
[  1.00 s] RGBLED(GPIO9,10,11) color=(1.00, 1.00, 0.00)
yellow
[  2.00 s] RGBLED(GPIO9,10,11) color=(0.50, 0.00, 0.50)
purple
[  3.00 s] RGBLED(GPIO9,10,11) color=(1.00, 1.00, 1.00)
white
[  4.00 s] RGBLED(GPIO9,10,11) color=(0.00, 0.00, 0.00)
```

PRACTICE CHECKPOINT

Build a pedestrian crossing: traffic lights, a push button and a buzzer, controlled with gpiozero event callbacks.

CROSSING.PY (CHECKPOINT SOLUTION)

```
from gpiozero import TrafficLights, Button, Buzzer
from time import sleep

lights = TrafficLights(22, 27, 17)
button = Button(2)
buzzer = Buzzer(23)
requested = False

def request():
    global requested
    requested = True
    print("crossing requested")

button.when_pressed = request
lights.green.on()
while True:
    if requested:
        sleep(1)
        lights.green.off(); lights.amber.on(); sleep(2)
        lights.amber.off(); lights.red.on()
        buzzer.beep(0.2, 0.2, n=10, background=False)   # 4 s of beeps: cross now
        lights.amber.on(); sleep(1)
        lights.red.off(); lights.amber.off(); lights.green.on()
        requested = False
    sleep(0.1)
```

SIMULATED RUN (hardware events and printed output)

```
[  0.00 s] LED(GPIO17) on
crossing requested
[  3.00 s] LED(GPIO17) off
[  3.00 s] LED(GPIO27) on
[  5.00 s] LED(GPIO27) off
[  5.00 s] LED(GPIO22) on
[  5.00 s] Buzzer(GPIO23) on
[  5.20 s] Buzzer(GPIO23) off
[  5.40 s] Buzzer(GPIO23) on
[  5.60 s] Buzzer(GPIO23) off
[  5.80 s] Buzzer(GPIO23) on
[  6.00 s] Buzzer(GPIO23) off
[  6.20 s] Buzzer(GPIO23) on
[  6.40 s] Buzzer(GPIO23) off
[  6.60 s] Buzzer(GPIO23) on
[  6.80 s] Buzzer(GPIO23) off
[  7.00 s] Buzzer(GPIO23) on
[  7.20 s] Buzzer(GPIO23) off
[  7.40 s] Buzzer(GPIO23) on
[  7.60 s] Buzzer(GPIO23) off
[  7.80 s] Buzzer(GPIO23) on
[  8.00 s] Buzzer(GPIO23) off
[  8.20 s] Buzzer(GPIO23) on
[  8.40 s] Buzzer(GPIO23) off
[  8.60 s] Buzzer(GPIO23) on
[  8.80 s] Buzzer(GPIO23) off
[  9.00 s] LED(GPIO27) on
[ 10.00 s] LED(GPIO22) off
```

```
[ 10.00 s] LED(GPIO27) off
[ 10.00 s] LED(GPIO17) on
```

7.8 Practice questions

7.8.1 Basic level

Q1 Which physical pin supplies 3.3 V?

(a) pin 2 (b) pin 6 (c) pin 40 (d) pin 1

Q2 GPIO17 is which physical pin?

(a) 17 (b) 11 (c) 12 (d) 7

Q3 What is the logic HIGH voltage of a Pi GPIO pin?

(a) 3.3 V (b) 5 V (c) 1.8 V (d) 12 V

Q4 Which library is recommended for GPIO in Python on all Pi models?

(a) RPi.GPIO (b) wiringPi (c) gpiozero (d) Arduino.h

Q5 What does a pull-up resistor do?

(a) holds an input HIGH when nothing else drives it
(b) limits LED current
(c) raises the supply to 5 V
(d) makes outputs stronger

Q6 How should a button be wired for gpiozero's Button(2) with default settings?

(a) between GPIO2 and 5 V (b) between GPIO2 and GND
(c) between 3.3 V and 5 V (d) between GPIO2 and GPIO3

Q7 Which method makes an LED flash in the background?

(a) led.flash() (b) led.toggle(1) (c) led.pwm() (d) led.blink()

Q8 Which numbering does gpiozero use by default?

(a) physical pin numbers (b) wiringPi numbers
(c) BCM (GPIO) numbers (d) alphabetical

7.8.2 Intermediate level

Q9 What resistor is needed for a red LED (2.0 V) at 10 mA from a 3.3 V pin?

(a) 330 Ω (b) 13 Ω (c) 130 Ω (d) 200 Ω

Q10 A blue LED (3.0 V) has a 100 Ω resistor on a 3.3 V pin. What current flows?

(a) 33 mA (b) 30 mA (c) 0.3 mA (d) 3 mA

Q11 Why does RPi.GPIO fail on a Raspberry Pi 5?

(a) the Pi 5 has no GPIO pins
(b) the header pins are driven by the RP1 chip, not the Broadcom registers RPi.GPIO writes to
(c) Python 3 is not supported
(d) it needs root on the Pi 5

Q12 Which pins can produce hardware PWM?

(a) GPIO12, 13, 18 and 19
(b) GPIO2 and 3
(c) any GPIO
(d) GPIO0 and 1

Q13 What is wrong with button.when_pressed = led.on()?

(a) nothing
(b) led.on() runs once now and None is assigned, so later presses do nothing
(c) it turns the LED off
(d) it raises a syntax error

Q14 What does bounce_time=0.05 do?

(a) ignores further edges for 50 ms after one is detected
(b) waits 50 ms before starting
(c) blinks every 50 ms
(d) limits current to 50 mA

Q15 Which command sets GPIO17 as an output and drives it high from the terminal?

(a) pinctrl get 17
(b) gpio write 17
(c) pinctrl set 17 op dh
(d) sudo led 17 on

Q16 An LED at 25 % duty cycle PWM looks

(a) four times brighter
(b) off
(c) flickering slowly at 25 Hz
(d) about a quarter as bright

7.8.3 Advanced level

Q17 Six LEDs each draw 8 mA from separate GPIO pins at once. Is this within the guideline of 50 mA total?

(a) no: 64 mA
(b) no: 80 mA
(c) yes: 8 mA
(d) yes: 48 mA

Q18 Which value is the brightness of PWMLED after led.value = 0.6?

(a) 6 % duty cycle
(b) 0.6 V
(c) 60 % duty cycle
(d) 60 Hz

Q19 A 5 V sensor output must go to a GPIO input. Which divider makes about 3.3 V from 5 V?

(a) 1 kΩ on top, 2 kΩ to GND
(b) 2 kΩ on top, 1 kΩ to GND
(c) 10 kΩ on top, 1 kΩ to GND
(d) no divider is needed

Q20 Make an LED on GPIO17 toggle each time a button on GPIO2 is pressed, using a callback.

Q21 Fade an LED on GPIO18 up from 0 to 100 % in 10 steps of 0.1 s, then down again, and print the final value.

Q22 Count how many times a button on GPIO3 is pressed in 5 seconds and print the total and the presses per second.

Q23 Show a "held" action: when a button on GPIO2 is held for 2 seconds, print "shutdown requested"; a short press prints "short press".

Q24 Using an RGB LED on GPIO 9, 10, 11, show a temperature as colour: blue below 20 °C, green 20–30 °C, red above 30 °C, for temperatures 15, 25 and 35.

Q25 Which is the safest first test of a newly wired LED on GPIO17 without writing Python?

(a) connect the LED to 5 V directly
(b) run sudo rm /dev/gpiochip0
(c) touch the pin with a finger
(d) pinctrl set 17 op dh, then pinctrl set 17 op dl

7.9 Solutions

Q1 **Answer: (d) pin 1**
Pins **1** and 17 are 3.3 V; 2 and 4 are 5 V; 6 is GND.

Q2 **Answer: (b) 11**
GPIO17 = **pin 11**.

Q3 **Answer: (a) 3.3 V**
3.3 V, and inputs are not 5 V tolerant.

Q4 **Answer: (c) gpiozero**
gpiozero.

Q5 **Answer: (a) holds an input HIGH when nothing else drives it**
It stops the input **floating**.

Q6 **Answer: (b) between GPIO2 and GND**
Default **pull-up**, so the button connects the pin to GND.

Q7 **Answer: (d) led.blink()**
`blink(on_time, off_time)`.

Q8 **Answer: (c) BCM (GPIO) numbers**
LED(17) means **GPIO17**.

Q9 **Answer: (c) 130 Ω**
(3.3 - 2.0) ÷ 0.01 = **130 Ω**.

Q10 **Answer: (d) 3 mA**
(3.3 - 3.0) ÷ 100 = 0.003 A = **3 mA** (dim; blue LEDs are tricky at 3.3 V).

Q11 **Answer: (b) the header pins are driven by the RP1 chip, not the Broadcom registers RPi.GPIO writes to**
Use gpiozero, lgpio or the **rpi-lgpio** drop-in.

Q12 **Answer: (a) GPIO12, 13, 18 and 19**

Hardware PWM channels map to **12, 13, 18, 19**.

Q13 **Answer: (b) led.on() runs once now and None is assigned, so later presses do nothing**

Assign the **function** (led.on), not its result.

Q14 **Answer: (a) ignores further edges for 50 ms after one is detected**

It filters contact **bounce**.

Q15 **Answer: (c) pinctrl set 17 op dh**

`op` output, `dh` drive high.

Q16 **Answer: (d) about a quarter as bright**

Average power ∝ **duty cycle**.

Q17 **Answer: (d) yes: 48 mA**

6 × 8 = **48 mA**, just under 50 mA.

Q18 **Answer: (c) 60 % duty cycle**

value 0–1 = **duty cycle**.

Q19 **Answer: (a) 1 kΩ on top, 2 kΩ to GND**

V_{out} = 5 × 2/(1+2) = **3.33 V**.

Q20 **Answer: Solution program**

led.toggle switches between on and off; it runs on every press. The simulation pressed the button three times.

```
from gpiozero import LED, Button
from signal import pause

led = LED(17)
button = Button(2, bounce_time=0.05)
button.when_pressed = led.toggle
pause()
```

SIMULATED RUN

```
[  1.00 s] LED(GPIO17) on
[  2.00 s] LED(GPIO17) off
[  3.00 s] LED(GPIO17) on
```

Q21 **Answer: Solution program**

value goes 0.0, 0.1 … 1.0 then back to 0.0. (gpiozero also offers led.pulse() for this.)

```
from gpiozero import PWMLED
from time import sleep

led = PWMLED(18)
for i in range(11):
    led.value = i / 10
    sleep(0.1)
for i in range(10, -1, -1):
    led.value = i / 10
    sleep(0.1)
print("final value", led.value)
```

OUTPUT

```
final value 0.0
```

Q22 **Answer: Solution program**

A callback counts presses while the main program simply sleeps for 5 s. The simulation pressed 7 times.

```
from gpiozero import Button
from time import sleep

count = 0
def pressed():
    global count
    count += 1

b = Button(3)
b.when_pressed = pressed
sleep(5)
print("presses:", count, "rate:", count / 5, "per second")
```

OUTPUT

```
presses: 7 rate: 1.4 per second
```

Q23 **Answer: Solution program**

when_held fires after hold_time seconds; the flag stops the release of a long press also counting as a short press. Press 1 lasted 0.3 s, press 2 lasted 3 s.

```
from gpiozero import Button
from signal import pause

held_flag = False
def held():
    global held_flag
    held_flag = True
    print("shutdown requested")
def released():
    global held_flag
    if not held_flag:
        print("short press")
    held_flag = False

b = Button(2, hold_time=2)
b.when_held = held
b.when_released = released
pause()
```

OUTPUT

```
short press
shutdown requested
```

Q24 **Answer: Solution program**

Each (r, g, b) tuple sets one colour fully on.

```
from gpiozero import RGBLED
from time import sleep

rgb = RGBLED(9, 10, 11)
def show(t):
    if t < 20:
        rgb.color = (0, 0, 1)
    elif t <= 30:
        rgb.color = (0, 1, 0)
    else:
        rgb.color = (1, 0, 0)
for t in (15, 25, 35):
    show(t)
    print(t, "C ->", rgb.color)
    sleep(1)
```

OUTPUT

```
15 C -> (0.0, 0.0, 1.0)
25 C -> (0.0, 1.0, 0.0)
35 C -> (1.0, 0.0, 0.0)
```

Q25 **Answer: (d) pinctrl set 17 op dh, then pinctrl set 17 op dl**

pinctrl drives the pin high then low.

3.3 V logic, never 5 V into a GPIO; keep each pin under about 16 mA. LED resistor R = (3.3 - V_F) / I (220 Ω is a safe default). gpiozero uses BCM numbers: LED, Button (pull-up, wire to GND), Buzzer, PWMLED, RGBLED. Use bounce_time and callbacks (assign the function, not its result). Hardware PWM on GPIO12/13/18/19.

MODULE 08 · BEGINNER

Sensors and basic input devices

Minimum duration **2 days** | Level **Beginner**

Reading the environment turns a Pi into a measuring and monitoring device. This module reads digital sensors (PIR motion, IR obstacle, reed and tilt switches), measures distance with the HC-SR04 and a voltage divider for its 5 V echo, reads temperature and humidity with the DHT11/DHT22 and the DS18B20 on the 1-Wire bus, adds analog input through the MCP3008 ADC for potentiometers, LDRs and soil sensors, uses the Sense HAT, and logs and plots readings with CSV and matplotlib, followed by 25 solved questions and programs.

IN THIS MODULE

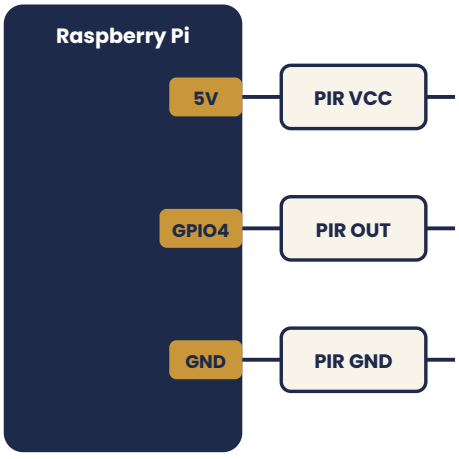
8.1 Digital sensors
8.2 Distance measurement
8.3 Temperature and humidity
8.4 Analog input
8.5 Sense HAT
8.6 Logging and plotting data
8.7 Practice questions
8.8 Solutions

8.1 Digital sensors

A **digital sensor module** outputs HIGH or LOW, so it connects to any GPIO like a button. Most modules run from 3.3 V or 5 V; if a module is powered from 5 V, make sure its output is 3.3 V (or divide it down) before it reaches the Pi.

8.1.1 PIR motion sensor

A **PIR** (passive infrared) sensor such as the HC-SR501 detects moving warm bodies. Its output goes HIGH for a time set by a trimmer (about 3 s to 5 min) when it sees motion. Power it from 5 V; its output is 3.3 V, safe for the Pi. It needs about 30–60 s to settle after power-up.



HC-SR501 PIR WIRING: VCC TO 5 V (PIN 2), OUT TO GPIO4 (PIN 7), GND TO GND (PIN 6)

PIR_LIGHT.PY

```
from gpiozero import MotionSensor, LED
from signal import pause

pir = MotionSensor(4)
led = LED(17)

def motion():
    print("Motion detected")
    led.on()

def no_motion():
    print("All quiet")
    led.off()

pir.when_motion = motion
pir.when_no_motion = no_motion
pause()
```

SIMULATED RUN (hardware events and printed output)

```
Motion detected
[  2.00 s] LED(GPIO17) on
All quiet
[  5.00 s] LED(GPIO17) off
Motion detected
[  8.00 s] LED(GPIO17) on
All quiet
[  9.50 s] LED(GPIO17) off
```

8.1.2 IR obstacle sensor

An IR obstacle module shines infrared light and detects its reflection. Its output is usually **LOW when an object is near** (active-low); a trimmer sets the range (2–30 cm). Use `DigitalInputDevice(pin, pull_up=True)` and treat value 0 as "object", or `LineSensor` for line-following modules.

8.1.3 Reed switch and tilt switch

A **reed switch** closes when a magnet is near: door and window sensors. A **tilt switch** closes (or opens) when tilted. Both are plain switches: wire between GPIO and GND and read with `Button` (pull-up), adding `bounce_time`.

DOOR_MONITOR.PY

```
from gpiozero import Button
from datetime import datetime
from signal import pause

door = Button(17, bounce_time=0.1)     # reed switch: closed = door shut

door.when_released = lambda: print(datetime.now().strftime("%H:%M:%S"), "DOOR OPENED")
door.when_pressed = lambda: print(datetime.now().strftime("%H:%M:%S"), "door closed")
pause()
```

OUTPUT

```
09:00:00 door closed
09:00:03 DOOR OPENED
09:00:10 door closed
```

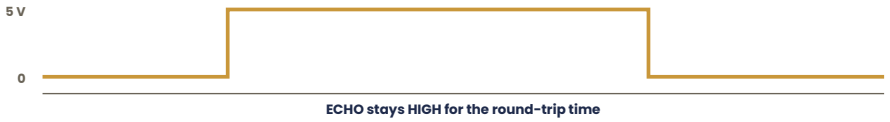
8.2 Distance measurement

8.2.1 HC-SR04 ultrasonic sensor

The HC-SR04 sends a 40 kHz ultrasonic burst when its TRIG pin gets a 10 µs pulse, then holds ECHO HIGH for as long as the sound takes to go out and back. Sound travels at about **343 m/s** in air at 20 °C, so:

distance = echo time × 343 ÷ 2 (metres), or distance in cm = echo time in µs ÷ 58.3

TRIG PULSE STARTS A MEASUREMENT…



…AND THE ECHO PULSE WIDTH IS PROPORTIONAL TO DISTANCE

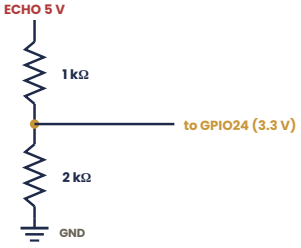
WORKED EXAMPLE

Echo time to distance

Echo HIGH for 1166 µs: distance = 0.001166 × 343 ÷ 2 = **0.200 m = 20.0 cm**. Range is about 2 cm to 4 m; the echo pulse for 4 m is about 23 ms.

8.2.2 Voltage divider for the 5 V echo pin

The HC-SR04 runs at 5 V and its ECHO output is **5 V**, too high for a GPIO. A divider of **1 kΩ** (from ECHO) and **2 kΩ** (to GND) gives 5 × 2 ÷ 3 = **3.33 V** at the junction. TRIG can be driven directly by the Pi's 3.3 V.



DIVIDER ON THE ECHO LINE

8.2.3 gpiozero DistanceSensor

DISTANCE.PY

```
from gpiozero import DistanceSensor
from time import sleep

sensor = DistanceSensor(echo=24, trigger=23, max_distance=2)
for _ in range(5):
    print(f"Distance: {sensor.distance * 100:.1f} cm")
    sleep(1)
```

OUTPUT

```
Distance: 85.0 cm
Distance: 85.0 cm
Distance: 42.0 cm
Distance: 18.0 cm
Distance: 5.0 cm
```

`distance` is in metres (capped at max_distance). gpiozero averages several readings internally to remove noise. Readings jitter more when the CPU is busy, because the timing is done in software.

8.3 Temperature and humidity

8.3.1 DHT11 and DHT22

	DHT11	DHT22 (AM2302)
Temperature	0–50 °C, ±2 °C	-40–80 °C, ±0.5 °C
Humidity	20–80 %, ±5 %	0–100 %, ±2–5 %
Resolution	1 °C, 1 %	0.1 °C, 0.1 %
Read at most	once per second	once every 2 seconds

DHT sensors use their own one-wire timing protocol on one GPIO (with a 10 kΩ pull-up, built into most modules). Use the CircuitPython library in a venv: `pip install adafruit-circuitpython-dht`. Reads fail occasionally because the timing is strict in Linux, so **retry** on RuntimeError:

DHT22.PY

```
import time
import board
import adafruit_dht

dht = adafruit_dht.DHT22(board.D4)
good = 0
while good < 4:
    try:
        t = dht.temperature
        h = dht.humidity
        print(f"Temp {t:.1f} C   Humidity {h:.1f} %")
        good += 1
    except RuntimeError as e:
        print("retrying:", e)
    time.sleep(2.0)
dht.exit()
```

OUTPUT

```
Temp 24.6 C   Humidity 61.2 %
Temp 24.6 C   Humidity 61.2 %
retrying: Checksum did not validate. Try again.
Temp 24.8 C   Humidity 61.2 %
Temp 24.8 C   Humidity 61.2 %
```

8.3.2 DS18B20 on the 1-Wire bus

The DS18B20 is a digital thermometer (-55 to 125 °C, ±0.5 °C) that uses the **1-Wire** bus: data on GPIO4 with a **4.7 kΩ pull-up to 3.3 V**. Waterproof probe versions are ideal for water, soil and outdoor use. Many sensors can share one wire because each has a unique 64-bit ID. Enable the bus with `dtoverlay=w1-gpio` in config.txt (or raspi-config → Interface Options → 1-Wire), reboot, and each sensor appears as a folder:

READING THE SENSOR FROM THE SHELL

```
pi@raspberrypi:~ $ ls /sys/bus/w1/devices/
28-3c01d607e348  w1_bus_master1
pi@raspberrypi:~ $ cat /sys/bus/w1/devices/28-3c01d607e348/temperature
23125
```

DS18B20.PY

```
import glob, time

def read_ds18b20():
    readings = {}
    for dev in glob.glob("/sys/bus/w1/devices/28*"):
        with open(dev + "/temperature") as f:
            readings[dev.split("/")[-1]] = int(f.read()) / 1000
    return readings

for _ in range(2):
    for sensor_id, t in read_ds18b20().items():
        print(f"{sensor_id}: {t:.3f} C")
    time.sleep(2)
```

```
OUTPUT
28-3c01d607e348: 23.125 C
28-3c01d6071a2b: 18.500 C
28-3c01d607e348: 23.250 C
28-3c01d6071a2b: 18.500 C
```

8.4 Analog input

8.4.1 Why the Pi has no built-in ADC

GPIO pins are purely digital: they read only HIGH or LOW. Sensors such as potentiometers, LDRs, soil-moisture probes and many gas sensors produce a **varying voltage**. To read them, add an **analog-to-digital converter (ADC)** chip.

8.4.2 MCP3008 ADC over SPI

The **MCP3008** has 8 analog inputs (CH0–CH7) and 10-bit resolution: a reading from 0 to 1023 for 0 V to V_{REF} (connect V_{REF} to 3.3 V). It talks over SPI, so enable SPI and connect it to the SPI0 pins.

MCP3008 pin	Connect to
16 VDD, 15 VREF	3.3 V
14 AGND, 9 DGND	GND
13 CLK	GPIO11 SCLK (pin 23)
12 DOUT	GPIO9 MISO (pin 21)
11 DIN	GPIO10 MOSI (pin 19)
10 CS/SHDN	GPIO8 CE0 (pin 24)
1–8 CH0–CH7	the analog sensors

voltage = reading ÷ 1023 × 3.3 V. gpiozero's MCP3008(channel).value gives reading ÷ 1023 (0.0–1.0) directly.

8.4.3 Potentiometer, LDR and soil moisture sensor

A **potentiometer** connects between 3.3 V and GND with its wiper to CH0. An **LDR** (light-dependent resistor) needs a fixed resistor (10 kΩ) to form a divider. A capacitive **soil moisture** sensor gives a lower voltage when the soil is wetter; calibrate it in dry air and in water.

ANALOG.PY

```
from gpiozero import MCP3008, PWMLED
from time import sleep

pot = MCP3008(channel=0)
ldr = MCP3008(channel=1)
led = PWMLED(18)
for _ in range(4):
    led.value = pot.value                  # knob sets LED brightness
    volts = pot.value * 3.3
    light = "dark" if ldr.value < 0.3 else "bright"
    print(f"pot {pot.raw_value:4d} = {volts:.2f} V   LED {led.value:.0%}   light {ldr.value:.2f}
({light})")
    sleep(1)
```

OUTPUT

```
pot    0 = 0.00 V   LED 0%   light 0.80 (bright)
pot  338 = 1.09 V   LED 33%   light 0.80 (bright)
pot  767 = 2.47 V   LED 75%   light 0.80 (bright)
pot 1023 = 3.30 V   LED 100%   light 0.12 (dark)
```

WORKED EXAMPLE

Calibrating a soil sensor

Dry reading 820, fully wet 400. For a reading of 610: moisture = (820 - 610) ÷ (820 - 400) × 100 = 210 ÷ 420 × 100 = **50 %**.

8.5 Sense HAT

8.5.1 LED matrix, joystick and environmental sensors

The Sense HAT (module 2) needs no wiring. The `sense_hat` library is pre-installed. Its temperature sensor sits close to the Pi and reads a few degrees high; mount it on a GPIO extension cable for accurate room temperature.

SENSE_BASIC.PY

```
from sense_hat import SenseHat
sense = SenseHat()

t = round(sense.get_temperature(), 1)
h = round(sense.get_humidity(), 1)
p = round(sense.get_pressure(), 1)
print(f"{t} C  {h} %  {p} hPa")
colour = (255, 0, 0) if t > 28 else (0, 255, 0)
sense.show_message(f"{t}C", scroll_speed=0.05, text_colour=colour)
sense.set_pixel(0, 0, (0, 0, 255))
```

SIMULATED RUN (hardware events and printed output)

```
29.4 C  48.0 %  1009.6 hPa
[  0.00 s] LED matrix scrolls "29.4C" colour (255, 0, 0)
[  1.50 s] pixel (0,0) = (0, 0, 255)
```

8.6 Logging and plotting data

8.6.1 Saving readings to CSV

CSV_LOGGER.PY

```
import csv, os, time
from datetime import datetime
from gpiozero import MCP3008

light = MCP3008(channel=1)
FILE = "light_log.csv"
new = not os.path.exists(FILE)
with open(FILE, "a", newline="") as f:
    w = csv.writer(f)
    if new:
        w.writerow(["timestamp", "light"])          # header only once
    for _ in range(5):
        w.writerow([datetime.now().isoformat(timespec="seconds"), round(light.value, 3)])
        f.flush()                                   # data is safe even if the Pi loses power
        time.sleep(60)
print(open(FILE).read())
```

OUTPUT

```
timestamp,light
2026-03-15T09:00:00,0.62
2026-03-15T09:01:00,0.64
2026-03-15T09:02:00,0.58
2026-03-15T09:03:00,0.41
2026-03-15T09:04:00,0.37
```

8.6.2 Plotting with matplotlib

Install with `sudo apt install python3-matplotlib`. On a headless Pi, save the plot to a PNG file instead of showing a window:

PLOT_ROOM.PY

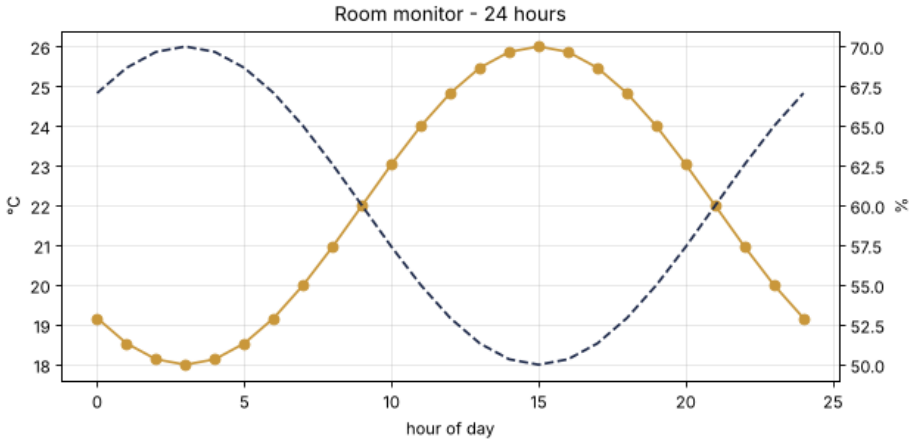
```
import matplotlib
matplotlib.use("Agg")                    # no screen needed
import matplotlib.pyplot as plt
import math

hours = list(range(25))
temp = [22 + 4 * math.sin((h - 9) / 24 * 2 * math.pi) for h in hours]
hum = [60 - 10 * math.sin((h - 9) / 24 * 2 * math.pi) for h in hours]

fig, ax1 = plt.subplots(figsize=(8, 4))
ax1.plot(hours, temp, color="#C9973A", marker="o", label="temperature")
ax1.set_xlabel("hour of day"); ax1.set_ylabel("°C")
ax2 = ax1.twinx()
ax2.plot(hours, hum, color="#1E2A4A", linestyle="--", label="humidity")
ax2.set_ylabel("%")
ax1.set_title("Room monitor - 24 hours")
ax1.grid(alpha=0.3)
fig.tight_layout()
fig.savefig("room.png", dpi=80)
print("max temp", round(max(temp), 1), "at hour", hours[temp.index(max(temp))])
```

OUTPUT

```
max temp 26.0 at hour 15
```



THE PNG CREATED BY THE PROGRAM

PRACTICE CHECKPOINT

Build a room monitor that reads temperature, humidity, light and motion, logs data to CSV and plots a 24-hour graph.

8.7 Practice questions

8.7.1 Basic level

Q1 What does a PIR sensor detect?

(a) distance by sound
(b) moving warm bodies (infrared)
(c) light level
(d) magnetic fields

Q2 Which gpiozero class suits a PIR sensor?

(a) DistanceSensor (b) MotionSensor (c) LED (d) MCP3008

Q3 A reed switch is triggered by

(a) light (b) heat (c) sound (d) a magnet

Q4 Why does the Pi need an MCP3008 to read a potentiometer?

(a) the Pi has no analog-to-digital converter
(b) the Pi has no SPI
(c) potentiometers need 12 V
(d) GPIO pins are too fast

Q5 What resolution does the MCP3008 have?

(a) 8-bit (0–255)
(b) 12-bit (0–4095)
(c) 10-bit (0–1023)
(d) 16-bit

Q6 Which bus does the DS18B20 use?

(a) I2C (b) SPI (c) UART (d) 1-Wire

Q7 What is the speed of sound used for HC-SR04 calculations at 20 °C?

(a) about 34 m/s (b) about 343 m/s (c) about 3430 m/s (d) 300 000 km/s

Q8 Which is more accurate, DHT11 or DHT22?

(a) DHT22
(b) DHT11
(c) they are identical
(d) neither measures temperature

8.7.2 Intermediate level

Q9 Why does the HC-SR04 ECHO line need a voltage divider?

(a) it needs more current
(b) to make it faster
(c) it outputs 5 V, which would damage a 3.3 V GPIO
(d) it outputs negative voltage

Q10 An HC-SR04 echo lasts 2332 µs. What is the distance?

(a) 40 cm (b) 80 cm (c) 20 cm (d) 4 cm

Q11 With a 1 kΩ top and 2 kΩ bottom resistor, what voltage does a 5 V echo give?

(a) 2.5 V (b) 3.33 V (c) 1.67 V (d) 5 V

Q12 An MCP3008 (VREF = 3.3 V) reads 512. What is the input voltage?

(a) 3.3 V (b) 0.5 V (c) 2.2 V (d) 1.65 V

Q13 Why do DHT22 programs use try/except around each read?

(a) the sensor needs exceptions to start
(b) to make it faster
(c) reads sometimes fail because of strict timing, raising RuntimeError
(d) Python requires it for GPIO

Q14 Where does a DS18B20 reading appear on Raspberry Pi OS?

(a) /dev/i2c-1
(b) /etc/ds18b20.conf
(c) /sys/bus/w1/devices/28-xxxx/temperature
(d) /boot/firmware/temp.txt

Q15 Why should you call f.flush() when logging to CSV?

(a) to delete old data
(b) to sort the file
(c) to compress the file
(d) so each line is written to the card at once and is not lost if power fails

Q16 Why does matplotlib.use("Agg") matter on a headless Pi?

(a) it speeds up the CPU
(b) it draws to image files without needing a screen
(c) it enables SPI
(d) it installs matplotlib

8.7.3 Advanced level

Q17 What is the maximum echo pulse length for 4 m?

(a) about 23.3 ms (b) about 2.3 ms (c) about 233 ms (d) about 11.7 ms

Q18 Soil sensor: dry = 800, wet = 380. What moisture % is a reading of 590?

(a) 74 % (b) 50 % (c) 26 % (d) 59 %

Q19 An MCP3008 must read a 0–5 V sensor with VREF = 3.3 V. What should you do?

(a) divide the sensor voltage down so 5 V becomes at most 3.3 V
(b) connect it directly
(c) set VREF to 12 V
(d) use a DHT22 instead

Q20 A PIR on GPIO4 should count motion events and print the count with a timestamp each time.

Q21 Make a parking sensor: an HC-SR04 (echo 24, trigger 23) and a buzzer on GPIO 22. Print the distance and beep faster as the car gets closer (off above 50 cm, on continuously below 10 cm).

Q22 Read the MCP3008 channel 2 every second and convert it to volts; print "LOW BATTERY" when the voltage of a battery connected through a 2:1 divider (so actual = 2 × measured) falls below 6.0 V.

Q23 Read a DS18B20 and a DHT22 together every 5 s, three times, and print both temperatures and

their difference. Handle DHT failures by printing "DHT read failed".

Q24 Read a list of 24 hourly temperatures from temps.csv (columns hour,temp), plot them with matplotlib to day.png and print the min and max with their hours.

Q25 In the room monitor, which design reads all sensors reliably?

(a) a separate program per sensor writing to the same file at random times

(b) reading the DHT22 ten times a second

(c) one loop that reads each sensor with error handling, writes one CSV row per minute with a timestamp, and flushes

(d) printing values without saving

8.8 Solutions

Q1 **Answer: (b) moving warm bodies (infrared)**

Passive **infrared** motion.

Q2 **Answer: (b) MotionSensor**

MotionSensor with when_motion / when_no_motion.

Q3 **Answer: (d) a magnet**

Magnet: used on doors and windows.

Q4 **Answer: (a) the Pi has no analog-to-digital converter**

No built-in **ADC**.

Q5 **Answer: (c) 10-bit (0–1023)**

10-bit.

Q6 **Answer: (d) 1-Wire**

1-Wire, on GPIO4 by default with a 4.7 kΩ pull-up.

Q7 **Answer: (b) about 343 m/s**

343 m/s.

Q8 **Answer: (a) DHT22**

DHT22: ±0.5 °C vs ±2 °C.

Q9 **Answer: (c) it outputs 5 V, which would damage a 3.3 V GPIO**

Bring 5 V down to **3.3 V**.

Q10 **Answer: (a) 40 cm**

0.002332 × 343 ÷ 2 = 0.400 m = **40 cm**.

Q11 **Answer: (b) 3.33 V**

5 × 2 ÷ 3 = **3.33 V**.

Q12 **Answer: (d) 1.65 V**

512 ÷ 1023 × 3.3 = **1.65 V**.

Q13 **Answer: (c) reads sometimes fail because of strict timing, raising RuntimeError**

Catch the error and **retry** after 2 s.

Q14 **Answer: (c) /sys/bus/w1/devices/28-xxxx/temperature**

Each sensor has a **28-…** folder; temperature is in millidegrees.

Q15 **Answer: (d) so each line is written to the card at once and is not lost if power fails**

flush() pushes buffered data to the file.

Q16 **Answer: (b) it draws to image files without needing a screen**

The **Agg** backend renders to PNG with no display.

Q17 **Answer: (a) about 23.3 ms**

Round trip 8 m ÷ 343 m/s = 0.0233 s = **23.3 ms**.

Q18 **Answer: (b) 50 %**

(800 - 590) ÷ (800 - 380) × 100 = 210 ÷ 420 × 100 = **50 %**.

Q19 **Answer: (a) divide the sensor voltage down so 5 V becomes at most 3.3 V**

Inputs must stay within 0–VREF (and 3.3 V supply).

Q20 **Answer: Solution program**

A global counter is updated in the callback. The simulation produced three motion events.

```
from gpiozero import MotionSensor
from datetime import datetime
from signal import pause

count = 0
def seen():
    global count
    count += 1
    print(datetime.now().strftime("%H:%M:%S"), "motion #", count)

pir = MotionSensor(4)
pir.when_motion = seen
pause()
```

OUTPUT

```
09:00:03 motion # 1
09:00:15 motion # 2
09:00:30 motion # 3
```

Q21 **Answer: Solution program**

The beep gap is proportional to the distance. beep() runs in the background, so the loop keeps measuring.

```
from gpiozero import DistanceSensor, Buzzer
from time import sleep

sensor = DistanceSensor(echo=24, trigger=23)
bz = Buzzer(22)
for _ in range(5):
    cm = sensor.distance * 100
    if cm > 50:
        bz.off(); action = "silent"
    elif cm < 10:
        bz.on(); action = "continuous"
    else:
        gap = cm / 100                    # 50 cm -> 0.5 s, 20 cm -> 0.2 s
        bz.beep(on_time=0.05, off_time=gap); action = f"beep every {gap + 0.05:.2f} s"
    print(f"{cm:5.1f} cm  {action}")
    sleep(1)
```

OUTPUT

```
 75.0 cm  silent
 40.0 cm  beep every 0.45 s
 22.0 cm  beep every 0.27 s
  8.0 cm  continuous
  6.0 cm  continuous
```

Q22 **Answer: Solution program**

The divider halves the battery voltage to keep it under 3.3 V; the program multiplies back by 2.

```
from gpiozero import MCP3008
from time import sleep

ch = MCP3008(channel=2)
for _ in range(4):
    measured = ch.value * 3.3
    battery = measured * 2
    flag = "LOW BATTERY" if battery < 6.0 else "ok"
    print(f"ADC {ch.raw_value}  measured {measured:.2f} V  battery {battery:.2f} V  {flag}")
    sleep(1)
```

OUTPUT

```
ADC 992  measured 3.20 V  battery 6.40 V  ok
ADC 962  measured 3.10 V  battery 6.21 V  ok
ADC 921  measured 2.97 V  battery 5.94 V  LOW BATTERY
ADC 900  measured 2.90 V  battery 5.81 V  LOW BATTERY
```

Q23 **Answer: Solution program**

Two different buses on different pins; the DHT read failed once (every 2nd read in the simulation).

```
import glob, time, board, adafruit_dht

dht = adafruit_dht.DHT22(board.D17)
dev = glob.glob("/sys/bus/w1/devices/28*")[0]
for _ in range(3):
    with open(dev + "/temperature") as f:
        t1 = int(f.read()) / 1000
    try:
        t2 = dht.temperature
        print(f"DS18B20 {t1:.2f} C  DHT22 {t2:.1f} C  diff {abs(t1 - t2):.2f}")
    except RuntimeError:
        print(f"DS18B20 {t1:.2f} C  DHT read failed")
    time.sleep(5)
```

OUTPUT

```
DS18B20 24.38 C  DHT22 24.1 C  diff 0.27
DS18B20 24.38 C  DHT read failed
DS18B20 24.38 C  DHT22 24.1 C  diff 0.27
```

Q24 **Answer: Solution program**

zip(*rows) splits the list of pairs into two sequences for plotting; min/max with key compare by temperature.

```
import csv
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

with open("temps.csv", newline="") as f:
    rows = [(int(r["hour"]), float(r["temp"])) for r in csv.DictReader(f)]
hours, temps = zip(*rows)
plt.plot(hours, temps, marker=".")
plt.xlabel("hour"); plt.ylabel("°C"); plt.title("24 h")
plt.savefig("day.png", dpi=60)
lo = min(rows, key=lambda r: r[1]); hi = max(rows, key=lambda r: r[1])
print(f"min {lo[1]} C at {lo[0]}:00, max {hi[1]} C at {hi[0]}:00")
```

OUTPUT

```
min 16.8 C at 5:00, max 25.4 C at 14:00
```

Q25 **Answer: (c) one loop that reads each sensor with error handling, writes one CSV row per minute with a timestamp, and flushes**

One timestamped row per interval keeps the data aligned and safe.

Digital sensors (PIR, IR, reed, tilt) read like buttons. HC-SR04: distance = time × 343 ÷ 2, and divide the 5 V echo (1 kΩ/2 kΩ). DHT22: retry failed reads, 2 s apart. DS18B20: 1-Wire on GPIO4, 4.7 kΩ pull-up, read /sys/bus/w1/devices/28-*/temperature. Analog needs an ADC: MCP3008, V = reading ÷ 1023 × 3.3. Log with timestamps and flush; plot with matplotlib (Agg backend).

MODULE 09 · BEGINNER

Beginner projects

Minimum duration **2 days** | Level **Beginner**

This module brings the beginner level together into complete, working mini-projects. It shows how to plan a project with requirements, a circuit diagram and a flowchart, builds five projects (reaction-time game, motion alarm, weather station, smart night light, digital dice and quiz buzzer), makes programs start at boot with cron and systemd, and documents and publishes a project on GitHub, followed by 25 solved questions and programs.

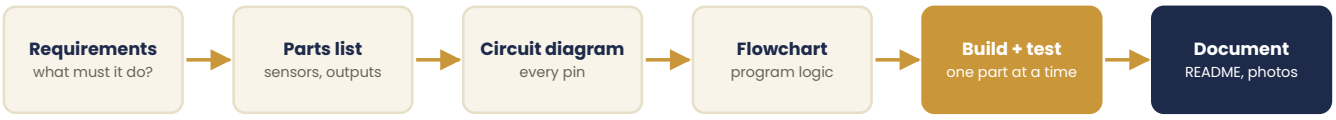
IN THIS MODULE

9.1	Project planning	9.2	Project ideas
9.3	Running programs at boot	9.4	Documenting a project
9.5	Practice questions	9.6	Solutions

9.1 Project planning

9.1.1 Requirements, circuit diagram and flowchart

Before wiring anything, write down what the project must do. Good requirements are **specific and testable**: not "an alarm", but "when motion is detected between 22:00 and 06:00, sound the buzzer for 10 s and log the time".



PROJECT WORKFLOW

FLOWCHART OF THE MOTION ALARM

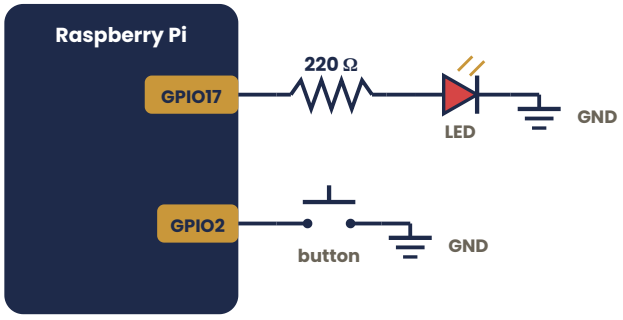
9.1.2 Drawing circuits in Fritzing or similar tools

Fritzing draws breadboard views that look like the real build; **KiCad** draws proper schematics; online simulators such as **Wokwi** can even run a Pi Pico or ESP32 virtually. A neat hand-drawn diagram with pin numbers is fine for small projects. Always label BCM and physical pin numbers.

9.2 Project ideas

9.2.1 Reaction-time game

An LED lights after a random delay; the player presses the button as fast as possible. Pressing too early is a foul.



REACTION GAME WIRING

REACTION.PY

```
from gpiozero import LED, Button
from time import sleep, monotonic
import random

led = LED(17)
button = Button(2)
random.seed(7)
best = None
for rnd in range(1, 4):
    print(f"Round {rnd}: get ready...")
    sleep(random.uniform(2, 5))
    if button.is_pressed:
        print("  Foul! You pressed too early.")
        continue
    led.on()
    start = monotonic()
    button.wait_for_press()
    reaction = monotonic() - start
    led.off()
    print(f"  Reaction time: {reaction * 1000:.0f} ms")
    best = reaction if best is None else min(best, reaction)
    button.wait_for_release()
print(f"Best: {best * 1000:.0f} ms")
```

OUTPUT

```
Round 1: get ready...
  Reaction time: 314 ms
```

```
Round 2: get ready...
  Reaction time: 269 ms
Round 3: get ready...
  Reaction time: 293 ms
Best: 269 ms
```

9.2.2 Motion-activated alarm with buzzer

ALARM.PY

```
from gpiozero import MotionSensor, Buzzer, LED
from datetime import datetime
from signal import pause

pir = MotionSensor(4)
buzzer = Buzzer(23)
armed = LED(17)          # status light: armed
armed.on()

def alarm():
    stamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    print("INTRUDER at", stamp)
    with open("alarm.log", "a") as f:
        f.write(stamp + "\n")
    buzzer.beep(0.25, 0.25, n=6)          # 3 s of beeping in the background

pir.when_motion = alarm
pause()
```

SIMULATED RUN (hardware events and printed output)

```
[  0.00 s] LED(GPIO17) on
INTRUDER at 2026-03-15 09:00:05
[  5.00 s] Buzzer(GPIO23) on
[  5.25 s] Buzzer(GPIO23) off
[  5.50 s] Buzzer(GPIO23) on
[  5.75 s] Buzzer(GPIO23) off
[  6.00 s] Buzzer(GPIO23) on
[  6.25 s] Buzzer(GPIO23) off
[  6.50 s] Buzzer(GPIO23) on
[  6.75 s] Buzzer(GPIO23) off
[  7.00 s] Buzzer(GPIO23) on
[  7.25 s] Buzzer(GPIO23) off
[  7.50 s] Buzzer(GPIO23) on
[  7.75 s] Buzzer(GPIO23) off
```

9.2.3 Weather station with Sense HAT or sensors

WEATHER.PY

```
from sense_hat import SenseHat
import time

sense = SenseHat()
for _ in range(3):
    t = sense.get_temperature()
    h = sense.get_humidity()
    p = sense.get_pressure()
    trend = "rain likely" if p < 1005 else "fair"
    print(f"{t:5.1f} C  {h:4.0f} %  {p:7.1f} hPa  {trend}")
    time.sleep(600)                      # every 10 minutes
```

OUTPUT

```
 24.2 C    58 %   1007.4 hPa  fair
 24.6 C    58 %   1005.9 hPa  fair
 23.9 C    66 %   1003.8 hPa  rain likely
```

SHORTCUT TRICK

Falling pressure means worsening weather: a drop of more than about 1 hPa per hour, or pressure below about 1005 hPa, often means rain on the way. Rising pressure means clearing.

9.2.4 Smart night light

NIGHT_LIGHT.PY

```
from gpiozero import MCP3008, MotionSensor, PWMLED
from time import sleep

ldr = MCP3008(channel=0)        # bright = high value
pir = MotionSensor(4)
lamp = PWMLED(18)
for _ in range(6):
    dark = ldr.value < 0.25
    if dark and pir.motion_detected:
        lamp.value = 1.0            # someone is here in the dark: full light
    elif dark:
        lamp.value = 0.1            # dark but empty: dim glow
    else:
        lamp.value = 0              # daytime: off
    print(f"light {ldr.value:.2f}  motion {pir.motion_detected!s:5}  lamp {lamp.value:.0%}")
    sleep(5)
```

OUTPUT

```
light 0.70  motion False  lamp 0%
light 0.70  motion False  lamp 0%
light 0.18  motion False  lamp 10%
light 0.18  motion False  lamp 10%
light 0.18  motion True   lamp 100%
light 0.18  motion False  lamp 10%
```

9.2.5 Digital dice and quiz buzzer

DICE.PY

```
from gpiozero import Button
from signal import pause
import random

random.seed(3)
FACES = {1: "    o    ", 2: "o       o", 3: "o   o   o", 4: "o o   o o", 5: "o o o o o", 6: "ooo
ooo"}
def roll():
    n = random.randint(1, 6)
    print(f"[{FACES[n]}]  {n}")

Button(2).when_pressed = roll
pause()
```

OUTPUT

```
[o       o]  2
[o o o o o]  5
[o o o o o]  5
[o       o]  2
```

QUIZ_BUZZER.PY

```
from gpiozero import Button, LED
from time import sleep

players = {"Team A": (Button(5), LED(17)), "Team B": (Button(6), LED(27))}
winner = None
print("Question: What does GPIO stand for?")
while winner is None:
    for name, (btn, led) in players.items():
        if btn.is_pressed:
            winner = name
            led.on()
            break
    sleep(0.001)
print(winner, "buzzed first!")
```

SIMULATED RUN (hardware events and printed output)

```
Question: What does GPIO stand for?
[  2.41 s] LED(GPIO27) on
Team B buzzed first!
```

The loop checks both buttons every millisecond, so it can tell apart presses only 18 ms apart (Team B at 2.413 s beat Team A at 2.431 s).

9.3 Running programs at boot

9.3.1 cron with @reboot

cron runs commands on a schedule. Edit your user's table with `crontab -e` and add:

CRONTAB LINE: START THE ALARM 20 S AFTER EVERY BOOT

```
@reboot sleep 20 && /usr/bin/python3 /home/pi/alarm/alarm.py >> /home/pi/alarm/cron.log 2>&1
```

Use **full paths** (cron has a minimal PATH and starts in your home folder), redirect output to a log file so you can see errors, and the short `sleep` gives the network and hardware time to come up.

9.3.2 Introduction to systemd services

systemd starts the whole system. A service file gives better control than cron: automatic restart after crashes, logs in the journal, start/stop commands. Module 19 goes deeper.

/ETC/SYSTEMD/SYSTEM/ALARM.SERVICE

```
[Unit]
Description=Motion alarm
After=network-online.target

[Service]
User=pi
WorkingDirectory=/home/pi/alarm
ExecStart=/usr/bin/python3 /home/pi/alarm/alarm.py
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

MANAGING THE SERVICE

```
sudo systemctl daemon-reload        # read the new file
sudo systemctl enable --now alarm    # start now and at every boot
systemctl status alarm               # running? last log lines
journalctl -u alarm -f               # follow its output
sudo systemctl stop alarm            # stop it
```

SHORTCUT TRICK

cron @reboot for quick personal scripts; **systemd** for anything that must keep running (restart on crash, proper logs). In both, use absolute paths.

9.4 Documenting a project

9.4.1 README, photos and circuit diagram

README.MD (MARKDOWN)

```
# Motion Alarm (Raspberry Pi)

Sounds a buzzer and logs the time when motion is detected.

## Hardware
| Part | Pin |
|------|-----|
| HC-SR501 PIR | OUT -> GPIO4 (pin 7), VCC -> 5 V, GND |
| Active buzzer module | GPIO23 (pin 16) |
| Status LED + 220 ohm | GPIO17 (pin 11) |

![circuit](docs/circuit.png)

## Install
    git clone https://github.com/<you>/motion-alarm.git
    cd motion-alarm && python3 alarm.py

## Demo
Video: docs/demo.mp4
```

9.4.2 Publishing code on GitHub

CREATING A REPOSITORY AND THE FIRST COMMIT

```
pi@raspberrypi:~/motion-alarm $ cd motion-alarm && git init -q -b main && git config user.name
"Lohit" && git config user.email "lohit@example.com"
pi@raspberrypi:~/motion-alarm $ cd motion-alarm && git add . && git commit -q -m "First working
version" && git log --oneline
pi@raspberrypi:~/motion-alarm $ cd motion-alarm && git status --short && echo "(clean)"
```

Then create an empty repository on github.com and connect it: `git remote add origin https://github.com/you/motion-alarm.git` and `git push -u origin main` (GitHub needs a personal access token or SSH key, not your password). Add a `.gitignore` listing `env/`, `__pycache__/`, `*.log` and any file with passwords.

WATCH OUT

Never commit passwords, Wi-Fi keys or API tokens to GitHub, even in a private repository. Keep them in a separate file that is listed in .gitignore (module 21).

PRACTICE CHECKPOINT

Build one beginner project end to end, make it start automatically on boot, and publish it on GitHub with a README, diagram and demo video.

9.5 Practice questions

9.5.1 Basic level

Q1 What should you write first when starting a project?

(a) the final README
(b) the GitHub repository
(c) the systemd file
(d) clear, testable requirements

Q2 Which crontab entry runs a script at every boot?

(a) @boot python app.py
(b) reboot = app.py
(c) * * * * * reboot
(d) @reboot /usr/bin/python3 /home/pi/app.py

Q3 Which tool draws breadboard-style circuit diagrams?

(a) nano **(b)** cron **(c)** Fritzing **(d)** git

Q4 Which file explains a GitHub project to visitors?

(a) README.md **(b)** .gitignore **(c)** crontab **(d)** config.txt

Q5 Which command starts a systemd service now and at every boot?

(a) sudo systemctl start name only
(b) sudo service boot name
(c) crontab -e name
(d) sudo systemctl enable --now name

Q6 Which command shows a service's log output?

(a) cat name.log only
(b) journalctl -u name
(c) ls /var/name
(d) git log name

Q7 What does git commit do?

(a) uploads code to GitHub
(b) records a snapshot of the staged changes in the local repository
(c) deletes old files
(d) installs a package

Q8 Which shape usually represents a decision in a flowchart?

(a) a rectangle **(b)** an oval **(c)** an arrow **(d)** a diamond

9.5.2 Intermediate level

Q9 Why add sleep 20 before a program in a cron @reboot line?

(a) to give the network and hardware time to start
(b) to save power
(c) cron requires it
(d) to debounce buttons

Q10 Why must cron commands use full paths?

(a) cron cannot read relative paths at all on any system

(b) to make them faster
(c) cron runs with a minimal PATH and environment
(d) cron runs as root

Q11 What does Restart=on-failure in a service file do?

(a) restarts the Pi on failure
(b) retries apt update
(c) restarts at midnight
(d) restarts the program automatically if it crashes

Q12 In the reaction game, why check button.is_pressed before lighting the LED?

(a) to turn on the pull-up
(b) to detect a foul (pressing too early)
(c) to debounce
(d) to save power

Q13 Which should be listed in .gitignore?

(a) env/, __pycache__/, *.log and secrets files
(b) README.md
(c) the main program
(d) the circuit diagram

Q14 Why measure reaction time with time.monotonic() rather than time.time()?

(a) time.time() is not available on a Pi
(b) monotonic is in milliseconds
(c) monotonic never jumps when the clock is adjusted
(d) time.time() needs internet

Q15 Which pressure reading suggests rain is likely?

(a) falling, below about 1005 hPa
(b) rising, above 1025 hPa
(c) steady at 1013 hPa
(d) pressure does not relate to weather

Q16 A night light should be dim when dark and empty, bright when dark with motion, off in daylight. How many different lamp levels are needed?

(a) 2 (b) 3 (c) 4 (d) 1

9.5.3 Advanced level

Q17 Reaction times were 412, 356 and 389 ms. What are the best and the average?

(a) 412 ms and 385.7 ms
(b) 356 ms and 379 ms
(c) 389 ms and 386 ms
(d) 356 ms and 385.7 ms

Q18 A quiz buzzer loop checks buttons every 1 ms. What is the smallest press-time difference it can always resolve?

(a) about 1 s (b) about 100 ms (c) about 1 ms (d) it cannot tell

Q19 Make a "Simon says" step: flash LEDs on GPIO 17, 27, 22 in a random order of length 4 (seed 5), printing the sequence.

Q20 Write a dice roller that rolls two dice each time a button on GPIO2 is pressed, prints both and the total, and prints "DOUBLE!" for doubles (seed 11).

Q21 The alarm should only sound at night. Write is_night(hour) for 22:00–06:00 and test it for hours 21, 22, 3, 6 and 12.

Q22 Create a git repository for a project, commit a file, change it, and show the difference before committing again.

Q23 Log each alarm trigger to alarm.log, then write a short report program that reads the log and prints how many triggers happened and the first and last time.

Q24 Your alarm runs from cron @reboot but nothing happens after boot. What should you check first?

(a) buy a new PIR
(b) reinstall Raspberry Pi OS
(c) the redirected log file for errors (full paths, missing modules)
(d) change the GPIO library

Q25 Which order publishes a new project on GitHub?

(a) git push → git init → git commit
(b) git commit → git init → git add
(c) git clone → git push → git init
(d) git init → git add → git commit → git remote add origin … → git push -u origin main

9.6 Solutions

Q1 **Answer: (d) clear, testable requirements**

Know **what it must do** before building.

Q2 **Answer: (d) @reboot /usr/bin/python3 /home/pi/app.py**

@reboot with full paths.

Q3 **Answer: (c) Fritzing**

Fritzing.

Q4 **Answer: (a) README.md**

README.md, written in Markdown.

Q5 **Answer: (d) sudo systemctl enable --now name**

`enable --now`.

Q6 **Answer: (b) journalctl -u name**

The systemd journal: `journalctl -u name`.

Q7 **Answer: (b) records a snapshot of the staged changes in the local repository**

Commit is local; **push** uploads.

Q8 **Answer: (d) a diamond**

Diamond = yes/no question.

Q9 **Answer: (a) to give the network and hardware time to start**

Services may not be ready the instant cron runs.

Q10 **Answer: (c) cron runs with a minimal PATH and environment**

Use **/usr/bin/python3 /home/pi/….**

Q11 **Answer: (d) restarts the program automatically if it crashes**

systemd restarts a **crashed** service.

Q12 **Answer: (b) to detect a foul (pressing too early)**

Pressing before the LED is a **foul**.

Q13 **Answer: (a) env/, __pycache__/, *.log and secrets files**

Generated files and **secrets**.

Q14 **Answer: (c) monotonic never jumps when the clock is adjusted**

Intervals need a **steady** clock.

Q15 **Answer: (a) falling, below about 1005 hPa**

Low and **falling** pressure.

Q16 **Answer: (b) 3**

Off, dim, full: **3**.

Q17 **Answer: (d) 356 ms and 385.7 ms**

min = 356; mean = 1157 ÷ 3 = **385.7 ms**.

Q18 **Answer: (c) about 1 ms**

Resolution ≈ the polling interval.

Q19 **Answer: Solution program**

random.choice picks a pin each step; seeding makes the run repeatable. The run log shows each LED flash.

```
from gpiozero import LED
from time import sleep
import random

random.seed(5)
leds = {17: LED(17), 27: LED(27), 22: LED(22)}
seq = [random.choice(list(leds)) for _ in range(4)]
print("sequence:", seq)
for pin in seq:
    leds[pin].on(); sleep(0.5)
    leds[pin].off(); sleep(0.25)
```

SIMULATED RUN

```
sequence: [22, 27, 22, 27]
[  0.00 s] LED(GPIO22) on
[  0.50 s] LED(GPIO22) off
[  0.75 s] LED(GPIO27) on
[  1.25 s] LED(GPIO27) off
[  1.50 s] LED(GPIO22) on
[  2.00 s] LED(GPIO22) off
[  2.25 s] LED(GPIO27) on
[  2.75 s] LED(GPIO27) off
```

Q20 **Answer: Solution program**

Two randint calls per press; a conditional expression adds the double message.

```
from gpiozero import Button
from signal import pause
import random

random.seed(11)
def roll():
    a, b = random.randint(1, 6), random.randint(1, 6)
    print(a, "+", b, "=", a + b, "DOUBLE!" if a == b else "")

Button(2).when_pressed = roll
pause()
```

OUTPUT

```
4 + 5 = 9
4 + 4 = 8 DOUBLE!
5 + 5 = 10 DOUBLE!
2 + 2 = 4 DOUBLE!
5 + 4 = 9
```

Q21 **Answer: Solution program**

The night window crosses midnight, so it is "after 22 OR before 6", not AND.

```
def is_night(hour):
    return hour >= 22 or hour < 6

for h in (21, 22, 3, 6, 12):
    print(f"{h:02d}:00 night? {is_night(h)}")
```

OUTPUT

```
21:00 night? False
22:00 night? True
03:00 night? True
06:00 night? False
12:00 night? False
```

Q22 **Answer: Commands**

git diff shows exactly what changed since the last commit; commit -a stages modified tracked files.

```
pi@raspberrypi:~ $ mkdir proj && cd proj && git init -q -b main && git config user.email a@b.c &&
git config user.name A && echo "LED_PIN = 17" > config.py && git add . && git commit -qm "start" &&
echo committed
committed
pi@raspberrypi:~/proj $ cd proj && echo "LED_PIN = 27" > config.py && git diff --stat && git diff |
tail -2
pi@raspberrypi:~/proj $ cd proj && git commit -qam "use GPIO27" && git log --oneline | wc -l
```

Q23 **Answer: Solution program**

Read lines, strip newlines and skip blank lines; the first and last list items are the earliest and latest entries.

```
log_lines = """2026-03-14 22:41:05
2026-03-15 01:12:44
2026-03-15 03:58:10
"""
with open("alarm.log", "w") as f:
    f.write(log_lines)

with open("alarm.log") as f:
    times = [line.strip() for line in f if line.strip()]
print("triggers:", len(times))
print("first:", times[0])
print("last: ", times[-1])
```

OUTPUT

```
triggers: 3
first: 2026-03-14 22:41:05
last:  2026-03-15 03:58:10
```

Q24 **Answer: (c) the redirected log file for errors (full paths, missing modules)**

Redirect output with >> log 2>&1 and read it.

Q25 **Answer: (d) git init → git add → git commit → git remote add origin … → git push -u origin main**

Create, stage, commit, connect, push.

Plan first: requirements → parts → circuit with pin numbers → flowchart → build and test one part at a time. Start programs at boot with cron @reboot (full paths, log output) or a systemd service (Restart=on-failure, journalctl). Document with a README, diagram and demo, and publish with git; never commit secrets.

LEVEL 2

Intermediate level

Use I2C, SPI and UART devices, displays, motors and relays, the camera and audio, build web dashboards, log data to databases, connect devices with MQTT, automate a home and administer Linux reliably

10 MODULES · 19 DAYS MINIMUM

Module 10 · Communication protocols: I2C, SPI and UART	2 days
Module 11 · Displays and user interfaces	2 days
Module 12 · Motors, relays and actuators	2 days
Module 13 · Camera and image capture	2 days
Module 14 · Audio and multimedia	1 day
Module 15 · Web servers and web dashboards	2 days
Module 16 · Databases and data logging	2 days
Module 17 · IoT with MQTT and cloud platforms	2 days
Module 18 · Home automation	2 days
Module 19 · Linux system administration for Raspberry Pi	2 days

MODULE 10 · INTERMEDIATE

Communication protocols: I2C, SPI and UART

Minimum duration **2 days** | Level **Intermediate**

Most advanced sensors, displays and modules talk over a few standard buses. This module explains I2C (addressing, i2cdetect, smbus2, BME280, MPU6050 and BH1750 sensors, address conflicts), SPI (the four lines, spidev and the MCP3008 at register level), UART serial (baud rate, enabling the port, pyserial, GPS NMEA sentences and talking to an Arduino or ESP32), 1-Wire, logic level shifting for 5 V devices, device tree overlays in config.txt, and the Adafruit Blinka libraries, followed by 25 solved questions and programs.

IN THIS MODULE

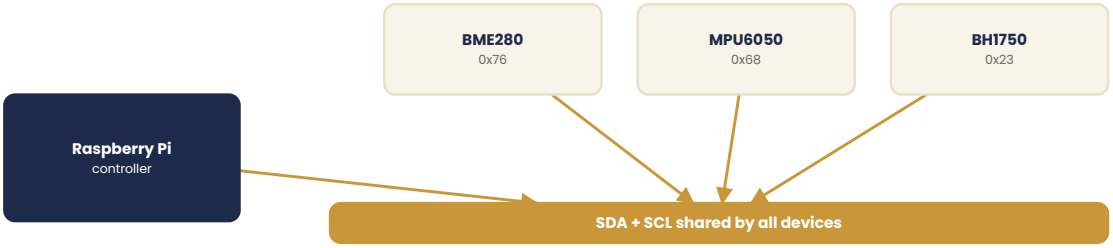
10.1	I2C	10.2	SPI
10.3	UART (serial)	10.4	1-Wire
10.5	Logic level shifting	10.6	Device tree overlays
10.7	Adafruit Blinka and CircuitPython libraries	10.8	Practice questions
10.9	Solutions		

	I2C	SPI	UART	1-Wire
Wires (plus GND)	2: SDA, SCL	4: MOSI, MISO, SCLK, CE	2: TX, RX (crossed)	1: DQ
Devices	up to ~112 by 7-bit address	one CE line per device	one device per port	many, by 64-bit ID
Typical speed	100 / 400 kHz	1–30 MHz	9600–115 200 baud	~16 kbit/s
Clock	shared (synchronous)	shared (synchronous)	none: both sides agree on baud rate	timing slots
Typical devices	BME280, MPU6050, OLED, PCA9685	MCP3008, TFT displays, SD cards	GPS, Arduino, Bluetooth modules	DS18B20

10.1 I2C

10.1.1 SDA, SCL and addressing

I2C (Inter-Integrated Circuit, said "I-squared-C") uses two shared lines: **SDA** (data, GPIO2, pin 3) and **SCL** (clock, GPIO3, pin 5). The Pi is the **controller** and starts every transfer; each **target** device has a 7-bit **address** (0x08–0x77), such as 0x76 for a BME280. Both lines need pull-up resistors; the Pi has 1.8 kΩ pull-ups on GPIO2/3 built in.



MANY DEVICES ON ONE I2C BUS, EACH WITH ITS OWN ADDRESS

A transfer: START → address + read/write bit → ACK from the device → register number → data bytes (each acknowledged) → STOP. Most sensors expose numbered **registers**: write to configure, read to get measurements.

10.1.2 Enabling I2C and i2cdetect

Enable with `sudo raspi-config nonint do_i2c 0` (adds `dtparam=i2c_arm=on`), install tools with `sudo apt install i2c-tools`, then scan bus 1:

SCANNING THE BUS (DEVICES FOUND AT 0X23, 0X68 AND 0X76)

```
pi@raspberrypi:~ $ i2cdetect -y 1
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:          -- -- -- -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- 23 -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- 68 -- -- -- -- -- -- --
70: -- -- -- -- -- -- 76 --
```

SHORTCUT TRICK

If i2cdetect shows nothing: check I2C is enabled (`ls /dev/i2c*`), swap SDA/SCL, check 3.3 V and GND, and look up the address in the datasheet; some modules have an address jumper (BME280 0x76/0x77, MPU6050 0x68/0x69).

10.1.3 smbus2 in Python

Method	Does
SMBus(1)	open bus 1 (/dev/i2c-1)
read_byte_data(addr, reg)	read one register
write_byte_data(addr, reg, value)	write one register
read_i2c_block_data(addr, reg, n)	read n consecutive registers
write_i2c_block_data(addr, reg, [..])	write several bytes

10.1.4 Sensors: BME280, MPU6050, BH1750

BME280 (0x76): temperature, humidity and pressure. Its raw values need calibration maths, so use a library (`pip install RPi.bme280`):

BME280_READ.PY

```
import smbus2, bme280

bus = smbus2.SMBus(1)
ADDR = 0x76
print("chip id:", hex(bus.read_byte_data(ADDR, 0xD0)))      # 0x60 = BME280
cal = bme280.load_calibration_params(bus, ADDR)
data = bme280.sample(bus, ADDR, cal)
print(f"{data.temperature:.2f} C  {data.humidity:.1f} %  {data.pressure:.2f} hPa")
```

OUTPUT

```
chip id: 0x60
23.48 C  52.3 %  1011.24 hPa
```

MPU6050 (0x68): 3-axis accelerometer and gyroscope. Wake it by writing 0 to register 0x6B, then read 14 bytes from 0x3B: accel X, Y, Z, temperature, gyro X, Y, Z, each a signed 16-bit big-endian number. At the default range, **16384 = 1 g** and **131 = 1 °/s**.

MPU6050.PY

```
from smbus2 import SMBus
import math

def s16(hi, lo):
    v = (hi << 8) | lo
    return v - 65536 if v >= 32768 else v            # two's complement

with SMBus(1) as bus:
    print("WHO_AM_I:", hex(bus.read_byte_data(0x68, 0x75)))
    bus.write_byte_data(0x68, 0x6B, 0)                # wake up
    d = bus.read_i2c_block_data(0x68, 0x3B, 14)
ax, ay, az = (s16(d[i], d[i + 1]) / 16384 for i in (0, 2, 4))
temp = s16(d[6], d[7]) / 340 + 36.53
gz = s16(d[12], d[13]) / 131
pitch = math.degrees(math.atan2(ax, math.sqrt(ay ** 2 + az ** 2)))
print(f"accel g: x={ax:.2f} y={ay:.2f} z={az:.2f}   temp {temp:.1f} C   gyro z {gz:.1f} deg/s")
print(f"tilt (pitch): {pitch:.1f} degrees")
```

OUTPUT

```
WHO_AM_I: 0x68
accel g: x=0.50 y=0.00 z=0.87   temp 27.4 C   gyro z -12.5 deg/s
tilt (pitch): 30.0 degrees
```

BH1750 (0x23): ambient light in lux. Send a measurement command (0x10 = continuous high-resolution), wait 180 ms, read 2 bytes: lux = raw ÷ 1.2.

BH1750.PY

```
from smbus2 import SMBus
import time

with SMBus(1) as bus:
    bus.write_byte(0x23, 0x10)
    time.sleep(0.18)
    hi, lo = bus.read_i2c_block_data(0x23, 0x10, 2)
raw = (hi << 8) | lo
print(f"raw {raw}  ->  {raw / 1.2:.1f} lux")
```

OUTPUT

```
raw 543  ->  452.5 lux
```

10.1.5 Multiple devices and address conflicts

Two identical sensors have the same address. Solutions: use the address jumper if the module has one (0x76 and 0x77), use a second I2C bus (10.6), or add a **TCA9548A multiplexer** (address 0x70) that switches between 8 separate sub-buses. A missing device raises `OSError: [Errno 121] Remote I/O error`:

PROBE.PY

```
from smbus2 import SMBus
with SMBus(1) as bus:
    for addr in (0x76, 0x77):
        try:
            bus.read_byte_data(addr, 0xD0)
            print(hex(addr), "present")
        except OSError as e:
            print(hex(addr), "not responding:", e)
```

OUTPUT

```
0x76 present
0x77 not responding: [Errno 121] Remote I/O error
```

10.2 SPI

10.2.1 MOSI, MISO, SCLK and CE lines

SPI (Serial Peripheral Interface) is a fast, full-duplex bus: **MOSI** (controller out, GPIO10), **MISO** (controller in, GPIO9), **SCLK** (clock, GPIO11) and one **chip-enable** line per device (CE0 = GPIO8, CE1 = GPIO7). The Pi pulls a device's CE line LOW to talk to it; every clock pulse sends one bit out and receives one bit in at the same time. Enable with `dtparam=spi=on`; the devices appear as `/dev/spidev0.0` and `/dev/spidev0.1`.

10.2.2 spidev in Python

Reading the MCP3008 at register level shows how SPI works: send 3 bytes and get 3 back. Byte 1 = start bit (1); byte 2 = single-ended mode + channel in the top 4 bits; byte 3 = don't care. The 10-bit result is in the last 2 bits of the second reply byte and the whole third byte.

THE THREE BYTES SENT TO THE MCP3008

MCP3008_SPIDEV.PY

```
import spidev

spi = spidev.SpiDev()
spi.open(0, 0)                  # bus 0, CE0
spi.max_speed_hz = 1_350_000

def read_adc(ch):
    r = spi.xfer2([1, (8 + ch) << 4, 0])
    return ((r[1] & 3) << 8) | r[2]

for ch in range(3):
    raw = read_adc(ch)
    print(f"CH{ch}: {raw:4d}  {raw / 1023 * 3.3:.2f} V")
spi.close()
```

OUTPUT

```
CH0:  256  0.83 V
CH1:  512  1.65 V
CH2: 1022  3.30 V
```

10.2.3 SPI ADCs and displays

Besides ADCs (MCP3008, MCP3208 12-bit, ADS1256 24-bit), SPI drives small TFT colour displays (ST7735, ILI9341), e-paper displays and the MAX7219 LED matrix, where speed matters: a 320 × 240 × 16-bit frame is 153 600 bytes, about 0.05 s at 24 MHz.

10.3 UART (serial)

10.3.1 TX, RX and baud rate

A **UART** sends bytes one bit at a time with no clock line: both ends must use the same **baud rate** (bits per second). Connect **TX of one side to RX of the other** and share GND. The common "8N1" format sends 10 bits per byte (1 start, 8 data, 1 stop), so 9600 baud carries 960 bytes per second.

TX GOES TO RX IN BOTH DIRECTIONS, PLUS A COMMON GND

10.3.2 Enabling the serial port and disabling the serial console

By default a Linux login console may use the GPIO serial port. In raspi-config → Interface Options → Serial Port answer **No** to "login shell over serial" and **Yes** to "serial port hardware" (adds `enable_uart=1`). Then use `/dev/serial0`, an alias that always points at the GPIO UART on any model. On the Pi 5 the separate 3-pin debug connector has its own UART (`/dev/ttyAMA10`). Your user must be in the **dialout** group.

10.3.3 pyserial

PYSERIAL BASICS (PIP INSTALL PYSERIAL, OR APT PYTHON3-SERIAL)

```
import serial
ser = serial.Serial("/dev/serial0", baudrate=9600, timeout=1)
ser.write(b"hello\n")              # send bytes
line = ser.readline()              # read up to \n (or timeout)
print(line.decode().strip())
ser.close()
```

10.3.4 GPS modules

GPS modules such as the NEO-6M send text **NMEA sentences** at 9600 baud once per second. `$GPGGA` gives time, position, fix quality, satellites and altitude. Latitude is in **ddmm.mmmm** format: degrees = dd + mm.mmmm ÷ 60. The two hex digits after * are a checksum: the XOR of every character between $ and *.

GPS.PY

```
import serial

def checksum_ok(s):
    body, _, cs = s[1:].partition("*")
    x = 0
    for ch in body:
        x ^= ord(ch)
    return f"{x:02X}" == cs

def to_deg(value, hemi, deg_digits):
    deg = int(value[:deg_digits]) + float(value[deg_digits:]) / 60
    return -deg if hemi in "SW" else deg

with serial.Serial("/dev/serial0", 9600, timeout=2) as gps:
    for _ in range(3):
        s = gps.readline().decode(errors="ignore").strip()
        if not s.startswith("$GPGGA"):
            continue
        if not checksum_ok(s):
            print("bad checksum"); continue
        f = s.split(",")
        lat, lon = to_deg(f[2], f[3], 2), to_deg(f[4], f[5], 3)
        print(f"time {f[1][:2]}:{f[1][2:4]}:{f[1][4:6]} UTC  lat {lat:.5f}  lon {lon:.5f}  sats
{int(f[7])}  alt {f[9]} m")
```

OUTPUT

```
time 09:30:15 UTC  lat 12.97100  lon 77.59405  sats 8  alt 920.4 m
time 09:30:16 UTC  lat 12.97101  lon 77.59406  sats 9  alt 920.6 m
```

RAW SENTENCES RECEIVED (BENGALURU)

```
$GPGGA,093015.00,1258.2602,N,07735.6431,E,1,08,0.9,920.4,M,-86.3,M,,*42
$GPRMC,093015.00,A,1258.2602,N,07735.6431,E,0.12,,150326,,,A*40
```

WORKED EXAMPLE

Converting NMEA latitude

1258.2602 N → 12° + 58.2602′ ÷ 60 = 12 + 0.97100 = **12.97100° N**. Longitude 07735.6431 E has three degree digits: 77 + 35.6431 ÷ 60 = **77.59405° E**.

10.3.5 Talking to an Arduino or ESP32 over serial and USB

Plugged in by USB, an Arduino appears as `/dev/ttyACM0` (or `/dev/ttyUSB0` with a USB-serial chip): no level shifting needed, and it is powered too. A simple text protocol of one command per line is robust. Note that opening the port **resets** most Arduinos, so wait about 2 s before sending.

PI_TO_ARDUINO.PY

```
import serial, time

with serial.Serial("/dev/ttyACM0", 115200, timeout=1) as ard:
    time.sleep(2)                                  # Arduino resets when the port opens
    for cmd in ("LED ON", "TEMP?", "LED OFF"):
        ard.write((cmd + "\n").encode())
        reply = ard.readline().decode().strip()
        print(f"sent {cmd!r:10} got {reply!r}")
```

OUTPUT

```
sent 'LED ON'   got 'OK LED ON'
sent 'TEMP?'    got 'T=26.5'
sent 'LED OFF'  got 'OK LED OFF'
```

WATCH OUT

A 5 V Arduino Uno's TX pin outputs 5 V. Never connect it straight to the Pi's RX (GPIO15): use USB, a level shifter, or a 1 kΩ/2 kΩ divider on that line. The Pi's 3.3 V TX is usually read correctly by a 5 V Arduino.

10.4 1-Wire

Covered in module 8 with the DS18B20: one data line with a 4.7 kΩ pull-up, enabled by `dtoverlay=w1-gpio` (default GPIO4; change with `dtoverlay=w1-gpio,gpiopin=17`). Each device has a factory-set 64-bit ROM ID, so dozens can share one wire, each appearing as /sys/bus/w1/devices/28-xxxxxxxxxxxx.

10.5 Logic level shifting

10.5.1 Connecting 5 V devices safely

Situation	Solution
5 V output → Pi input (one direction)	voltage divider: 1 kΩ + 2 kΩ gives 3.33 V

Situation	Solution
I2C between 3.3 V Pi and 5 V device (bidirectional)	MOSFET level shifter (BSS138 module)
Fast signals (SPI, NeoPixels)	74AHCT125 / 74HCT245 buffer (3.3 V in, 5 V out)
Pi output → 5 V input	often works directly (5 V CMOS reads ≥ 3.0–3.5 V as HIGH); check the datasheet or use 74AHCT

WORKED EXAMPLE

Divider check

With R1 = 10 kΩ (top) and R2 = 20 kΩ (bottom), V_{out} = 5 × 20 ÷ 30 = **3.33 V**; with 4.7 kΩ and 10 kΩ, 5 × 10 ÷ 14.7 = **3.40 V**: both safe. Smaller values give faster edges; larger values waste less current.

10.6 Device tree overlays

10.6.1 config.txt and dtoverlay

The kernel learns which hardware is attached from the **device tree**. `/boot/firmware/config.txt` switches features on with `dtparam` (parameters of the base tree) and `dtoverlay` (extra hardware descriptions). Edit with sudo nano and **reboot** to apply.

COMMON LINES

```
# /boot/firmware/config.txt (extract)
dtparam=i2c_arm=on                 # I2C bus 1 on GPIO2/3
dtparam=i2c_arm_baudrate=400000    # fast mode 400 kHz
dtparam=spi=on                     # SPI0
enable_uart=1                      # GPIO serial port
dtoverlay=w1-gpio,gpiopin=4        # 1-Wire on GPIO4
dtoverlay=pwm-2chan                # hardware PWM on GPIO18 and 19
dtoverlay=i2c-gpio,bus=3,i2c_gpio_sda=23,i2c_gpio_scl=24   # extra (software) I2C bus 3
```

10.6.2 Enabling extra I2C, SPI and UART buses

The SoC has more buses than the default pins expose. Overlays such as `i2c-gpio` (any two pins, appears as /dev/i2c-3), `spi1-1cs` (SPI1 on GPIO19–21) and `uart2` … `uart5` (Pi 4/5) add them. `dtoverlay -h name` or /boot/firmware/overlays/README lists every overlay and its parameters; `pinout` and `pinctrl` show which pins they use.

10.7 Adafruit Blinka and CircuitPython libraries

Blinka (`pip install adafruit-blinka` in a venv) lets the hundreds of Adafruit **CircuitPython** device libraries run on a Pi. You write the same code that runs on a microcontroller: `board` gives pin names, `busio` the buses, and each sensor has a driver class.

THE SAME BME280 THROUGH BLINKA

```
import board, busio
import adafruit_bme280.basic as adafruit_bme280   # pip install adafruit-circuitpython-bme280

i2c = busio.I2C(board.SCL, board.SDA)
sensor = adafruit_bme280.Adafruit_BME280_I2C(i2c, address=0x76)
print(sensor.temperature, sensor.humidity, sensor.pressure)
```

PRACTICE CHECKPOINT

Read a BME280 over I2C, an MCP3008 over SPI and a GPS module over UART in a single Python program, and print combined readings.

ALL_BUSES.PY (CHECKPOINT SOLUTION)

```
import smbus2, bme280, spidev, serial

bus = smbus2.SMBus(1); cal = bme280.load_calibration_params(bus, 0x76)
spi = spidev.SpiDev(); spi.open(0, 0); spi.max_speed_hz = 1_000_000
gps = serial.Serial("/dev/serial0", 9600, timeout=2)

def adc(ch):
    r = spi.xfer2([1, (8 + ch) << 4, 0]); return ((r[1] & 3) << 8) | r[2]

def gps_fix():
    for _ in range(5):
        f = gps.readline().decode(errors="ignore").strip().split(",")
        if f[0] == "$GPGGA" and f[6] != "0":
            lat = int(f[2][:2]) + float(f[2][2:]) / 60
            lon = int(f[4][:3]) + float(f[4][3:]) / 60
            return f"{lat:.4f},{lon:.4f}"
    return "no fix"

b = bme280.sample(bus, 0x76, cal)
print(f"BME280  {b.temperature:.1f} C  {b.humidity:.0f} %  {b.pressure:.1f} hPa")
print(f"MCP3008 soil {adc(0)}  light {adc(1)}")
print(f"GPS     {gps_fix()}")
```

OUTPUT

```
BME280  23.5 C  52 %  1011.2 hPa
MCP3008 soil 419  light 788
GPS     12.9710,77.5941
```

10.8 Practice questions

10.8.1 Basic level

Q1 Which two lines does I2C use?

(a) MOSI and MISO
(b) SDA and SCL
(c) TX and RX
(d) DQ and GND only

Q2 Which GPIO pins are I2C bus 1?

(a) GPIO2 (SDA) and GPIO3 (SCL)
(b) GPIO14 and GPIO15
(c) GPIO10 and GPIO11
(d) GPIO17 and GPIO27

Q3 Which command scans I2C bus 1 for devices?

(a) i2cscan 1
(b) i2cdetect -y 1
(c) lsusb
(d) spidev -l

Q4 How are UART TX and RX connected between two devices?

(a) TX to RX and RX to TX, plus common GND
(b) TX to TX and RX to RX
(c) only TX to TX
(d) TX to SDA

Q5 Which SPI line selects which device is being talked to?

(a) MISO
(b) SCLK
(c) CE (chip enable / chip select)
(d) SDA

Q6 Which Python package talks to serial ports?

(a) smbus2
(b) spidev
(c) gpiozero
(d) pyserial (import serial)

Q7 Which file holds dtparam and dtoverlay lines?

(a) /etc/fstab
(b) /etc/hostname
(c) ~/.profile
(d) /boot/firmware/config.txt

Q8 What address does a BME280 usually have?

(a) 0x68
(b) 0x23
(c) 0x76 (or 0x77)
(d) 0x3C

10.8.2 Intermediate level

Q9 How many bytes per second does 9600 baud 8N1 carry?

(a) 960
(b) 9600
(c) 1200
(d) 96

Q10 Two identical sensors have the same fixed I2C address. Which is a valid fix?

(a) connect them to the same address anyway
(b) use SPI CE lines on I2C
(c) swap SDA and SCL on one
(d) a TCA9548A I2C multiplexer

Q11 What does OSError [Errno 121] Remote I/O error mean with smbus2?

(a) the bus is too fast
(b) no device acknowledged at that address
(c) Python is outdated
(d) the file system is full

Q12 Which device path should a program use for the GPIO serial port on any Pi model?

(a) /dev/i2c-1 **(b)** /dev/serial0 **(c)** /dev/spidev0.0 **(d)** /dev/gpiochip0

Q13 An MPU6050 accelerometer register reads 8192 at the ±2 g range. What acceleration is that?

(a) 2 g **(b)** 1 g **(c)** 0.25 g **(d)** 0.5 g

Q14 A BH1750 returns bytes 0x01 and 0xE0. What is the light level?

(a) 400 lux **(b)** 480 lux **(c)** 576 lux **(d)** 120 lux

Q15 Which bidirectional solution suits I2C between the Pi and a 5 V device?

(a) a single resistor divider **(b)** connecting directly
(c) a MOSFET (BSS138) level shifter **(d)** a diode

Q16 Why must the serial login console be disabled before using the UART for a GPS?

(a) it uses too much power
(b) it changes the baud rate to 5 V
(c) it disables I2C
(d) otherwise Linux sends login prompts on it and reads the GPS data as typing

10.8.3 Advanced level

Q17 An MPU6050 register pair reads 0xFC 0x18. What is the signed value?

(a) 64536 **(b)** -1000 **(c)** 1000 **(d)** -64536

Q18 Convert the NMEA latitude 2236.5400,N to decimal degrees.

(a) 22.60900 **(b)** 22.36540 **(c)** 23.60900 **(d)** 22.54000

Q19 Convert the NMEA longitude 08822.8800,E to decimal degrees.

(a) 88.22880 **(b)** 82.28800 **(c)** 88.38133 **(d)** 88.48000

Q20 Compute the NMEA checksum (XOR of characters between $ and *) for "GPGLL,1258.26,N,07735.64,E,093015,A" and print the full sentence.

Q21 Scan addresses 0x03–0x77 on bus 1 with smbus2 (read_byte) and print the addresses that answer, like a mini i2cdetect.

Q22 Read MCP3008 channel 0 ten times with spidev and print the average raw value and voltage.

Q23 Read an MPU6050 and print "LEVEL" if the tilt (pitch) is within ±5°, otherwise "TILTED" with the angle.

Q24 Send "PING" to an ESP32 on /dev/ttyUSB0 at 115200 baud every second, three times, and print each reply or "timeout" if none arrives.

Q25 A project needs an OLED (I2C), an MCP3008 (SPI), a GPS (UART) and two DS18B20s. How many GPIO signal pins does that use at minimum?

(a) 9: SDA, SCL, MOSI, MISO, SCLK, CE0, TX, RX, 1-Wire
(b) 5
(c) 12
(d) 7

10.9 Solutions

Q1 **Answer: (b) SDA and SCL**

I2C: **SDA** (data) and **SCL** (clock).

Q2 **Answer: (a) GPIO2 (SDA) and GPIO3 (SCL)**

Physical pins 3 and 5.

Q3 **Answer: (b) i2cdetect -y 1**

`i2cdetect -y 1` from i2c-tools.

Q4 **Answer: (a) TX to RX and RX to TX, plus common GND**

Lines **cross over**.

Q5 **Answer: (c) CE (chip enable / chip select)**

One **CE** line per device.

Q6 **Answer: (d) pyserial (import serial)**

pyserial.

Q7 **Answer: (d) /boot/firmware/config.txt**

config.txt.

Q8 **Answer: (c) 0x76 (or 0x77)**

0x76/0x77; 0x68 MPU6050, 0x23 BH1750, 0x3C OLED.

Q9 **Answer: (a) 960**

10 bits per byte: 9600 ÷ 10 = **960**.

Q10 **Answer: (d) a TCA9548A I2C multiplexer**

The multiplexer puts them on separate sub-buses.

Q11 **Answer: (b) no device acknowledged at that address**

Wrong address, wiring or power.

Q12 **Answer: (b) /dev/serial0**

/dev/serial0 always points at the GPIO UART.

Q13 **Answer: (d) 0.5 g**

8192 ÷ 16384 = **0.5 g**.

Q14 **Answer: (a) 400 lux**

raw = 0x01E0 = 480; 480 ÷ 1.2 = **400 lux**.

Q15 **Answer: (c) a MOSFET (BSS138) level shifter**

I2C lines carry data in **both directions**.

Q16 **Answer: (d) otherwise Linux sends login prompts on it and reads the GPS data as typing**

The port must be free for your program.

Q17 **Answer: (b) -1000**

0xFC18 = 64536; ≥ 32768 so subtract 65536: **-1000**.

Q18 **Answer: (a) 22.60900**

22 + 36.54 ÷ 60 = 22 + 0.609 = **22.60900**.

Q19 **Answer: (c) 88.38133**

Three degree digits: 88 + 22.88 ÷ 60 = **88.38133**.

Q20 **Answer: Solution program**

XOR every character code, then format as two upper-case hex digits.

```
body = "GPGLL,1258.26,N,07735.64,E,093015,A"
cs = 0
for ch in body:
    cs ^= ord(ch)
print(f"${body}*{cs:02X}")
```

OUTPUT

```
$GPGLL,1258.26,N,07735.64,E,093015,A*2A
```

Q21 **Answer: Solution program**

Absent addresses raise OSError and are skipped. (Some devices dislike a plain read; i2cdetect chooses the safest probe for each address.)

```
from smbus2 import SMBus

found = []
with SMBus(1) as bus:
    for addr in range(0x03, 0x78):
        try:
            bus.read_byte(addr)
            found.append(hex(addr))
        except OSError:
            pass
print("devices:", found)
```

OUTPUT
```
devices: ['0x3c', '0x48', '0x76']
```

Q22 **Answer: Solution program**

Channel 0 → second byte (8 + 0) << 4 = 0x80. Averaging reduces noise on real hardware.

```
import spidev
spi = spidev.SpiDev(); spi.open(0, 0); spi.max_speed_hz = 1_000_000
vals = []
for _ in range(10):
    r = spi.xfer2([1, 0x80, 0])            # 0x80 = (8 + 0) << 4
    vals.append(((r[1] & 3) << 8) | r[2])
avg = sum(vals) / len(vals)
print(f"average {avg:.1f}  = {avg / 1023 * 3.3:.3f} V")
spi.close()
```

OUTPUT
```
average 634.0  = 2.045 V
```

Q23 **Answer: Solution program**

With gravity split between x and z the pitch is atan2(ax, √(ay² + az²)). Here ax = −0.26 g, az = 0.97 g → about −15°.

```
from smbus2 import SMBus
import math
def s16(h, l):
    v = (h << 8) | l
    return v - 65536 if v > 32767 else v
with SMBus(1) as bus:
    bus.write_byte_data(0x68, 0x6B, 0)
    d = bus.read_i2c_block_data(0x68, 0x3B, 6)
ax, ay, az = (s16(d[i], d[i + 1]) / 16384 for i in (0, 2, 4))
pitch = math.degrees(math.atan2(ax, math.hypot(ay, az)))
print("LEVEL" if abs(pitch) <= 5 else f"TILTED {pitch:.1f} deg")
```

OUTPUT
```
TILTED -15.1 deg
```

Q24 **Answer: Solution program**

readline() returns b"" when the timeout expires. In the simulation the ESP32 answered only the first ping.

```
import serial
with serial.Serial("/dev/ttyUSB0", 115200, timeout=1) as esp:
    for i in range(3):
        esp.write(b"PING\n")
        reply = esp.readline().decode().strip()
        print(i + 1, reply if reply else "timeout")
```

OUTPUT

```
1 PONG
2 timeout
3 timeout
```

Q25 **Answer: (a) 9: SDA, SCL, MOSI, MISO, SCLK, CE0, TX, RX, 1-Wire**

2 (I2C) + 4 (SPI) + 2 (UART) + 1 (1-Wire, shared by both sensors) = **9**.

I2C: SDA/SCL (GPIO2/3), 7-bit addresses, i2cdetect -y 1, smbus2 register reads. SPI: MOSI/MISO/SCLK + CE per device, spidev xfer2. UART: TX→RX crossed, same baud, /dev/serial0, console off, pyserial readline. 1-Wire: DS18B20. Shift 5 V signals down (divider or BSS138). Enable everything in config.txt with dtparam/dtoverlay and reboot.

MODULE 11 · INTERMEDIATE

Displays and user interfaces

Minimum duration **2 days** | Level **Intermediate**

Displays and interfaces let users see and control what the Pi is doing. This module drives a 16x2 LCD with an I2C backpack, an SSD1306 OLED, 7-segment and MAX7219 LED displays and e-paper, uses the official Touch Display and HDMI/SPI touchscreens with a browser in kiosk mode, powers and level-shifts NeoPixel strips, builds desktop GUIs with Tkinter and guizero, and reads keypads, rotary encoders and game controllers, followed by 25 solved questions and programs.

IN THIS MODULE

11.1 Small displays
11.2 Touchscreens
11.3 Addressable LEDs
11.4 Desktop GUIs in Python
11.5 Input devices
11.6 Practice questions
11.7 Solutions

11.1 Small displays

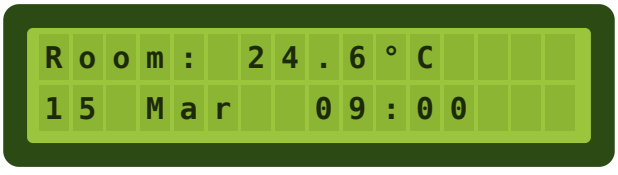
11.1.1 16x2 LCD with I2C backpack

The classic **16x2 character LCD** (HD44780 controller) shows 2 rows of 16 characters. On its own it needs 6+ GPIO pins; an **I2C backpack** (PCF8574 chip, address 0x27 or 0x3F) reduces that to SDA and SCL. Power it from 5 V; the backpack's I2C pull-ups go to 5 V, so for safety use a level shifter or a backpack with 3.3 V-compatible pull-ups. The blue potentiometer on the back sets the contrast: if you see only blocks or nothing, turn it.

LCD16X2.PY

```
from RPLCD.i2c import CharLCD
from datetime import datetime

lcd = CharLCD("PCF8574", 0x27, cols=16, rows=2)
lcd.clear()
lcd.write_string("Room: 24.6\xdfC")          # \xdf is the degree sign on the LCD
lcd.cursor_pos = (1, 0)                       # row 1, column 0
lcd.write_string(datetime.now().strftime("%d %b  %H:%M"))
```



WHAT THE LCD SHOWS (DRAWN FROM THE PROGRAM'S FINAL SCREEN BUFFER)

Install with `pip install RPLCD smbus2` in a venv. Strings longer than 16 characters wrap to the next row, so format them first: `f"{t:5.1f}"` keeps a fixed width so old digits are overwritten.

11.1.2 OLED SSD1306

The **SSD1306** 0.96" OLED has 128 × 64 pixels, each lit individually, so it can show graphics, icons and several font sizes. It runs on 3.3 V over I2C (usually address 0x3C). The **luma.oled** library (`pip install luma.oled`) draws with Pillow: everything you draw inside `with canvas(device)` appears when the block ends.

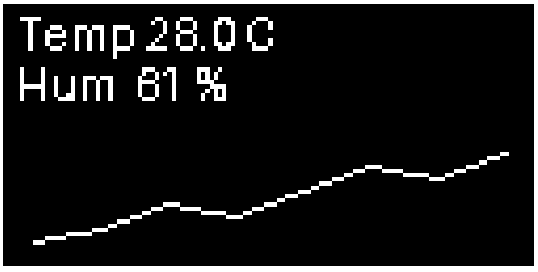
OLED_CHART.PY

```
from luma.core.interface.serial import i2c
from luma.oled.device import ssd1306
from luma.core.render import canvas
from PIL import ImageFont

device = ssd1306(i2c(port=1, address=0x3C))
font = ImageFont.load_default()
temps = [21, 22, 24, 23, 25, 27, 26, 28]
with canvas(device) as draw:
    draw.rectangle((0, 0, 127, 63), outline=1)
    draw.text((4, 2), "Temp 28.0 C", font=font, fill=1)
    draw.text((4, 14), "Hum  61 %", font=font, fill=1)
    for i in range(len(temps) - 1):                    # mini line chart
        x1, x2 = 8 + i * 16, 8 + (i + 1) * 16
        y1, y2 = 60 - (temps[i] - 20) * 3, 60 - (temps[i + 1] - 20) * 3
        draw.line((x1, y1, x2, y2), fill=1)
print("frame sent to OLED")
```

OUTPUT

```
frame sent to OLED
```



THE 128 × 64 FRAME PRODUCED BY THE PROGRAM (SCALED UP)

11.1.3 7-segment and MAX7219 LED matrix

A **7-segment** display shows digits with segments a–g. Driving a 4-digit display directly needs 12 pins and multiplexing, so use a driver: the **TM1637** (2 wires, own protocol) for clocks, or the **MAX7219** (SPI) which drives 8 digits or an **8 × 8 LED matrix**, and can be chained. luma.led_matrix supports MAX7219 matrices with scrolling text.

DIGIT 1 LIGHTS ONLY SEGMENTS B AND C

Digit	Segments on	Pattern gfedcba (binary)	Hex
0	a b c d e f	0111111	0x3F
1	b c	0000110	0x06
7	a b c	0000111	0x07
8	all seven	1111111	0x7F

11.1.4 E-paper displays

E-paper (e-ink) displays keep their image with **no power** and are readable in sunlight, but refresh slowly (1–15 s) and flicker on full refresh. They suit weather stations, price tags and battery dashboards that change every few minutes. Waveshare and Pimoroni Inky boards connect by SPI and come with Python libraries that draw from a Pillow image.

11.2 Touchscreens

11.2.1 Official Raspberry Pi Touch Display

The official **Touch Display 2** (7" 720 × 1280, and a 5" version) connects to the DSI connector with a flat cable and takes power from the GPIO 5 V and GND pins. It works with no drivers: touch acts like a mouse, and the desktop has an on-screen keyboard. The original 7" 800 × 480 display is still common.

11.2.2 HDMI and SPI touchscreens

Type	How it connects	Notes
HDMI touchscreen	HDMI for video, USB for touch	fast, plug and play, any size
SPI touchscreen (3.5")	GPIO header (SPI)	cheap but slow (≈ 20 fps), needs a driver overlay, blocks many GPIO pins
DSI display	flat cable to DSI	official option, frees HDMI and GPIO

11.2.3 Kiosk mode in the browser

A **kiosk** shows one full-screen web page (a dashboard) with no browser controls. Start Chromium at login, for example from a desktop autostart entry:

LAUNCH COMMAND (RASPBERRY PI OS: CHROMIUM OR CHROMIUM-BROWSER)

```
chromium-browser --kiosk --noerrdialogs --disable-infobars --incognito http://localhost:5000
```

Combine with auto-login (raspi-config → System → Boot / Auto Login) and screen-blanking off (raspi-config → Display → Screen Blanking). Module 15 builds the Flask page to show.

11.3 Addressable LEDs

11.3.1 WS2812/NeoPixel strips

Each **WS2812B** ("NeoPixel") LED contains a tiny controller: one data wire carries 24 bits (green, red, blue) per LED down the chain, so every LED can have its own colour. Timing is strict (800 kHz). On the Pi 4 the rpi_ws281x library uses PWM/DMA on GPIO18; on the Pi 5 use the SPI method (`adafruit-circuitpython-neopixel-spi`) or the Pi5Neo library on GPIO10 (MOSI).

RAINBOW.PY

```
import board, neopixel

N = 12
pixels = neopixel.NeoPixel(board.D18, N, brightness=0.5, auto_write=False)

def wheel(pos):                      # 0-255 -> a colour of the rainbow
    if pos < 85:
        return (255 - pos * 3, pos * 3, 0)
    if pos < 170:
        pos -= 85
        return (0, 255 - pos * 3, pos * 3)
    pos -= 170
    return (pos * 3, 0, 255 - pos * 3)

for i in range(N):
    pixels[i] = wheel(i * 256 // N)
pixels.show()
print("first three:", list(pixels)[:3])
```

OUTPUT

```
first three: [(255, 0, 0), (192, 63, 0), (129, 126, 0)]
```

THE 12 COLOURS THE PROGRAM SENT (DRAWN AT FULL BRIGHTNESS)

11.3.2 Power requirements and level shifting

WORKED EXAMPLE

Power budget for a strip

Each WS2812B draws up to **60 mA** at full white (20 mA per colour). 60 LEDs × 60 mA = **3.6 A** at 5 V = 18 W: far beyond the Pi. Use a separate 5 V supply rated about 20 % above (≈ 4.5 A), connect its GND to the Pi's GND, and inject power at both ends of long strips. Limiting brightness to 30 % in software cuts the worst case to about 1.1 A.

The strip expects a 5 V data signal; the Pi gives 3.3 V. It often works with short wires, but a **74AHCT125** level shifter makes it reliable. Add a 300–500 Ω resistor in the data line and a 1000 µF capacitor across the strip's power input.

WATCH OUT
Never power more than a few NeoPixels from the Pi's 5 V pin. A full-white strip can draw several amps, causing undervoltage resets and damaging traces. Always share GND between the LED supply and the Pi.

11.4 Desktop GUIs in Python

11.4.1 Tkinter

Tkinter is Python's built-in GUI toolkit. A window is a tree of widgets (labels, buttons, sliders); `mainloop()` waits for events. Use `after(ms, func)` instead of sleep() to update periodically, or the window freezes.

PANEL_TK.PY (NEEDS THE DESKTOP)

```
import tkinter as tk
from gpiozero import LED, CPUTemperature

led = LED(17); cpu = CPUTemperature()
root = tk.Tk(); root.title("Control panel")
temp_label = tk.Label(root, text="--", font=("Arial", 28))
temp_label.pack(padx=20, pady=10)
tk.Button(root, text="LED on/off", font=("Arial", 18), command=led.toggle).pack(pady=10)

def refresh():
    temp_label.config(text=f"{cpu.temperature:.1f} °C")
    root.after(1000, refresh)          # call again in 1 s, never block

refresh()
root.mainloop()
```

11.4.2 guizero

guizero (pre-installed, by the Raspberry Pi Foundation) wraps Tkinter in a much simpler API designed for beginners and touchscreens:

PANEL_GUIZERO.PY (NEEDS THE DESKTOP)

```
from guizero import App, Text, PushButton
from gpiozero import LED, CPUTemperature

led = LED(17); cpu = CPUTemperature()
app = App(title="Control panel", width=320, height=200)
temp = Text(app, text="--", size=30)
PushButton(app, text="LED on/off", command=led.toggle, width=12)
def refresh():
    temp.value = f"{cpu.temperature:.1f} C"
temp.repeat(1000, refresh)             # every 1000 ms
app.display()
```

11.4.3 Introduction to PyQt or Kivy

PyQt/PySide (Qt) gives professional-looking, themeable interfaces with a visual designer (Qt Designer). **Kivy** is built for multi-touch and runs well full-screen on touch displays without a desktop. For many

projects a **web page** in kiosk mode (modules 11.2.3 and 15) is the easiest GUI of all, and it also works from phones.

11.5 Input devices

11.5.1 Keypads and rotary encoders

A **4 × 4 matrix keypad** has 8 wires: 4 rows and 4 columns. Scan it by driving one row LOW at a time and reading which column goes LOW. A **rotary encoder** outputs two square waves (A and B) 90° apart: the order in which they change gives the direction. gpiozero has `RotaryEncoder`:

CHANNEL A

CHANNEL B

ENCODER_DIMMER.PY

```
from gpiozero import RotaryEncoder, PWMLED
from signal import pause

enc = RotaryEncoder(a=21, b=20, max_steps=10)
led = PWMLED(18)

def changed():
    led.value = (enc.steps + 10) / 20        # steps -10..10 -> 0..1
    print(f"steps {enc.steps:+d}  brightness {led.value:.0%}")

enc.when_rotated = changed
pause()
```

OUTPUT

```
steps +1  brightness 55%
steps +2  brightness 60%
steps +3  brightness 65%
steps +2  brightness 60%
```

When A rises while B is LOW the knob moved clockwise (+1); when B is already HIGH it moved anticlockwise (-1). The simulation turned three clicks clockwise, then one back.

KEYPAD_LOGIC.PY

```
KEYS = [["1", "2", "3", "A"],
        ["4", "5", "6", "B"],
        ["7", "8", "9", "C"],
        ["*", "0", "#", "D"]]

def scan(pressed):                # pressed = set of (row, col) closed switches
    for r in range(4):            # drive row r LOW ...
        for c in range(4):        # ... and see which column reads LOW
            if (r, c) in pressed:
                return KEYS[r][c]
    return None

for test in [{(0, 1)}, {(2, 2)}, {(3, 0)}, set()]:
    print(test or "{}", "->", scan(test))
```

OUTPUT

```
{(0, 1)} -> 2
{(2, 2)} -> 9
{(3, 0)} -> *
{} -> None
```

11.5.2 USB game controllers

USB and Bluetooth gamepads appear as input devices (`/dev/input/event*`). The **evdev** library reads their buttons and sticks; pygame's joystick module is an alternative. Great for driving a robot (module 25).

GAMEPAD.PY (REFERENCE)

```
from evdev import InputDevice, ecodes        # pip install evdev
pad = InputDevice("/dev/input/event0")
for e in pad.read_loop():
    if e.type == ecodes.EV_KEY and e.value == 1:
        print("button", e.code)
    elif e.type == ecodes.EV_ABS and e.code == ecodes.ABS_Y:
        print("left stick Y", e.value)
```

PRACTICE CHECKPOINT

Build a touchscreen control panel that shows live sensor readings and has buttons to switch LEDs and a relay.

11.6 Practice questions

11.6.1 Basic level

Q1 How many I2C wires does a 16x2 LCD with a PCF8574 backpack need (besides power)?

(a) 6 **(b)** 2 **(c)** 4 **(d)** 8

Q2 What is the usual I2C address of an SSD1306 OLED?

(a) 0x27 **(b)** 0x76 **(c)** 0x68 **(d)** 0x3C

Q3 What is the resolution of a common 0.96" SSD1306?

(a) 16 × 2 **(b)** 320 × 240 **(c)** 128 × 64 **(d)** 800 × 480

Q4 Which display keeps its image with no power?

(a) OLED **(b)** LCD with backlight
(c) e-paper **(d)** NeoPixel

Q5 What does the potentiometer on an LCD backpack adjust?

(a) I2C address **(b)** brightness of the Pi
(c) baud rate **(d)** contrast

Q6 Which chip drives an 8 × 8 LED matrix over SPI?

(a) PCF8574 **(b)** MAX7219 **(c)** MCP3008 **(d)** BME280

Q7 What does Chromium's --kiosk option do?

(a) shows one page full screen without browser controls
(b) blocks the internet
(c) opens a terminal
(d) enables touch

Q8 Which GUI library is pre-installed and designed for beginners?

(a) PyQt6 **(b)** guizero **(c)** Kivy **(d)** Electron

11.6.2 Intermediate level

Q9 How much current can one WS2812B draw at full white?

(a) about 60 mA **(b)** about 6 mA **(c)** about 600 mA **(d)** about 1 mA

Q10 Why use after() instead of sleep() in a Tkinter program?

(a) after() is more accurate
(b) sleep() is not allowed in Python
(c) sleep() blocks mainloop so the window freezes
(d) after() uses less power

Q11 Which hex pattern shows digit 7 on a 7-segment display (bits gfedcba)?

(a) 0x3F **(b)** 0x7F **(c)** 0x06 **(d)** 0x07

Q12 A rotary encoder's channel A rises while B is LOW. Which way did it turn (with the convention above)?

(a) anticlockwise (b) no movement
(c) it pressed the button (d) clockwise

Q13 A 4 × 4 keypad uses how many GPIO pins?

(a) 16 (b) 4 (c) 8 (d) 2

Q14 Why should a long NeoPixel strip have its own 5 V supply with a shared GND?

(a) it can draw amps, far more than the Pi can supply, and data needs a common reference
(b) it needs 12 V
(c) the Pi cannot send data otherwise
(d) to make colours brighter

Q15 Which interface does the official Touch Display use for video?

(a) HDMI (b) SPI (c) USB (d) DSI

Q16 Why format LCD numbers as f"{t:5.1f}"?

(a) the LCD needs floats
(b) fixed width overwrites old characters so no stray digits remain
(c) it makes text scroll
(d) it saves power

Q17 A strip has 144 LEDs at full white. What current could it draw?

(a) 864 mA (b) 8.64 A (c) 2.88 A (d) 14.4 A

Q18 The same 144-LED strip is limited to 25 % brightness in software. What is the worst-case current?

(a) 8.64 A (b) 0.54 A (c) 4.32 A (d) 2.16 A

11.6.3 Advanced level

Q19 Write a function seg(d) that returns the 7-segment hex pattern for digits 0–9 and print the patterns for 2026.

Q20 Show a two-line status on a 16x2 LCD at 0x27: line 1 "CPU xx.x°C", line 2 "OK" or "HOT" (above 70 °C).

Q21 Light a 10-LED NeoPixel bar as a level meter: for a value 0–100, light that proportion of LEDs green, the rest off. Test with 70.

Q22 Calculate the supply needed for 60 NeoPixels at 40 % maximum brightness with a 20 % safety margin, and print the current and power.

Q23 Use a rotary encoder (A = 21, B = 20, max_steps 5) to select a menu item from ["Temp", "Hum", "Light", "Motion"], printing the current item at each click.

Q24 Which Python library reads buttons and sticks of a USB gamepad from /dev/input/event*?

(a) evdev (b) smbus2 (c) spidev (d) RPLCD

Q25 A kiosk dashboard must start automatically, never blank and show a local Flask page. Which set of steps is right?

(a) VNC + SSH
(b) cron @reboot running Tkinter as root without a desktop
(c) auto-login + screen blanking off + Chromium --kiosk http://localhost:5000 at login

(d) e-paper refresh every second

11.7 Solutions

Q1 **Answer: (b) 2**

SDA and SCL.

Q2 **Answer: (d) 0x3C**

0x3C (sometimes 0x3D).

Q3 **Answer: (c) 128 × 64**

128 × 64 pixels.

Q4 **Answer: (c) e-paper**

E-paper.

Q5 **Answer: (d) contrast**

Contrast.

Q6 **Answer: (b) MAX7219**

MAX7219.

Q7 **Answer: (a) shows one page full screen without browser controls**

Kiosk = full-screen single page.

Q8 **Answer: (b) guizero**

guizero.

Q9 **Answer: (a) about 60 mA**

20 mA per colour × 3.

Q10 **Answer: (c) sleep() blocks mainloop so the window freezes**

Never block the GUI **event loop**.

Q11 **Answer: (d) 0x07**

Segments a, b, c → 0000111 = **0x07**.

Q12 **Answer: (d) clockwise**

B LOW at A's rising edge = clockwise.

Q13 **Answer: (c) 8**

4 rows + 4 columns.

Q14 **Answer: (a) it can draw amps, far more than the Pi can supply, and data needs a common reference**

Power separately; **share GND**.

Q15 **Answer: (d) DSI**

The DSI ribbon connector.

Q16 **Answer: (b) fixed width overwrites old characters so no stray digits remain**

5 characters every time.

Q17 **Answer: (b) 8.64 A**

144 × 0.06 = **8.64 A**.

Q18 **Answer: (d) 2.16 A**

8.64 × 0.25 = **2.16 A**.

Q19 **Answer: Solution program**

A lookup table indexed by the digit is the standard approach (bit 0 = segment a … bit 6 = g).

```
PATTERNS = [0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F]
def seg(d):
    return PATTERNS[d]
print([hex(seg(int(ch))) for ch in "2026"])
```

OUTPUT

```
['0x5b', '0x3f', '0x5b', '0x7d']
```

Q20 **Answer: Solution program**

The program prints the LCD's screen buffer so you can see the result.

```
from RPLCD.i2c import CharLCD
from gpiozero import CPUTemperature

lcd = CharLCD("PCF8574", 0x27, cols=16, rows=2)
t = CPUTemperature().temperature
lcd.clear()
lcd.write_string(f"CPU {t:4.1f}\xdfC")
lcd.cursor_pos = (1, 0)
lcd.write_string("HOT" if t > 70 else "OK")
print(open("lcd.txt").read().replace("\xdf", "°"))
```

OUTPUT

```
CPU 72.4°C
HOT
```

Q21 **Answer: Solution program**

lit = round(70 ÷ 100 × 10) = 7. auto_write=False with one show() updates the strip in a single transfer.

```
import board, neopixel
N = 10
px = neopixel.NeoPixel(board.D18, N, auto_write=False)
def meter(value):
    lit = round(value / 100 * N)
    for i in range(N):
        px[i] = (0, 200, 0) if i < lit else (0, 0, 0)
    px.show()
    return lit
print("LEDs lit:", meter(70))
print(open("strip.txt").read())
```

OUTPUT

```
LEDs lit: 7
0,200,0;0,200,0;0,200,0;0,200,0;0,200,0;0,200,0;0,200,0;0,200,0;0,200,0;0,0,0;0,0,0;0,0,0;0,0,0
```

Q22 **Answer: Solution program**

60 × 0.06 × 0.4 = 1.44 A; with 20 % margin 1.73 A, so a 5 V 2 A supply is enough.

```
leds, per_led, brightness, margin = 60, 0.060, 0.40, 1.2
current = leds * per_led * brightness * margin
print(f"supply: {current:.2f} A  ({current * 5:.1f} W at 5 V)")
```

OUTPUT

```
supply: 1.73 A  (8.6 W at 5 V)
```

Q23 **Answer: Solution program**

max_steps=0 means unlimited counting; steps % 4 wraps the menu round. Five clockwise clicks: Hum, Light, Motion, Temp, Hum.

```
from gpiozero import RotaryEncoder
from signal import pause
MENU = ["Temp", "Hum", "Light", "Motion"]
enc = RotaryEncoder(21, 20, max_steps=0)
def show():
    print("selected:", MENU[enc.steps % len(MENU)])
enc.when_rotated = show
pause()
```

OUTPUT

```
selected: Hum
selected: Light
```

```
selected: Motion
selected: Temp
selected: Hum
```

Q24 **Answer: (a) evdev**

evdev (or pygame's joystick module).

Q25 **Answer: (c) auto-login + screen blanking off + Chromium --kiosk http://localhost:5000 at login**

Auto-login, no blanking, kiosk browser.

16x2 LCD: I2C backpack 0x27, RPLCD, contrast pot. OLED SSD1306: 0x3C, 128×64, luma.oled + Pillow canvas. MAX7219 for LED matrices, e-paper for low-power static screens. NeoPixels: 60 mA each at white, separate 5 V supply, shared GND, level shifter. GUIs: guizero/Tkinter with after(), or a web page in kiosk mode. Encoders: A/B phase gives direction.

MODULE 12 · INTERMEDIATE

Motors, relays and actuators

Minimum duration **2 days** | Level **Intermediate**

This module covers making things move and switching real loads safely: why GPIO cannot drive motors, transistors and MOSFETs, flyback diodes and separate supplies with a common ground; DC motors with L298N and TB6612FNG drivers and gpiozero Motor and Robot; servos with PWM and the PCA9685 16-channel driver; steppers (28BYJ-48 with ULN2003, NEMA 17 with A4988/DRV8825); relays, optocouplers, mains safety and solid-state relays; and motor HATs, followed by 25 solved questions and programs.

IN THIS MODULE

12.1 Driving loads safely
12.2 DC motors
12.3 Servo motors
12.4 Stepper motors
12.5 Relays
12.6 Motor HATs
12.7 Practice questions
12.8 Solutions

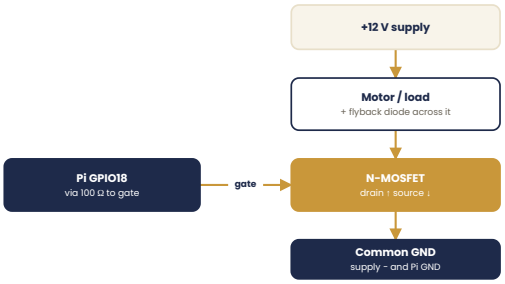
12.1 Driving loads safely

12.1.1 Why GPIO cannot drive motors directly

A GPIO pin gives 3.3 V at a few milliamps. A small hobby motor needs 5–12 V and 100 mA to several amps, with a **stall current** 5–10 times its running current. Motors also generate voltage spikes and electrical noise. Connecting one directly would at best not turn and at worst destroy the pin. The GPIO pin only gives the **signal**; a transistor, MOSFET or driver chip switches the **power**.

12.1.2 Transistors and MOSFETs

An **NPN transistor** (2N2222, BC337) used as a switch: a small base current (through a 1 kΩ resistor from the GPIO) lets a much larger collector current flow, up to about 500–800 mA. A logic-level **N-channel MOSFET** (IRLZ44N, AO3400) is switched by gate *voltage*, wastes very little power and handles several amps; it must be "logic level", fully on at 3.3 V gate voltage. Both are placed on the **low side**: between the load and GND.



LOW-SIDE MOSFET SWITCH: THE PI CONTROLS THE GATE; THE 12 V SUPPLY POWERS THE LOAD

WORKED EXAMPLE

Transistor base resistor

Load 200 mA, transistor gain $h_{FE} \approx 100$ → base current needed 2 mA; use 5× more to saturate it: 10 mA would overload the pin, so aim for about 4 mA: R = (3.3 − 0.7) ÷ 0.004 = **650 Ω** → 680 Ω. For bigger loads use a MOSFET, which needs almost no gate current.

12.1.3 Flyback diodes

A motor, relay coil or solenoid is an **inductor**. When the current is switched off, the collapsing magnetic field produces a large reverse voltage spike (tens to hundreds of volts) that destroys transistors. A **flyback diode** (1N4007, or a fast 1N5819) across the load, with its stripe (cathode) towards +V, gives that current a safe path. Driver boards and relay modules include them.

12.1.4 Separate power supplies and common ground

Power motors from their own supply (battery pack or adapter), not from the Pi's 5 V pin: motor current surges cause undervoltage resets. Always connect the motor supply's **GND to the Pi's GND**, otherwise the control signals have no reference and nothing works (or works randomly).

WATCH OUT

The most common motor project mistakes: no common ground; motors on the Pi's 5 V pin; no flyback diode; a driver whose logic inputs need 5 V. Check all four before powering up.

12.2 DC motors

12.2.1 L298N and TB6612FNG drivers

To run a DC motor in **both directions** you need an **H-bridge**: four switches that can connect each motor terminal to + or −. Driver chips contain two H-bridges, so each runs two motors.

	L298N module	TB6612FNG
Motor voltage	5–35 V	4.5–13.5 V
Current per motor	2 A (gets hot)	1.2 A (3.2 A peak)
Voltage drop	about 2 V (bipolar transistors)	about 0.5 V (MOSFETs)
Logic	IN1, IN2 (direction), ENA (PWM speed)	AIN1, AIN2, PWMA, STBY
Best for	large cheap robots, 12 V motors	small battery robots: efficient, small

IN1	IN2	Motor
1	0	forward
0	1	backward
0	0	coast (free-wheel)
1	1	brake

12.2.2 Speed and direction control with gpiozero Motor and Robot

`Motor(forward, backward)` drives the two direction pins with PWM: speed 0–1. `Robot(left=(a, b), right=(c, d))` combines two motors for a two-wheel-drive chassis.

MOTOR.PY

```
from gpiozero import Motor
from time import sleep

m = Motor(forward=17, backward=27)
m.forward(0.5); sleep(2)      # half speed forward
m.forward(1.0); sleep(2)      # full speed
m.stop();       sleep(0.5)    # always stop before reversing
m.backward(0.7); sleep(2)
m.stop()
```

SIMULATED RUN (hardware events and printed output)

```
[  0.00 s] Motor(GPIO17,27) forward 0.50
[  2.00 s] Motor(GPIO17,27) forward 1.00
[  4.00 s] Motor(GPIO17,27) stop
[  4.50 s] Motor(GPIO17,27) backward 0.70
[  6.50 s] Motor(GPIO17,27) stop
```

SQUARE.PY

```
from gpiozero import Robot
from time import sleep

bot = Robot(left=(7, 8), right=(9, 10))
for _ in range(4):            # drive a square
    bot.forward(0.6); sleep(1.5)
    bot.right(0.5);   sleep(0.45)   # tuned so that this is about 90 degrees
bot.stop()
```

SIMULATED RUN (hardware events and printed output)

```
[  0.00 s] Robot forward 0.60
[  1.50 s] Robot turn right 0.50
[  1.95 s] Robot forward 0.60
[  3.45 s] Robot turn right 0.50
[  3.90 s] Robot forward 0.60
[  5.40 s] Robot turn right 0.50
[  5.85 s] Robot forward 0.60
[  7.35 s] Robot turn right 0.50
[  7.80 s] Robot stop
```

SHORTCUT TRICK

Robots drift because motors are never identical. Tune with `bot.forward(0.6, curve_right=0.05)` or, properly, with wheel encoders and a PID loop (module 25). Timed turns depend on battery charge.

12.3 Servo motors

12.3.1 Servo control with PWM

A hobby **servo** (SG90, MG996R) turns to an **angle** (usually 0–180°) set by the width of a pulse repeated every **20 ms** (50 Hz): about **1.0 ms = 0°**, **1.5 ms = 90°** (centre), **2.0 ms = 180°**. Exact limits vary (often 0.5–2.5 ms), so calibrate. Power servos from a separate 5–6 V supply: an SG90 draws up to ~700 mA when stalled.

SERVO CONTROL SIGNAL

pulse width (ms) = 1.0 + angle ÷ 180 (for a 1–2 ms servo); duty cycle = pulse ÷ 20 ms

SERVO.PY

```
from gpiozero import AngularServo
from time import sleep

servo = AngularServo(18, min_angle=0, max_angle=180,
                     min_pulse_width=0.5 / 1000, max_pulse_width=2.5 / 1000)
for angle in (0, 45, 90, 135, 180, 90):
    servo.angle = angle
    sleep(0.6)
servo.detach()               # stop sending pulses: no jitter, less power
```

SIMULATED RUN (hardware events and printed output)

```
[  0.60 s] AngularServo(GPIO18) angle 45° -> pulse 1.00 ms
[  1.20 s] AngularServo(GPIO18) angle 90° -> pulse 1.50 ms
[  1.80 s] AngularServo(GPIO18) angle 135° -> pulse 2.00 ms
[  2.40 s] AngularServo(GPIO18) angle 180° -> pulse 2.50 ms
[  3.00 s] AngularServo(GPIO18) angle 90° -> pulse 1.50 ms
[  3.60 s] AngularServo(GPIO18) detached
```

Software PWM can make servos **jitter**. Use a hardware PWM pin (GPIO12/13/18/19) with gpiozero's pigpio or lgpio factories, or better, a dedicated servo driver.

12.3.2 PCA9685 16-channel servo driver

The **PCA9685** board (I2C address 0x40) generates 16 independent, rock-steady PWM signals in hardware, and boards can be chained for up to 992 channels. Servos plug straight onto its 3-pin headers, powered by the board's V+ terminal. Use Adafruit's ServoKit (`pip install adafruit-circuitpython-servokit`):

PANTILT_PCA9685.PY

```
from adafruit_servokit import ServoKit
import time

kit = ServoKit(channels=16)
pan, tilt = kit.servo[0], kit.servo[1]
for p_, t_ in [(90, 90), (30, 60), (150, 60), (90, 120)]:
    pan.angle, tilt.angle = p_, t_
    time.sleep(0.5)
```

SIMULATED RUN (hardware events and printed output)

```
[  0.00 s] PCA9685 ch0 angle 90 deg -> pulse 1500 us
[  0.00 s] PCA9685 ch1 angle 90 deg -> pulse 1500 us
[  0.50 s] PCA9685 ch0 angle 30 deg -> pulse 1167 us
[  0.50 s] PCA9685 ch1 angle 60 deg -> pulse 1333 us
[  1.00 s] PCA9685 ch0 angle 150 deg -> pulse 1833 us
[  1.00 s] PCA9685 ch1 angle 60 deg -> pulse 1333 us
[  1.50 s] PCA9685 ch0 angle 90 deg -> pulse 1500 us
[  1.50 s] PCA9685 ch1 angle 120 deg -> pulse 1667 us
```

12.4 Stepper motors

A **stepper motor** moves in exact steps when its coils are energised in sequence, so it can position precisely without a sensor (open loop): 3D printers, CNC machines, camera sliders.

12.4.1 28BYJ-48 with ULN2003

The cheap **28BYJ-48** (5 V) has an internal gearbox: **4096 half-steps** (2048 full steps) per output revolution. The ULN2003 board contains Darlington drivers and LEDs; four GPIO pins drive IN1–IN4 in an 8-step half-step sequence:

STEPPER_28BYJ.PY

```
from gpiozero import OutputDevice
from time import sleep

pins = [OutputDevice(p) for p in (5, 6, 13, 19)]         # IN1-IN4
SEQ = [(1,0,0,0), (1,1,0,0), (0,1,0,0), (0,1,1,0),
       (0,0,1,0), (0,0,1,1), (0,0,0,1), (1,0,0,1)]        # half-step
STEPS_PER_REV = 4096

def rotate(degrees, delay=0.001):
    steps = int(STEPS_PER_REV * abs(degrees) / 360)
    order = SEQ if degrees > 0 else SEQ[::-1]
    for i in range(steps):
        for pin, v in zip(pins, order[i % 8]):
            pin.value = v
        sleep(delay)
    for pin in pins:
        pin.off()                                        # de-energise: stays cool
    return steps

n = rotate(90)
print(f"90 degrees = {n} half-steps, took about {n * 0.001:.1f} s")
n = rotate(-45)
print(f"-45 degrees = {n} half-steps")
```

OUTPUT

```
90 degrees = 1024 half-steps, took about 1.0 s
-45 degrees = 512 half-steps
```

12.4.2 NEMA 17 with A4988 or DRV8825

A **NEMA 17** (1.8° per step = 200 steps/rev, 12–24 V, 1–2 A) needs a **current-limiting driver** like the A4988 or DRV8825. The Pi only sends two signals: **DIR** (direction) and **STEP** (one pulse = one step). **Microstepping** (MS1–MS3 pins) divides each step into 2, 4, 8, 16 (A4988) or 32 (DRV8825) for smoother motion. Set the driver's current limit with its potentiometer before use.

WORKED EXAMPLE

Steps for a linear move

Leadscrew 8 mm per revolution, 200 steps/rev, 16× microstepping: 3200 microsteps per 8 mm = **400 microsteps per mm**. Moving 25 mm needs 25 × 400 = **10 000 pulses**.

NEMA17.PY

```
from gpiozero import OutputDevice
from time import sleep

step, direction = OutputDevice(20), OutputDevice(21)

def move(steps, clockwise=True, rpm=60, steps_per_rev=200):
    direction.value = clockwise
    half = 60 / (rpm * steps_per_rev) / 2           # half period per step
    for _ in range(steps):
        step.on(); sleep(half); step.off(); sleep(half)
    return steps * 2 * half

t = move(200, rpm=60)
print(f"one revolution at 60 rpm took {t:.2f} s")
t = move(100, clockwise=False, rpm=120)
print(f"half a revolution back at 120 rpm took {t:.2f} s")
```

OUTPUT

```
one revolution at 60 rpm took 1.00 s
half a revolution back at 120 rpm took 0.25 s
```

12.5 Relays

12.5.1 Relay modules and optocouplers

A **relay** is an electrically operated switch: a small coil current pulls a contact closed, completely isolating the control side from the load side. Relay **modules** include a transistor, flyback diode and LED, and often an **optocoupler** (an LED and phototransistor in one package) for extra isolation. Contacts: **COM** (common), **NO** (normally open: connected only when energised) and **NC** (normally closed).

Many modules are **active-low** (the relay switches on when IN is pulled LOW) and need 5 V for the coil. Use `OutputDevice(pin, active_high=False, initial_value=False)` so "on" means energised and the relay stays off at start-up.

RELAY_SCHEDULE.PY

```
from gpiozero import OutputDevice
from datetime import datetime
import time

lamp = OutputDevice(26, active_high=False, initial_value=False)   # active-low module
SCHEDULE = [(9, 0, 5, True), (9, 0, 15, False), (9, 0, 25, True)]  # (h, m, s, on?)

done = set()
while len(done) < len(SCHEDULE):
    now = datetime.now()
    for i, (h, m, s, state) in enumerate(SCHEDULE):
        if i not in done and (now.hour, now.minute, now.second) >= (h, m, s):
            lamp.value = state
            print(now.strftime("%H:%M:%S"), "lamp", "ON" if state else "OFF")
            done.add(i)
    time.sleep(1)
```

OUTPUT

```
09:00:05 lamp ON
09:00:15 lamp OFF
09:00:25 lamp ON
```

12.5.2 Mains voltage safety rules

WATCH OUT

Mains electricity (230 V in India) can kill. Do not work on mains wiring unless you are qualified. Never touch the screw terminals of a powered relay board. Use enclosed, certified smart plugs or an enclosed relay box for mains loads, keep low-voltage and mains wiring physically separated, use fuses, and unplug before any change. For learning, switch 12 V LED strips or small DC loads instead.

12.5.3 Solid-state relays

A **solid-state relay** (SSR) switches with semiconductors (a triac for AC) driven through an internal optocoupler: silent, no contact wear, very fast, and the input often works from 3–32 V directly. Downsides: a small leakage current when "off", heat at high current (needs a heatsink), and AC SSRs cannot switch DC loads. Always rate an SSR well above the load current.

	Mechanical relay	Solid-state relay
Switching	moving contact	semiconductor
Noise	clicks	silent
Life	~100 000 operations	practically unlimited
Off state	fully open	small leakage
AC/DC	both	specific to AC or DC

12.6 Motor HATs

Motor HATs combine drivers and power input on one board that plugs onto the header, saving wiring: for example, a TB6612/DRV8833 HAT for 2–4 DC motors, the Adafruit DC & Stepper Motor HAT (PCA9685 + two TB6612, I2C, so it uses only two GPIO pins), servo HATs with a PCA9685, and robot boards with encoder inputs. Check the motor voltage and current ratings, the GPIO pins used, and whether it powers the Pi as well.

PRACTICE CHECKPOINT

Build a pan-tilt mount with two servos controlled from the keyboard, and a relay-controlled lamp switched by a schedule.

PANTILT_KEYS.PY (CHECKPOINT SOLUTION)

```
from gpiozero import AngularServo

pan = AngularServo(12, min_angle=-90, max_angle=90)
tilt = AngularServo(13, min_angle=-45, max_angle=45)
STEP = 15
KEYS = {"a": (-STEP, 0), "d": (STEP, 0), "w": (0, STEP), "s": (0, -STEP)}

def clamp(v, lo, hi):
    return max(lo, min(hi, v))

while True:
    k = input("move (w/a/s/d, c = centre, q = quit): ").strip().lower()
    if k == "q":
        break
    if k == "c":
        pan.angle, tilt.angle = 0, 0
    elif k in KEYS:
        dp, dt = KEYS[k]
        pan.angle = clamp(pan.angle + dp, -90, 90)
        tilt.angle = clamp(tilt.angle + dt, -45, 45)
    print(f"  pan {pan.angle:+.0f}  tilt {tilt.angle:+.0f}")
```

OUTPUT

```
move (w/a/s/d, c = centre, q = quit): d
  pan +15  tilt +0
move (w/a/s/d, c = centre, q = quit): d
  pan +30  tilt +0
move (w/a/s/d, c = centre, q = quit): w
  pan +30  tilt +15
move (w/a/s/d, c = centre, q = quit): w
  pan +30  tilt +30
move (w/a/s/d, c = centre, q = quit): w
  pan +30  tilt +45
move (w/a/s/d, c = centre, q = quit): w
  pan +30  tilt +45
move (w/a/s/d, c = centre, q = quit): a
  pan +15  tilt +45
move (w/a/s/d, c = centre, q = quit): c
  pan +0  tilt +0
move (w/a/s/d, c = centre, q = quit): q
```

12.7 Practice questions

12.7.1 Basic level

Q1 Why can't a GPIO pin drive a motor directly?

(a) motors need AC
(b) GPIO pins are inputs only
(c) motors need I2C
(d) it supplies only a few mA at 3.3 V; motors need much more current and create spikes

Q2 What is the job of a flyback diode?

(a) to make the motor faster
(b) to give the inductive spike from a motor or coil a safe path when switched off
(c) to reverse the motor
(d) to convert AC to DC

Q3 Which circuit lets a DC motor run in both directions?

(a) an H-bridge **(b)** a voltage divider
(c) a pull-up resistor **(d)** a single diode

Q4 What pulse width puts a standard servo at its centre?

(a) 0.5 ms **(b)** 20 ms **(c)** 1.5 ms **(d)** 15 ms

Q5 How often is a servo pulse repeated?

(a) every 20 ms (50 Hz) **(b)** every 1 s
(c) every 1 ms **(d)** only once

Q6 What do the relay terminals NO, NC and COM mean?

(a) north, centre, common **(b)** normally open, normally closed, common
(c) no output, negative, collector **(d)** node, neutral, current

Q7 Which gpiozero class controls a two-wheel-drive robot?

(a) Motor **(b)** Servo **(c)** TrafficLights **(d)** Robot

Q8 Which I2C chip gives 16 hardware PWM servo channels?

(a) ULN2003 **(b)** MAX7219 **(c)** PCA9685 **(d)** L298N

12.7.2 Intermediate level

Q9 Why must the motor supply GND be connected to the Pi GND?

(a) to charge the battery
(b) to make the motor reverse
(c) control signals need a shared reference voltage
(d) it is not necessary

Q10 L298N: IN1 = 0, IN2 = 1. What does the motor do?

(a) runs forward **(b)** brakes **(c)** coasts **(d)** runs backward

Q11 What makes a MOSFET "logic level"?

(a) it runs on mains
(b) it switches fully on with a 3.3–5 V gate voltage
(c) it is an optocoupler
(d) it has a built-in diode only

Q12 A relay module turns on when its IN pin is LOW. Which gpiozero setting is correct?

(a) OutputDevice(pin, active_high=False) **(b)** OutputDevice(pin)
(c) Button(pin) **(d)** PWMLED(pin)

Q13 Which is an advantage of a solid-state relay?

(a) zero leakage when off **(b)** silent, with no contact wear
(c) switches both AC and DC with one type **(d)** never gets hot

Q14 How many half-steps make one output revolution of a 28BYJ-48?

(a) 4096 **(b)** 200 **(c)** 360 **(d)** 64

Q15 A NEMA 17 has 1.8° per step. How many full steps per revolution?

(a) 180 **(b)** 360 **(c)** 200 **(d)** 100

Q16 Which two signals does an A4988 driver need from the Pi?

(a) SDA and SCL **(b)** TX and RX **(c)** IN1 to IN4 **(d)** STEP and DIR

12.7.3 Advanced level

Q17 What pulse width sets a 1–2 ms servo to 45°?

(a) 1.45 ms **(b)** 0.45 ms **(c)** 1.75 ms **(d)** 1.25 ms

Q18 A servo pulse of 2.0 ms every 20 ms has what duty cycle?

(a) 2 % **(b)** 20 % **(c)** 10 % **(d)** 50 %

Q19 A NEMA 17 (200 steps/rev) with 16× microstepping drives an 8 mm leadscrew. How many pulses move 10 mm?

(a) 4000 **(b)** 400 **(c)** 2500 **(d)** 32 000

Q20 How long does one 28BYJ-48 revolution take at 1 ms per half-step?

(a) about 0.4 s **(b)** about 41 s **(c)** about 1 s **(d)** about 4.1 s

Q21 Ramp a DC motor (forward = 17, backward = 27) smoothly from 0 to full speed in steps of 0.25 every 0.5 s, then stop.

Q22 Write angle_to_pulse(angle) for a servo with 0.5 ms at 0° and 2.5 ms at 180°, and print the pulse and duty cycle (20 ms period) for 0, 90 and 180°.

Q23 Sweep a servo on GPIO18 from -90° to 90° in 30° steps with AngularServo, then detach it.

Q24 Calculate the steps and time for a camera slider to move 300 mm with a GT2 belt (40 mm per pulley revolution), 200 steps/rev, 8× microstepping, at 2000 steps per second.

Q25 A 12 V, 2 A solenoid valve must be switched by a Pi. Which circuit is right?

(a) GPIO pin straight to the valve

(b) logic-level N-MOSFET on the low side, flyback diode across the valve, separate 12 V supply with common GND

(c) NPN 2N2222 from the 3.3 V pin with no diode

(d) the Pi 5 V pin and a relay with no diode

12.8 Solutions

Q1 **Answer: (d) it supplies only a few mA at 3.3 V; motors need much more current and create spikes**

The GPIO is a **signal**, not a power source.

Q2 **Answer: (b) to give the inductive spike from a motor or coil a safe path when switched off**

Protects the transistor from **reverse voltage spikes**.

Q3 **Answer: (a) an H-bridge**

H-bridge (L298N, TB6612FNG).

Q4 **Answer: (c) 1.5 ms**

1.5 ms every 20 ms.

Q5 **Answer: (a) every 20 ms (50 Hz)**

50 Hz.

Q6 **Answer: (b) normally open, normally closed, common**

NO connects to COM only when energised.

Q7 **Answer: (d) Robot**

Robot(left=..., right=...).

Q8 **Answer: (c) PCA9685**

PCA9685 at 0x40.

Q9 **Answer: (c) control signals need a shared reference voltage**

Without a **common ground** the logic levels are meaningless.

Q10 **Answer: (d) runs backward**

0/1 = backward; 1/0 forward; 1/1 brake; 0/0 coast.

Q11 **Answer: (b) it switches fully on with a 3.3–5 V gate voltage**

Check $R_{DS(on)}$ at V_{GS} = 3.3 V or 4.5 V.

Q12 **Answer: (a) OutputDevice(pin, active_high=False)**

Active-low input.

Q13 **Answer: (b) silent, with no contact wear**

SSRs are **silent and long-lived**.

Q14 **Answer: (a) 4096**

4096 half-steps (2048 full steps).

Q15 **Answer: (c) 200**

360 ÷ 1.8 = **200**.

Q16 **Answer: (d) STEP and DIR**

One pulse per step and a direction level.

Q17 **Answer: (d) 1.25 ms**

1.0 + 45 ÷ 180 = **1.25 ms**.

Q18 **Answer: (c) 10 %**

2 ÷ 20 = **10 %**.

Q19 **Answer: (a) 4000**

3200 per 8 mm = 400/mm; × 10 = **4000**.

Q20 **Answer: (d) about 4.1 s**

4096 × 1 ms = **4.096 s**.

Q21 **Answer: Solution program**

Gradual acceleration reduces current surges and wheel slip.

```
from gpiozero import Motor
from time import sleep
m = Motor(17, 27)
for i in range(1, 5):
    m.forward(i / 4)
    sleep(0.5)
m.stop()
```

SIMULATED RUN

```
[  0.00 s] Motor(GPIO17,27) forward 0.25
[  0.50 s] Motor(GPIO17,27) forward 0.50
[  1.00 s] Motor(GPIO17,27) forward 0.75
[  1.50 s] Motor(GPIO17,27) forward 1.00
[  2.00 s] Motor(GPIO17,27) stop
```

Q22 **Answer: Solution program**

Linear interpolation between the end points; duty = pulse ÷ period.

```
def angle_to_pulse(angle, lo=0.5, hi=2.5):
    return lo + (hi - lo) * angle / 180

for a in (0, 90, 180):
    p = angle_to_pulse(a)
    print(f"{a:3d} deg -> {p:.2f} ms, duty {p / 20:.1%}")
```

OUTPUT

```
  0 deg -> 0.50 ms, duty 2.5%
 90 deg -> 1.50 ms, duty 7.5%
180 deg -> 2.50 ms, duty 12.5%
```

Q23 **Answer: Solution program**

range(-90, 91, 30) gives -90 … 90. Detaching stops the pulses so the servo stops buzzing.

```
from gpiozero import AngularServo
from time import sleep
s = AngularServo(18, min_angle=-90, max_angle=90)
for a in range(-90, 91, 30):
    s.angle = a
    sleep(0.3)
s.detach()
```

SIMULATED RUN

```
[  0.00 s] AngularServo(GPIO18) angle -90° -> pulse 1.00 ms
[  0.30 s] AngularServo(GPIO18) angle -60° -> pulse 1.17 ms
[  0.60 s] AngularServo(GPIO18) angle -30° -> pulse 1.33 ms
[  0.90 s] AngularServo(GPIO18) angle 0° -> pulse 1.50 ms
[  1.20 s] AngularServo(GPIO18) angle 30° -> pulse 1.67 ms
[  1.50 s] AngularServo(GPIO18) angle 60° -> pulse 1.83 ms
[  1.80 s] AngularServo(GPIO18) angle 90° -> pulse 2.00 ms
[  2.10 s] AngularServo(GPIO18) detached
```

Q24 **Answer: Solution program**

300 ÷ 40 = 7.5 rev × 1600 = 12 000 steps; at 2000/s that is 6 s; each microstep moves 0.025 mm.

```
mm, mm_per_rev, steps_rev, micro, rate = 300, 40, 200, 8, 2000
steps = mm / mm_per_rev * steps_rev * micro
print(f"steps {steps:.0f}, time {steps / rate:.1f} s, resolution {mm_per_rev / (steps_rev *
micro):.3f} mm/step")
```

OUTPUT

```
steps 12000, time 6.0 s, resolution 0.025 mm/step
```

Q25 **Answer: (b) logic-level N-MOSFET on the low side, flyback diode across the valve, separate 12 V supply with common GND**

MOSFET for the current, diode for the spike, separate supply, shared ground.

GPIO gives signals; transistors, MOSFETs, H-bridges and drivers give power. Always: separate motor supply, common GND, flyback diodes. DC motors: L298N/TB6612 with gpiozero Motor/Robot. Servos: 1–2 ms pulse every 20 ms, PCA9685 for many. Steppers: ULN2003 (28BYJ-48, 4096 half-steps) or A4988/DRV8825 (STEP/DIR, microstepping). Relays isolate loads; mains is for qualified people only.

MODULE 13 · INTERMEDIATE

Camera and image capture

Minimum duration **2 days** | Level **Intermediate**

The camera turns the Pi into a vision device for security, time-lapse and AI projects. This module compares Camera Module 3, the High Quality, Global Shutter and AI cameras and USB webcams, connects ribbon cables on the Pi 4 and Pi 5, uses the rpicam command-line tools with resolution, exposure, focus and HDR options, programs the Picamera2 library (configurations, stills, video encoders, autofocus and exposure controls), and builds time-lapse, motion-triggered, live-streaming and photo-booth projects, followed by 25 solved questions and programs.

IN THIS MODULE

13.1 Camera hardware
13.2 Command-line tools
13.3 Picamera2 library
13.4 Camera projects
13.5 Practice questions
13.6 Solutions

13.1 Camera hardware

Camera	Sensor / resolution	Special feature	Use
13.1.1 Camera Module 3	Sony IMX708, 12 MP	autofocus, HDR; standard (75°) or wide (120°); NoIR versions without IR filter	general projects, security, robots
13.1.2 High Quality Camera	Sony IMX477, 12.3 MP, larger pixels	interchangeable C/CS-mount (and M12) lenses	microscopes, telescopes, photography
13.1.3 Global Shutter Camera	Sony IMX296, 1.6 MP	all pixels exposed at the same instant: no distortion of fast motion	machine vision, fast robots, sports
13.1.4 AI Camera	Sony IMX500, 12.3 MP	neural network runs **on the sensor**: outputs detections, not just images	low-power smart cameras (module 23)
13.1.6 USB webcam	varies	plug in, appears as /dev/video0 (UVC)	quick tests, video calls, OpenCV

13.1.5 Connecting ribbon cables (Pi 4 vs Pi 5 connectors)

Switch off the Pi. Lift the connector's plastic latch, insert the flat cable fully and squarely, press the latch down. On a **Pi 4** (15-pin, 1 mm pitch CAMERA port) the blue stiffener faces the Ethernet/USB ports; on a **Pi 5** the two 22-pin (0.5 mm) CAM/DISP connectors need a **22-to-15-pin** cable for most cameras, with the contacts facing the board. Check detection with `rpicam-hello --list-cameras`.

WATCH OUT
Ribbon cables are fragile: do not fold them sharply, and never plug or unplug a camera while the Pi is powered. Most "no cameras available" errors are a cable inserted backwards or not fully seated.

A **rolling shutter** (most cameras) reads the image row by row over ~10–30 ms, so a fast-moving object or spinning propeller looks slanted or bent. A **global shutter** captures every pixel at once.

13.2 Command-line tools

13.2.1 rpicam-hello, rpicam-still, rpicam-vid

Command	Does
rpicam-hello -t 5000	preview for 5 s (0 = forever); tests the camera
rpicam-hello --list-cameras	list detected cameras and their modes
rpicam-jpeg -o test.jpg	simplest still capture
rpicam-still -o photo.jpg	full-featured stills (raw DNG with -r, timelapse, HDR…)
rpicam-vid -t 10000 -o clip.h264	10 s of H.264 video (add --codec libav -o clip.mp4 for MP4)
rpicam-raw	raw sensor frames

These replace the old `raspistill` / `raspivid` (and are also available under the older names `libcamera-still` etc.). Add `-n` (no preview) on a headless Pi.

13.2.2 Resolution, exposure, focus and HDR options

USEFUL OPTIONS

```
rpicam-still -n -o small.jpg --width 1920 --height 1080   # resolution
rpicam-still -n -o dark.jpg --shutter 200000 --gain 4       # 0.2 s exposure, analogue gain 4
rpicam-still -n -o ev.jpg --ev -1                            # one stop darker
rpicam-still -n -o near.jpg --autofocus-mode manual --lens-position 5   # focus at 1/5 m = 20 cm
rpicam-still -n -o hdr.jpg --hdr                             # Camera Module 3 HDR mode
rpicam-still -n -o tl_%04d.jpg --timelapse 10000 -t 3600000   # one photo every 10 s for an hour
rpicam-vid -n -t 0 --inline --listen -o tcp://0.0.0.0:8888   # stream over the network
```

SHORTCUT TRICK
Lens position is in **dioptres** = 1 ÷ distance in metres: 0 = infinity, 2 = 50 cm, 10 = 10 cm. Shutter time is in **microseconds**: 10 000 µs = 1/100 s.

13.3 Picamera2 library

Picamera2 (pre-installed: `python3-picamera2`) is the official Python library. Remember `--system-site-packages` when you create a venv.

13.3.1 Previews and configurations

A **configuration** sets the output streams: `main` (the image you use), an optional low-resolution `lores` stream (fast analysis) and `raw`. There are ready-made templates for preview, still (full resolution, high quality) and video (smaller, fast):

CONFIGURE.PY

```
from picamera2 import Picamera2

picam2 = Picamera2()
cfg = picam2.create_still_configuration(main={"size": (2304, 1296)})
print("main stream:", cfg["main"])
picam2.configure(cfg)
picam2.start()
```

SIMULATED RUN (hardware events and printed output)

```
main stream: {'size': (2304, 1296), 'format': 'BGR888'}
[  0.00 s] camera configured: still (2304, 1296)
[  0.50 s] camera started
```

13.3.2 Capturing images

CAPTURE.PY

```
from picamera2 import Picamera2
from datetime import datetime
import time

picam2 = Picamera2()
picam2.configure(picam2.create_still_configuration())
picam2.start()
time.sleep(2)                       # let auto-exposure and white balance settle
name = datetime.now().strftime("photo_%Y%m%d_%H%M%S.jpg")
picam2.capture_file(name)
frame = picam2.capture_array()      # the image as a NumPy array for processing
print(name, "saved; array shape", frame.shape, "dtype", frame.dtype)
picam2.stop()
```

SIMULATED RUN (hardware events and printed output)

```
[  0.00 s] camera configured: still (4608, 2592)
[  0.50 s] camera started
[  2.80 s] captured photo_20260315_090002.jpg (4608x2592)
photo_20260315_090002.jpg saved; array shape (2592, 4608, 3) dtype uint8
[  2.83 s] camera stopped
```

`capture_array()` returns a NumPy array (height × width × 3 channels), ready for OpenCV (module 22). Always give the camera 1–2 s after start() before the first capture.

13.3.3 Recording video with encoders

RECORD.PY

```
from picamera2 import Picamera2
from picamera2.encoders import H264Encoder
from picamera2.outputs import FfmpegOutput
import time

picam2 = Picamera2()
picam2.configure(picam2.create_video_configuration(main={"size": (1280, 720)}))
encoder = H264Encoder(bitrate=5_000_000)              # 5 Mbit/s
picam2.start_recording(encoder, FfmpegOutput("clip.mp4"))
time.sleep(10)
picam2.stop_recording()
```

SIMULATED RUN (hardware events and printed output)

```
[  0.00 s] camera configured: video (1280, 720)
[  0.00 s] recording started -> FfmpegOutput('clip.mp4')
[ 10.00 s] recording stopped (FfmpegOutput('clip.mp4'))
```

WORKED EXAMPLE

Video file size

Size = bitrate × time ÷ 8. At 5 Mbit/s, 10 s = 50 Mbit = **6.25 MB**; an hour = 18 000 Mbit = **2.25 GB**. For 24/7 recording use lower bitrates, motion triggering or an SSD.

13.3.4 Controls: autofocus, exposure, white balance

SETTING CONTROLS (REFERENCE)

```
from picamera2 import Picamera2
from libcamera import controls

picam2 = Picamera2(); picam2.start()
picam2.set_controls({"AfMode": controls.AfModeEnum.Continuous})        # Camera Module 3
picam2.set_controls({"AfMode": controls.AfModeEnum.Manual, "LensPosition": 2.0})   # focus at 50 cm
picam2.set_controls({"ExposureTime": 20000, "AnalogueGain": 2.0})      # fixed 1/50 s, gain 2
picam2.set_controls({"AwbEnable": False, "ColourGains": (1.8, 1.5)})   # fixed white balance
picam2.set_controls({"FrameDurationLimits": (33333, 33333)})           # 30 fps
```

Automatic exposure and white balance are best for people; **fixed** settings are better for machine vision and time-lapse, where brightness must not jump between frames.

13.4 Camera projects

13.4.1 Time-lapse photography

TIMELAPSE.PY

```
from picamera2 import Picamera2
import time

INTERVAL, COUNT = 30, 5                    # one frame every 30 s
picam2 = Picamera2()
picam2.configure(picam2.create_still_configuration(main={"size": (1920, 1080)}))
picam2.start(); time.sleep(2)
picam2.set_controls({"AeEnable": False, "AwbEnable": False})   # lock exposure: no flicker
start = time.monotonic()
for i in range(COUNT):
    picam2.capture_file(f"tl_{i:04d}.jpg")
    time.sleep(max(0, start + (i + 1) * INTERVAL - time.monotonic()))   # no drift
print("frames:", COUNT)
```

SIMULATED RUN (hardware events and printed output)

```
[  0.00 s] camera configured: still (1920, 1080)
[  0.50 s] camera started
[  2.50 s] controls set: AeEnable=False, AwbEnable=False
[  2.80 s] captured tl_0000.jpg (1920x1080)
[ 32.80 s] captured tl_0001.jpg (1920x1080)
[ 62.80 s] captured tl_0002.jpg (1920x1080)
[ 92.80 s] captured tl_0003.jpg (1920x1080)
[122.80 s] captured tl_0004.jpg (1920x1080)
frames: 5
```

Join the frames into a video: `ffmpeg -framerate 25 -i tl_%04d.jpg -c:v libx264 -pix_fmt yuv420p timelapse.mp4`.

WORKED EXAMPLE

Planning a time-lapse

A flower opening over 4 hours shown in a 20 s clip at 25 fps needs 20 × 25 = 500 frames; interval = 4 × 3600 ÷ 500 = **28.8 s**. At about 1 MB per 1080p JPEG that is 500 MB of storage.

13.4.2 Motion-triggered security camera

Compare consecutive low-resolution frames: if many pixels change, something moved. The **mean squared difference** of two greyscale frames is a simple, effective measure. Use a **CircularOutput** to keep the last few seconds in memory so the saved clip starts *before* the motion.

MOTION_CAMERA.PY

```
from picamera2 import Picamera2
from picamera2.encoders import H264Encoder
from picamera2.outputs import CircularOutput
from datetime import datetime
import numpy as np, time

picam2 = Picamera2()
picam2.configure(picam2.create_video_configuration(main={"size": (1280, 720)}, lores={"size": (320,
240)}))
encoder = H264Encoder(2_000_000)
circ = CircularOutput(buffersize=150)              # about 5 s at 30 fps
picam2.start_recording(encoder, circ)
prev, recording, last_motion = None, False, 0
while True:
    cur = picam2.capture_array("lores").astype(np.int16)
    if prev is not None:
        mse = np.square(cur - prev).mean()
        now = time.monotonic()
        if mse > 20:
            last_motion = now
            if not recording:
                name = datetime.now().strftime("motion_%H%M%S.h264")
                circ.fileoutput = name; circ.start(); recording = True
                print(f"motion (MSE {mse:.0f}) -> recording {name}")
        elif recording and now - last_motion > 5:
            circ.stop(); recording = False
            print("no motion for 5 s -> clip closed")
    prev = cur
```

OUTPUT

```
motion (MSE 66) -> recording motion_090003.h264
no motion for 5 s -> clip closed
```

13.4.3 Live video streaming to a browser

The Picamera2 examples include `mjpeg_server.py`: an MJPEGEncoder writes JPEG frames to a small HTTP server and any browser shows the live view at `http://pi-address:8000`. For lower bandwidth and more viewers, stream H.264 through **MediaMTX** (RTSP/WebRTC). Never expose a camera stream directly to the internet; use a VPN (module 15.6).

13.4.4 Photo booth with a button

PHOTOBOOTH.PY

```
from picamera2 import Picamera2
from gpiozero import Button, LED
from datetime import datetime
from signal import pause
import time

picam2 = Picamera2()
picam2.configure(picam2.create_still_configuration(main={"size": (1920, 1080)}))
picam2.start()
light = LED(17)
count = 0

def take():
    global count
    for _ in range(3):                          # 3-2-1 countdown on the LED
        light.on(); time.sleep(0.5); light.off(); time.sleep(0.5)
    count += 1
    picam2.capture_file(datetime.now().strftime(f"booth_{count:03d}_%H%M%S.jpg"))

Button(2).when_pressed = take
pause()
```

SIMULATED RUN (hardware events and printed output)

```
[  0.00 s] camera configured: still (1920, 1080)
[  0.50 s] camera started
[  3.00 s] LED(GPIO17) on
[  3.50 s] LED(GPIO17) off
[  4.00 s] LED(GPIO17) on
[  4.50 s] LED(GPIO17) off
[  5.00 s] LED(GPIO17) on
[  5.50 s] LED(GPIO17) off
[  6.30 s] captured booth_001_090006.jpg (1920x1080)
```

PRACTICE CHECKPOINT

Build a motion-detection camera that records a short clip when motion is detected and saves it with a timestamp.

13.5 Practice questions

13.5.1 Basic level

Q1 Which camera has autofocus?

(a) High Quality Camera
(b) Camera Module 3
(c) Global Shutter Camera
(d) Camera Module 1

Q2 What does "NoIR" mean on a camera module?

(a) no image rotation
(b) no internal RAM
(c) not for indoor use
(d) no infrared filter, so it can see IR light at night with IR LEDs

Q3 Which command lists the cameras the Pi detects?

(a) rpicam-hello --list-cameras
(b) lsusb -c
(c) camera --scan
(d) vcgencmd list_cameras

Q4 Which command records video?

(a) rpicam-still (b) rpicam-hello (c) rpicam-vid (d) rpicam-jpeg

Q5 Which Python library is the official way to use the camera?

(a) picamera (legacy)
(b) smbus2
(c) gpiozero
(d) Picamera2

Q6 What does capture_array() return?

(a) a JPEG file
(b) a NumPy array of pixel values
(c) a video stream
(d) a list of file names

Q7 Which camera lets you change lenses?

(a) High Quality Camera
(b) Camera Module 3
(c) AI Camera
(d) a laptop webcam

Q8 Which camera runs a neural network on the sensor itself?

(a) Global Shutter Camera
(b) High Quality Camera
(c) AI Camera (IMX500)
(d) NoIR Camera Module 3

13.5.2 Intermediate level

Q9 Why does a fast propeller look bent in photos from most cameras?

(a) rolling shutter reads rows at slightly different times
(b) the lens is cheap
(c) autofocus is too slow
(d) JPEG compression

Q10 What cable does a Camera Module 3 need on a Pi 5?

(a) the same 15-pin cable as the Pi 4
(b) a 22-pin to 15-pin camera cable

(c) USB-C (d) micro-HDMI

Q11 What lens position (dioptres) focuses at 25 cm?

(a) 0.25 (b) 25 (c) 2.5 (d) 4

Q12 What --shutter value gives 1/250 s?

(a) 250 (b) 40 (c) 4000 (d) 400 000

Q13 Why wait 1–2 seconds after picam2.start() before capturing?

(a) the camera must warm up to 40 °C
(b) to save power
(c) auto-exposure and white balance need time to settle
(d) the file system is slow

Q14 Why lock exposure and white balance for time-lapse?

(a) to make files smaller
(b) to save the battery
(c) to enable HDR
(d) so brightness and colour do not flicker between frames

Q15 What does a CircularOutput give a security camera?

(a) infinite storage
(b) the clip includes a few seconds from before motion was detected
(c) colour correction
(d) faster autofocus

Q16 Why analyse the lores stream rather than the main stream for motion?

(a) small frames are much faster to compare
(b) the main stream has no colour
(c) lores has better quality
(d) main stream cannot be read

Q17 What file size does 30 s of 8 Mbit/s H.264 video produce?

(a) 240 MB (b) 30 MB (c) 3.75 MB (d) 8 MB

13.5.3 Advanced level

Q18 A 6-hour sunset-to-night time-lapse must play for 24 s at 25 fps. What interval is needed?

(a) 36 s (b) 24 s (c) 6 s (d) 60 s

Q19 Calculate how many hours of 1080p video at 4 Mbit/s fit on a 64 GB card with 20 % kept free.

Q20 Take 3 photos 5 seconds apart with Picamera2, naming them with the date and time.

Q21 Write a motion score function: given two greyscale frames as NumPy arrays, return the percentage of pixels that changed by more than 25 levels. Test with synthetic frames where a 20 × 20 block changes in a 100 × 100 image.

Q22 Record a 10-second clip with Picamera2 only while a button on GPIO2 is pressed (start on press, stop on release), naming it with the time.

Q23 Which command streams the camera over TCP so another computer can view it?

(a) rpicam-still --stream
(b) rpicam-hello -o tcp
(c) rpicam-vid -t 0 --inline --listen -o tcp://0.0.0.0:8888
(d) ssh -X rpicam

Q24 A wildlife camera must work at night without visible light. Which setup works?

(a) standard Camera Module 3 with a white LED flash
(b) Global Shutter Camera with no light
(c) High Quality Camera at ISO 100
(d) Camera Module 3 NoIR with 850 nm IR LEDs

Q25 A conveyor inspection system must photograph parts moving quickly without distortion. Which camera fits best?

(a) Camera Module 3 Wide
(b) USB webcam
(c) AI Camera
(d) Global Shutter Camera

13.6 Solutions

Q1 **Answer: (b) Camera Module 3**
CM3 (IMX708) has **autofocus**.

Q2 **Answer: (d) no infrared filter, so it can see IR light at night with IR LEDs**
NoIR = **No IR filter**.

Q3 **Answer: (a) rpicam-hello --list-cameras**
`rpicam-hello --list-cameras`.

Q4 **Answer: (c) rpicam-vid**
rpicam-vid.

Q5 **Answer: (d) Picamera2**
Picamera2; the old picamera does not work on current OS.

Q6 **Answer: (b) a NumPy array of pixel values**
Height × width × channels array.

Q7 **Answer: (a) High Quality Camera**
C/CS-mount lenses.

Q8 **Answer: (c) AI Camera (IMX500)**
On-sensor inference.

Q9 **Answer: (a) rolling shutter reads rows at slightly different times**

Use a **global shutter** for fast motion.

Q10 **Answer: (b) a 22-pin to 15-pin camera cable**

Pi 5 has 22-pin 0.5 mm connectors.

Q11 **Answer: (d) 4**

1 ÷ 0.25 m = **4**.

Q12 **Answer: (c) 4000**

1/250 s = 0.004 s = **4000 µs**.

Q13 **Answer: (c) auto-exposure and white balance need time to settle**

First frames may be too dark or tinted.

Q14 **Answer: (d) so brightness and colour do not flicker between frames**

Consistency between frames.

Q15 **Answer: (b) the clip includes a few seconds from before motion was detected**

It keeps a rolling buffer in RAM.

Q16 **Answer: (a) small frames are much faster to compare**

320 × 240 = 76 800 pixels vs 921 600 at 720p.

Q17 **Answer: (b) 30 MB**

8 × 30 = 240 Mbit ÷ 8 = **30 MB**.

Q18 **Answer: (a) 36 s**

24 × 25 = 600 frames; 21 600 s ÷ 600 = **36 s**.

Q19 **Answer: Solution program**

64 × 0.8 = 51.2 GB = 409 600 Mbit; ÷ 4 Mbit/s = 102 400 s = 28.4 h.

```
card_gb, free, bitrate = 64, 0.20, 4          # GB, fraction, Mbit/s
usable_mbit = card_gb * (1 - free) * 8000     # 1 GB = 8000 Mbit
hours = usable_mbit / bitrate / 3600
print(f"about {hours:.1f} hours")
```

OUTPUT

```
about 28.4 hours
```

Q20 **Answer: Solution program**

strftime builds a unique, sortable name for each photo.

```
from picamera2 import Picamera2
from datetime import datetime
import time
cam = Picamera2()
cam.configure(cam.create_still_configuration(main={"size": (1920, 1080)}))
cam.start(); time.sleep(2)
for _ in range(3):
    name = datetime.now().strftime("%Y-%m-%d_%H-%M-%S.jpg")
    cam.capture_file(name)
    print("saved", name)
    time.sleep(5)
cam.stop()
```

OUTPUT

```
saved 2026-03-15_09-00-02.jpg
saved 2026-03-15_09-00-07.jpg
saved 2026-03-15_09-00-13.jpg
```

Q21 **Answer: Solution program**

Cast to int16 before subtracting (uint8 would wrap round). 400 of 10 000 pixels = 4 %.

```
import numpy as np
def changed_percent(a, b, threshold=25):
    diff = np.abs(a.astype(np.int16) - b.astype(np.int16))
    return (diff > threshold).mean() * 100

f1 = np.full((100, 100), 80, dtype=np.uint8)
f2 = f1.copy(); f2[40:60, 40:60] = 200
print(f"{changed_percent(f1, f2):.1f} % changed")
print(f"{changed_percent(f1, f1):.1f} % changed (same frame)")
```

OUTPUT

```
4.0 % changed
0.0 % changed (same frame)
```

Q22 **Answer: Solution program**

Callbacks start and stop the encoder; the button was held from 2 s to 6 s.

```
from picamera2 import Picamera2
from picamera2.encoders import H264Encoder
from gpiozero import Button
from datetime import datetime
from signal import pause

cam = Picamera2()
cam.configure(cam.create_video_configuration())
enc = H264Encoder(3_000_000)
def start():
    cam.start_recording(enc, datetime.now().strftime("clip_%H%M%S.h264"))
def stop():
    cam.stop_recording()
b = Button(2)
b.when_pressed = start
b.when_released = stop
pause()
```

SIMULATED RUN

```
[  0.00 s] camera configured: video (1280, 720)
[  2.00 s] recording started -> clip_090002.h264
[  6.00 s] recording stopped (clip_090002.h264)
```

Q23 **Answer: (c) rpicam-vid -t 0 --inline --listen -o tcp://0.0.0.0:8888**

`--listen` waits for a client on that port.

Q24 **Answer: (d) Camera Module 3 NoIR with 850 nm IR LEDs**

NoIR + infrared illumination.

Q25 **Answer: (d) Global Shutter Camera**

Global shutter removes rolling-shutter distortion.

Choose the camera for the job (CM3 autofocus/NoIR, HQ lenses, Global Shutter for motion, AI Camera on-sensor AI). Connect cables with power off (Pi 5: 22-pin). Test with rpicam-hello; capture with rpicam-still/vid. In Python: Picamera2 configure → start → wait → capture_file/capture_array; H264Encoder for video; lock exposure for time-lapse; frame differencing + CircularOutput for motion clips.

MODULE 14 · INTERMEDIATE

Audio and multimedia

Minimum duration **1 day** | Level **Intermediate**

This module covers sound output through HDMI, USB and I2S DAC and amplifier HATs, playing sounds from Python, recording from USB microphones, text-to-speech with espeak and Piper, offline speech recognition with Vosk and Whisper, the Pi as a Kodi/LibreELEC media centre and music server, and retro gaming with RetroPie and the legal use of game files, followed by 25 solved questions and programs.

IN THIS MODULE

14.1 Audio output
14.2 Audio input
14.3 Speech
14.4 Media centre
14.5 Retro gaming
14.6 Practice questions
14.7 Solutions

14.1 Audio output

14.1.1 HDMI and USB audio

Raspberry Pi OS uses **PipeWire** to manage sound. Audio goes to the HDMI monitor's speakers by default; plug in a USB sound card, USB speaker or Bluetooth speaker and choose it by right-clicking the volume icon. The Pi 5 has **no 3.5 mm audio jack** (the Pi 4 and earlier do, but its quality is modest). From the terminal:

CHECKING AUDIO DEVICES

```
aplay -l                        # list playback devices (ALSA)
wpctl status                    # PipeWire sinks and sources
wpctl set-default 47            # make sink id 47 the default
speaker-test -c 2 -t wav        # say "front left", "front right"
aplay /usr/share/sounds/alsa/Front_Center.wav
```

14.1.2 I2S DAC and amplifier HATs

I2S is a digital audio bus (clock GPIO18, frame sync GPIO19, data out GPIO21). A **DAC HAT** (HiFiBerry DAC+, Raspberry Pi DAC+, IQaudio) converts it to high-quality line-level audio; an **amplifier HAT** (Raspberry Pi DigiAMP+, or a MAX98357A board) drives speakers directly. Enable with an overlay such as `dtoverlay=hifiberry-dacplus` in config.txt; current Raspberry Pi HATs are detected automatically from their EEPROM.

14.1.3 Playing sounds from Python

Generate or load audio as samples. A WAV file stores samples as integers: **sample rate** (samples per second, e.g. 44 100 Hz) × **bit depth** (16 bits) × **channels**. This program makes a real WAV file of a two-note chime that any player can play:

MAKE_CHIME.PY

```
import wave, struct, math

RATE = 44100
def tone(freq, seconds, volume=0.5):
    n = int(RATE * seconds)
    return [int(volume * 32767 * math.sin(2 * math.pi * freq * i / RATE)
                * min(1, (n - i) / 2000)) for i in range(n)]     # short fade-out

samples = tone(659.25, 0.4) + tone(523.25, 0.6)                 # E5 then C5
with wave.open("chime.wav", "wb") as w:
    w.setnchannels(1); w.setsampwidth(2); w.setframerate(RATE)
    w.writeframes(struct.pack(f"<{len(samples)}h", *samples))

with wave.open("chime.wav") as w:
    secs = w.getnframes() / w.getframerate()
    print(f"{w.getnchannels()} channel, {w.getframerate()} Hz, {8 * w.getsampwidth()}-bit,
{secs:.2f} s")
import os; print("file size:", os.path.getsize("chime.wav"), "bytes")
```

OUTPUT

```
1 channel, 44100 Hz, 16-bit, 1.00 s
file size: 88244 bytes
```

Play it with `aplay chime.wav`, or from Python with `subprocess.run(["aplay", "chime.wav"])`, **pygame.mixer** (games, several sounds at once), or **simpleaudio**/**sounddevice**. For MP3s use `mpg123` or `mpv`.

WORKED EXAMPLE

Audio data rate

CD quality = 44 100 samples/s × 16 bits × 2 channels = **1 411 200 bit/s ≈ 176 kB/s**, so a 3-minute song is about 31.8 MB as WAV, compared with about 4.3 MB as a 192 kbit/s MP3.

14.2 Audio input

14.2.1 USB microphones

The Pi has no microphone input. Use a **USB microphone**, a USB sound card with a mic jack, a USB webcam's mic, or an I2S MEMS microphone (INMP441, SPH0645). For voice assistants, a **microphone array** (ReSpeaker) picks up speech from across a room. `arecord -l` lists capture devices.

14.2.2 Recording audio

RECORDING FROM THE TERMINAL

```
arecord -l                                  # list capture devices
arecord -D plughw:1,0 -f S16_LE -r 16000 -c 1 -d 5 test.wav   # 5 s, 16 kHz mono
aplay test.wav
alsamixer                                   # F4 to adjust capture level
```

Speech recognisers usually want **16 kHz, 16-bit, mono**. In Python, `sounddevice.rec()` returns a NumPy array, handy for measuring loudness (RMS) to detect claps or noise levels.

CLAP_DETECT.PY

```
import numpy as np

RATE = 16000
t = np.arange(RATE) / RATE                                  # 1 second
quiet = 0.01 * np.random.default_rng(1).standard_normal(RATE)
clap = quiet.copy(); clap[8000:8400] += 0.8 * np.exp(-np.arange(400) / 60)

def rms_db(x):
    return 20 * np.log10(np.sqrt(np.mean(x ** 2)) + 1e-12)

for name, sig in (("quiet room", quiet), ("with a clap", clap)):
    blocks = sig.reshape(-1, 800)                           # 50 ms blocks
    loud = [i * 50 for i, b in enumerate(blocks) if rms_db(b) > -25]
    print(f"{name:12} overall {rms_db(sig):6.1f} dBFS  loud blocks at ms: {loud}")
```

OUTPUT

```
quiet room   overall  -40.0 dBFS  loud blocks at ms: []
with a clap  overall  -28.8 dBFS  loud blocks at ms: [500]
```

14.3 Speech

14.3.1 Text-to-speech (espeak, Piper)

Engine	Quality	Notes
espeak-ng	robotic but clear	`sudo apt install espeak-ng`; `espeak-ng -v en-gb -s 150 "Hello"`; tiny and instant
Piper	natural neural voices	fast enough on a Pi 4/5, fully offline; download a voice model (.onnx)
Cloud TTS	very natural	needs internet and an API key; data leaves the device

PIPER FROM THE COMMAND LINE

```
echo "Motion detected in the garage" | piper --model en_GB-alan-medium.onnx --output_file alert.wav
&& aplay alert.wav
```

14.3.2 Offline speech recognition (Vosk, Whisper models)

Vosk (Kaldi-based) runs fully offline with small models (about 50 MB for English) and recognises speech in real time on a Pi; you can restrict it to a short list of command words for high accuracy. **Whisper** (OpenAI's model, via whisper.cpp or faster-whisper) is more accurate on free speech but slower: the tiny/base models suit a Pi 5.

VOSK_COMMANDS.PY (REFERENCE: NEEDS A MICROPHONE)

```
from vosk import Model, KaldiRecognizer        # pip install vosk sounddevice
import sounddevice as sd, json, queue
q = queue.Queue()
model = Model("vosk-model-small-en-us-0.15")
rec = KaldiRecognizer(model, 16000, '["light on", "light off", "what time is it", "[unk]"]')
with sd.RawInputStream(samplerate=16000, blocksize=8000, dtype="int16", channels=1,
                       callback=lambda data, *a: q.put(bytes(data))):
    while True:
        if rec.AcceptWaveform(q.get()):
            text = json.loads(rec.Result())["text"]
            if text: print("heard:", text)
```

Once speech is text, handling commands is ordinary Python. Keep the matching tolerant:

COMMANDS.PY

```
from gpiozero import LED
from datetime import datetime

light = LED(17)
def handle(text):
    t = text.lower()
    if "light" in t and "on" in t.split():
        light.on();  return "Turning the light on"
    if "light" in t and "off" in t.split():
        light.off(); return "Turning the light off"
    if "time" in t:
        return datetime.now().strftime("It is %H:%M")
    return "Sorry, I did not understand"

for heard in ["light on", "please switch the light off", "what time is it", "play music"]:
    print(f"{heard!r:32} -> {handle(heard)}")
```

SIMULATED RUN (hardware events and printed output)

```
[  0.00 s] LED(GPIO17) on
'light on'                       -> Turning the light on
[  0.00 s] LED(GPIO17) off
'please switch the light off'    -> Turning the light off
'what time is it'                -> It is 09:00
'play music'                     -> Sorry, I did not understand
```

14.4 Media centre

14.4.1 Kodi and LibreELEC

Kodi is a full-screen media centre: films, TV, music and photos from USB drives or network shares, with remote control from a phone app or the TV remote (HDMI-CEC). **LibreELEC** is a minimal OS that boots straight into Kodi in seconds; or install Kodi on Raspberry Pi OS with `sudo apt install kodi`. The Pi 5 decodes 4K HEVC video in hardware.

14.4.2 Music streaming servers

A Pi with a DAC HAT makes an excellent network music player: **Volumio** or **moOde** (complete OSes with web interfaces), **Mopidy** or **MPD** (music servers controlled from apps), **Raspotify** (Spotify Connect receiver), and **shairport-sync** (AirPlay receiver). Jellyfin or Navidrome serve your own library to phones.

14.5 Retro gaming

14.5.1 RetroPie and emulation basics

An **emulator** is a program that imitates old console hardware so its games run on the Pi. **RetroPie**, Recalbox and Batocera bundle many emulators (RetroArch cores) behind a game-pad-friendly menu (EmulationStation). A Pi 4/5 handles 8- and 16-bit consoles easily and many 32-bit systems. Use USB or Bluetooth controllers and good cooling.

14.5.2 Legal use of game files

WATCH OUT

Game files (ROMs) are copyrighted. Downloading ROMs of games you do not own is illegal in most countries, including India. Legal sources: games you have dumped from cartridges you own (where local law allows), free homebrew games, and titles the rights holder has released for free. Emulators themselves are legal.

PRACTICE CHECKPOINT

Build a voice-controlled assistant that recognises three spoken commands offline and answers with speech while switching an LED or relay.

14.6 Practice questions

14.6.1 Basic level

Q1 Which Raspberry Pi model has no 3.5 mm audio jack?

(a) Raspberry Pi 4
(b) Raspberry Pi 3B+
(c) Raspberry Pi 5
(d) Raspberry Pi 400 only

Q2 Which command lists playback devices?

(a) aplay -l
(b) arecord -d
(c) lsusb -a
(d) alsamixer -x

Q3 Which digital audio bus do DAC HATs use?

(a) I2C
(b) SPI
(c) UART
(d) I2S

Q4 Which TTS engine is small, instant and robotic-sounding?

(a) Piper
(b) espeak-ng
(c) Whisper
(d) Kodi

Q5 Which tool works fully offline for speech recognition with small models?

(a) a cloud API
(b) Vosk
(c) Kodi
(d) aplay

Q6 What is LibreELEC?

(a) a speech recogniser
(b) a GPIO library
(c) an audio HAT
(d) a minimal OS that boots straight into Kodi

Q7 Which is legal for retro gaming?

(a) free homebrew games and games released free by their owners
(b) downloading any ROM from the internet
(c) sharing ROMs of commercial games
(d) selling pre-loaded SD cards

Q8 Which sound server manages audio in current Raspberry Pi OS?

(a) DirectSound
(b) Core Audio
(c) PipeWire
(d) Kodi

14.6.2 Intermediate level

Q9 What sample format do speech recognisers usually want?

(a) 192 kHz, 32-bit, stereo
(b) 8-bit, 8 kHz, stereo
(c) MP3 at 320 kbit/s
(d) 16 kHz, 16-bit, mono

Q10 Why restrict a Vosk recogniser to a short list of command words?

(a) it makes recording louder
(b) it greatly improves accuracy for those commands
(c) it is required by the licence
(d) it enables English

Q11 Which is more accurate on free speech but slower: Vosk small or Whisper?

(a) Whisper (b) Vosk small (c) espeak-ng (d) Piper

Q12 Which HAT lets the Pi drive passive speakers directly?

(a) a DAC HAT with line-out only (b) a Sense HAT
(c) an amplifier HAT (e.g. DigiAMP+) (d) a PoE HAT

Q13 Which software makes a Pi a Spotify Connect receiver?

(a) Raspotify (b) RetroPie (c) Pi-hole (d) Vosk

Q14 What is an emulator?

(a) a type of ROM
(b) software that imitates another computer or console so its programs run
(c) a speaker driver
(d) a game controller

Q15 Which command records 5 s of 16 kHz mono audio?

(a) aplay -r 16000 test.wav
(b) speaker-test -d 5
(c) rpicam-vid -t 5000
(d) arecord -f S16_LE -r 16000 -c 1 -d 5 test.wav

Q16 What does HDMI-CEC allow with Kodi?

(a) 4K decoding
(b) Wi-Fi streaming
(c) controlling the Pi with the TV's remote
(d) faster boot

Q17 What is the data rate of 48 kHz, 16-bit stereo audio?

(a) 768 kbit/s (b) 96 kbit/s (c) 1 536 kbit/s (d) 3 072 kbit/s

14.6.3 Advanced level

Q18 How big is a 10-second 16 kHz 16-bit mono WAV (data only)?

(a) 160 000 bytes (b) 32 000 bytes (c) 1.6 MB (d) 320 000 bytes

Q19 A signal has RMS amplitude 0.1 of full scale. What level is that in dBFS?

(a) -10 dBFS (b) -20 dBFS (c) -40 dBFS (d) +20 dBFS

Q20 Write the A major scale (440 Hz, ratio 2^(n/12)) frequencies for steps 0, 2, 4, 5, 7, 9, 11, 12 and print them rounded to 2 decimals.

Q21 Create a 1-second 1 kHz beep WAV file (8 kHz, 16-bit mono) and print its number of frames and size.

Q22 Make a "talking clock" phrase: given the time, return text like "It is twenty past nine" for minutes 0, 15, 20, 30, 45 (otherwise "It is 9:07").

Q23 Count claps in a recording: given loudness values (dBFS) per 50 ms block, count rising crossings of -20 dBFS. Test with a list containing two claps.

Q24 Why can a Raspberry Pi 5 play 4K HEVC films smoothly in Kodi?

(a) it has a hardware HEVC video decoder
(b) the CPU decodes everything in software only
(c) it streams from the cloud
(d) it lowers the film to 720p

Q25 Which pipeline makes a private offline voice assistant?

(a) USB mic → cloud API → Kodi
(b) USB mic → Vosk with a command grammar → Python logic → GPIO + Piper speech
(c) HDMI audio → espeak → RetroPie
(d) Sense HAT → Whisper in the cloud → aplay

14.7 Solutions

Q1 **Answer: (c) Raspberry Pi 5**

The **Pi 5** dropped the analogue jack.

Q2 **Answer: (a) aplay -l**

`aplay -l`; `arecord -l` lists capture devices.

Q3 **Answer: (d) I2S**

I2S (GPIO18, 19, 21).

Q4 **Answer: (b) espeak-ng**

espeak-ng.

Q5 **Answer: (b) Vosk**

Vosk.

Q6 **Answer: (d) a minimal OS that boots straight into Kodi**

Media-centre OS.

Q7 **Answer: (a) free homebrew games and games released free by their owners**

Respect copyright.

Q8 **Answer: (c) PipeWire**

PipeWire (with wpctl).

Q9 **Answer: (d) 16 kHz, 16-bit, mono**

Speech needs little bandwidth.

Q10 **Answer: (b) it greatly improves accuracy for those commands**

A small grammar = fewer mistakes.

Q11 **Answer: (a) Whisper**

Whisper is more accurate, needs more compute.

Q12 **Answer: (c) an amplifier HAT (e.g. DigiAMP+)**

A DAC gives line level; an **amplifier** drives speakers.

Q13 **Answer: (a) Raspotify**

Raspotify.

Q14 **Answer: (b) software that imitates another computer or console so its programs run**

It emulates the original hardware.

Q15 **Answer: (d) arecord -f S16_LE -r 16000 -c 1 -d 5 test.wav**

`-d 5` = duration.

Q16 **Answer: (c) controlling the Pi with the TV's remote**

CEC carries remote-control commands over HDMI.

Q17 **Answer: (c) 1 536 kbit/s**

48 000 × 16 × 2 = **1 536 000 bit/s**.

Q18 **Answer: (d) 320 000 bytes**

16 000 × 2 bytes × 10 = **320 000 bytes**.

Q19 **Answer: (b) -20 dBFS**

20 × log10(0.1) = **−20 dBFS**.

Q20 **Answer: Solution program**

Each semitone multiplies the frequency by 2^(1/12) ≈ 1.0595; 12 semitones doubles it (880 Hz).

```
steps = [0, 2, 4, 5, 7, 9, 11, 12]
freqs = [round(440 * 2 ** (n / 12), 2) for n in steps]
print(freqs)
```

OUTPUT

```
[440.0, 493.88, 554.37, 587.33, 659.26, 739.99, 830.61, 880.0]
```

Q21 **Answer: Solution program**

8000 samples × 2 bytes = 16 000 bytes of data plus a 44-byte WAV header.

```
import wave, struct, math, os
RATE = 8000
data = [int(16000 * math.sin(2 * math.pi * 1000 * i / RATE)) for i in range(RATE)]
with wave.open("beep.wav", "wb") as w:
    w.setnchannels(1); w.setsampwidth(2); w.setframerate(RATE)
    w.writeframes(struct.pack(f"<{RATE}h", *data))
print("frames:", RATE, "size:", os.path.getsize("beep.wav"), "bytes (16 000 data + 44 header)")
```

OUTPUT

```
frames: 8000 size: 16044 bytes (16 000 data + 44 header)
```

Q22 **Answer: Solution program**

A dictionary maps minutes to phrases; "quarter to" uses the next hour. Feed the text to espeak-ng or Piper to speak it.

```
from datetime import time
NUM = {1: "one", 2: "two", 3: "three", 4: "four", 5: "five", 6: "six", 7: "seven",
       8: "eight", 9: "nine", 10: "ten", 11: "eleven", 12: "twelve"}
def phrase(t):
    h12 = lambda h: NUM[(h - 1) % 12 + 1]
    words = {0: "{h} o'clock", 15: "quarter past {h}", 20: "twenty past {h}", 30: "half past {h}"}
    if t.minute in words:
        return "It is " + words[t.minute].format(h=h12(t.hour))
    if t.minute == 45:
        return f"It is quarter to {h12(t.hour + 1)}"
    return f"It is {t.hour}:{t.minute:02d}"
for t in (time(9, 0), time(9, 20), time(21, 45), time(9, 7)):
    print(phrase(t))
```

OUTPUT

```
It is nine o'clock
It is twenty past nine
It is quarter to ten
It is 9:07
```

Q23 **Answer: Solution program**

Count only the moment the level rises above the threshold, so a loud block that lasts two readings is still one clap.

```
levels = [-48, -47, -12, -15, -40, -46, -45, -10, -30, -47]
claps, above = 0, False
for lv in levels:
    if lv > -20 and not above:
        claps += 1
    above = lv > -20
print("claps:", claps)
```

OUTPUT

```
claps: 2
```

Q24 **Answer: (a) it has a hardware HEVC video decoder**

Hardware decoding keeps the CPU almost idle.

Q25 **Answer: (b) USB mic → Vosk with a command grammar → Python logic → GPIO + Piper speech**

Everything runs on the Pi; no audio leaves it.

Audio out: HDMI, USB, Bluetooth or an I2S DAC/amp HAT (Pi 5 has no jack); manage with PipeWire (wpctl) and test with aplay/speaker-test. Audio in: USB or I2S microphones, arecord at 16 kHz mono for speech. Speech: espeak-ng/Piper out, Vosk/Whisper in, all offline. Kodi/LibreELEC for media, Volumio/Raspotify for music, RetroPie for emulation with legal games only.

MODULE 15 · INTERMEDIATE

Web servers and web dashboards

Minimum duration **2 days** | Level **Intermediate**

Serving web pages from the Pi lets any phone or laptop monitor and control a project. This module explains HTTP, clients and servers with an HTML/CSS/JavaScript refresher, builds Flask apps with routes, templates, forms that switch GPIO, JSON APIs and live updates with fetch, compares FastAPI, puts Nginx in front as a reverse proxy with Gunicorn under systemd, draws dashboards with Chart.js, Node-RED and Grafana, and reaches the Pi from outside safely with VPNs and tunnels instead of port forwarding, followed by 25 solved questions and programs. Every Flask example was run with Flask's test client, and the responses shown are real.

IN THIS MODULE

15.1 Web basics
15.2 Flask
15.3 FastAPI as an alternative
15.4 Web servers
15.5 Dashboards
15.6 Accessing the Pi from outside the home network
15.7 Practice questions
15.8 Solutions

15.1 Web basics

15.1.1 HTTP, clients and servers

A **client** (browser, phone app, Python script) sends an **HTTP request** to a **server**, which sends back a **response**. A request has a **method** (GET to read, POST to send or change something), a **path** (/api/temp) and headers; a response has a **status code**, headers and a body (HTML, JSON, an image).

ONE REQUEST, ONE RESPONSE

Code	Meaning
200 OK	success
201 Created	a new item was created (POST)
301/302	redirect to another URL
400 Bad Request	the client sent invalid data
401 / 403	not logged in / not allowed

Code	Meaning
404 Not Found	no such page
500 Internal Server Error	the server code crashed

15.1.2 HTML, CSS and JavaScript refresher

Language	Role	Example
HTML	structure and content	Room <button id="led">LED</button>
CSS	appearance	button { font-size: 2em; background: #C9973A; }
JavaScript	behaviour in the browser	fetch("/api/temp").then(r => r.json())

15.2 Flask

Flask is a small Python web framework: `sudo apt install python3-flask` (or pip in a venv). A **route** connects a URL path to a Python function; whatever the function returns is the response.

15.2.1 Routes and templates

APP_BASIC.PY

```
from flask import Flask, render_template_string

app = Flask(__name__)
PAGE = """<!doctype html><title>Pi</title>
<h1>Hello from {{ name }}</h1><p>CPU temperature: {{ temp }} &deg;C</p>
{% if temp > 60 %}<p style="color:red">Hot!</p>{% endif %}"""

@app.route("/")
def home():
    return render_template_string(PAGE, name="labpi", temp=47.2)

@app.route("/hello/<user>")            # variable part of the URL
def hello(user):
    return f"Hi {user}!"

if __name__ == "__main__":
    c = app.test_client()               # simulate a browser for this book
    r = c.get("/"); print(r.status, "|", r.get_data(as_text=True).replace(chr(10), " "))
    r = c.get("/hello/Asha"); print(r.status, "|", r.get_data(as_text=True))
    r = c.get("/nothing"); print(r.status)
    # on the Pi you would run:  app.run(host="0.0.0.0", port=5000)
```

OUTPUT

```
200 OK | <!doctype html><title>Pi</title> <h1>Hello from labpi</h1><p>CPU temperature: 47.2
&deg;C</p>
200 OK | Hi Asha!
404 NOT FOUND
```

`host="0.0.0.0"` makes the server listen on all network interfaces, so other devices can open `http://labpi.local:5000`; the default 127.0.0.1 only accepts connections from the Pi itself. Normally templates live as files in a `templates/` folder and are rendered with `render_template("index.html", …)`. The **Jinja2** template language fills in `{{ values }}` and runs `{% if %}` / `{% for %}` blocks. Values are HTML-escaped automatically, which protects against code injection.

15.2.2 Forms and buttons to control GPIO

GPIO_WEB.PY

```
from flask import Flask, request, redirect, render_template_string
from gpiozero import LED

app = Flask(__name__)
leds = {"red": LED(17), "green": LED(27)}
PAGE = """{% for name, led in leds.items() %}
<form method="post" action="/led/{{ name }}">{{ name }}: {{ "ON" if led.is_lit else "off" }}
<button name="state" value="on">On</button><button name="state" value="off">Off</button></form>{%
endfor %}"""

@app.route("/")
def index():
    return render_template_string(PAGE, leds=leds)

@app.route("/led/<name>", methods=["POST"])
def set_led(name):
    if name not in leds:
        return "unknown LED", 404
    leds[name].on() if request.form["state"] == "on" else leds[name].off()
    return redirect("/")                                  # back to the page

c = app.test_client()
print(c.post("/led/red", data={"state": "on"}).status)
print(" ".join(c.get("/").get_data(as_text=True).split())[:150], "...")
print(c.post("/led/blue", data={"state": "on"}).status)
```

SIMULATED RUN (hardware events and printed output)

```
[  0.00 s] LED(GPIO17) on
302 FOUND
<form method="post" action="/led/red">red: ON <button name="state" value="on">On</button><button
name="state" value="off">Off</button></form> <form me ...
404 NOT FOUND
```

Use **POST** for anything that changes state (switching a relay); browsers, link previews and search engines may fetch GET URLs automatically. Redirecting after a POST stops the action repeating when the page is refreshed.

15.2.3 JSON APIs

API.PY

```
from flask import Flask, jsonify, request
from gpiozero import CPUTemperature, LED

app = Flask(__name__)
cpu, fan = CPUTemperature(), LED(18)

@app.get("/api/status")
def status():
    return jsonify(cpu_temp=round(cpu.temperature, 1), fan=fan.is_lit)

@app.post("/api/fan")
def set_fan():
    data = request.get_json(silent=True) or {}
    if "on" not in data or not isinstance(data["on"], bool):
        return jsonify(error="send {\"on\": true|false}"), 400
    fan.value = data["on"]
    return jsonify(fan=fan.is_lit)

c = app.test_client()
print(c.get("/api/status").get_json())
print(c.post("/api/fan", json={"on": True}).get_json())
r = c.post("/api/fan", json={"speed": 3}); print(r.status_code, r.get_json())
print(c.get("/api/status").get_json())
```

OUTPUT

```
{'cpu_temp': 52.3, 'fan': False}
{'fan': True}
400 {'error': 'send {"on": true|false}'}
{'cpu_temp': 52.3, 'fan': True}
```

A **JSON API** returns data instead of HTML, so any client (a JavaScript dashboard, a phone app, another Pi, Home Assistant) can use it. Validate every input and return a 4xx code with a clear error message for bad requests.

15.2.4 Live updates with JavaScript fetch or WebSockets

TEMPLATES/INDEX.HTML: POLLING THE JSON API

```
<h2>CPU: <span id="t">--</span> °C</h2>
<button onclick="toggle()">Fan</button>
<script>
async function refresh() {
  const r = await fetch("/api/status");          // ask the Pi for fresh data
  const d = await r.json();
  document.getElementById("t").textContent = d.cpu_temp;
}
async function toggle() {
  await fetch("/api/fan", {method: "POST", headers: {"Content-Type": "application/json"},
                           body: JSON.stringify({on: true})});
}
setInterval(refresh, 2000);                      // poll every 2 s
refresh();
</script>
```

Polling with fetch every few seconds is simple and fine for a few users. For instant updates (many readings per second), a **WebSocket** keeps one connection open so the server can push data (Flask-SocketIO, or FastAPI's built-in WebSockets). **Server-Sent Events** are a simpler one-way alternative.

15.3 FastAPI as an alternative

FastAPI builds JSON APIs from Python type hints: it validates input automatically, generates interactive documentation at `/docs`, supports async and WebSockets, and runs on the **uvicorn** server (`uvicorn main:app --host 0.0.0.0`).

MAIN.PY (FASTAPI VERSION OF THE SAME API)

```
from fastapi import FastAPI
from pydantic import BaseModel
app = FastAPI()

class Fan(BaseModel):
    on: bool                          # wrong type -> automatic 422 error

@app.get("/api/status")
def status():
    return {"cpu_temp": 47.2}

@app.post("/api/fan")
def set_fan(f: Fan):
    return {"fan": f.on}
```

	Flask	FastAPI
Style	simple, flexible, huge ecosystem	type hints, automatic validation and docs
Best for	HTML pages + small APIs	JSON APIs, async, WebSockets
Server	Gunicorn (WSGI)	Uvicorn (ASGI)

15.4 Web servers

15.4.1 Nginx and Apache

Flask's built-in server is for development only: single-threaded, not hardened. In production a real web server faces the network. **Nginx** (light, fast, the usual choice on a Pi) and **Apache** serve static files (HTML, images) directly and pass dynamic requests to your app. `sudo apt install nginx`, then browse to the Pi to see its welcome page.

15.4.2 Reverse proxy to a Python app

PRODUCTION STACK: ONLY NGINX IS REACHABLE FROM THE NETWORK

/ETC/NGINX/SITES-AVAILABLE/DASHBOARD

```
server {
    listen 80;
    server_name labpi.local;

    location /static/ {
        alias /home/pi/dashboard/static/;      # Nginx serves these directly
    }
    location / {
        proxy_pass http://127.0.0.1:8000;      # everything else -> Gunicorn
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

ENABLING THE SITE

```
sudo ln -s /etc/nginx/sites-available/dashboard /etc/nginx/sites-enabled/
sudo rm /etc/nginx/sites-enabled/default
sudo nginx -t                # test the configuration
sudo systemctl reload nginx
```

15.4.3 Running the app as a systemd service with Gunicorn

/ETC/SYSTEMD/SYSTEM/DASHBOARD.SERVICE

```
[Unit]
Description=Sensor dashboard (Gunicorn)
After=network.target

[Service]
User=pi
WorkingDirectory=/home/pi/dashboard
ExecStart=/home/pi/dashboard/env/bin/gunicorn --workers 2 --bind 127.0.0.1:8000 app:app
Restart=always

[Install]
WantedBy=multi-user.target
```

`app:app` means "module app.py, object app". A common rule is 2 × CPU cores + 1 workers for CPU-light apps, but with GPIO keep **one** worker (or one process owning the hardware), because each worker is a separate process and two processes cannot claim the same pin.

WATCH OUT
Gunicorn with several workers runs your app several times. Hardware objects (LED(17)) created at import would be claimed by each worker and fail with "pin already in use". Use --workers 1 with threads (--threads 4), or move hardware control into one separate service that the web app talks to.

15.5 Dashboards

15.5.1 Charts with Chart.js

A LIVE LINE CHART FED BY A JSON ENDPOINT

```
<canvas id="c"></canvas>
<script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.1/dist/chart.umd.min.js"></script>
<script>
const chart = new Chart(document.getElementById("c"), {
  type: "line",
  data: {labels: [], datasets: [{label: "Temperature °C", data: [], borderColor: "#C9973A"}]}
});
async function update() {
  const d = await (await fetch("/api/history?n=60")).json();   // [{t:"09:00", v:24.1}, ...]
  chart.data.labels = d.map(p => p.t);
  chart.data.datasets[0].data = d.map(p => p.v);
  chart.update();
}
setInterval(update, 5000); update();
</script>
```

15.5.2 Node-RED dashboards

Node-RED is a browser-based visual tool: drag nodes (GPIO in, MQTT, function, chart, gauge) and wire them into flows; the dashboard nodes build a mobile-friendly UI with no HTML. Install with the official script (`bash <(curl -sL https://github.com/node-red/linux-`

`installers/releases/latest/download/update-nodejs-and-nodered-deb`) and open port 1880. Module 17 uses it with MQTT.

15.5.3 Grafana

Grafana draws professional time-series dashboards from databases such as InfluxDB, Prometheus, SQLite or PostgreSQL, with alerts. Best when you store long histories (modules 16 and 20); it runs on port 3000.

15.6 Accessing the Pi from outside the home network

15.6.1 Port forwarding and its risks

Port forwarding tells the home router to send traffic arriving on a public port to the Pi. It works, but it exposes the Pi to the entire internet, where automated bots scan every address constantly for weak passwords and unpatched software. Many Indian ISPs also use **CGNAT** (no public IPv4 per home), so forwarding may not even be possible.

15.6.2 Secure alternatives: VPN and tunnels (WireGuard, Tailscale)

Option	How it works	Notes
WireGuard VPN	your phone joins your home network through an encrypted tunnel	one UDP port forwarded; fast, modern (module 20)
Tailscale	WireGuard-based mesh; devices reach each other by name	no port forwarding, works behind CGNAT; free for personal use
Cloudflare Tunnel	the Pi makes an outgoing connection to Cloudflare, which publishes the site	good for public dashboards; add Cloudflare Access login
Raspberry Pi Connect	browser access to the Pi desktop/shell	remote admin, not for hosting sites

SHORTCUT TRICK

Rule: **admin pages, cameras and GPIO controls stay private** behind a VPN such as Tailscale or WireGuard. Only pages meant for the public go through a tunnel or reverse proxy, with HTTPS and a login.

PRACTICE CHECKPOINT

Build a Flask dashboard that shows live sensor charts and controls two GPIO outputs, running as a systemd service behind Nginx.

APP.PY (CHECKPOINT SOLUTION CORE)

```
from flask import Flask, jsonify, request, render_template_string
from gpiozero import LED, CPUTemperature
from collections import deque
from datetime import datetime
import threading, time

app = Flask(__name__)
outputs = {"lamp": LED(17), "fan": LED(27)}
cpu = CPUTemperature()
history = deque(maxlen=120)                     # last 120 readings

def sampler():                                   # background thread: one reading per 5 s
    while True:
        history.append({"t": datetime.now().strftime("%H:%M:%S"), "v": round(cpu.temperature, 1)})
        time.sleep(5)

@app.get("/api/history")
def api_history():
    n = request.args.get("n", default=60, type=int)
    return jsonify(list(history)[-n:])

@app.post("/api/out/<name>")
def api_out(name):
    if name not in outputs:
        return jsonify(error="no such output"), 404
    outputs[name].toggle()
    return jsonify({k: v.is_lit for k, v in outputs.items()})

@app.get("/")
def index():
    return render_template_string("<h1>Dashboard</h1><!-- Chart.js page from 15.5.1 -->")

if __name__ == "__main__":
    threading.Thread(target=sampler, daemon=True).start()
    time.sleep(16)                               # (book demo: let 4 samples accumulate)
    c = app.test_client()
    print(c.get("/api/history?n=3").get_json())
    print(c.post("/api/out/lamp").get_json())
    print(c.post("/api/out/heater").status_code)
```

OUTPUT

```
[{'t': '09:00:05', 'v': 48.5}, {'t': '09:00:10', 'v': 51.2}, {'t': '09:00:15', 'v': 49.9}]
{'fan': False, 'lamp': True}
404
```

15.7 Practice questions

15.7.1 Basic level

Q1 Which HTTP method should be used to switch a relay on?

(a) POST **(b)** GET **(c)** HEAD **(d)** OPTIONS

Q2 What status code means "not found"?

(a) 200 **(b)** 500 **(c)** 404 **(d)** 301

Q3 What does @app.route("/") do in Flask?

(a) starts the server
(b) creates a template
(c) opens port 80
(d) connects the URL path / to the function below it

Q4 Why run app.run(host="0.0.0.0")?

(a) to make it faster
(b) to use HTTPS
(c) to hide the server
(d) so other devices on the network can connect

Q5 What format do most web APIs return?

(a) CSV only **(b)** PDF **(c)** JSON **(d)** WAV

Q6 Which web server is the usual reverse proxy on a Pi?

(a) Nginx **(b)** Kodi **(c)** Mosquitto **(d)** Vosk

Q7 What does Gunicorn do?

(a) draws charts
(b) encrypts traffic
(c) reads GPIO
(d) runs a Python (WSGI) web app with several workers in production

Q8 Which tool builds flows and dashboards by wiring nodes in a browser?

(a) Gunicorn **(b)** Node-RED **(c)** Nginx **(d)** Jinja2

15.7.2 Intermediate level

Q9 What does {{ temp }} mean in a Jinja2 template?

(a) a comment
(b) insert the value of the variable temp (HTML-escaped)
(c) a JavaScript function
(d) a CSS rule

Q10 Why redirect after handling a form POST?

(a) to log the user out

(b) to make the page load faster
(c) HTML requires it
(d) so refreshing the page does not repeat the action

Q11 Why should Flask's development server not be used in production?

(a) it is single-threaded and not hardened for the internet
(b) it cannot serve HTML
(c) it only works on Windows
(d) it needs a GPU

Q12 What does proxy_pass http://127.0.0.1:8000 in Nginx do?

(a) blocks port 8000
(b) serves files from a folder
(c) forwards requests to the app server on port 8000
(d) creates a VPN

Q13 Which update method lets the server push data instantly to the browser?

(a) polling every 10 minutes
(b) printing to the terminal
(c) cron
(d) WebSockets

Q14 What does FastAPI generate automatically at /docs?

(a) a database
(b) interactive API documentation
(c) an SSL certificate
(d) a systemd service

Q15 Why is port forwarding a dashboard risky?

(a) it exposes the Pi to bots scanning the whole internet
(b) it slows the Wi-Fi
(c) it uses more electricity
(d) it disables HTTPS

Q16 Which option works behind CGNAT with no port forwarding?

(a) a static IP from the router
(b) port forwarding to 443
(c) Tailscale
(d) mDNS

Q17 A dashboard polls a 200-byte JSON endpoint every 2 s. How much data does one client use per hour (bodies only)?

(a) 360 kB
(b) 36 kB
(c) 3.6 MB
(d) 720 kB

15.7.3 Advanced level

Q18 Gunicorn runs 3 workers and each creates LED(17) at start-up. What happens?

(a) the LED is three times brighter
(b) the second worker fails because the pin is already claimed
(c) all three share the pin safely
(d) Nginx merges them

Q19 Write a Flask route /api/convert that takes ?c=… and returns JSON with Celsius and Fahrenheit, or a 400 error if c is missing or not a number. Test it with c=25, c=abc and no parameter.

Q20 Make a Flask page that lists three sensors from a Python list of dicts in an HTML table using a Jinja2 for-loop, and print the rendered rows.

Q21 Add simple protection to a Flask API: require the header X-API-Key to equal a secret read from an environment variable, returning 401 otherwise.

Q22 Keep the last 5 temperature readings in a deque and expose /api/stats returning min, max and average. Add readings 21, 23, 22, 26, 24, 25 and query it.

Q23 Which command checks an Nginx configuration for errors before reloading?

(a) sudo nginx --run
(b) systemctl test nginx
(c) nginx -check-all
(d) sudo nginx -t

Q24 Which order of layers is right for a production Pi dashboard reachable from your phone anywhere?

(a) phone → port 5000 forwarded → Flask dev server
(b) phone → Gunicorn on port 80 → Nginx
(c) phone → Tailscale VPN → Nginx → Gunicorn (1 worker) → Flask app → GPIO
(d) phone → SSH → Kodi

Q25 A Grafana dashboard is best suited to

(a) switching a single LED
(b) serving static HTML only
(c) long time-series histories from a database with alerts
(d) replacing SSH

15.8 Solutions

Q1 **Answer: (a) POST**

Use **POST** for actions that change state.

Q2 **Answer: (c) 404**

404.

Q3 **Answer: (d) connects the URL path / to the function below it**

A route maps a path to a view function.

Q4 **Answer: (d) so other devices on the network can connect**

The default 127.0.0.1 accepts only local connections.

Q5 **Answer: (c) JSON**

JSON.

Q6 **Answer: (a) Nginx**

Nginx.

Q7 **Answer: (d) runs a Python (WSGI) web app with several workers in production**

A production **WSGI server**.

Q8 **Answer: (b) Node-RED**

Node-RED.

Q9 **Answer: (b) insert the value of the variable temp (HTML-escaped)**

Jinja2 substitution.

Q10 **Answer: (d) so refreshing the page does not repeat the action**

Post/Redirect/Get pattern.

Q11 **Answer: (a) it is single-threaded and not hardened for the internet**

Use Gunicorn behind Nginx.

Q12 **Answer: (c) forwards requests to the app server on port 8000**

Reverse proxy to Gunicorn.

Q13 **Answer: (d) WebSockets**

A WebSocket keeps a two-way connection open.

Q14 **Answer: (b) interactive API documentation**

Swagger UI from your type hints.

Q15 **Answer: (a) it exposes the Pi to bots scanning the whole internet**

Prefer a VPN or tunnel.

Q16 **Answer: (c) Tailscale**

Tailscale connects outward and meshes devices.

Q17 **Answer: (a) 360 kB**

3600 ÷ 2 = 1800 requests × 200 B = **360 000 B**.

Q18 **Answer: (b) the second worker fails because the pin is already claimed**

Each worker is a separate process; use one worker for hardware.

Q19 **Answer: Solution program**

request.args holds query parameters; a missing key raises KeyError and bad text raises ValueError; both become 400.

```
from flask import Flask, jsonify, request
app = Flask(__name__)

@app.get("/api/convert")
def convert():
    try:
        c = float(request.args["c"])
    except (KeyError, ValueError):
        return jsonify(error="give ?c=<number>"), 400
    return jsonify(c=c, f=round(c * 9 / 5 + 32, 1))

client = app.test_client()
for url in ("/api/convert?c=25", "/api/convert?c=abc", "/api/convert"):
    r = client.get(url)
    print(url, r.status_code, r.get_json())
```

OUTPUT

```
/api/convert?c=25 200 {'c': 25.0, 'f': 77.0}
/api/convert?c=abc 400 {'error': 'give ?c=<number>'}
/api/convert 400 {'error': 'give ?c=<number>'}
```

Q20 **Answer: Solution program**

Jinja2 loops over the list; s.name works for dictionary keys too.

```
from flask import Flask, render_template_string
app = Flask(__name__)
SENSORS = [{"name": "Kitchen", "t": 26.1}, {"name": "Bedroom", "t": 23.4}, {"name": "Garage", "t":
31.8}]
T = """<table>{% for s in sensors %}<tr><td>{{ s.name }}</td><td>{{ s.t }}</td>
<td>{{ "HOT" if s.t > 30 else "" }}</td></tr>{% endfor %}</table>"""

@app.get("/")
def page():
    return render_template_string(T, sensors=SENSORS)

html = app.test_client().get("/").get_data(as_text=True)
for row in html.split("<tr>")[1:]:
    print("<tr>" + row.strip().replace(chr(10), ""))
```

OUTPUT

```
<tr><td>Kitchen</td><td>26.1</td><td></td></tr>
<tr><td>Bedroom</td><td>23.4</td><td></td></tr>
<tr><td>Garage</td><td>31.8</td><td>HOT</td></tr></table>
```

Q21 **Answer: Solution program**

before_request runs before every route; returning a response stops the request. Keep the key out of the code (module 21) and use HTTPS so it is not sent in clear text.

```
import os
from flask import Flask, jsonify, request
os.environ.setdefault("API_KEY", "s3cret-demo")       # normally set in the systemd unit
app = Flask(__name__)

@app.before_request
def check_key():
    if request.headers.get("X-API-Key") != os.environ["API_KEY"]:
        return jsonify(error="unauthorised"), 401

@app.get("/api/relay")
def relay():
    return jsonify(relay="off")

c = app.test_client()
print(c.get("/api/relay").status_code)
print(c.get("/api/relay", headers={"X-API-Key": "wrong"}).status_code)
print(c.get("/api/relay", headers={"X-API-Key": "s3cret-demo"}).get_json())
```

OUTPUT

```
401
401
{'relay': 'off'}
```

Q22 **Answer: Solution program**

deque(maxlen=5) drops the oldest value automatically: 21 is gone, leaving 23, 22, 26, 24, 25 (average 24.0).

```
from collections import deque
from flask import Flask, jsonify
app = Flask(__name__)
readings = deque(maxlen=5)
for v in (21, 23, 22, 26, 24, 25):
    readings.append(v)

@app.get("/api/stats")
def stats():
    return jsonify(n=len(readings), min=min(readings), max=max(readings),
                   avg=round(sum(readings) / len(readings), 2))

print(app.test_client().get("/api/stats").get_json())
```

OUTPUT

```
{'avg': 24.0, 'max': 26, 'min': 22, 'n': 5}
```

Q23 **Answer: (d) sudo nginx -t**

`nginx -t` tests the syntax; then reload.

Q24 **Answer: (c) phone → Tailscale VPN → Nginx → Gunicorn (1 worker) → Flask app → GPIO**

Private access, reverse proxy, production server, one hardware owner.

Q25 **Answer: (c) long time-series histories from a database with alerts**

Grafana reads InfluxDB/Prometheus/SQL.

HTTP: GET reads, POST changes; know 200/400/404/500. Flask: routes, Jinja2 templates, forms with POST + redirect, jsonify APIs with validation, fetch polling or WebSockets for live data. Production: Nginx reverse proxy → Gunicorn (one worker if it owns GPIO) under systemd. Charts with Chart.js, Node-RED or Grafana. Reach it from outside through Tailscale/WireGuard, not port forwarding.

MODULE 16 · INTERMEDIATE

Databases and data logging

Minimum duration **2 days** | Level **Intermediate**

Storing data properly lets projects show trends, history and alerts. This module logs to CSV and JSON files with rotation, uses SQLite from Python (tables, inserts, queries, aggregates and indexes), introduces MariaDB/MySQL and PostgreSQL servers, stores time series in InfluxDB and graphs them in Grafana, sends data to cloud services, and protects the SD card by reducing writes or moving heavy logging to an SSD, followed by 25 solved questions and programs. Every SQL example was run with SQLite.

IN THIS MODULE

16.1 File-based logging
16.2 SQLite
16.3 Server databases
16.4 Time-series databases
16.5 Cloud data storage
16.6 Protecting the SD card
16.7 Practice questions
16.8 Solutions

16.1 File-based logging

16.1.1 CSV and JSON logs

CSV (module 6) is compact and opens in any spreadsheet, but every row must have the same columns. **JSON Lines** (one JSON object per line, .jsonl) allows different fields per record and nested data, and appending a line never breaks earlier lines.

JSONL_LOG.PY

```
import json, time
from datetime import datetime
from gpiozero import CPUTemperature

cpu = CPUTemperature()
with open("events.jsonl", "a") as f:
    for i in range(3):
        rec = {"ts": datetime.now().isoformat(timespec="seconds"), "cpu": round(cpu.temperature, 1)}
        if i == 2:
            rec["alert"] = {"type": "hot", "limit": 50}      # extra field only when needed
        f.write(json.dumps(rec) + "\n")
        time.sleep(30)

with open("events.jsonl") as f:
    records = [json.loads(line) for line in f]
print(open("events.jsonl").read().strip())
print("alerts:", [r["ts"] for r in records if "alert" in r])
```

OUTPUT

```
{"ts": "2026-03-15T09:00:00", "cpu": 47.1}
{"ts": "2026-03-15T09:00:30", "cpu": 48.9}
{"ts": "2026-03-15T09:01:00", "cpu": 51.3, "alert": {"type": "hot", "limit": 50}}
alerts: ['2026-03-15T09:01:00']
```

16.1.2 Log rotation

A log that grows forever eventually fills the card. **Rotation** starts a new file when the current one is too big (or each day) and deletes the oldest. Python's `logging` module does it for you; system logs are rotated by **logrotate** (/etc/logrotate.d/) and the systemd journal (module 19).

ROTATING_LOG.PY

```
import logging
from logging.handlers import RotatingFileHandler
import os

log = logging.getLogger("sensor")
log.setLevel(logging.INFO)
h = RotatingFileHandler("sensor.log", maxBytes=500, backupCount=2)   # tiny limit for the demo
h.setFormatter(logging.Formatter("%(asctime)s %(levelname)s %(message)s"))
log.addHandler(h)

for i in range(30):
    log.info("reading %d temp=%.1f", i, 20 + i * 0.1)
log.warning("sensor timeout")

for name in sorted(f for f in os.listdir() if f.startswith("sensor.log")):
    print(name, os.path.getsize(name), "bytes")
print("last line:", open("sensor.log").read().strip().splitlines()[-1])
```

OUTPUT

```
sensor.log 147 bytes
sensor.log.1 450 bytes
sensor.log.2 450 bytes
last line: 2026-03-15 09:00:00,000 WARNING sensor timeout
```

Only sensor.log, .1 and .2 exist: older data was discarded automatically. In real use choose maxBytes such as 1–5 MB, or `TimedRotatingFileHandler(when="midnight", backupCount=7)` for one file per day kept for a week.

16.2 SQLite

SQLite is a complete SQL database in a single file, built into Python (`import sqlite3`): no server to install, perfect for one Pi. It handles millions of rows and makes queries such as hourly averages one line of SQL. Inspect a database from the terminal with `sudo apt install sqlite3`, then `sqlite3 data.db`.

16.2.1 Creating tables

TABLE READINGS: ONE ROW PER MEASUREMENT

SQL type	Holds
INTEGER	whole numbers; INTEGER PRIMARY KEY auto-numbers rows
REAL	decimal numbers
TEXT	strings, including ISO timestamps "2026-03-15 09:00:00"
BLOB	raw bytes (small images)

16.2.2 Inserting and querying data from Python

SQLITE_DEMO.PY

```
import sqlite3, random
from datetime import datetime, timedelta

con = sqlite3.connect("data.db")
con.execute("""CREATE TABLE IF NOT EXISTS readings (
                id INTEGER PRIMARY KEY, ts TEXT NOT NULL,
                sensor TEXT NOT NULL, value REAL NOT NULL)""")
con.execute("CREATE INDEX IF NOT EXISTS idx_ts ON readings(ts)")

random.seed(4)
start = datetime(2026, 3, 15, 0, 0)
rows = []
for m in range(0, 24 * 60, 10):                         # one reading every 10 minutes
    t = start + timedelta(minutes=m)
    temp = 22 + 4 * (1 - abs(t.hour - 15) / 12) + random.uniform(-0.3, 0.3)
    rows.append((t.strftime("%Y-%m-%d %H:%M:%S"), "temp", round(temp, 2)))
with con:                                               # one transaction, commits at the end
    con.executemany("INSERT INTO readings (ts, sensor, value) VALUES (?, ?, ?)", rows)

print("rows:", con.execute("SELECT COUNT(*) FROM readings").fetchone()[0])
print("min/max/avg:", con.execute(
    "SELECT MIN(value), MAX(value), ROUND(AVG(value), 2) FROM readings WHERE sensor =
'temp'").fetchone())
print("hourly averages 12:00-15:00:")
for hour, avg, n in con.execute("""
        SELECT strftime('%H', ts) AS h, ROUND(AVG(value), 2), COUNT(*)
        FROM readings WHERE ts BETWEEN '2026-03-15 12:00' AND '2026-03-15 15:59'
        GROUP BY h ORDER BY h"""):
    print(f"  {hour}:00  {avg}  ({n} readings)")
hot = con.execute("SELECT ts, value FROM readings ORDER BY value DESC LIMIT 1").fetchone()
print("hottest:", hot)
con.close()
```

OUTPUT

```
rows: 144
min/max/avg: (20.74, 26.28, 23.82)
hourly averages 12:00-15:00:
  12:00  25.03  (6 readings)
  13:00  25.41  (6 readings)
  14:00  25.69  (6 readings)
  15:00  25.93  (6 readings)
hottest: ('2026-03-15 15:30:00', 26.28)
```

WATCH OUT
Always pass values with **? placeholders**: `execute("... WHERE sensor = ?", (name,))`. Building SQL by pasting strings together (f"... = '{name}'") allows **SQL injection**: a value such as `x'; DROP TABLE readings; --` would run as code.

SQL	Does
CREATE TABLE … IF NOT EXISTS	make a table once
INSERT INTO t (cols) VALUES (?, ?)	add a row
SELECT cols FROM t WHERE … ORDER BY … LIMIT n	read rows
COUNT, MIN, MAX, AVG, SUM	aggregate functions
GROUP BY	one result row per group (per hour, per sensor)
UPDATE t SET col = ? WHERE …	change rows
DELETE FROM t WHERE ts < ?	remove old rows
CREATE INDEX	speeds up WHERE/ORDER BY on that column

SHORTCUT TRICK
Commit in batches. Each commit forces a write to the card: inserting 1000 rows with 1000 commits can take tens of seconds and wears the SD card; one transaction (`with con:` + executemany) takes milliseconds.

16.3 Server databases

16.3.1 MariaDB/MySQL

MariaDB (a drop-in replacement for MySQL) is a database **server**: many clients and devices can read and write at once over the network, with user accounts and permissions. Install with `sudo apt install mariadb-server`, secure it with `sudo mysql_secure_installation`, and connect from Python with `pymysql` or `mysql-connector-python`. The SQL is almost identical to SQLite.

MARIADB FROM PYTHON (NOTE %S PLACEHOLDERS)

```
import pymysql
con = pymysql.connect(host="localhost", user="logger", password=os.environ["DB_PASS"],
database="home")
with con.cursor() as cur:
    cur.execute("INSERT INTO readings (ts, sensor, value) VALUES (NOW(), %s, %s)", ("temp", 24.6))
con.commit()
```

16.3.2 PostgreSQL

PostgreSQL is a powerful open-source server database with strict data types, rich SQL and extensions such as **TimescaleDB** (time series) and PostGIS (maps). Use `psycopg` from Python. Choose it when you need advanced queries or many concurrent writers.

	SQLite	MariaDB	PostgreSQL
Server needed	no (a file)	yes	yes
Many devices writing	one Pi only	yes	yes
Set-up effort	none	medium	medium
Best for	one Pi, simple logging, apps	web apps, several clients	complex data, extensions

16.4 Time-series databases

16.4.1 InfluxDB

Sensor data is a **time series**: values stamped with times, mostly appended and queried by time range. **InfluxDB** is built for this: fast writes, automatic downsampling and **retention policies** (for example keep raw data 7 days and hourly averages for a year). Each point has a **measurement**, **tags** (indexed labels like room=kitchen), **fields** (the values) and a timestamp. Its text format is "line protocol":

INFLUXDB LINE PROTOCOL: MEASUREMENT,TAGS FIELDS TIMESTAMP(NS)

```
climate,room=kitchen,sensor=bme280 temp=24.6,hum=58.2,press=1009.4 1773565200000000000
climate,room=garage,sensor=dht22 temp=31.8,hum=40.1 1773565200000000000
```

LINE_PROTOCOL.PY

```
def line_protocol(measurement, tags, fields, ts_ns):
    t = ",".join(f"{k}={v}" for k, v in sorted(tags.items()))
    f = ",".join(f"{k}={v}" for k, v in fields.items())
    return f"{measurement},{t} {f} {ts_ns}"

print(line_protocol("climate", {"room": "kitchen", "sensor": "bme280"},
                    {"temp": 24.6, "hum": 58.2}, 1773565200 * 10**9))
```

OUTPUT

```
climate,room=kitchen,sensor=bme280 temp=24.6,hum=58.2 1773565200000000000
```

With the official client (`pip install influxdb-client`), write points with `write_api.write(bucket="home", record=Point("climate").tag("room", "kitchen").field("temp", 24.6))`. Telegraf, InfluxDB's agent, can collect Pi system metrics and MQTT data with no code.

16.4.2 Grafana dashboards on InfluxDB

Add InfluxDB as a **data source** in Grafana (http://pi:3000), create a dashboard, and build panels with queries such as "mean of temp grouped by 5 minutes for the last 24 hours, per room". Grafana adds zoom, thresholds (colour turns red above 30 °C) and alert rules that can notify you by email or Telegram.

16.5 Cloud data storage

16.5.1 Google Sheets, Firebase and similar services

Service	Good for	Python access
Google Sheets	sharing small datasets with non-programmers	gspread + a service-account key
Firebase Realtime DB / Firestore	syncing live data to web and mobile apps	firebase-admin
ThingSpeak, Adafruit IO	IoT dashboards with free tiers	simple HTTP or MQTT (module 17)
AWS S3 / Google Cloud Storage	archiving files: images, daily CSVs	boto3 / google-cloud-storage

Cloud storage needs internet and has rate limits, so **buffer locally** (SQLite) and upload in batches; if the network drops, nothing is lost. Never put service-account keys in your code repository.

16.6 Protecting the SD card

16.6.1 Reducing writes

Flash memory wears out after a limited number of write cycles per block, and cheap cards wear fastest. A card can also be corrupted if power is cut while it is writing.

Technique	How
Batch writes	collect readings in memory and write every few minutes, in one transaction
Log less often	one reading per minute is plenty for room temperature
Logs in RAM	mount /tmp or the log folder as tmpfs; log2ram syncs to disk periodically
Volatile journal	Storage=volatile in journald.conf (module 19)
Read-only root	overlay file system (module 19.6.3)
Good card	A2, name brand, high endurance models for cameras

WORKED EXAMPLE

How much does logging write?

One SQLite insert ≈ 4 KB written (a page plus journal). Every second: 4 KB × 86 400 = **346 MB per day**.
Batched once a minute (60 rows per transaction, ~8 KB): 8 KB × 1440 = **11.5 MB per day**, about 30 times less.

16.6.2 Using SSDs for heavy logging

An SSD (USB 3 or NVMe on a Pi 5) has far higher endurance, wear-levelling and speed. Put databases, Docker volumes and camera recordings on it, or boot entirely from it (module 3.6). Keep backups anyway: any storage can fail.

PRACTICE CHECKPOINT

Log sensor data to SQLite every minute for a day, then display hourly averages and min/max values in a Grafana or Flask dashboard.

16.7 Practice questions

16.7.1 Basic level

Q1 Which Python module gives SQLite with no installation?

(a) pymysql (b) psycopg (c) influxdb (d) sqlite3

Q2 Which SQL command adds a row?

(a) SELECT (b) INSERT (c) CREATE (d) GROUP

Q3 Which SQL keyword filters rows?

(a) WHERE (b) FROM (c) ORDER (d) VALUES

Q4 What is SQLite?

(a) a cloud service
(b) a SQL database stored in a single file, with no server
(c) a type of SD card
(d) a web server

Q5 Which database is designed for time-series data?

(a) InfluxDB (b) SQLite (c) Google Sheets (d) MariaDB

Q6 What does log rotation do?

(a) encrypts logs
(b) emails logs
(c) starts new log files and deletes the oldest so logs do not fill the disk
(d) sorts logs

Q7 Which tool draws dashboards from InfluxDB?

(a) Nginx (b) Mosquitto (c) Thonny (d) Grafana

Q8 What does JSON Lines (.jsonl) store on each line?

(a) one CSV column (b) one SQL table
(c) a binary image (d) one complete JSON object

Q9 Which file format allows a different set of fields in each record?

(a) CSV (b) a fixed-width text file
(c) JSON Lines (d) WAV

16.7.2 Intermediate level

Q10 Why use ? placeholders in SQL from Python?

(a) to make queries shorter
(b) SQLite needs them for numbers only
(c) to prevent SQL injection and handle quoting safely
(d) to create indexes

Q11 Which query gives the average temperature per hour?

(a) SELECT strftime('%H', ts), AVG(value) FROM readings GROUP BY 1
(b) SELECT AVG(value) FROM readings
(c) SELECT * FROM readings ORDER BY ts
(d) SELECT COUNT(*) FROM readings

Q12 Why insert many rows in one transaction?

(a) transactions are required by Python
(b) it encrypts the data
(c) it sorts the rows
(d) one commit is much faster and writes the card far less

Q13 What does CREATE INDEX on the ts column speed up?

(a) inserts only
(b) queries that filter or sort by ts
(c) deleting the database
(d) Wi-Fi

Q14 When is MariaDB a better choice than SQLite?

(a) for one script on one Pi
(b) when many devices or apps must write over the network at once
(c) when you have no internet
(d) for storing one CSV file

Q15 In InfluxDB, what are tags?

(a) the measured values
(b) the timestamps
(c) the user passwords
(d) indexed labels such as room=kitchen

Q16 Which reduces SD card wear most?

(a) batching writes and keeping logs in RAM or on an SSD
(b) logging every 10 ms
(c) using a bigger font
(d) disabling Wi-Fi

Q17 Why buffer data locally before uploading to the cloud?

(a) the cloud refuses single values
(b) to make it slower
(c) so nothing is lost when the internet drops, and uploads can be batched
(d) to avoid using SQL

16.7.3 Advanced level

Q18 A sensor logs one 4 KB write every 10 s. How much is written per day?

(a) about 346 MB
(b) about 3.5 MB
(c) about 4 KB
(d) about 34.6 MB

Q19 Readings every 10 minutes for 30 days: how many rows?

(a) 720
(b) 4320
(c) 43 200
(d) 1440

Q20 Using the table below, print the average, minimum and maximum temperature per hour.

Q21 Delete readings older than 10:00 and print how many rows were removed and how many remain.

Q22 Show the danger of SQL injection: build a query with an f-string from the "name" x' OR '1'='1 and compare it with a parameterised query.

Q23 Buffer readings in memory and write them to SQLite in batches of 5 (one transaction each). Simulate 12 readings and print when each batch is written.

Q24 Count how many temperature readings were above 23 °C and for how many minutes, assuming readings are 30 minutes apart.

Q25 Which design suits a weather station that logs 5 sensors every minute for years and shows graphs?

(a) SQLite or InfluxDB on an SSD, batched writes, retention or downsampling, Grafana or Flask charts

(b) one giant CSV rewritten every minute on the SD card

(c) Google Sheets with one API call per reading

(d) print() to the terminal only

16.8 Solutions

Q1 **Answer: (d) sqlite3**

Built in: **sqlite3**.

Q2 **Answer: (b) INSERT**

INSERT INTO ….

Q3 **Answer: (a) WHERE**

WHERE.

Q4 **Answer: (b) a SQL database stored in a single file, with no server**

Serverless, one file.

Q5 **Answer: (a) InfluxDB**

InfluxDB (or TimescaleDB).

Q6 **Answer: (c) starts new log files and deletes the oldest so logs do not fill the disk**

Limits disk use.

Q7 **Answer: (d) Grafana**

Grafana.

Q8 **Answer: (d) one complete JSON object**

One record per line.

Q9 **Answer: (c) JSON Lines**

Each JSON object can have its own keys.

Q10 **Answer: (c) to prevent SQL injection and handle quoting safely**

Values are never treated as SQL code.

Q11 **Answer: (a) SELECT strftime('%H', ts), AVG(value) FROM readings GROUP BY 1**

GROUP BY the hour.

Q12 **Answer: (d) one commit is much faster and writes the card far less**

Batch commits.

Q13 **Answer: (b) queries that filter or sort by ts**

Indexes make lookups fast (with a small insert cost).

Q14 **Answer: (b) when many devices or apps must write over the network at once**

A **server** handles many clients.

Q15 **Answer: (d) indexed labels such as room=kitchen**

Tags are indexed; fields hold values.

Q16 **Answer: (a) batching writes and keeping logs in RAM or on an SSD**

Fewer, larger writes.

Q17 **Answer: (c) so nothing is lost when the internet drops, and uploads can be batched**

Store-and-forward.

Q18 **Answer: (d) about 34.6 MB**

8640 writes × 4 KB = 34 560 KB ≈ **34.6 MB**.

Q19 **Answer: (b) 4320**

144 per day × 30 = **4320**.

Q20 **Answer: Solution program**

GROUP BY the hour string; WHERE keeps only temperature rows.

```
import sqlite3
con = sqlite3.connect(":memory:")
con.execute("CREATE TABLE readings (id INTEGER PRIMARY KEY, ts TEXT, sensor TEXT, value REAL)")
con.executemany("INSERT INTO readings (ts, sensor, value) VALUES (?, ?, ?)", [
    ("2026-03-15 09:00:00", "temp", 22.1), ("2026-03-15 09:30:00", "temp", 22.9),
    ("2026-03-15 10:00:00", "temp", 24.0), ("2026-03-15 10:30:00", "temp", 25.2),
    ("2026-03-15 09:00:00", "hum", 61.0), ("2026-03-15 10:00:00", "hum", 55.5)])

for h, avg, lo, hi in con.execute("""
        SELECT strftime('%H', ts), ROUND(AVG(value), 2), MIN(value), MAX(value)
        FROM readings WHERE sensor = 'temp' GROUP BY 1 ORDER BY 1"""):
    print(f"{h}:00  avg {avg}  min {lo}  max {hi}")
```

OUTPUT

```
09:00  avg 22.5  min 22.1  max 22.9
10:00  avg 24.6  min 24.0  max 25.2
```

Q21 **Answer: Solution program**

ISO timestamps sort correctly as text, so < works. rowcount reports rows affected. This is how a retention policy is done in SQLite.

```
import sqlite3
con = sqlite3.connect(":memory:")
con.execute("CREATE TABLE readings (id INTEGER PRIMARY KEY, ts TEXT, sensor TEXT, value REAL)")
con.executemany("INSERT INTO readings (ts, sensor, value) VALUES (?, ?, ?)", [
    ("2026-03-15 09:00:00", "temp", 22.1), ("2026-03-15 09:30:00", "temp", 22.9),
    ("2026-03-15 10:00:00", "temp", 24.0), ("2026-03-15 10:30:00", "temp", 25.2),
    ("2026-03-15 09:00:00", "hum", 61.0), ("2026-03-15 10:00:00", "hum", 55.5)])

with con:
    cur = con.execute("DELETE FROM readings WHERE ts < ?", ("2026-03-15 10:00:00",))
print("deleted:", cur.rowcount)
print("remaining:", con.execute("SELECT COUNT(*) FROM readings").fetchone()[0])
```

OUTPUT

```
deleted: 3
remaining: 3
```

Q22 **Answer: Solution program**

The pasted quote changes the meaning of the SQL so the condition is always true. With ? the whole string is just a value that matches nothing.

```
import sqlite3
con = sqlite3.connect(":memory:")
con.execute("CREATE TABLE readings (id INTEGER PRIMARY KEY, ts TEXT, sensor TEXT, value REAL)")
con.executemany("INSERT INTO readings (ts, sensor, value) VALUES (?, ?, ?)", [
    ("2026-03-15 09:00:00", "temp", 22.1), ("2026-03-15 09:30:00", "temp", 22.9),
    ("2026-03-15 10:00:00", "temp", 24.0), ("2026-03-15 10:30:00", "temp", 25.2),
    ("2026-03-15 09:00:00", "hum", 61.0), ("2026-03-15 10:00:00", "hum", 55.5)])

name = "x' OR '1'='1"
unsafe = f"SELECT COUNT(*) FROM readings WHERE sensor = '{name}'"
print("unsafe SQL:", unsafe)
print("unsafe result:", con.execute(unsafe).fetchone()[0], "rows (all of them!)")
safe = con.execute("SELECT COUNT(*) FROM readings WHERE sensor = ?", (name,)).fetchone()[0]
print("safe result:", safe, "rows")
```

OUTPUT

```
unsafe SQL: SELECT COUNT(*) FROM readings WHERE sensor = 'x' OR '1'='1'
unsafe result: 6 rows (all of them!)
safe result: 0 rows
```

Q23 **Answer: Solution program**

Two full batches of 5 and a final flush of 2. Always flush at exit (try/finally) so the last readings are not lost.

```
import sqlite3
con = sqlite3.connect(":memory:")
con.execute("CREATE TABLE r (n INTEGER, v REAL)")
buffer = []
def flush():
    if buffer:
        with con:
            con.executemany("INSERT INTO r VALUES (?, ?)", buffer)
        print(f"wrote batch of {len(buffer)} (total {con.execute('SELECT COUNT(*) FROM
r').fetchone()[0]})")
        buffer.clear()
for n in range(12):
    buffer.append((n, 20 + n / 10))
    if len(buffer) == 5:
        flush()
flush()                     # write the remainder at shutdown
```

OUTPUT

```
wrote batch of 5 (total 5)
wrote batch of 5 (total 10)
wrote batch of 2 (total 12)
```

Q24 **Answer: Solution program**

COUNT with two conditions joined by AND; each reading represents 30 minutes.

```
import sqlite3
con = sqlite3.connect(":memory:")
con.execute("CREATE TABLE readings (id INTEGER PRIMARY KEY, ts TEXT, sensor TEXT, value REAL)")
con.executemany("INSERT INTO readings (ts, sensor, value) VALUES (?, ?, ?)", [
    ("2026-03-15 09:00:00", "temp", 22.1), ("2026-03-15 09:30:00", "temp", 22.9),
    ("2026-03-15 10:00:00", "temp", 24.0), ("2026-03-15 10:30:00", "temp", 25.2),
    ("2026-03-15 09:00:00", "hum", 61.0), ("2026-03-15 10:00:00", "hum", 55.5)])

n = con.execute("SELECT COUNT(*) FROM readings WHERE sensor='temp' AND value > 23").fetchone()[0]
print(f"{n} readings above 23 C = about {n * 30} minutes")
```

OUTPUT

```
2 readings above 23 C = about 60 minutes
```

Q25 **Answer: (a) SQLite or InfluxDB on an SSD, batched writes, retention or downsampling, Grafana or Flask charts**

Right store, batched, bounded size, good visualisation.

Log to CSV/JSON Lines with rotation, or better, to SQLite: CREATE TABLE IF NOT EXISTS, ? placeholders, batched transactions, indexes on time, GROUP BY for hourly stats, DELETE old rows. Use MariaDB/PostgreSQL for many clients, InfluxDB + Grafana for long time series, and the cloud with local buffering. Protect the SD card: fewer, larger writes, logs in RAM, SSD for heavy data.

MODULE 17 · INTERMEDIATE

IoT with MQTT and cloud platforms

Minimum duration **2 days** | Level **Intermediate**

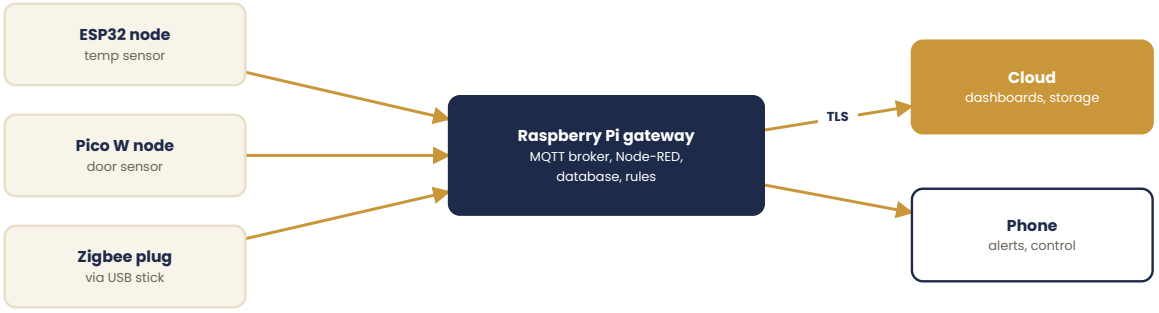
This module connects Pi projects to each other and to the cloud using industry-standard IoT protocols: devices, gateways and cloud; publish/subscribe versus request/response; MQTT brokers, topics, wildcards, QoS and retained messages; the Mosquitto broker on the Pi; paho-mqtt in Python; ESP32 and Pico W nodes; authentication and TLS; Node-RED flows and dashboards; cloud platforms (ThingSpeak, Blynk, Adafruit IO, AWS IoT Core, Azure IoT Hub); and email, Telegram and push notifications, followed by 25 solved questions and programs. MQTT programs ran against a simulated in-process broker.

IN THIS MODULE

17.1 IoT fundamentals
17.2 MQTT
17.3 Node-RED
17.4 Cloud IoT platforms
17.5 Notifications
17.6 Practice questions
17.7 Solutions

17.1 IoT fundamentals

17.1.1 Devices, gateways and cloud



TYPICAL IOT ARCHITECTURE: DEVICES → GATEWAY → CLOUD AND USERS

Devices (sensor nodes, actuators) are small and often battery-powered. A **gateway** such as a Pi collects their data, translates protocols, stores data, runs local rules and forwards selected data to the **cloud**. Local processing keeps things working when the internet is down and keeps private data at home ("edge computing").

17.1.2 Publish/subscribe vs request/response

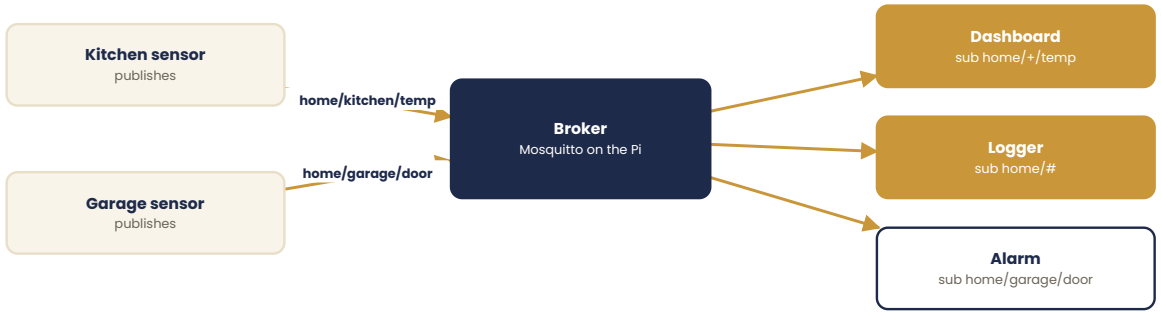
	Request/response (HTTP)	Publish/subscribe (MQTT)
Pattern	client asks, server answers	publishers send to topics; the broker forwards to all subscribers

	Request/response (HTTP)	Publish/subscribe (MQTT)
Coupling	client must know the server and ask	senders and receivers never know each other
Updates	poll repeatedly	pushed instantly when they happen
Overhead	large headers per request	tiny (2-byte header), one long connection
Best for	web pages, APIs, commands with answers	telemetry from many devices, events, low bandwidth

17.2 MQTT

17.2.1 Brokers, topics, QoS and retained messages

MQTT (Message Queuing Telemetry Transport) is the standard IoT messaging protocol. A central **broker** receives every message. A message has a **topic**, a hierarchical string such as `home/livingroom/temp`, and a **payload** (text, JSON or bytes). Clients **subscribe** to topics, optionally with wildcards.



PUBLISH/SUBSCRIBE THROUGH A BROKER

Wildcard	Matches	Example
+	exactly one level	home/+/temp matches home/kitchen/temp, not home/kitchen/fridge/temp
#	any number of levels (last character only)	home/# matches everything under home

QoS	Guarantee	Use
0	at most once ("fire and forget")	frequent sensor readings: losing one is fine
1	at least once (acknowledged; duplicates possible)	commands, alerts
2	exactly once (4-step handshake)	billing, counters where duplicates matter

A **retained** message is stored by the broker and sent immediately to any new subscriber: ideal for "last known state" such as `home/heater/state = on`. A **Last Will** message is published by the broker if a client disconnects unexpectedly (for example `nodes/esp1/status = offline`).

SHORTCUT TRICK

Design topics from general to specific: **site/room/device/measurement** (home/kitchen/bme280/temp). Use lower case, no spaces, no leading slash, and put commands under a separate branch such as **home/kitchen/light/set**.

17.2.2 Installing Mosquitto on the Pi

INSTALL AND TEST

```
sudo apt install mosquitto mosquitto-clients
sudo systemctl enable --now mosquitto
mosquitto_sub -h localhost -t "home/#" -v          # terminal 1: watch everything
mosquitto_pub -h localhost -t home/test -m "hello"  # terminal 2: send
```

SINCE MOSQUITTO 2.0 REMOTE CLIENTS NEED AN EXPLICIT LISTENER (AND SHOULD NEED A PASSWORD)

```
# /etc/mosquitto/conf.d/local.conf
listener 1883
allow_anonymous false
password_file /etc/mosquitto/passwd
```

17.2.3 paho-mqtt in Python

MQTT_PUBLISH.PY

```
import paho.mqtt.client as mqtt
import json, time
from gpiozero import CPUTemperature

cpu = CPUTemperature()
client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2, client_id="labpi")

def on_connect(client, userdata, flags, reason_code, properties):
    print("connected:", reason_code)

client.on_connect = on_connect
client.will_set("nodes/labpi/status", "offline", retain=True)
client.connect("localhost", 1883)
client.loop_start()                              # network thread handles traffic
client.publish("nodes/labpi/status", "online", retain=True)
for _ in range(3):
    payload = json.dumps({"cpu": round(cpu.temperature, 1), "ts": int(time.time())})
    client.publish("home/labpi/cpu", payload, qos=0)
    time.sleep(10)
client.loop_stop(); client.disconnect()
```

SIMULATED RUN (hardware events and printed output)

```
[  0.00 s] MQTT labpi connect localhost:1883 -> accepted
connected: Success
[  0.00 s] MQTT publish nodes/labpi/status online [retained]
[  0.00 s] MQTT publish home/labpi/cpu {"cpu": 47.4, "ts": 1773565200}
[ 10.00 s] MQTT publish home/labpi/cpu {"cpu": 48.1, "ts": 1773565210}
[ 20.00 s] MQTT publish home/labpi/cpu {"cpu": 49.0, "ts": 1773565220}
[ 30.00 s] MQTT labpi disconnected
```

MQTT_LED.PY

```
import paho.mqtt.client as mqtt
from gpiozero import LED

led = LED(17)
def on_connect(client, userdata, flags, rc, properties):
    client.subscribe("home/labpi/led/set", qos=1)   # subscribe here: re-done after reconnects

def on_message(client, userdata, msg):
    cmd = msg.payload.decode().strip().lower()
    print(f"{msg.topic} -> {cmd}")
    if cmd in ("on", "off"):
        led.on() if cmd == "on" else led.off()
        client.publish("home/labpi/led/state", cmd, retain=True)

client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
client.on_connect, client.on_message = on_connect, on_message
client.connect("localhost")
client.loop_forever()
```

SIMULATED RUN (hardware events and printed output)

```
[  0.00 s] MQTT client connect localhost:1883 -> accepted
[  0.00 s] MQTT client subscribed home/labpi/led/set (QoS 1)
home/labpi/led/set -> on
[  2.00 s] LED(GPIO17) on
[  2.00 s] MQTT publish home/labpi/led/state on [retained]
home/labpi/led/set -> blink
home/labpi/led/set -> off
[  8.01 s] LED(GPIO17) off
[  8.01 s] MQTT publish home/labpi/led/state off [retained]
```

Subscribe inside `on_connect` so subscriptions are restored automatically after a reconnection. Publish the new **state** on a separate topic (retained), so dashboards always show reality, not just the last command.

17.2.4 Connecting ESP32 or Pico W nodes to the Pi broker

Microcontroller nodes use small MQTT libraries: **PubSubClient** on ESP32/Arduino, **umqtt.simple** in MicroPython on the Pico W (module 24). They connect to the Pi by IP address or hostname, publish readings, and sleep between readings to save battery.

PICO W NODE (REFERENCE)

```
from umqtt.simple import MQTTClient          # MicroPython on a Pico W
import network, time, machine
wlan = network.WLAN(network.STA_IF); wlan.active(True); wlan.connect("HomeNet", "password")
while not wlan.isconnected(): time.sleep(0.5)
c = MQTTClient("pico1", "192.168.1.42", user="sensor", password="secret")
c.connect()
adc = machine.ADC(4)                                  # internal temperature sensor
t = 27 - (adc.read_u16() * 3.3 / 65535 - 0.706) / 0.001721
c.publish(b"home/pico1/temp", b"%.1f" % t)
```

17.2.5 Username, password and TLS

ADDING A USER

```
sudo mosquitto_passwd -c /etc/mosquitto/passwd sensor    # create user (asks for password)
sudo systemctl restart mosquitto
```

AUTH_TEST.PY

```
import paho.mqtt.client as mqtt
for user, pw in (("sensor", "wrong"), ("sensor", "s3cret")):
    c = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
    c.username_pw_set(user, pw)
    c.on_connect = lambda cl, u, f, rc, p: print(f"{user}/{pw}: {rc}")
    c.connect("192.168.1.42", 1883)
    c.loop_start()
```

OUTPUT

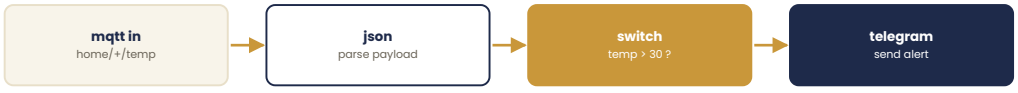
```
sensor/wrong: Not authorized
sensor/s3cret: Success
```

Passwords alone travel in clear text. For anything beyond your own LAN enable **TLS** on port **8883** (`listener 8883`, `certfile`, `keyfile` in mosquitto.conf; `client.tls_set(ca_certs="ca.crt")` in paho). Cloud brokers require TLS, and AWS IoT Core uses client certificates instead of passwords.

17.3 Node-RED

17.3.1 Flows, nodes and dashboards

In **Node-RED** (http://pi:1880) you drag **nodes** onto a canvas and wire them into **flows**. Messages (`msg.payload`) travel along the wires. Common nodes: inject (timer), mqtt in/out, function (JavaScript), switch (if/else), change, delay, debug, and the dashboard nodes gauge, chart, button and text (install @flowfuse/node-red-dashboard).



A NODE-RED FLOW: ALERT WHEN ANY ROOM IS ABOVE 30 °C

NODE-RED FUNCTION NODE (JAVASCRIPT)

```
// function node: convert to °F and add a status
msg.payload = { c: msg.payload.temp, f: msg.payload.temp * 9 / 5 + 32,
                status: msg.payload.temp > 30 ? "HOT" : "ok" };
return msg;
```

17.3.2 GPIO and MQTT nodes

The **rpi-gpio in/out** nodes (on the Pi 5 use the node-red-node-pi-gpiod or the "rpi gpio lgpio" nodes) read buttons and switch outputs directly from a flow. The **mqtt in/out** nodes connect to the broker, so a dashboard button can publish to home/labpi/led/set and the Python program above reacts.

17.4 Cloud IoT platforms

17.4.1 ThingSpeak, Blynk, Adafruit IO

Platform	What it gives	How the Pi sends data
ThingSpeak (MathWorks)	channels with 8 fields, charts, MATLAB analysis; free tier ~ one update per 15 s	HTTP GET/POST with a write API key, or MQTT
Blynk	drag-and-drop phone app dashboards	Blynk library or HTTP API
Adafruit IO	feeds, dashboards, triggers; free tier limits per minute	MQTT (io.adafruit.com:8883) or REST

THINGSPEAK.PY

```
import requests, time, os
from gpiozero import CPUTemperature

KEY = os.environ.get("TS_KEY", "DEMOKEY123")              # never hard-code real keys
cpu = CPUTemperature()
for _ in range(2):
    r = requests.get("https://api.thingspeak.com/update",
                     params={"api_key": KEY, "field1": round(cpu.temperature, 1)}, timeout=10)
    print("status", r.status_code, "entry id", r.text)
    time.sleep(20)                                        # free tier: at most one update per 15 s
```

SIMULATED RUN (hardware events and printed output)

```
[  0.15 s] HTTP GET https://api.thingspeak.com/update {'api_key': 'DEMOKEY123', 'field1': 48.3}
status 200 entry id 41
[ 20.30 s] HTTP GET https://api.thingspeak.com/update {'api_key': 'DEMOKEY123', 'field1': 49.1}
status 200 entry id 42
```

17.4.2 AWS IoT Core and Azure IoT Hub overview

Enterprise platforms for fleets of thousands of devices. **AWS IoT Core**: devices connect by MQTT over TLS with X.509 **client certificates**; "device shadows" store desired and reported state; rules route messages to databases, Lambda functions and analytics. **Azure IoT Hub** offers the same ideas (device twins, per-device identities, MQTT/AMQP) and connects to Azure services. Both charge per message, so aggregate data on the Pi first.

17.5 Notifications

17.5.1 Email alerts

EMAIL_ALERT.PY

```
import smtplib, os
from email.message import EmailMessage

def send_alert(subject, body):
    msg = EmailMessage()
    msg["Subject"], msg["From"], msg["To"] = subject, "labpi@example.com", "lohit@example.com"
    msg.set_content(body)
    with smtplib.SMTP_SSL("smtp.gmail.com", 465) as s:
        s.login("labpi@example.com", os.environ.get("MAIL_APP_PASSWORD", "app-password"))
        s.send_message(msg)

send_alert("Garage HOT: 41.2 C", "Garage temperature crossed 40 C at 14:05.")
print("alert sent")
```

SIMULATED RUN (hardware events and printed output)

```
[  0.30 s] SMTP login labpi@example.com
[  0.30 s] SMTP sent "Garage HOT: 41.2 C" to lohit@example.com
alert sent
```

Gmail and most providers need an **app password** (with 2-step verification on), not your normal password. Rate-limit alerts: send once when a threshold is crossed, not every minute while it stays crossed.

17.5.2 Telegram bots

Create a bot by chatting with **@BotFather** in Telegram (it gives a token), send your bot a message, then find your chat id. Sending is a single HTTPS request; bots can also receive commands (python-telegram-bot library).

TELEGRAM_ALERT.PY

```
import requests, os
TOKEN = os.environ.get("TG_TOKEN", "123456:ABC-demo")
CHAT_ID = os.environ.get("TG_CHAT", "987654321")

def telegram(text):
    r = requests.post(f"https://api.telegram.org/bot{TOKEN}/sendMessage",
                      json={"chat_id": CHAT_ID, "text": text}, timeout=10)
    return r.json()["ok"]

print("delivered:", telegram("Motion detected at the front door"))
```

SIMULATED RUN (hardware events and printed output)

```
[  0.15 s] HTTP POST https://api.telegram.org/bot<TOKEN>/sendMessage {'chat_id': '987654321',
'text': 'Motion detected at the front door'}
delivered: True
```

17.5.3 Push notifications

Services such as **ntfy** (open source, self-hostable: `requests.post("https://ntfy.sh/my-secret-topic", data="Door opened")` and subscribe in the ntfy phone app), Pushover and Home Assistant's companion app deliver native phone notifications. Use long random topic names on public ntfy servers, because anyone who knows the topic can read it.

PRACTICE CHECKPOINT

Build an MQTT network where two sensor nodes publish data to a Mosquitto broker on the Pi, Node-RED displays it, and a Telegram alert fires when a threshold is crossed.

ALERT_SERVICE.PY (CHECKPOINT SOLUTION)

```
import paho.mqtt.client as mqtt, requests, json, os

LIMIT = 30.0
alerted = {}                                # room -> already alerted?
def telegram(text):
    requests.post(f"https://api.telegram.org/bot{os.environ.get('TG_TOKEN', 'x')}/sendMessage",
                  json={"chat_id": os.environ.get("TG_CHAT", "1"), "text": text}, timeout=10)

def on_connect(c, u, f, rc, p):
    c.subscribe("home/+/temp")

def on_message(c, u, msg):
    room = msg.topic.split("/")[1]
    t = json.loads(msg.payload)["t"]
    print(f"{room:8} {t:5.1f} C")
    if t > LIMIT and not alerted.get(room):
        telegram(f"{room} is {t} C (limit {LIMIT})"); alerted[room] = True
    elif t < LIMIT - 1:                     # 1 degree hysteresis before re-arming
        alerted[room] = False

c = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
c.on_connect, c.on_message = on_connect, on_message
c.connect("localhost"); c.loop_forever()
```

SIMULATED RUN (hardware events and printed output)

```
[  0.00 s] MQTT client connect localhost:1883 -> accepted
[  0.00 s] MQTT client subscribed home/+/temp (QoS 0)
kitchen   28.5 C
garage    30.4 C
[  2.15 s] HTTP POST https://api.telegram.org/bot<TOKEN>/sendMessage {'chat_id': '1', 'text':
'garage is 30.4 C (limit 30.0)'}
garage    31.2 C
kitchen   29.0 C
garage    28.7 C
garage    30.9 C
[  6.16 s] HTTP POST https://api.telegram.org/bot<TOKEN>/sendMessage {'chat_id': '1', 'text':
'garage is 30.9 C (limit 30.0)'}
```

The garage crossed 30 °C twice but alerted only on the first crossing of each episode: the **hysteresis** (re-arm only after dropping below 29 °C) prevents a flood of alerts when a value hovers around the limit.

17.6 Practice questions

17.6.1 Basic level

Q1 What does MQTT use to route messages?

(a) topics through a broker
(b) IP addresses only
(c) Bluetooth pairing
(d) email addresses

Q2 Which popular MQTT broker runs on a Pi?

(a) Nginx (b) Mosquitto (c) Grafana (d) Kodi

Q3 What is the default unencrypted MQTT port?

(a) 80 (b) 8883 (c) 22 (d) 1883

Q4 Which wildcard matches exactly one topic level?

(a) # (b) * (c) + (d) ?

Q5 Which QoS level is "fire and forget"?

(a) 1 (b) 2 (c) 0 (d) 3

Q6 Which Python library is the standard MQTT client?

(a) smbus2 (b) flask (c) picamera2 (d) paho-mqtt

Q7 Which tool lets you wire IoT flows visually in a browser?

(a) Mosquitto (b) Node-RED (c) Vosk (d) rpicam

Q8 Which app do you message to create a Telegram bot?

(a) @BotFather (b) @MQTTBot (c) @Grafana (d) @PiBot

Q9 Which command-line tool subscribes to topics for testing?

(a) mosquitto_pub (b) mqtt-ls (c) mosquitto_sub (d) curl -mqtt

17.6.2 Intermediate level

Q10 What is a retained message?

(a) a message that failed
(b) the last message on a topic, stored by the broker and sent to new subscribers
(c) an encrypted message
(d) a message with QoS 2

Q11 What is a Last Will message for?

(a) the broker publishes it if a client disconnects unexpectedly
(b) to delete old messages
(c) to log in
(d) to encrypt payloads

Q12 Does home/+/temp match home/kitchen/fridge/temp?

(a) yes (b) only with QoS 2

(c) no, + matches only one level (d) only if retained

Q13 Why subscribe inside on_connect?

(a) it is faster
(b) paho forbids subscribing elsewhere
(c) to enable TLS
(d) so subscriptions are restored automatically after a reconnect

Q14 Which port is MQTT over TLS?

(a) 1883 (b) 443 only (c) 8080 (d) 8883

Q15 Which is an advantage of pub/sub over HTTP polling for sensor data?

(a) it needs no network
(b) it always uses QoS 2
(c) updates are pushed instantly with tiny overhead
(d) it cannot lose data

Q16 AWS IoT Core authenticates devices mainly with

(a) X.509 client certificates over TLS (b) MAC addresses
(c) no authentication (d) SSH keys

Q17 Why add hysteresis to a temperature alert?

(a) to make readings more accurate
(b) to save broker memory
(c) MQTT requires it
(d) to avoid repeated alerts when the value hovers around the limit

Q18 Which topic design is best?

(a) /Home Kitchen Temp (b) home/kitchen/bme280/temp
(c) temp (d) HOME#KITCHEN

17.6.3 Advanced level

Q19 ThingSpeak's free tier allows one update every 15 s. How many updates per day at most?

(a) 1440 (b) 5760 (c) 86 400 (d) 960

Q20 Ten nodes each publish a 60-byte message every 30 s. How many messages per day reach the broker?

(a) 2880 (b) 288 000 (c) 14 400 (d) 28 800

Q21 Write topic_match(sub, topic) supporting + and #, and test it on four cases.

Q22 Publish a retained "online" status and three JSON readings with paho, then have a second client subscribe to "nodes/#" late and show that it still receives the retained status.

Q23 Subscribe to home/+/temp and print a running average per room for the incoming readings.

Q24 Send a Telegram message with requests, retrying up to 3 times if the network fails, with 2 s between attempts.

Q25 A battery ESP32 sends a temperature every 5 minutes and a Pi must show it on a phone instantly and also alert by Telegram. Which design fits?

(a) ESP32 → MQTT (QoS 1) → Mosquitto on the Pi → Node-RED dashboard + Telegram node

(b) ESP32 → HTTP polling from the phone every second

(c) Pi polls the ESP32 by ping

(d) ESP32 emails the phone directly every 5 minutes

17.7 Solutions

Q1 **Answer: (a) topics through a broker**

Publishers and subscribers meet at **topics** on a **broker**.

Q2 **Answer: (b) Mosquitto**

Mosquitto.

Q3 **Answer: (d) 1883**

1883; 8883 is MQTT over TLS.

Q4 **Answer: (c) +**

+; # matches many levels.

Q5 **Answer: (c) 0**

QoS 0: at most once.

Q6 **Answer: (d) paho-mqtt**

paho-mqtt.

Q7 **Answer: (b) Node-RED**

Node-RED.

Q8 **Answer: (a) @BotFather**

@BotFather gives the token.

Q9 **Answer: (c) mosquitto_sub**

`mosquitto_sub -t "home/#" -v`.

Q10 **Answer: (b) the last message on a topic, stored by the broker and sent to new subscribers**

Last known state.

Q11 **Answer: (a) the broker publishes it if a client disconnects unexpectedly**

E.g. status = offline.

Q12 **Answer: (c) no, + matches only one level**

Use home/# or home/+/+/temp.

Q13 **Answer: (d) so subscriptions are restored automatically after a reconnect**

Reconnects start a fresh session.

Q14 **Answer: (d) 8883**

8883.

Q15 **Answer: (c) updates are pushed instantly with tiny overhead**

Push vs poll.

Q16 **Answer: (a) X.509 client certificates over TLS**

Per-device **certificates**.

Q17 **Answer: (d) to avoid repeated alerts when the value hovers around the limit**

Re-arm only after it moves clearly back.

Q18 **Answer: (b) home/kitchen/bme280/temp**

Hierarchical, lower case, no spaces or leading slash.

Q19 **Answer: (b) 5760**

86 400 ÷ 15 = **5760**.

Q20 **Answer: (d) 28 800**

10 × 2880 = **28 800**.

Q21 **Answer: Solution program**

accepts the rest of the topic; + must match exactly one level; otherwise every level must match and the lengths must be equal.

```
def topic_match(sub, topic):
    s, t = sub.split("/"), topic.split("/")
    for i, part in enumerate(s):
        if part == "#":
            return True
        if i >= len(t) or (part != "+" and part != t[i]):
            return False
    return len(s) == len(t)

for sub, topic in [("home/+/temp", "home/kitchen/temp"), ("home/+/temp",
"home/kitchen/fridge/temp"),
                   ("home/#", "home/garage/door"), ("home/garage/door", "home/garage")]:
    print(f"{sub:18} {topic:26} {topic_match(sub, topic)}")
```

OUTPUT

```
home/+/temp        home/kitchen/temp          True
home/+/temp        home/kitchen/fridge/temp   False
home/#             home/garage/door           True
home/garage/door   home/garage                False
```

Q22 **Answer: Solution program**

The late subscriber missed the readings (not retained) but immediately received the retained status.

```
import paho.mqtt.client as mqtt, json, time
pub = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2, client_id="node1")
pub.connect("localhost"); pub.loop_start()
pub.publish("nodes/node1/status", "online", retain=True)
for t in (21.5, 21.7, 21.6):
    pub.publish("nodes/node1/temp", json.dumps({"t": t}))
time.sleep(1)
late = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2, client_id="late")
late.on_message = lambda c, u, m: print("late subscriber got", m.topic, m.payload.decode(), "
(retained)" if m.retain else "")
late.connect("localhost"); late.loop_start()
late.subscribe("nodes/#")
time.sleep(1)
```

OUTPUT

```
late subscriber got nodes/node1/status online (retained)
```

Q23 **Answer: Solution program**

defaultdict keeps a separate sum and count for each room taken from the topic.

```
import paho.mqtt.client as mqtt
from collections import defaultdict
sums, counts = defaultdict(float), defaultdict(int)
def on_message(c, u, msg):
    room = msg.topic.split("/")[1]
    sums[room] += float(msg.payload); counts[room] += 1
    print(f"{room}: avg {sums[room] / counts[room]:.2f} over {counts[room]}")
c = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
c.on_message = on_message
c.on_connect = lambda c, u, f, rc, p: c.subscribe("home/+/temp")
c.connect("localhost"); c.loop_forever()
```

OUTPUT

```
hall: avg 22.00 over 1
bed: avg 24.00 over 1
hall: avg 22.50 over 2
hall: avg 23.17 over 3
```

Q24 **Answer: Solution program**

Catch RequestException (connection errors, timeouts, HTTP errors); here the first two attempts failed.

```
import requests, time
def send(text, tries=3):
    for attempt in range(1, tries + 1):
        try:
            r = requests.post("https://api.telegram.org/botTOKEN/sendMessage",
                              json={"chat_id": "1", "text": text}, timeout=10)
            r.raise_for_status()
            print("sent on attempt", attempt); return True
        except requests.RequestException as e:
            print("attempt", attempt, "failed:", e)
            time.sleep(2)
    return False
send("Water leak detected!")
```

OUTPUT

```
attempt 1 failed: HTTPSConnectionPool: Max retries exceeded with url:
https://api.telegram.org/botTOKEN/sendMessage
attempt 2 failed: HTTPSConnectionPool: Max retries exceeded with url:
https://api.telegram.org/botTOKEN/sendMessage
sent on attempt 3
```

Q25 **Answer: (a) ESP32 → MQTT (QoS 1) → Mosquitto on the Pi → Node-RED dashboard + Telegram node**

Light pub/sub from the node; the gateway handles UI and alerts.

MQTT = topics + broker (Mosquitto, port 1883 / 8883 TLS). Wildcards + (one level) and # (rest). QoS 0/1/2. Retained = last state; Last Will = offline notice. paho-mqtt: subscribe in on_connect, loop_start/loop_forever, publish state separately from commands. Secure with users, passwords and TLS. Node-RED for flows and dashboards; ThingSpeak/Adafruit IO/AWS/Azure for cloud; alerts by email, Telegram or ntfy with hysteresis.

MODULE 18 · INTERMEDIATE

Home automation

Minimum duration **2 days** | Level **Intermediate**

The Pi is a popular hub for smart homes, and this module builds a complete local system: sensors, actuators, scenes and automations; local versus cloud control; Home Assistant installation options, integrations, entities, automations, scripts and dashboards; Wi-Fi devices with ESPHome, Zigbee with a USB coordinator and Zigbee2MQTT, and an overview of Z-Wave, Thread and Matter; DIY switches, door sensors and irrigation; local and commercial voice control; and electrical safety, followed by 25 solved questions and programs.

IN THIS MODULE

18.1 Home automation concepts
18.2 Home Assistant
18.3 Smart home protocols
18.4 DIY smart devices
18.5 Voice control
18.6 Electrical safety in home projects
18.7 Practice questions
18.8 Solutions

18.1 Home automation concepts

18.1.1 Sensors, actuators, scenes and automations

Term	Meaning	Example
Sensor	reports a state or measurement	motion, door open, temperature, power use
Actuator	changes something	light, smart plug, valve, blind motor
Scene	a saved set of states applied together	"Movie": lights 20 %, TV on, blinds closed
Automation	trigger + conditions + actions	WHEN motion AND after sunset THEN hall light on for 2 min
Script	a reusable sequence of actions	"Leaving home": all off, lock door, notify

EVERY AUTOMATION HAS THE SAME THREE PARTS

18.1.2 Local control vs cloud control

	Local control	Cloud control
Works without internet	yes	no

	Local control	Cloud control
Speed	tens of milliseconds	often 0.5–2 s
Privacy	data stays at home	data on company servers
Lifetime	as long as you run it	ends if the company shuts the service
Set-up	more work	phone app, quick

SHORTCUT TRICK

Prefer devices that can be controlled **locally** (Zigbee, ESPHome, Matter, Tasmota, Shelly). Use the cloud only for what truly needs it (remote access, voice assistants), and reach your hub remotely through a VPN or Home Assistant Cloud.

18.2 Home Assistant

Home Assistant (HA) is the leading open-source home automation platform: thousands of integrations, a polished web and phone app, local control and a huge community. A Pi 4/5 with an SSD runs it well.

18.2.1 Installation options: Home Assistant OS vs container

Option	What you get	Choose when
Home Assistant OS	a dedicated OS image (write with Imager); add-ons (Mosquitto, Zigbee2MQTT, ESPHome, file editor) install with one click; automatic backups	the Pi is only a smart-home hub (recommended for most)
Home Assistant Container	HA in Docker on Raspberry Pi OS; no add-on store: run Mosquitto etc. as your own containers	the Pi also runs other services and you know Docker (module 20)

After installation open `http://homeassistant.local:8123`, create the owner account and set your home location (needed for sunrise/sunset). Use an SSD rather than a microSD card: HA writes to its database constantly.

18.2.2 Integrations, entities and devices

An **integration** connects HA to a brand or protocol (Philips Hue, MQTT, ESPHome, Shelly, Google Cast, weather). It creates **devices** (a physical thing) with **entities** (each controllable or readable aspect). Entity IDs look like `domain.name`: `light.hall`, `sensor.kitchen_temperature`, `binary_sensor.front_door`, `switch.pump`. Every entity has a **state** ("on", "22.4") and attributes.

Your own Pi programs can appear in HA automatically through **MQTT discovery**: publish a retained config message and HA creates the entity.

HA_DISCOVERY.PY

```
import paho.mqtt.client as mqtt, json

c = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2, client_id="labpi")
c.connect("homeassistant.local"); c.loop_start()
config = {
    "name": "Lab temperature", "unique_id": "labpi_temp",
    "state_topic": "labpi/sensor/temp", "unit_of_measurement": "°C",
    "device_class": "temperature",
    "device": {"identifiers": ["labpi"], "name": "Lab Pi", "manufacturer": "LBF"},
}
c.publish("homeassistant/sensor/labpi_temp/config", json.dumps(config, ensure_ascii=False),
retain=True)
c.publish("labpi/sensor/temp", "24.6")
```

SIMULATED RUN (hardware events and printed output)

```
[  0.00 s] MQTT labpi connect homeassistant.local:1883 -> accepted
[  0.00 s] MQTT publish homeassistant/sensor/labpi_temp/config {"name": "Lab temperature",
"unique_id": "labpi_temp", "state_topic": "labpi/sensor/temp", "unit_of_measurement": "°C",
"device_class": "temperature", "device": {"identifiers": ["labpi"], "name": "Lab Pi",
"manufacturer": "LBF"}} [retained]
[  0.00 s] MQTT publish labpi/sensor/temp 24.6
```

18.2.3 Automations and scripts

Build automations in the visual editor (Settings → Automations) or in YAML. The same automation in YAML:

AUTOMATIONS.YAML

```
alias: Hall light on motion after sunset
triggers:
  - trigger: state
    entity_id: binary_sensor.hall_motion
    to: "on"
conditions:
  - condition: sun
    after: sunset
    after_offset: "-00:30:00"          # from 30 min before sunset
actions:
  - action: light.turn_on
    target: {entity_id: light.hall}
    data: {brightness_pct: 60}
  - wait_for_trigger:
      - trigger: state
        entity_id: binary_sensor.hall_motion
        to: "off"
        for: "00:02:00"
  - action: light.turn_off
    target: {entity_id: light.hall}
mode: restart                          # new motion restarts the timer
```

The logic is the same as any program. Here it is in Python, tested against a sequence of events:

AUTOMATION_LOGIC.PY

```
from datetime import time

SUNSET = time(18, 24)                    # from the location's sun calculation
def is_dark(now):
    return now >= time(17, 54) or now < time(6, 10)      # 30 min before sunset .. sunrise

light_on, off_at = False, None
events = [(time(17, 30), "motion"), (time(18, 5), "motion"), (time(18, 6), "clear"),
          (time(18, 7), "motion"), (time(18, 8), "clear"), (time(18, 11), "tick")]
for now, ev in events:
    mins = now.hour * 60 + now.minute
    if off_at is not None and mins >= off_at:
        print(f"{off_at // 60:02d}:{off_at % 60:02d} timer expired -> light OFF")
        light_on, off_at = False, None
    if ev == "motion" and is_dark(now):
        light_on, off_at = True, None
        print(f"{now:%H:%M} motion, dark -> light ON (timer cancelled)")
    elif ev == "motion":
        print(f"{now:%H:%M} motion, daylight -> no action")
    elif ev == "clear" and light_on:
        off_at = mins + 2
        print(f"{now:%H:%M} motion clear -> light off at {off_at // 60:02d}:{off_at % 60:02d} unless
motion returns")
```

OUTPUT

```
17:30 motion, daylight -> no action
18:05 motion, dark -> light ON (timer cancelled)
18:06 motion clear -> light off at 18:08 unless motion returns
18:07 motion, dark -> light ON (timer cancelled)
18:08 motion clear -> light off at 18:10 unless motion returns
18:10 timer expired -> light OFF
```

18.2.4 Dashboards

Dashboards are built from **cards**: entities, gauges, history graphs, thermostats, camera feeds, buttons, maps. Make a dedicated mobile dashboard with the most-used controls, and install the HA Companion app, which also turns your phone into a sensor (location for "nobody home" automations, battery level) and receives notifications.

18.3 Smart home protocols

18.3.1 Wi-Fi devices and ESPHome

ESPHome turns an ESP32/ESP8266 into a custom smart device from a short YAML file, with no programming: it compiles firmware, flashes it (first by USB, then over Wi-Fi) and connects to HA natively. Commercial Wi-Fi devices (Shelly relays, Tasmota-flashed plugs) can also be controlled locally.

GARAGE-NODE.YAML (ESPHOME)

```
esphome:
  name: garage-node
esp32:
  board: esp32dev
wifi:
  ssid: !secret wifi_ssid
  password: !secret wifi_password
api:                                  # native Home Assistant connection
ota:
  - platform: esphome
sensor:
  - platform: dht
    pin: GPIO4
    model: DHT22
    temperature: {name: "Garage temperature"}
    humidity: {name: "Garage humidity"}
    update_interval: 60s
binary_sensor:
  - platform: gpio
    pin: {number: GPIO5, mode: INPUT_PULLUP, inverted: true}
    name: "Garage door"
    device_class: garage_door
switch:
  - platform: gpio
    pin: GPIO26
    name: "Garage light relay"
```

18.3.2 Zigbee with a USB coordinator (Zigbee2MQTT)

Zigbee is a low-power 2.4 GHz **mesh** protocol: battery sensors last 1–2 years, and mains-powered devices (plugs, bulbs) act as **routers** that extend the network. Plug a **coordinator** USB stick (e.g. Sonoff ZBDongle, SLZB-06) into the Pi, preferably on a USB extension cable away from USB 3 ports and Wi-Fi, which cause 2.4 GHz interference. **Zigbee2MQTT** bridges every Zigbee device to MQTT topics (zigbee2mqtt/<name>), supports thousands of devices from many brands, and HA picks them up by MQTT discovery. HA's built-in ZHA integration is the simpler alternative.

ZIGBEE2MQTT/FRONT_DOOR: A DOOR SENSOR REPORTS IT IS OPEN (CONTACT FALSE)

```
{"contact": false, "battery": 87, "linkquality": 120, "voltage": 2995}
```

18.3.3 Overview of Z-Wave, Thread and Matter

Protocol	Key points
Z-Wave	sub-GHz (865.2 MHz in India), less interference, long range mesh, certified interoperable devices; needs a Z-Wave USB stick
Thread	low-power IPv6 mesh network (like Zigbee but IP-based); needs a border router (e.g. an Apple TV, Google Nest Hub, or a Thread USB stick)
Matter	an application standard backed by Apple, Google, Amazon and Samsung; devices work with all ecosystems, controlled locally, over Wi-Fi, Ethernet or Thread

18.4 DIY smart devices

18.4.1 Smart switches and plugs

Buy certified smart plugs and in-wall relays (Shelly, Sonoff with local firmware) for mains loads, and control them locally from HA. Build your own only for **low-voltage** loads (12 V LED strips, fans, pumps) with relays or MOSFETs (module 12).

18.4.2 Door and window sensors

A reed switch and magnet (module 8) on a Pi, ESP32 or Zigbee sensor reports open/closed. Combine them: "if any window is open for 10 minutes while the AC is on, send a reminder"; "if the front door opens while everyone is away, sound the alarm".

18.4.3 Smart irrigation

IRRIGATION_RULES.PY

```
def water_minutes(soil_pct, rain_mm_next_24h, temp_max_c):
    if rain_mm_next_24h >= 5 or soil_pct >= 60:
        return 0                                  # skip: rain coming or soil wet
    base = (60 - soil_pct) / 2                    # drier soil -> longer
    if temp_max_c > 35:
        base *= 1.3                               # hot day: 30 % more
    return round(min(base, 25))                   # never more than 25 minutes

for soil, rain, temp in [(25, 0, 33), (25, 8, 33), (40, 0, 38), (65, 0, 30), (5, 0, 40)]:
    print(f"soil {soil:2d} %, rain {rain} mm, max {temp} C -> water {water_minutes(soil, rain,
temp)} min")
```

OUTPUT

```
soil 25 %, rain 0 mm, max 33 C -> water 18 min
soil 25 %, rain 8 mm, max 33 C -> water 0 min
soil 40 %, rain 0 mm, max 38 C -> water 13 min
soil 65 %, rain 0 mm, max 30 C -> water 0 min
soil  5 %, rain 0 mm, max 40 C -> water 25 min
```

In practice: capacitive soil sensors on an ESP32 (ESPHome), a 12 V solenoid valve through a relay or MOSFET with a flyback diode, a weather forecast integration for rain, and a HA automation each morning that runs the valve for the computed time, with a safety maximum.

18.5 Voice control

18.5.1 Local voice assistants

Home Assistant's **Assist** understands commands like "turn on the hall light" fully locally: **Whisper** or **Speech-to-Phrase** for speech-to-text, Assist's intent engine, **Piper** for replies, and **openWakeWord** for a wake word, with voice satellites such as the Home Assistant Voice Preview Edition or an ESP32-S3 box. Your voice never leaves the house.

18.5.2 Integration with commercial assistants

Alexa and Google Assistant can control HA entities through Home Assistant Cloud (a paid subscription that also funds HA) or a manual cloud setup, and Matter devices pair with all of them. Expose only the entities you want to speak to, and never expose locks or alarms without a PIN.

18.6 Electrical safety in home projects

WATCH OUT

Mains (230 V AC) can kill and start fires. Only qualified electricians should work on fixed wiring, switchboards or in-wall devices. Use certified (BIS/CE) smart plugs and enclosed modules; never leave exposed mains terminals; use correctly rated wire, fuses and enclosures; keep low-voltage and mains wiring separate; and switch off at the breaker before any change.

Rule	Why
Low voltage for DIY	5–24 V DC projects are safe to build and touch
Rated parts	a relay or SSR rated well above the load current (inductive loads like motors draw several times their running current at start)
Fuses	protect wiring from overload and short circuits
Enclosures	no exposed live parts; strain relief on cables
Fail-safe design	heaters and pumps must turn off if the Pi crashes (timeouts, hardware thermostats, float switches)

PRACTICE CHECKPOINT

Set up Home Assistant with at least three devices, create two automations (for example lights on motion after sunset) and a mobile dashboard.

18.7 Practice questions

18.7.1 Basic level

Q1 In home automation, what is an actuator?

(a) a temperature sensor
(b) the internet router
(c) a dashboard
(d) a device that changes something, such as a light or valve

Q2 What are the three parts of an automation?

(a) sensor, cloud, phone
(b) trigger, conditions, actions
(c) input, output, power
(d) scene, script, entity

Q3 What is the default Home Assistant web address?

(a) http://homeassistant.local:8123
(b) http://pi:5000
(c) http://localhost:1880
(d) http://ha:80

Q4 Which tool builds ESP32 smart devices from YAML?

(a) Zigbee2MQTT **(b)** Kodi **(c)** ESPHome **(d)** Nginx

Q5 Which protocol is a low-power 2.4 GHz mesh used by battery sensors?

(a) Zigbee **(b)** HDMI **(c)** UART **(d)** Ethernet

Q6 Which standard is backed by Apple, Google, Amazon and Samsung for cross-ecosystem devices?

(a) Z-Wave **(b)** Matter **(c)** MQTT **(d)** I2C

Q7 What is a scene?

(a) a camera
(b) a trigger
(c) a log file
(d) a saved set of device states applied together

Q8 What does an entity ID like binary_sensor.front_door describe?

(a) a light dimmer
(b) a number sensor
(c) an on/off sensor named front_door
(d) a user account

Q9 Which Home Assistant feature turns a phone into a location sensor and notification target?

(a) Zigbee2MQTT
(b) the Companion app
(c) ESPHome
(d) Grafana

18.7.2 Intermediate level

Q10 Why prefer local control?

(a) it is always cheaper to buy
(b) it needs no hub
(c) it works without internet, is fast and keeps data private

(d) it cannot be automated

Q11 What is the main advantage of Home Assistant OS over the container install?

(a) it uses less RAM than any OS
(b) it runs other servers better
(c) it needs no network
(d) one-click add-ons and built-in backups

Q12 Why put a Zigbee USB coordinator on an extension cable?

(a) to make it charge
(b) to reduce 2.4 GHz interference from USB 3 ports and Wi-Fi
(c) to change its address
(d) to speed up the Pi

Q13 What do mains-powered Zigbee devices do for the network?

(a) act as routers that extend the mesh
(b) nothing
(c) replace the coordinator
(d) connect to the cloud

Q14 How can your own Python program create an entity in HA automatically?

(a) edit the HA database
(b) publish a retained MQTT discovery config message
(c) email Home Assistant
(d) use Bluetooth

Q15 Which frequency does Z-Wave use in India?

(a) 865.2 MHz
(b) 2.4 GHz
(c) 5 GHz
(d) 433 MHz

Q16 Why mode: restart in a motion-light automation?

(a) to reboot Home Assistant
(b) to restart the light bulb
(c) new motion restarts the off timer instead of being ignored
(d) to repeat forever

Q17 Which is the safest DIY mains approach?

(a) solder relays to house wiring
(b) use bare relay boards on the wall
(c) switch mains with a GPIO pin directly
(d) use certified smart plugs or enclosed relays and leave fixed wiring to electricians

18.7.3 Advanced level

Q18 A battery Zigbee sensor draws 20 µA average from a 220 mAh CR2032. Roughly how long does it last?

(a) about 15 days
(b) about 15 years
(c) about 1 week
(d) about 15 months

Q19 A motor pump starts at 6 × its running current. If it runs at 1.5 A on 230 V, what start current must the relay handle, and is a 10 A relay enough?

Q20 Write a function that decides whether to send a "window open" reminder: AC on, any window open for at least 10 minutes. Test three situations.

Q21 Publish an MQTT discovery config for a switch "Pump" with command and state topics, then simulate HA sending ON and the program confirming the state.

Q22 Parse a Zigbee2MQTT door-sensor message and print "OPEN" or "closed" plus a low-battery warning below 20 %.

Q23 Which option needs a border router?

(a) Z-Wave devices
(b) ESPHome Wi-Fi devices
(c) Thread devices
(d) a wired Ethernet switch

Q24 Which voice setup keeps all audio inside the home?

(a) HA Assist with Whisper (or Speech-to-Phrase), Piper and openWakeWord
(b) a commercial smart speaker
(c) a cloud speech API
(d) Telegram voice notes

Q25 A heater controlled by a Pi relay must not stay on if the Pi crashes. Which design is fail-safe?

(a) a Python try/except
(b) a faster SD card
(c) a retained MQTT message
(d) a hardware thermostat or timer in series that cuts power independently

18.8 Solutions

Q1 **Answer: (d) a device that changes something, such as a light or valve**

Actuators **act**.

Q2 **Answer: (b) trigger, conditions, actions**

WHEN … IF … THEN.

Q3 **Answer: (a) http://homeassistant.local:8123**

Port **8123**.

Q4 **Answer: (c) ESPHome**

ESPHome.

Q5 **Answer: (a) Zigbee**

Zigbee.

Q6 **Answer: (b) Matter**

Matter.

Q7 **Answer: (d) a saved set of device states applied together**

E.g. "Movie" mode.

Q8 **Answer: (c) an on/off sensor named front_door**

domain.name.

Q9 **Answer: (b) the Companion app**

HA Companion app.

Q10 **Answer: (c) it works without internet, is fast and keeps data private**

No cloud dependency.

Q11 **Answer: (d) one-click add-ons and built-in backups**

The add-on store.

Q12 **Answer: (b) to reduce 2.4 GHz interference from USB 3 ports and Wi-Fi**

USB 3 is very noisy at 2.4 GHz.

Q13 **Answer: (a) act as routers that extend the mesh**

Plugs and bulbs route messages.

Q14 **Answer: (b) publish a retained MQTT discovery config message**

homeassistant/<component>/<id>/config.

Q15 **Answer: (a) 865.2 MHz**

Sub-GHz band.

Q16 **Answer: (c) new motion restarts the off timer instead of being ignored**

Keeps the light on while people move.

Q17 **Answer: (d) use certified smart plugs or enclosed relays and leave fixed wiring to electricians**

Safety first.

Q18 **Answer: (d) about 15 months**

220 ÷ 0.020 = 11 000 h ≈ 458 days ≈ **15 months**.

Q19 **Answer: Solution program**

1.5 × 6 = 9 A, inside 10 A but with little margin; a 16 A relay or contactor is the better choice for long life.

```
run_a, factor, relay_a = 1.5, 6, 10
start_a = run_a * factor
print(f"start current {start_a} A, running power {run_a * 230:.0f} W")
print("10 A relay OK" if start_a <= relay_a else "choose a bigger relay or a contactor")
```

OUTPUT

```
start current 9.0 A, running power 345 W
10 A relay OK
```

Q20 **Answer: Solution program**

Both conditions must hold; the list comprehension finds windows open 10 minutes or more.

```
def remind(ac_on, windows):            # windows: {name: minutes_open or 0}
    open_long = [w for w, m in windows.items() if m >= 10]
    if ac_on and open_long:
        return "Close " + ", ".join(open_long) + ": the AC is on"
    return None
print(remind(True, {"bedroom": 12, "kitchen": 3}))
print(remind(True, {"bedroom": 5}))
print(remind(False, {"hall": 30}))
```

OUTPUT

```
Close bedroom: the AC is on
None
None
```

Q21 **Answer: Solution program**

HA publishes ON/OFF to the command topic; the device acts (here prints) and reports the real state.

```
import paho.mqtt.client as mqtt, json
c = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2, client_id="pumpnode")
def on_message(cl, u, msg):
    state = msg.payload.decode()
    print("command:", state)
    cl.publish("pumpnode/pump/state", state, retain=True)
c.on_message = on_message
c.connect("homeassistant.local"); c.loop_start()
c.publish("homeassistant/switch/pump/config", json.dumps({
    "name": "Pump", "unique_id": "pumpnode_pump",
    "command_topic": "pumpnode/pump/set", "state_topic": "pumpnode/pump/state"}), retain=True)
c.subscribe("pumpnode/pump/set")
import time; time.sleep(3)
```

SIMULATED RUN

```
[  0.00 s] MQTT pumpnode connect homeassistant.local:1883 -> accepted
[  0.00 s] MQTT publish homeassistant/switch/pump/config {"name": "Pump", "unique_id":
"pumpnode_pump", "command_topic": "pumpnode/pump/set", "state_topic": "pumpnode/pump/state"}
[retained]
[  0.00 s] MQTT pumpnode subscribed pumpnode/pump/set (QoS 0)
```

```
command: ON
[  1.50 s] MQTT publish pumpnode/pump/state ON [retained]
```

Q22 **Answer: Solution program**

contact = true means the magnet is near (closed).

```
import json
for raw in ('{"contact": false, "battery": 87}', '{"contact": true, "battery": 14}'):
    m = json.loads(raw)
    state = "closed" if m["contact"] else "OPEN"
    warn = "  LOW BATTERY" if m["battery"] < 20 else ""
    print(f"front door {state}{warn}")
```

OUTPUT

```
front door OPEN
front door closed  LOW BATTERY
```

Q23 **Answer: (c) Thread devices**

Thread joins the home IP network through a **border router**.

Q24 **Answer: (a) HA Assist with Whisper (or Speech-to-Phrase), Piper and openWakeWord**

Fully local pipeline.

Q25 **Answer: (d) a hardware thermostat or timer in series that cuts power independently**

Independent hardware protection.

Automations = trigger + conditions + actions; scenes set many states; scripts reuse sequences. Run Home Assistant (OS for a dedicated hub, Container alongside other services) on an SSD. Prefer local protocols: ESPHome (Wi-Fi), Zigbee2MQTT (mesh, battery sensors), Z-Wave, Thread/Matter. Integrate your own Pi code with MQTT discovery. Keep DIY to low voltage; use certified devices and fail-safe design for mains.

MODULE 19 · INTERMEDIATE

Linux system administration for Raspberry Pi

Minimum duration **2 days** | Level **Intermediate**

Reliable projects need a well-administered system, and these skills transfer directly to servers and DevOps. This module covers systemd units, services, targets, restart policies, dependencies and journalctl; scheduling with cron and systemd timers; mounting USB drives and SSDs, fstab and file system types; users and groups; boot configuration in config.txt and cmdline.txt and the risks of overclocking; backups with rsync, imaging, and a read-only overlay root file system; and advanced Bash scripting with functions, arguments and exit codes, followed by 25 solved questions and tasks.

IN THIS MODULE

19.1 systemd
19.2 Scheduling
19.3 Storage
19.4 Users and groups management
19.5 Boot configuration
19.6 Backups and recovery
19.7 Advanced Bash scripting
19.8 Practice questions
19.9 Solutions

19.1 systemd

19.1.1 Units, services and targets

systemd is the first process the kernel starts (PID 1); it starts and supervises everything else. Each thing it manages is a **unit** described by a file: `.service` (a program), `.timer` (a schedule), `.mount` (a file system), `.target` (a group of units, like a run level: `multi-user.target` = normal console system, `graphical.target` = desktop).

Command	Does
systemctl status name	state, PID, memory, last log lines
sudo systemctl start / stop / restart name	control now
sudo systemctl enable / disable name	start (or not) at boot
sudo systemctl enable --now name	both at once
systemctl list-units --type=service --state=running	what is running
systemctl --failed	units that failed
sudo systemctl daemon-reload	re-read unit files after editing them

19.1.2 Writing service files for Python apps

/ETC/SYSTEMD/SYSTEM/MONITOR.SERVICE

```
[Unit]
Description=Room monitor logger
After=network-online.target mosquitto.service
Wants=network-online.target

[Service]
Type=simple
User=pi
Group=pi
WorkingDirectory=/home/pi/monitor
Environment=PYTHONUNBUFFERED=1
EnvironmentFile=/home/pi/monitor/.env
ExecStart=/home/pi/monitor/env/bin/python monitor.py
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Line	Why
User=pi	never run your app as root unless it must
WorkingDirectory	relative file paths in your code work
PYTHONUNBUFFERED=1	print() output reaches the journal immediately
EnvironmentFile	secrets (API keys) stay out of the code and the unit file
venv python in ExecStart	uses the project's packages with no "activate" needed
WantedBy=multi-user.target	what "enable" hooks it into

19.1.3 Restart policies and dependencies

Restart=	Restarts when
no	never (default)
on-failure	it exits with an error code, crashes or is killed (not after a clean exit 0)
always	it stops for any reason, even a clean exit
on-abnormal	crash or timeout only

`After=` sets the **order** (start after X); `Wants=` / `Requires=` pull X in too (Requires also stops this unit if X fails). Avoid restart storms with `StartLimitIntervalSec=300` and `StartLimitBurst=5` in [Unit]: at most 5 restarts in 5 minutes.

19.1.4 journalctl and logs

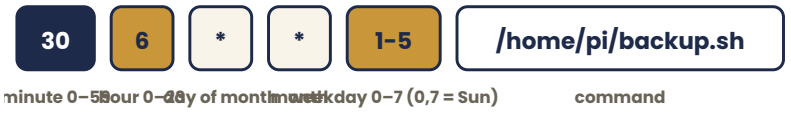
READING THE JOURNAL

```
journalctl -u monitor             # all logs of one service
journalctl -u monitor -f          # follow live (like tail -f)
journalctl -u monitor --since "1 hour ago"
journalctl -b -p err              # errors since this boot
journalctl --disk-usage
sudo journalctl --vacuum-size=100M # shrink stored logs
```

To keep logs in RAM only (protects the SD card, loses them at reboot) set `Storage=volatile` and limit size with `RuntimeMaxUse=50M` in `/etc/systemd/journald.conf`.

19.2 Scheduling

19.2.1 cron



CRONTAB LINE: 06:30 EVERY MONDAY TO FRIDAY

Expression	Runs
*/15 * * * *	every 15 minutes
0 * * * *	every hour on the hour
0 2 * * *	daily at 02:00
0 9 * * 1	Mondays at 09:00
0 0 1 * *	first day of each month at midnight
@reboot	once at boot

CRON_CHECK.PY

```
def matches(field, value, lo, hi):
    for part in field.split(","):
        step = 1
        if "/" in part:
            part, step = part.split("/"); step = int(step)
        if part == "*":
            start, end = lo, hi
        elif "-" in part:
            start, end = map(int, part.split("-"))
        else:
            start = end = int(part)
        if start <= value <= end and (value - start) % step == 0:
            return True
    return False

def runs(expr, minute, hour, weekday):          # weekday 0 = Sunday
    m, h, _, _, w = expr.split()
    return matches(m, minute, 0, 59) and matches(h, hour, 0, 23) and matches(w, weekday, 0, 6)

for expr in ("30 6 * * 1-5", "*/15 * * * *", "0 2 * * *"):
    hits = [f"{h:02d}:{m:02d}" for h in range(24) for m in range(60) if runs(expr, m, h, 3)]
    print(f"{expr:14} on a Wednesday: {len(hits):3d} runs, first {hits[:3]}")
```

OUTPUT

```
30 6 * * 1-5   on a Wednesday:   1 runs, first ['06:30']
*/15 * * * *   on a Wednesday:  96 runs, first ['00:00', '00:15', '00:30']
0 2 * * *      on a Wednesday:   1 runs, first ['02:00']
```

19.2.2 systemd timers

A **timer** unit starts a matching service on a schedule. Advantages over cron: logs in the journal, `Persistent=true` runs a missed job after the Pi was off, random delays to spread load, and dependencies.

BACKUP.TIMER (PAIRS WITH BACKUP.SERVICE OF TYPE=ONESHOT)

```
# /etc/systemd/system/backup.timer
[Unit]
Description=Nightly backup

[Timer]
OnCalendar=*-*-* 02:00:00
Persistent=true
RandomizedDelaySec=10min

[Install]
WantedBy=timers.target
```

USING TIMERS

```
sudo systemctl enable --now backup.timer
systemctl list-timers                       # next and last run of every timer
systemd-analyze calendar "Mon..Fri 06:30"   # check an OnCalendar expression
```

19.3 Storage

19.3.1 Mounting USB drives and SSDs

FINDING THE DRIVE

```
pi@raspberrypi:~ $ lsblk -f
NAME        FSTYPE FSVER LABEL  UUID                                 MOUNTPOINTS
sda
└─sda1      ext4   1.0   data   5c2e1f8a-3b7d-4e21-9a6f-0d8c4b7e2f13
mmcblk0
├─mmcblk0p1 vfat   FAT32 bootfs 4EF5-6F55                            /boot/firmware
└─mmcblk0p2 ext4   1.0   rootfs 7f7c1a2e-6d1b-4c55-8a3e-2b9e0c4d5f61 /
```

MOUNTING BY HAND

```
sudo mkdir -p /mnt/data
sudo mount /dev/sda1 /mnt/data
df -h /mnt/data
sudo umount /mnt/data            # always unmount before unplugging
```

19.3.2 fstab

`/etc/fstab` lists file systems to mount at boot. Always identify the drive by **UUID**: device names like /dev/sda1 can change when drives are plugged in a different order. Add `nofail` so the Pi still boots if the drive is missing.

/ETC/FSTAB LINES

```
# <device>                                <mount>        <type> <options>
<dump> <pass>
UUID=5c2e1f8a-3b7d-4e21-9a6f-0d8c4b7e2f13     /mnt/data      ext4   defaults,noatime,nofail        0
2
tmpfs                                          /var/log/app   tmpfs  defaults,size=32M,noatime
0      0
```

Test with `sudo mount -a` (mounts everything in fstab and reports errors) **before** rebooting; a broken fstab line without nofail can stop the Pi booting. `noatime` stops a write every time a file is read.

19.3.3 File system types

Type	Use	Notes
ext4	Linux drives, the Pi's root partition	journaling, permissions; not readable by Windows without extra tools
vfat (FAT32)	the boot partition, small USB sticks	every OS reads it; 4 GB file limit; no Linux permissions
exFAT	big USB drives shared with Windows/macOS	no 4 GB limit; no permissions
NTFS	Windows drives	works with the ntfs3 driver; fine for media
Btrfs	advanced: snapshots, checksums	more complex

Type	Use	Notes
tmpfs	files in RAM	fast, no wear, lost at reboot

19.4 Users and groups management

Command	Does
sudo adduser alice	create a user with home folder (asks for password)
sudo usermod -aG gpio,i2c alice	add to groups (-a = append!)
groups alice / id alice	show groups
sudo passwd alice	change password
sudo deluser --remove-home alice	delete user and home
sudo visudo	safely edit sudo rules (checks syntax)
sudo -u alice whoami	run a command as another user

WHO AM I AND WHICH GROUPS?

```
pi@raspberrypi:~ $ id
uid=30033(pi) gid=30033(pi) groups=30033(pi)
pi@raspberrypi:~ $ groups
pi
pi@raspberrypi:~ $ whoami
pi
pi@raspberrypi:~ $ ls -ld /home/pi
drwxr-x--- 2 pi pi 4096 Nov  2 00:25 /home/pi
```

Give each service its own system user where practical (`sudo adduser --system --group sensorsvc`) and only the groups it needs: if it is compromised, the damage is limited.

19.5 Boot configuration

19.5.1 config.txt and cmdline.txt

`/boot/firmware/config.txt` configures firmware and hardware (module 10.6): display, interfaces, overlays, overclocking. `/boot/firmware/cmdline.txt` is a **single line** of kernel options: root partition, console, quiet boot. A line break in cmdline.txt stops the Pi booting.

CMDLINE.TXT (ONE LINE)

```
console=serial0,115200 console=tty1 root=PARTUUID=6c586e13-02 rootfstype=ext4 fsck.repair=yes
rootwait quiet splash
```

19.5.2 Overclocking basics and risks

CONFIG.TXT OVERCLOCK

```
# Pi 5 example (needs good cooling)
arm_freq=2800
over_voltage_delta=50000      # +50 mV
```

Gains are modest (10–20 %) and come with more heat, higher power use, possible instability and file system corruption if the system crashes. Test with a stress tool while watching `vcgencmd measure_temp` and `get_throttled`. If the Pi will not boot, hold **Shift** at power-on (Pi 4/5) or edit config.txt on another computer. Never overclock a production device.

19.6 Backups and recovery

19.6.1 rsync backups

BACKUP.SH

```
#!/bin/bash
# backup.sh - mirror project folders to a backup drive, keeping a dated log
SRC="$HOME/projects/"
DEST="$HOME/backup/projects/"
LOG="$HOME/backup/backup.log"
mkdir -p "$DEST"
if rsync -a --delete "$SRC" "$DEST"; then
    echo "$(date '+%F %T') OK $(du -sh "$DEST" | cut -f1)" >> "$LOG"
else
    echo "$(date '+%F %T') FAILED" >> "$LOG"; exit 1
fi
tail -n 1 "$LOG"
find "$DEST" -type f | wc -l
```

TYPICAL OUTPUT

```
2026-03-15 02:00:04 OK 20K
3
```

Point DEST at a mounted USB drive or, better, another machine (`user@nas:/backups/labpi/`) so a failed SD card or a theft does not take the backup with it. The **3-2-1 rule**: 3 copies, on 2 different media, 1 off-site. Use `--link-dest` for space-efficient daily snapshots, or tools such as restic or Borg for encrypted, deduplicated backups.

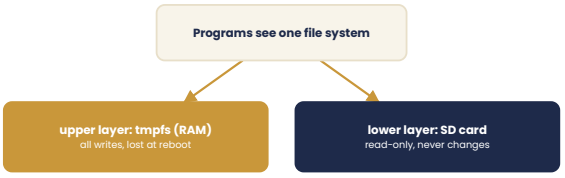
19.6.2 Imaging and restoring

A full image captures everything: OS, settings and data. Use the SD Card Copier, `dd` from another Linux machine (module 3.7), or `rpi-clone` to copy the running system to a USB card or SSD. Shrink images with PiShrink. Test restores: a backup you have never restored is only a hope.

19.6.3 Read-only root file system (overlay file system)

With the **overlay file system** enabled (raspi-config → Performance Options → Overlay File System), the root file system is mounted read-only and all writes go to RAM. Every reboot returns the Pi to a clean

state, and sudden power loss cannot corrupt the card. Ideal for kiosks, signage and appliances. Data that must persist goes to a separate writable partition, a USB drive or the network. Disable the overlay temporarily to install updates.



OVERLAY FILE SYSTEM

19.7 Advanced Bash scripting

19.7.1 Functions, arguments and exit codes

TEMPCHECK.SH

```
#!/bin/bash
set -euo pipefail           # stop on errors, unset variables and failed pipes

usage() { echo "Usage: $0 <warn_temp> [crit_temp]"; exit 2; }
log()   { echo "[$(date +%H:%M:%S)] $*"; }

check_temp() {
    local warn=$1 crit=${2:-80}           # second argument defaults to 80
    local t
    t=$(vcgencmd measure_temp | grep -o "[0-9.]*")
    if (( $(echo "$t >= $crit" | bc) )); then log "CRITICAL $t C"; return 2
    elif (( $(echo "$t >= $warn" | bc) )); then log "WARNING $t C"; return 1
    fi
    log "OK $t C"; return 0
}

[[ $# -lt 1 ]] && usage
status=0
check_temp "$@" || status=$?
echo "exit status: $status"
exit $status
```

OUTPUT

```
[00:44:04] WARNING 47.2 C
exit status: 1
```

Bash	Meaning
$0, $1 … $9, $#, "$@"	script name, arguments, how many, all (quoted)
${2:-80}	use $2, or 80 if it is missing
local var	variable visible only inside the function
return n / exit n	0 = success, 1–255 = error codes

Bash	Meaning
$?	exit status of the last command
cmd1 && cmd2 / cmd1 || cmd2	run cmd2 only on success / on failure
set -euo pipefail	strict mode: fail fast instead of continuing with bad data
[[…]]	safer test syntax than […]

19.7.2 Automation scripts

HEALTH.SH

```
#!/bin/bash
# health.sh - one-line health report for logs or alerts
temp=$(vcgencmd measure_temp | cut -d= -f2)
thr=$(vcgencmd get_throttled | cut -d= -f2)
disk=$(df -h / | awk 'NR==2 {print $5}')
load=$(cut -d" " -f1 /proc/loadavg)
host=$(hostname)
msg="$host temp=$temp throttled=$thr disk=$disk load=$load"
echo "$msg"
[[ "$thr" != "0x0" ]] && echo "ALERT: power or heat problem"
exit 0
```

OUTPUT

```
raspberrypi temp=47.2'C throttled=0x0 disk=29% load=0.15
```

Run it from a timer every 15 minutes and append to a log, or send the line to Telegram when there is an alert (module 17). The disk and load values above come from the machine that ran the script.

PRACTICE CHECKPOINT

Turn an earlier project into a robust appliance: a systemd service with automatic restart, a scheduled backup, log monitoring and an overlay file system to protect the SD card.

19.8 Practice questions

19.8.1 Basic level

Q1 What is PID 1 on Raspberry Pi OS?

(a) bash (b) systemd (c) cron (d) python3

Q2 Which command starts a service now and at every boot?

(a) sudo systemctl start name
(b) sudo service boot name
(c) crontab -e
(d) sudo systemctl enable --now name

Q3 Which command shows the logs of a service?

(a) journalctl -u name
(b) systemctl logs name
(c) cat /var/name
(d) dmesg -u name

Q4 What does the cron line 0 2 * * * mean?

(a) every 2 minutes
(b) at 00:02 every day
(c) every day at 02:00
(d) every second day

Q5 Which command shows the next run time of every systemd timer?

(a) crontab -l
(b) journalctl --timers
(c) at -l
(d) systemctl list-timers

Q6 Which file lists file systems to mount at boot?

(a) /etc/hosts
(b) /boot/firmware/config.txt
(c) /etc/crontab
(d) /etc/fstab

Q7 Which file system does the Pi's root partition use?

(a) FAT32 (b) ext4 (c) NTFS (d) exFAT

Q8 Which command adds alice to the gpio group without removing other groups?

(a) sudo usermod -aG gpio alice
(b) sudo usermod -G gpio alice
(c) sudo addgroup alice
(d) sudo chown gpio alice

Q9 What must cmdline.txt always be?

(a) a YAML file (b) empty (c) a single line (d) executable

19.8.2 Intermediate level

Q10 What does Restart=on-failure do?

(a) restarts the service if it crashes or exits with an error
(b) restarts the Pi
(c) restarts only at boot
(d) never restarts

Q11 Why identify drives by UUID in fstab?

(a) UUIDs are shorter
(b) device names like /dev/sda1 can change between boots

(c) it makes drives faster
(d) fstab requires it for FAT only

Q12 Why add nofail to a USB drive in fstab?

(a) to make it read-only
(b) to check it for errors
(c) to format it
(d) so the Pi still boots if the drive is missing

Q13 What does Persistent=true in a systemd timer do?

(a) keeps logs forever
(b) runs the job twice
(c) runs a missed job when the Pi starts again after being off
(d) stores files permanently

Q14 What is the benefit of the overlay (read-only root) file system?

(a) faster Wi-Fi
(b) more storage space
(c) power loss cannot corrupt the card and each reboot is clean
(d) automatic backups to the cloud

Q15 What does set -euo pipefail do in a Bash script?

(a) enables colours
(b) runs the script as root
(c) makes it faster
(d) stops on errors, unset variables and failures inside pipes

Q16 What does ${2:-80} give when no second argument is passed?

(a) 2 (b) 80
(c) an error (d) an empty string

Q17 Which is the 3-2-1 backup rule?

(a) 3 copies, 2 different media, 1 off-site
(b) 3 backups a day, 2 drives, 1 cloud
(c) 3 users, 2 groups, 1 root
(d) 3 weeks, 2 months, 1 year

Q18 How many times does */15 * * * * run per day?

(a) 15 (b) 96 (c) 24 (d) 60

19.8.3 Advanced level

Q19 Which cron expression runs at 18:45 on Saturdays and Sundays?

(a) 45 18 * * 0,6 (b) 18 45 * * 6,7
(c) 45 18 6,7 * * (d) * 18:45 * * weekend

Q20 Create a script that exits with status 3, run it, and print its exit status. Then show that || runs a fallback only on failure.

Q21 Write disk_alert.sh that takes a mount point and a percentage limit (default 80) and prints OK or ALERT with the current use.

Q22 Write a function-based script rotate.sh that keeps only the newest 3 files matching log_*.txt in a folder and deletes the rest, printing what it removes.

Q23 Given a list of service states (name, active, restarts), print the services needing attention: not active, or restarted more than 3 times.

Q24 Calculate how long a nightly rsync takes for 2 GB of changes at 30 MB/s over USB 3 vs 11 MB/s over Wi-Fi (1 GB = 1000 MB).

Q25 Which combination makes the most robust appliance from a Pi project?

(a) cron @reboot + no backups

(b) running the script by hand after each boot

(c) systemd service with Restart=on-failure + timer backups to another machine + journald limits + overlay root file system

(d) overclocking for speed

19.9 Solutions

Q1 **Answer: (b) systemd**

systemd starts and supervises everything.

Q2 **Answer: (d) sudo systemctl enable --now name**

`enable --now`.

Q3 **Answer: (a) journalctl -u name**

`journalctl -u`.

Q4 **Answer: (c) every day at 02:00**

minute 0, hour 2.

Q5 **Answer: (d) systemctl list-timers**

`systemctl list-timers`.

Q6 **Answer: (d) /etc/fstab**

/etc/fstab.

Q7 **Answer: (b) ext4**

Root is **ext4**; boot is FAT32.

Q8 **Answer: (a) sudo usermod -aG gpio alice**

-aG appends.

Q9 **Answer: (c) a single line**

A line break breaks booting.

Q10 **Answer: (a) restarts the service if it crashes or exits with an error**

Not after a clean exit 0.

Q11 **Answer: (b) device names like /dev/sda1 can change between boots**

Stable identification.

Q12 **Answer: (d) so the Pi still boots if the drive is missing**

Otherwise boot can stop at emergency mode.

Q13 **Answer: (c) runs a missed job when the Pi starts again after being off**

Catch-up for missed runs.

Q14 **Answer: (c) power loss cannot corrupt the card and each reboot is clean**

Writes go to RAM.

Q15 **Answer: (d) stops on errors, unset variables and failures inside pipes**

Strict mode.

Q16 **Answer: (b) 80**

Default value syntax.

Q17 **Answer: (a) 3 copies, 2 different media, 1 off-site**

Survives device failure, theft and fire.

Q18 **Answer: (b) 96**

4 per hour × 24 = **96**.

Q19 **Answer: (a) 45 18 * * 0,6**

minute 45, hour 18, weekdays 0 (Sun) and 6 (Sat).

Q20 **Answer: Commands**

$? holds the last exit status; || runs the right side only when the left side fails.

```
pi@raspberrypi:~ $ printf "#!/bin/bash\necho working\nexit 3\n" > job.sh && chmod +x job.sh
pi@raspberrypi:~ $ ./job.sh; echo "status=$?"
working
status=3
pi@raspberrypi:~ $ ./job.sh || echo "job failed, sending alert"
working
job failed, sending alert
pi@raspberrypi:~ $ true && echo "only after success"
only after success
```

Q21 **Answer: Solution script**

df --output=pcent prints just the use percentage; tr -dc keeps the digits. Run here as ./disk_alert.sh / 99 (the value comes from the machine that ran it).

```
#!/bin/bash
mount=${1:-/}
limit=${2:-80}
use=$(df --output=pcent "$mount" | tail -n 1 | tr -dc '0-9')
if (( use >= limit )); then
    echo "ALERT: $mount is ${use}% full (limit ${limit}%)"; exit 1
fi
echo "OK: $mount ${use}% used (limit ${limit}%)"
```

OUTPUT

```
OK: / 29% used (limit 99%)
```

Q22 **Answer: Solution script**

ls -1t sorts newest first; tail -n +4 skips the first 3; the rest are deleted.

```
#!/bin/bash
keep=3
cleanup() {
    local dir=$1
    ls -1t "$dir"/log_*.txt | tail -n +$((keep + 1)) | while read -r f; do
        echo "removing $f"; rm -- "$f"
    done
}
cleanup logs
echo "left:"; ls -1 logs
```

OUTPUT

```
removing logs/log_2.txt
removing logs/log_1.txt
left:
log_3.txt
log_4.txt
log_5.txt
```

Q23 **Answer: Solution program**

inactive is normal for a oneshot service run by a timer, so only "failed" and frequent restarts are flagged.

```
services = [("monitor", "active", 0), ("mosquitto", "active", 5),
            ("dashboard", "failed", 2), ("backup", "inactive", 0)]
for name, state, restarts in services:
    problems = []
    if state == "failed":
        problems.append("FAILED")
    if restarts > 3:
        problems.append(f"{restarts} restarts")
    if problems:
        print(f"{name:10} {', '.join(problems)}")
```

OUTPUT

```
mosquitto  5 restarts
dashboard  FAILED
```

Q24 **Answer: Solution program**

time = size ÷ speed. Only changed files are copied by rsync, so later nights are much faster.

```
changed_mb = 2000
for link, speed in (("USB 3 SSD", 30), ("Wi-Fi NAS", 11)):
    secs = changed_mb / speed
    print(f"{link:10} {secs:6.0f} s = {secs / 60:.1f} min")
```

OUTPUT

```
USB 3 SSD      67 s = 1.1 min
Wi-Fi NAS     182 s = 3.0 min
```

Q25 **Answer: (c) systemd service with Restart=on-failure + timer backups to another machine + journald limits + overlay root file system**

Supervision, backups, bounded logs, protected storage.

systemd: write a .service (User, WorkingDirectory, venv ExecStart, Restart=on-failure), daemon-reload, enable --now, read journalctl -u. Schedule with cron (m h dom mon dow) or timers (OnCalendar, Persistent). Mount drives by UUID in fstab with nofail and test with mount -a. Manage users with adduser/usermod -aG. Back up with rsync (3-2-1), test restores, use the overlay FS for appliances. Bash: functions, local, ${x:-default}, exit codes, set -euo pipefail.

LEVEL 3

Advanced level

Run the Pi as a secure server with Docker, build computer vision and edge AI systems, use the Pico with MicroPython, build robots with ROS 2, design for power and production, and complete a capstone project

8 MODULES · 21 DAYS MINIMUM

Module 20 · Raspberry Pi as a server	**2 days**
Module 21 · Security and hardening	**2 days**
Module 22 · Computer vision with OpenCV	**3 days**
Module 23 · AI and machine learning at the edge	**3 days**
Module 24 · Raspberry Pi Pico and MicroPython	**2 days**
Module 25 · Robotics with Raspberry Pi and ROS 2	**3 days**
Module 26 · Performance, power and product design	**2 days**
Module 27 · Capstone project and career path	**4 days**

MODULE 20 · ADVANCED

Raspberry Pi as a server

Minimum duration **2 days** | Level **Advanced**

A Pi makes an efficient, always-on home and lab server. This module runs network services (Pi-hole ad blocking, DNS and DHCP basics, a WireGuard VPN), file servers (Samba, an OpenMediaVault NAS, Nextcloud), Docker with ARM64 images, Compose stacks and Portainer, web hosting with HTTPS from Let's Encrypt, uptime and Prometheus/Grafana monitoring, and introduces Pi clusters and k3s, followed by 25 solved questions and programs.

IN THIS MODULE

20.1 Network services
20.2 File servers and NAS
20.3 Docker on Raspberry Pi
20.4 Web hosting
20.5 Monitoring
20.6 Clusters
20.7 Practice questions
20.8 Solutions

20.1 Network services

20.1.1 Pi-hole network-wide ad blocking

Pi-hole is a DNS server that refuses to resolve advertising and tracking domains, so every device on the network (phones, TVs, laptops) is protected without installing anything on them. Install with its script or Docker image, set the router's DHCP to hand out the Pi's IP as the DNS server (or let Pi-hole run DHCP), and watch the dashboard at `http://pi/admin`: queries, blocked percentage, top clients.

PI-HOLE ANSWERS BLOCKED DOMAINS WITH 0.0.0.0

20.1.2 DNS and DHCP basics

DNS turns names into IP addresses: a client asks a resolver, which asks root, TLD (.com) and authoritative servers in turn and **caches** answers for their TTL (time to live). **DHCP** hands out addresses, the gateway and the DNS server. Only **one** DHCP server may run on a network: if Pi-hole does DHCP, disable it on the router.

RESOLVE.PY

```
import socket
for name in ("localhost", "raspberrypi.local.invalid"):
    try:
        print(f"{name:28} -> {socket.gethostbyname(name)}")
    except socket.gaierror as e:
        print(f"{name:28} -> not found ({e.strerror})")
```

OUTPUT

```
localhost                    -> 127.0.0.1
raspberrypi.local.invalid    -> not found (Name or service not known)
```

Record	Meaning	Example
A	name → IPv4	labpi.home → 192.168.1.42
AAAA	name → IPv6	→ fd00::42
CNAME	alias to another name	grafana.home → labpi.home
MX	mail server for a domain	example.com → mail.example.com
TXT	text (verification, SPF)	"v=spf1 …"

Pi-hole's **Local DNS** page lets you add A/CNAME records so every service has a friendly name (nas.home, ha.home).

20.1.3 WireGuard VPN server

WireGuard is a modern VPN: small, fast and secure (Curve25519 keys, ChaCha20 encryption). Each peer has a key pair; the server lists the public keys allowed to connect. Install easily with **PiVPN** (`curl -L https://install.pivpn.io | bash`) or the wg-easy Docker image, forward **one UDP port** (51820) on the router, and scan the generated QR code with the WireGuard phone app. Your phone then reaches Pi-hole, Home Assistant and dashboards securely from anywhere.

/ETC/WIREGUARD/WG0.CONF (SERVER)

```
[Interface]
Address = 10.6.0.1/24
ListenPort = 51820
PrivateKey = <server private key>

[Peer]                       # Lohit's phone
PublicKey = <phone public key>
AllowedIPs = 10.6.0.2/32
```

SHORTCUT TRICK

If your ISP uses CGNAT (no public IP), port forwarding is impossible: use **Tailscale** (WireGuard-based, no open ports) instead.

20.2 File servers and NAS

20.2.1 Samba file sharing

Samba shares folders using SMB, the Windows file-sharing protocol that macOS, Linux, Android and iOS also speak. `sudo apt install samba`, add a share and a Samba password, and open `\\labpi\shared` in Windows Explorer or `smb://labpi/shared` on a Mac.

APPEND TO /ETC/SAMBA/SMB.CONF, THEN: SUDO SMBPASSWD -A LOHIT && SUDO SYSTEMCTL RESTART SMBD

```
[shared]
   path = /mnt/data/shared
   valid users = lohit
   read only = no
   browseable = yes
   create mask = 0664
   directory mask = 0775
```

20.2.2 NAS with SSDs (OpenMediaVault)

OpenMediaVault (OMV) turns a Pi into a NAS with a web interface for disks, RAID/mergerfs, SMB/NFS shares, users, scheduled jobs and SMART health. Use a Pi 5 with SSDs on USB 3 (or a SATA/NVMe HAT) and a powered hub or enclosure: drives must not be powered from the Pi's USB ports. RAID is **not a backup**: keep a separate copy.

WORKED EXAMPLE

Real throughput

Gigabit Ethernet = 1000 Mbit/s ÷ 8 = 125 MB/s raw; after protocol overhead SMB delivers about **110 MB/s**. A 10 GB folder copies in about 10 000 ÷ 110 ≈ **91 s**. Over Wi-Fi at 30 MB/s it takes 5½ minutes.

20.2.3 Nextcloud personal cloud

Nextcloud is a self-hosted Google Drive/Photos alternative: file sync clients, automatic phone photo upload, calendars, contacts and sharing links. Easiest on a Pi as **Nextcloud All-in-One** (Docker) or NextcloudPi. It needs a Pi 4/5 with 4 GB+, an SSD for data and HTTPS for remote access.

20.3 Docker on Raspberry Pi

Docker runs applications in **containers**: isolated packages that include the program and all its dependencies, sharing the host's kernel. One command starts Pi-hole, Grafana or Home Assistant with exactly the right versions, and removing it leaves no mess.

Term	Meaning
Image	a read-only template (e.g. grafana/grafana:11.2.0) downloaded from a registry such as Docker Hub
Container	a running instance of an image
Volume / bind mount	storage that survives when the container is replaced (databases, config)
Port mapping	-p 8080:80 = host port 8080 → container port 80

Term	Meaning
Compose	a YAML file describing several containers that work together

20.3.1 Installing Docker

INSTALLING AND FIRST COMMANDS

```
curl -fsSL https://get.docker.com | sh       # official convenience script
sudo usermod -aG docker $USER                  # use docker without sudo (log out and in)
docker run --rm hello-world

docker ps -a                                   # list containers
docker images                                  # list images
docker logs -f <name>                          # follow a container's output
```

WATCH OUT

Members of the **docker** group effectively have root access to the host. Add only trusted admin users, and never expose the Docker socket (/var/run/docker.sock) to untrusted containers or the network.

20.3.2 ARM64 images

The Pi's CPU is **ARM64** (aarch64), not x86-64. An image runs only if it is published for your architecture; most popular images are "multi-arch" and Docker picks the right one automatically. "exec format error" means the image is x86-only. Check with `docker manifest inspect image | grep architecture` or the tags on Docker Hub; build your own multi-arch images with `docker buildx`.

20.3.3 Docker Compose stacks

COMPOSE.YAML: THREE SERVICES IN ONE STACK (PASSWORDS COME FROM A .ENV FILE)

```
services:
  pihole:
    image: pihole/pihole:latest
    ports: ["53:53/tcp", "53:53/udp", "8081:80/tcp"]
    environment:
      TZ: Asia/Kolkata
      FTLCONF_webserver_api_password: ${PIHOLE_PASSWORD}
    volumes: ["./pihole:/etc/pihole"]
    restart: unless-stopped

  grafana:
    image: grafana/grafana:latest
    ports: ["3000:3000"]
    volumes: ["grafana-data:/var/lib/grafana"]
    restart: unless-stopped

  samba:
    image: dperson/samba:latest
    ports: ["445:445"]
    volumes: ["/mnt/data/shared:/share"]
    command: '-s "shared;/share;yes;no;no;lohit" -u "lohit;${SMB_PASSWORD}"'
    restart: unless-stopped

volumes:
  grafana-data:
```

MANAGING THE STACK

```
docker compose up -d          # create and start everything in the background
docker compose ps             # status
docker compose logs -f grafana
docker compose pull && docker compose up -d   # update to newer images
docker compose down           # stop and remove (volumes are kept)
```

CHECK_COMPOSE.PY

```
import yaml          # python3-yaml: check a compose file before deploying it
stack = yaml.safe_load(open("compose.yaml"))
used = {}
for name, svc in stack["services"].items():
    for p in svc.get("ports", []):
        host = p.split(":")[0]
        if host in used:
            print(f"CONFLICT: host port {host} used by {used[host]} and {name}")
        used[host] = name
    if svc.get("restart") not in ("unless-stopped", "always"):
        print(f"warning: {name} will not restart after a reboot")
print("services:", ", ".join(stack["services"]))
print("host ports:", sorted(used, key=int))
```

OUTPUT

```
CONFLICT: host port 3000 used by grafana and uptime
warning: uptime will not restart after a reboot
```

```
services: pihole, grafana, uptime
host ports: ['53', '3000', '8081']
```

20.3.4 Portainer

Portainer is a web interface for Docker: see, start, stop, update and inspect containers, volumes, logs and stacks from a browser (port 9443). Install it as a container itself: `docker run -d -p 9443:9443 --restart=always -v /var/run/docker.sock:/var/run/docker.sock -v portainer_data:/data portainer/portainer-ce:lts`.

20.4 Web hosting

20.4.1 Hosting websites

Nginx (module 15) serves static sites from `/var/www` very efficiently; a Pi 4/5 can serve thousands of static requests per second. For public sites, use a tunnel (Cloudflare Tunnel) or a reverse proxy such as **Caddy**, **Traefik** or **Nginx Proxy Manager** that routes several domain names to different containers.

20.4.2 HTTPS with Let's Encrypt

Let's Encrypt issues free TLS certificates valid for **90 days**. **Certbot** (`sudo certbot --nginx -d pi.example.com`) proves you control the domain (HTTP-01 challenge on port 80, or DNS-01 via a TXT record, which also works for private servers and wildcard certificates), installs the certificate and renews it automatically with a timer. Caddy does all of this with no configuration.

> WORKED EXAMPLE
> **Renewal timing**
> Certificates last 90 days and certbot renews any certificate with less than 30 days left, so in normal operation each one is replaced after about **60 days**. Check with `sudo certbot renew --dry-run`.

20.5 Monitoring

20.5.1 Uptime monitoring

Uptime Kuma (a container with a friendly web UI) checks HTTP, TCP, ping, DNS and Docker targets every minute and alerts by Telegram, email or ntfy when something goes down, with status pages. Also monitor the monitor: an external free service (e.g. Healthchecks.io "dead man's switch") alerts you when your Pi stops checking in.

20.5.2 Prometheus and Grafana for system metrics

Prometheus scrapes metrics over HTTP every 15 s from exporters (**node_exporter** for CPU, memory, disk, network and temperature; cAdvisor for containers) and stores them as time series; **Grafana** graphs them (the "Node Exporter Full" dashboard is ready-made). Exporters publish plain text, so your own Python program can expose metrics too:

METRICS_TEXT.PY

```
def metrics(temp, load, disk_pct, up):
    lines = ["# HELP pi_cpu_temperature_celsius CPU temperature",
             "# TYPE pi_cpu_temperature_celsius gauge",
             f"pi_cpu_temperature_celsius {temp}",
             f"pi_load1 {load}",
             f'pi_disk_used_percent{{mount="/"}} {disk_pct}',
             f'pi_service_up{{name="pihole"}} {int(up)}']
    return "\n".join(lines)

print(metrics(51.2, 0.42, 37, True))
```

OUTPUT

```
# HELP pi_cpu_temperature_celsius CPU temperature
# TYPE pi_cpu_temperature_celsius gauge
pi_cpu_temperature_celsius 51.2
pi_load1 0.42
pi_disk_used_percent{mount="/"} 37
pi_service_up{name="pihole"} 1
```

20.6 Clusters

20.6.1 Introduction to Pi clusters

A **cluster** joins several Pis (often PoE-powered, in a rack) so they work as one system: running many containers with automatic failover, parallel computing with MPI, or learning the tools data centres use. A cluster does not make one program faster unless it is written to split its work.

20.6.2 Lightweight Kubernetes (k3s) overview

Kubernetes schedules containers across machines, restarts failed ones, moves them when a node dies and scales them up and down. **k3s** is a certified, lightweight Kubernetes distribution that installs with one command and runs well on Pis: one **server** (control plane) and several **agents** (workers). You describe the desired state in YAML (Deployments, Services) and `kubectl apply` makes it so.

BUILDING A K3S CLUSTER

```
curl -sfL https://get.k3s.io | sh -                                    # on the server node
sudo cat /var/lib/rancher/k3s/server/node-token                        # join token
curl -sfL https://get.k3s.io | K3S_URL=https://192.168.1.50:6443 K3S_TOKEN=<token> sh -   # on each
agent
kubectl get nodes
```

PRACTICE CHECKPOINT

Deploy a Docker Compose stack with Pi-hole, a file share and a monitoring dashboard, accessible securely over a VPN.

20.7 Practice questions

20.7.1 Basic level

Q1 What does Pi-hole block?

(a) viruses on USB sticks
(b) all internet traffic
(c) Wi-Fi intruders
(d) advertising and tracking domains, by refusing to resolve them in DNS

Q2 Which protocol turns names into IP addresses?

(a) DHCP (b) SMB (c) DNS (d) SSH

Q3 Which UDP port does WireGuard use by default?

(a) 51820 (b) 22 (c) 443 (d) 1883

Q4 Which software shares folders with Windows computers?

(a) Mosquitto (b) Portainer (c) Certbot (d) Samba

Q5 What is a Docker image?

(a) a running program
(b) a read-only template from which containers are created
(c) a photo
(d) a virtual disk of Windows

Q6 Which file describes a multi-container Docker application?

(a) Dockerfile.txt (b) compose.yaml (c) fstab (d) crontab

Q7 How long are Let's Encrypt certificates valid?

(a) 1 year (b) 7 days (c) forever (d) 90 days

Q8 Which tool gives a web UI for managing Docker?

(a) Portainer (b) Pi-hole (c) Nextcloud (d) Samba

Q9 Which DNS record type maps a name to an IPv4 address?

(a) A (b) MX (c) TXT (d) CNAME

20.7.2 Intermediate level

Q10 Why must only one DHCP server run on a network?

(a) DHCP uses too much bandwidth
(b) DNS would stop
(c) two would hand out conflicting addresses and settings
(d) it is illegal

Q11 What does -p 8080:80 mean in docker run?

(a) container port 8080 forwards to host 80
(b) 8080 MB of RAM

(c) run 80 copies
(d) host port 8080 forwards to container port 80

Q12 What does "exec format error" usually mean on a Pi?

(a) the disk is full
(b) the image was built for x86, not ARM64
(c) Docker is not installed
(d) the password is wrong

Q13 Why use volumes for a database container?

(a) the data survives when the container is replaced or updated
(b) volumes are faster than RAM
(c) to share the CPU
(d) Docker forbids writing inside containers

Q14 Why is RAID not a backup?

(a) RAID is slower
(b) RAID cannot store photos
(c) it mirrors mistakes, deletions and ransomware instantly; it only protects against disk failure
(d) RAID needs Windows

Q15 What does Prometheus node_exporter provide?

(a) system metrics (CPU, memory, disk, network, temperature) over HTTP
(b) DNS blocking
(c) VPN tunnels
(d) file sharing

Q16 What is k3s?

(a) a Pi-hole blocklist
(b) a lightweight Kubernetes distribution
(c) a WireGuard client
(d) a Samba version

Q17 Which Let's Encrypt challenge works for a server not reachable from the internet?

(a) HTTP-01 on port 80
(b) TLS-ALPN
(c) SSH challenge
(d) DNS-01 (a TXT record)

Q18 How long does a 25 GB backup take over gigabit SMB at about 110 MB/s?

(a) about 38 minutes
(b) about 23 seconds
(c) about 3.8 minutes
(d) about 2.5 hours

20.7.3 Advanced level

Q19 With certbot renewing at 30 days before expiry, how often is a 90-day certificate actually replaced?

(a) every 90 days
(b) every 30 days
(c) about every 60 days
(d) every day

Q20 Pi-hole reports 48 230 queries in a day of which 9 646 were blocked. Print the blocked percentage and the average queries per minute.

Q21 Estimate whether a Pi 5 with 8 GB can run this stack, given typical memory use (MB): Pi-hole 120, Grafana 180, Prometheus 300, Nextcloud + DB 900, Home Assistant 600, Portainer 60, plus 800 for

the OS. Leave 25 % free.

Q22 Write a small health checker that tests TCP ports on localhost (for example 22, 53, 8123) and reports UP or DOWN for each, as Uptime Kuma does.

Q23 Generate the text of a WireGuard client config for a phone given its address, keys, the server endpoint and "route everything through the VPN".

Q24 Which design gives safe remote access to Pi-hole, Grafana and Home Assistant?

(a) forward ports 80, 3000 and 8123 to the Pi

(b) disable passwords for convenience

(c) put the Pi in the router DMZ

(d) only the WireGuard (or Tailscale) VPN is exposed; services stay on the LAN

Q25 A Docker stack must come back automatically after a power cut. Which setting ensures this?

(a) docker compose down

(b) restart: unless-stopped on each service and Docker enabled at boot

(c) restart: no

(d) running docker compose up without -d

20.8 Solutions

Q1 **Answer: (d) advertising and tracking domains, by refusing to resolve them in DNS**

A DNS sinkhole.

Q2 **Answer: (c) DNS**

DNS.

Q3 **Answer: (a) 51820**

51820.

Q4 **Answer: (d) Samba**

SMB via **Samba**.

Q5 **Answer: (b) a read-only template from which containers are created**

Image → container.

Q6 **Answer: (b) compose.yaml**

Docker Compose.

Q7 **Answer: (d) 90 days**

90 days, renewed automatically.

Q8 **Answer: (a) Portainer**

Portainer.

Q9 **Answer: (a) A**

A record; AAAA is IPv6.

Q10 **Answer: (c) two would hand out conflicting addresses and settings**

Disable the router's DHCP if Pi-hole does it.

Q11 **Answer: (d) host port 8080 forwards to container port 80**

host:container.

Q12 **Answer: (b) the image was built for x86, not ARM64**

Wrong architecture.

Q13 **Answer: (a) the data survives when the container is replaced or updated**

Containers are disposable; volumes persist.

Q14 **Answer: (c) it mirrors mistakes, deletions and ransomware instantly; it only protects against disk failure**

Keep separate copies.

Q15 **Answer: (a) system metrics (CPU, memory, disk, network, temperature) over HTTP**

Scraped every 15 s.

Q16 **Answer: (b) a lightweight Kubernetes distribution**

Certified, small Kubernetes.

Q17 **Answer: (d) DNS-01 (a TXT record)**

DNS-01 never contacts the server.

Q18 **Answer: (c) about 3.8 minutes**

25 000 ÷ 110 = 227 s ≈ **3.8 min**.

Q19 **Answer: (c) about every 60 days**

90 - 30 = **60**.

Q20 **Answer: Solution program**

9646 ÷ 48 230 = 20.0 %; 48 230 ÷ 1440 ≈ 33.5 per minute.

```
total, blocked = 48230, 9646
print(f"blocked {blocked / total:.1%}")
print(f"{total / (24 * 60):.1f} queries per minute")
```

OUTPUT

```
blocked 20.0%
33.5 queries per minute
```

Q21 **Answer: Solution program**

Sum the estimates and compare with 75 % of RAM. 2960 MB is 36 %: plenty of room (a 4 GB Pi would also manage).

```
use = {"OS": 800, "pihole": 120, "grafana": 180, "prometheus": 300,
       "nextcloud+db": 900, "homeassistant": 600, "portainer": 60}
total = sum(use.values()); ram = 8 * 1024
print(f"needed {total} MB of {ram} MB ({total / ram:.0%})")
print("fits with 25 % headroom" if total <= 0.75 * ram else "too tight")
```

OUTPUT

```
needed 2960 MB of 8192 MB (36%)
fits with 25 % headroom
```

Q22 **Answer: Solution program**

A successful TCP connection means something is listening. The demo starts a listening socket on port 18123 so one check succeeds.

```
import socket
def port_up(host, port, timeout=0.5):
    try:
        with socket.create_connection((host, port), timeout=timeout):
            return True
    except OSError:
        return False

srv = socket.socket(); srv.bind(("127.0.0.1", 18123)); srv.listen()   # pretend service
for name, port in (("home-assistant", 18123), ("unused", 18999)):
    print(f"{name:15} {port}: {'UP' if port_up('127.0.0.1', port) else 'DOWN'}")
```

OUTPUT

```
home-assistant  18123: UP
unused          18999: DOWN
```

Q23 **Answer: Solution program**

AllowedIPs 0.0.0.0/0 sends all traffic through the tunnel (and DNS through Pi-hole); a split tunnel would list only home subnets.

```
def client_conf(addr, priv, server_pub, endpoint, dns="10.6.0.1", full=True):
    allowed = "0.0.0.0/0, ::/0" if full else "10.6.0.0/24, 192.168.1.0/24"
    return f"""[Interface]
Address = {addr}/24
PrivateKey = {priv}
DNS = {dns}

[Peer]
PublicKey = {server_pub}
Endpoint = {endpoint}:51820
AllowedIPs = {allowed}
PersistentKeepalive = 25"""
print(client_conf("10.6.0.2", "<phone-private>", "<server-public>", "home.example.com"))
```

OUTPUT

```
[Interface]
Address = 10.6.0.2/24
PrivateKey = <phone-private>
DNS = 10.6.0.1

[Peer]
PublicKey = <server-public>
Endpoint = home.example.com:51820
AllowedIPs = 0.0.0.0/0, ::/0
PersistentKeepalive = 25
```

Q24 **Answer: (d) only the WireGuard (or Tailscale) VPN is exposed; services stay on the LAN**

One hardened entry point.

Q25 **Answer: (b) restart: unless-stopped on each service and Docker enabled at boot**

Restart policy + docker.service enabled.

Network services: Pi-hole (DNS sinkhole, one DHCP server only), WireGuard/PiVPN or Tailscale for remote access. Files: Samba, OpenMediaVault, Nextcloud on SSDs (RAID ≠ backup). Docker: ARM64 multi-arch images, volumes for data, Compose stacks with restart policies, Portainer UI; docker group = root. HTTPS with Let's Encrypt (90 days, auto-renew). Monitor with Uptime Kuma, Prometheus + Grafana. k3s for clusters.

MODULE 21 · ADVANCED

Security and hardening

Minimum duration **2 days** | Level **Advanced**

A connected Pi is a computer on the internet, so it must be secured like one. This module covers the threats to IoT devices (default passwords, open ports, botnets), account security (strong passwords, no default users, key-only SSH, sudo policies), network security (ufw and nftables firewalls, fail2ban, closing unused services), regular and unattended updates and firmware, securing applications (HTTPS, secrets outside code, MQTT authentication and TLS), and physical and data security (disk encryption, secure boot), followed by 25 solved questions and programs.

IN THIS MODULE

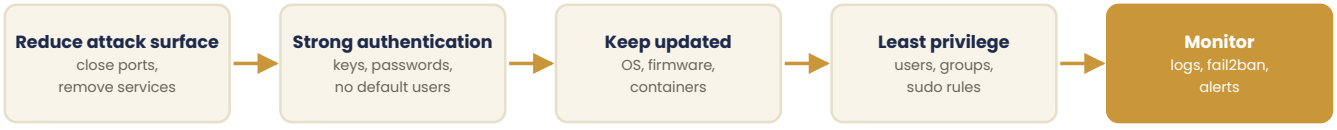
21.1	Threats to IoT devices	21.2	Account security
21.3	Network security	21.4	Updates
21.5	Securing applications	21.6	Physical and data security
21.7	Practice questions	21.8	Solutions

21.1 Threats to IoT devices

21.1.1 Default passwords, open ports and botnets

Automated scanners probe every IPv4 address on the internet within hours, trying default logins (pi/raspberry, admin/admin) on SSH, Telnet and web panels. A compromised device is silently added to a **botnet** such as Mirai and used for attacks, crypto-mining or spam, or as a foothold into your home network. Most incidents come from the same few mistakes:

Mistake	Fix
default or weak passwords	unique strong passwords, key-only SSH
services exposed to the internet	firewall, VPN instead of port forwarding
outdated software	regular and unattended updates
secrets in code or Git	environment files with tight permissions
everything running as root	least privilege: separate users, no needless sudo
no monitoring	log review, fail2ban, alerts



DEFENCE IN DEPTH: SEVERAL INDEPENDENT LAYERS

21.2 Account security

21.2.1 Strong passwords and no default users

Since 2022 Raspberry Pi OS has **no default "pi" user**: you create your own in Imager. Use long passphrases (four or more random words) or a password manager. Strength grows with length: a password's **entropy** is length × log2(alphabet size) bits.

PASSWORD_STRENGTH.PY

```
import math, secrets

def entropy_bits(length, alphabet):
    return length * math.log2(alphabet)

def crack_time(bits, guesses_per_sec=1e10):          # a fast offline attacker
    s = 2 ** bits / guesses_per_sec
    for unit, size in (("years", 3.15e7), ("days", 86400), ("hours", 3600), ("minutes", 60)):
        if s >= size:
            return f"{s / size:,.0f} {unit}"
    return f"{s:.0f} seconds"

for desc, length, alpha in [("8 lower-case letters", 8, 26), ("8 mixed + digits + symbols", 8, 94),
                            ("12 mixed + digits + symbols", 12, 94), ("4 random words (7776-word
list)", 4, 7776),
                            ("6 random words", 6, 7776)]:
    b = entropy_bits(length, alpha)
    print(f"{desc:34} {b:5.1f} bits  ~{crack_time(b)}")

WORDS = ["copper", "lantern", "mango", "orbit", "velvet", "tiger", "quartz", "harbor"]
print("example passphrase:", "-".join(secrets.choice(WORDS) for _ in range(4)), "(use a 7776-word
list!)")
```

OUTPUT

```
8 lower-case letters               37.6 bits  ~21 seconds
8 mixed + digits + symbols         52.4 bits  ~7 days
12 mixed + digits + symbols        78.7 bits  ~1,510,858 years
4 random words (7776-word list)    51.7 bits  ~4 days
6 random words                     77.5 bits  ~701,822 years
example passphrase: lantern-velvet-quartz-tiger (use a 7776-word list!)
```

Always use the **secrets** module (not random) for anything security-related: it uses the operating system's cryptographic random source.

21.2.2 Key-only SSH and disabling password login

Once your SSH key works (module 5), turn off password logins so guessing is impossible:

SSHD HARDENING DROP-IN FILE

```
# /etc/ssh/sshd_config.d/10-hardening.conf
PasswordAuthentication no
KbdInteractiveAuthentication no
PermitRootLogin no
PubkeyAuthentication yes
AllowUsers lohit
MaxAuthTries 3
```

APPLY SAFELY

```
sudo sshd -t                    # test the configuration (no output = OK)
sudo systemctl reload ssh
# keep your current session open and test a NEW login in a second terminal!
```

WATCH OUT

Before disabling passwords, confirm key login works in a **second** terminal while the first stays connected. If you lock yourself out of a headless Pi you need a keyboard and screen, or to edit the card on another computer.

21.2.3 sudo policies

Users in the **sudo** group can run anything as root. Raspberry Pi OS lets the first user use sudo **without a password** (file /etc/sudoers.d/010_pi-nopasswd); on a server, remove that file so an attacker with your session still needs the password. Grant service accounts only specific commands:

LEAST-PRIVILEGE SUDO RULE

```
# sudo visudo -f /etc/sudoers.d/backup
backupsvc ALL=(root) NOPASSWD: /usr/bin/rsync, /usr/bin/systemctl restart nginx
```

21.3 Network security

21.3.1 Firewalls: ufw and nftables

A firewall decides which network connections are allowed. **ufw** (Uncomplicated Firewall) is a friendly front end to the kernel's **nftables**. Start from **deny all incoming** and allow only what you need, preferably only from your LAN:

A DEFAULT-DENY FIREWALL

```
sudo apt install ufw
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow from 192.168.1.0/24 to any port 22 proto tcp     # SSH from the LAN only
sudo ufw allow 51820/udp                                        # WireGuard from anywhere
sudo ufw allow from 192.168.1.0/24 to any port 8123             # Home Assistant, LAN only
sudo ufw limit 22/tcp                                           # rate-limit repeated SSH attempts
sudo ufw enable
sudo ufw status numbered
```

WATCH OUT

Allow SSH **before** running `ufw enable` on a headless Pi, or you will cut off your own connection. Docker publishes container ports by editing firewall rules itself and can bypass ufw: bind private services to 127.0.0.1 (ports: "127.0.0.1:3000:3000") or the VPN interface.

FIREWALL_LOGIC.PY

```
import ipaddress

RULES = [  # (source network, port, protocol, action) - first match wins, default deny
    ("192.168.1.0/24", 22, "tcp", "allow"),
    ("0.0.0.0/0", 51820, "udp", "allow"),
    ("192.168.1.0/24", 8123, "tcp", "allow"),
]
def decide(src, port, proto):
    for net, p, pr, action in RULES:
        if ipaddress.ip_address(src) in ipaddress.ip_network(net) and p == port and pr == proto:
            return action
    return "deny (default)"

for src, port, proto in [("192.168.1.20", 22, "tcp"), ("203.0.113.9", 22, "tcp"),
                         ("203.0.113.9", 51820, "udp"), ("192.168.1.31", 3306, "tcp")]:
    print(f"{src:14} -> {port:5}/{proto}: {decide(src, port, proto)}")
```

OUTPUT

```
192.168.1.20   ->    22/tcp: allow
203.0.113.9    ->    22/tcp: deny (default)
203.0.113.9    -> 51820/udp: allow
192.168.1.31   ->  3306/tcp: deny (default)
```

21.3.2 fail2ban

fail2ban watches logs for repeated failures (bad SSH logins, web panel attacks) and bans the offending IP address in the firewall for a while. Configure jails in `/etc/fail2ban/jail.local`; on Raspberry Pi OS set `backend = systemd` because logs are in the journal.

/ETC/FAIL2BAN/JAIL.LOCAL

```
[sshd]
enabled  = true
backend  = systemd
maxretry = 5
findtime = 10m
bantime  = 1h
```

FAIL2BAN_LOGIC.PY

```
import re
from collections import defaultdict
from datetime import datetime, timedelta

LOG = """Mar 15 03:12:01 labpi sshd[811]: Failed password for invalid user admin from 203.0.113.9
port 51022 ssh2
Mar 15 03:12:04 labpi sshd[811]: Failed password for invalid user admin from 203.0.113.9 port 51022
ssh2
Mar 15 03:12:07 labpi sshd[815]: Failed password for root from 203.0.113.9 port 51030 ssh2
Mar 15 03:12:09 labpi sshd[815]: Failed password for root from 203.0.113.9 port 51030 ssh2
Mar 15 03:12:12 labpi sshd[820]: Failed password for pi from 203.0.113.9 port 51041 ssh2
Mar 15 08:30:44 labpi sshd[901]: Failed password for lohit from 192.168.1.20 port 60111 ssh2
Mar 15 08:30:51 labpi sshd[901]: Accepted publickey for lohit from 192.168.1.20 port 60111 ssh2"""

pat = re.compile(r"^(\w+ \d+ [\d:]+) .*Failed password for (?:invalid user )?(\S+) from (\S+)")
fails = defaultdict(list)
for line in LOG.splitlines():
    m = pat.search(line)
    if m:
        t = datetime.strptime("2026 " + m.group(1), "%Y %b %d %H:%M:%S")
        fails[m.group(3)].append((t, m.group(2)))

MAXRETRY, FINDTIME = 5, timedelta(minutes=10)
for ip, events in fails.items():
    times = [t for t, _ in events]
    worst = max(sum(1 for u in times if t <= u < t + FINDTIME) for t in times)
    users = sorted({u for _, u in events})
    verdict = "BAN" if worst >= MAXRETRY else "ok"
    print(f"{ip:14} {len(events)} failures, max {worst} in 10 min, users {users} -> {verdict}")
```

OUTPUT

```
203.0.113.9    5 failures, max 5 in 10 min, users ['admin', 'pi', 'root'] -> BAN
192.168.1.20   1 failures, max 1 in 10 min, users ['lohit'] -> ok
```

21.3.3 Closing unused services

FINDING AND REMOVING LISTENERS

```
sudo ss -tulpn                     # which programs listen on which ports
systemctl list-unit-files --state=enabled
sudo systemctl disable --now cups avahi-daemon   # examples: printing, mDNS if not needed
sudo apt purge telnetd vsftpd                     # never run plain-text login services
```

MINI_PORTSCAN.PY

```
import socket

def scan(host, ports, timeout=0.3):
    open_ports = []
    for p in ports:
        s = socket.socket(); s.settimeout(timeout)
        if s.connect_ex((host, p)) == 0:
            open_ports.append(p)
        s.close()
    return open_ports

listeners = []                                  # demo: start two "services"
for p in (18022, 18080):
    srv = socket.socket(); srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", p)); srv.listen(); listeners.append(srv)
found = scan("127.0.0.1", range(18000, 18100))
print("open ports:", found)
```

OUTPUT

```
open ports: [18022, 18080]
```

From another computer, **nmap** gives the attacker's view: `nmap -sV labpi.local` lists open ports and service versions. Scan only machines you own or are authorised to test.

21.4 Updates

21.4.1 Regular and unattended upgrades

KEEPING THE OS PATCHED

```
sudo apt update && sudo apt full-upgrade -y
sudo apt install unattended-upgrades
sudo dpkg-reconfigure --priority=low unattended-upgrades    # enable automatic security updates
cat /var/log/unattended-upgrades/unattended-upgrades.log
```

Unattended upgrades install security fixes daily; you still do full upgrades by hand regularly and reboot after kernel updates. Update containers too (`docker compose pull`), and Python packages in venvs (`pip list --outdated`).

21.4.2 Firmware and EEPROM updates

The bootloader EEPROM (Pi 4/5) receives security and stability fixes: `sudo rpi-eeprom-update` shows the status and `-a` applies updates at the next reboot. HATs and attached microcontrollers have firmware too; keep track of their versions.

21.5 Securing applications

21.5.1 HTTPS and certificates

HTTPS (TLS) encrypts traffic and proves the server's identity, so passwords and API keys cannot be read on the network. Use Let's Encrypt for public names (module 20), or Tailscale's built-in certificates or a private CA (mkcert, step-ca) for LAN-only services. Redirect HTTP to HTTPS and never ignore certificate warnings in your own code (`verify=False` disables the identity check).

21.5.2 Storing secrets outside code

SECRETS_DEMO.PY

```
import os, stat

with open(".env", "w") as f:                          # created once, by hand, on the Pi
    f.write("TG_TOKEN=123456:demo-token\nMQTT_PASSWORD=s3cret\n")
os.chmod(".env", 0o600)                               # owner read/write only

def load_env(path=".env"):
    mode = stat.S_IMODE(os.stat(path).st_mode)
    if mode & 0o077:
        raise PermissionError(f"{path} is readable by others ({oct(mode)}); run chmod 600")
    for line in open(path):
        if "=" in line and not line.startswith("#"):
            k, v = line.strip().split("=", 1)
            os.environ.setdefault(k, v)

load_env()
token = os.environ["TG_TOKEN"]
print("token loaded:", token[:6] + "..." + token[-4:])
print("permissions:", oct(stat.S_IMODE(os.stat(".env").st_mode)))
```

OUTPUT

```
token loaded: 123456...oken
permissions: 0o600
```

Add `.env` to `.gitignore`, load it with python-dotenv or systemd's `EnvironmentFile=`, and rotate (replace) any key that was ever committed: Git history keeps it forever.

21.5.3 MQTT authentication and TLS

Set `allow_anonymous false` with a password file, use **ACLs** so each device can only publish and subscribe to its own topics, and enable TLS on 8883 for anything leaving the LAN (module 17).

MOSQUITTO ACL: LEAST PRIVILEGE PER DEVICE (ACL_FILE /ETC/MOSQUITTO/ACL)

```
# /etc/mosquitto/acl
user esp_garage
topic write home/garage/#
topic read  home/garage/set

user dashboard
topic read  home/#
```

21.6 Physical and data security

21.6.1 Disk encryption basics

Anyone with physical access can remove the SD card and read every file on another computer, including Wi-Fi passwords and keys. **LUKS** (cryptsetup) encrypts a partition; the key must be entered at boot (or unlocked over SSH with dropbear-initramfs, or with a network-bound key such as Clevis/Tang). Encrypting just a data partition on a USB SSD is the easiest start. Expect some speed loss on Pis without hardware AES (Pi 4); the Pi 5 has AES instructions.

21.6.2 Secure boot on Pi 4/5 and Compute Modules overview

Secure boot lets the bootloader accept only boot images signed with your private key, so a modified card will not start. On the Pi 4, Pi 5 and Compute Modules this is done by writing a customer public key and locking settings in OTP (one-time-programmable) memory with the rpi-eeprom and usbboot tools. It is **irreversible**, intended for products (module 26). Combined with an encrypted root it protects devices deployed in public places.

PRACTICE CHECKPOINT

Audit and harden one of your Pi servers: run a port scan, configure a firewall and fail2ban, switch to key-only SSH and document every change.

21.7 Practice questions

21.7.1 Basic level

Q1 Why are default passwords dangerous?

(a) they are too long
(b) bots try them automatically on every reachable device
(c) they slow down the Pi
(d) they disable SSH

Q2 Which sshd setting disables password login?

(a) PasswordAuthentication no
(b) PermitRootLogin yes
(c) PubkeyAuthentication no
(d) Port 22

Q3 Which firewall front end is easiest on Raspberry Pi OS?

(a) nmap **(b)** fail2ban **(c)** ufw **(d)** iptables-save

Q4 What does fail2ban do?

(a) encrypts disks
(b) updates packages
(c) scans ports
(d) bans IP addresses that show repeated failed logins

Q5 Which command lists listening ports with their programs?

(a) ls /ports **(b)** ping -l **(c)** df -h **(d)** sudo ss -tulpn

Q6 Which package installs security updates automatically?

(a) fail2ban
(b) ufw
(c) unattended-upgrades
(d) certbot

Q7 Which Python module should generate passwords and tokens?

(a) secrets **(b)** random **(c)** math **(d)** time

Q8 What does LUKS provide?

(a) a firewall
(b) a VPN
(c) HTTPS certificates
(d) disk/partition encryption

Q9 Since 2022, what is true about the "pi" user on new Raspberry Pi OS installs?

(a) it still has the password raspberry
(b) it is the root account
(c) there is no default pi user; you create your own account
(d) it cannot use sudo

21.7.2 Intermediate level

Q10 What is the safe default incoming firewall policy?

(a) allow all

(b) deny all, then allow only needed ports
(c) allow all from the internet, deny the LAN
(d) no firewall on Linux

Q11 Why test a new SSH configuration in a second terminal?

(a) sshd requires two sessions
(b) so you can still fix it from the first session if login breaks
(c) to make it faster
(d) to share the key

Q12 Why can Docker-published ports bypass ufw?

(a) Docker disables the firewall
(b) ufw does not work on ARM
(c) containers use Bluetooth
(d) Docker inserts its own firewall rules ahead of ufw's

Q13 What file permission should a .env file with secrets have?

(a) 600 (b) 644 (c) 777 (d) 755

Q14 Why must a secret that was committed to Git be replaced?

(a) Git encrypts it badly
(b) it expires automatically
(c) it stays in the repository history even after deletion
(d) Git uploads it to the Pi

Q15 What does an MQTT ACL do?

(a) encrypts payloads
(b) compresses messages
(c) sets QoS
(d) limits which topics each user may read or write

Q16 Why is secure boot irreversible on Pi 4/5?

(a) the SD card is glued in
(b) the key hash is written into one-time-programmable memory
(c) the EEPROM is deleted
(d) the Pi stops accepting updates

Q17 What is the attacker's view tool for checking your open ports from another machine?

(a) nmap (b) htop (c) vcgencmd (d) raspi-config

Q18 How many bits of entropy does a 5-word passphrase from a 7776-word list have?

(a) about 38.8 (b) about 25 (c) about 64.6 (d) about 129

21.7.3 Advanced level

Q19 A rule allows SSH from 192.168.1.0/24 only. Is 192.168.2.15 allowed?

(a) no (b) yes

(c) only on weekends
(d) only with a password

Q20 Write check_password(p) returning a list of problems: shorter than 12, no digit, no upper case, or in a list of common passwords. Test three passwords.

Q21 Generate a secure 32-character API token and a 6-digit PIN with the secrets module.

Q22 Count failed SSH logins per IP address in auth-log lines and list addresses with 3 or more failures.

Q23 Check file permissions: list files in a folder that are readable by "others" and should be private (names ending in .env, .key or .pem).

Q24 Which hardening plan is complete for an internet-connected Pi server?

(a) change the SSH port only
(b) key-only SSH, no root login, ufw default deny, fail2ban, unattended upgrades, secrets in 600 files, services behind a VPN, documented changes
(c) install an antivirus and nothing else
(d) disable the firewall to avoid problems

Q25 Changing SSH from port 22 to 2222 mainly

(a) makes SSH unbreakable
(b) encrypts better
(c) disables passwords
(d) reduces log noise from bots, but is not real security

21.8 Solutions

Q1 **Answer: (b) bots try them automatically on every reachable device**
Mirai-style botnets.

Q2 **Answer: (a) PasswordAuthentication no**
Keys only.

Q3 **Answer: (c) ufw**
ufw over nftables.

Q4 **Answer: (d) bans IP addresses that show repeated failed logins**
Log-driven bans.

Q5 **Answer: (d) sudo ss -tulpn**
`ss -tulpn`.

Q6 **Answer: (c) unattended-upgrades**
unattended-upgrades.

Q7 **Answer: (a) secrets**

Cryptographically secure.

Q8 **Answer: (d) disk/partition encryption**

Linux Unified Key Setup.

Q9 **Answer: (c) there is no default pi user; you create your own account**

Created in Imager or the first-boot wizard.

Q10 **Answer: (b) deny all, then allow only needed ports**

Default deny.

Q11 **Answer: (b) so you can still fix it from the first session if login breaks**

Avoid lock-out.

Q12 **Answer: (d) Docker inserts its own firewall rules ahead of ufw's**

Bind private services to 127.0.0.1.

Q13 **Answer: (a) 600**

Owner read/write only.

Q14 **Answer: (c) it stays in the repository history even after deletion**

Rotate exposed keys.

Q15 **Answer: (d) limits which topics each user may read or write**

Least privilege per device.

Q16 **Answer: (b) the key hash is written into one-time-programmable memory**

OTP cannot be erased.

Q17 **Answer: (a) nmap**

Only scan your own systems.

Q18 **Answer: (c) about 64.6**

5 × log2(7776) = 5 × 12.92 = **64.6**.

Q19 **Answer: (a) no**

/24 fixes 192.168.1.

Q20 **Answer: Solution program**

Simple rules catch the worst passwords; length matters most.

```
COMMON = {"password", "raspberry", "admin123", "qwerty123"}
def check_password(p):
    problems = []
    if len(p) < 12: problems.append("too short")
    if not any(c.isdigit() for c in p): problems.append("no digit")
    if not any(c.isupper() for c in p): problems.append("no upper case")
    if p.lower() in COMMON: problems.append("common password")
    return problems or ["ok"]
for pw in ("raspberry", "Mango-Orbit-7-Velvet", "lanternharbor42"):
    print(f"{pw:22} {check_password(pw)}")
```

OUTPUT

```
raspberry              ['too short', 'no digit', 'no upper case', 'common password']
Mango-Orbit-7-Velvet   ['ok']
lanternharbor42        ['no upper case']
```

Q21 **Answer: Solution program**

62-character alphabet × 32 characters ≈ 190 bits. (The values themselves are random, so only their sizes are printed.)

```
import secrets, string
alphabet = string.ascii_letters + string.digits
token = "".join(secrets.choice(alphabet) for _ in range(32))
pin = f"{secrets.randbelow(10**6):06d}"
print("token length:", len(token), "| alphabet size:", len(alphabet))
print("PIN digits:", len(pin))
print("token_urlsafe example length:", len(secrets.token_urlsafe(24)))
```

OUTPUT

```
token length: 32 | alphabet size: 62
PIN digits: 6
token_urlsafe example length: 32
```

Q22 **Answer: Solution program**

A regular expression captures the IP; Counter tallies them.

```
import re
from collections import Counter
lines = ["Failed password for root from 198.51.100.7 port 4410 ssh2",
         "Failed password for invalid user test from 198.51.100.7 port 4412 ssh2",
         "Accepted publickey for lohit from 192.168.1.20 port 60111 ssh2",
         "Failed password for admin from 198.51.100.7 port 4419 ssh2",
         "Failed password for lohit from 192.168.1.20 port 60120 ssh2"]
c = Counter(m.group(1) for l in lines if (m := re.search(r"Failed password .* from (\S+)", l)))
print(c)
print("ban candidates:", [ip for ip, n in c.items() if n >= 3])
```

OUTPUT

```
Counter({'198.51.100.7': 3, '192.168.1.20': 1})
ban candidates: ['198.51.100.7']
```

Q23 **Answer: Solution program**

Mode bit 0o004 is "others can read"; group read (0o040) may be acceptable for a shared service group.

```
import os, stat
for name, mode in (("app.env", 0o644), ("server.key", 0o600), ("cert.pem", 0o640), ("ca.pem",
0o604)):
    open(name, "w").close(); os.chmod(name, mode)
for name in sorted(os.listdir()):
    if name.endswith((".env", ".key", ".pem")):
        m = stat.S_IMODE(os.stat(name).st_mode)
        flag = "FIX: chmod 600" if m & 0o004 else "ok"
        print(f"{name:12} {oct(m)}  {flag}")
```

OUTPUT

```
app.env      0o644  FIX: chmod 600
ca.pem       0o604  FIX: chmod 600
cert.pem     0o640  ok
server.key   0o600  ok
```

Q24 **Answer: (b) key-only SSH, no root login, ufw default deny, fail2ban, unattended upgrades, secrets in 600 files, services behind a VPN, documented changes**

Layers of defence, not one trick.

Q25 **Answer: (d) reduces log noise from bots, but is not real security**

Obscurity is not a substitute for keys and a firewall.

No default users; long passphrases from secrets. Key-only SSH (PasswordAuthentication no, PermitRootLogin no), tested in a second session. ufw default deny, allow only needed ports from the LAN; mind Docker's rules. fail2ban with backend=systemd. ss -tulpn to find and close services. Unattended upgrades + EEPROM updates. HTTPS everywhere, secrets in chmod 600 .env files, MQTT passwords + ACLs + TLS. LUKS for data at rest; secure boot for products.

MODULE 22 · ADVANCED

Computer vision with OpenCV

Minimum duration **3 days** | Level **Advanced**

Computer vision lets the Pi see and understand images, powering robots, counters and inspection systems. This module installs OpenCV and feeds it Picamera2 frames, explains pixels, channels and colour spaces, reads, saves, resizes, crops and rotates images, then applies thresholding, blurring, Canny edges, morphology and contours for shape detection, tracks objects by colour with HSV masks, detects faces, QR codes, ArUco markers and motion, and speeds everything up on the Pi with lower resolution, threading and FPS measurement, followed by 25 solved questions and programs. Every image in this module was produced by running the program shown with OpenCV.

IN THIS MODULE

22.1 OpenCV setup
22.2 Image basics
22.3 Image processing
22.4 Colour tracking
22.5 Detection
22.6 Performance on the Pi
22.7 Practice questions
22.8 Solutions

22.1 OpenCV setup

22.1.1 Installing OpenCV with apt or pip

TWO WAYS TO INSTALL

```
sudo apt install python3-opencv                    # system package: quick, matches the OS
# or, inside a venv created with --system-site-packages:
pip install opencv-python-headless                  # newer version, no GUI functions
python3 -c "import cv2; print(cv2.__version__)"
```

On a headless Pi, `cv2.imshow()` has no screen to draw on: save images with `cv2.imwrite()`, stream them to a web page (module 15), or use VNC. The apt package is the easiest; pip wheels are newer.

22.1.2 Using Picamera2 frames with OpenCV

PICAM_TO_CV.PY

```
from picamera2 import Picamera2
import cv2

picam2 = Picamera2()
picam2.configure(picam2.create_preview_configuration(main={"size": (640, 480), "format": "RGB888"}))
picam2.start()
frame = picam2.capture_array()                # "RGB888" is stored B,G,R in memory: OpenCV's order
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
print("frame", frame.shape, frame.dtype, "| gray", gray.shape, "| mean brightness",
round(float(gray.mean()), 1))
cv2.imwrite("snapshot.jpg", frame)
```

OUTPUT

```
frame (480, 640, 3) uint8 | gray (480, 640) | mean brightness 74.5
```

COLOUR ORDER

Picamera2's format names describe pixels as a 32-bit word, so "RGB888" arrives in memory as B, G, R: exactly OpenCV's BGR order. If colours look swapped (blue faces), convert with cv2.cvtColor(frame, cv2.COLOR_RGB2BGR).

22.2 Image basics

22.2.1 Pixels, channels and colour spaces (BGR, HSV, grayscale)

The examples in this module use a small test scene drawn by this helper file, so every result can be checked:

SCENE.PY

```
import cv2, numpy as np
def make_scene():
    img = np.full((240, 360, 3), (200, 205, 210), np.uint8)          # light grey background (BGR)
    cv2.rectangle(img, (20, 30), (110, 110), (60, 60, 200), -1)       # red square
    cv2.circle(img, (180, 75), 42, (40, 160, 40), -1)                 # green circle
    pts = np.array([[270, 30], [330, 120], [215, 120]], np.int32)
    cv2.fillPoly(img, [pts], (200, 110, 30))                          # blue triangle
    cv2.circle(img, (90, 185), 30, (0, 140, 255), -1)                 # orange ball
    cv2.rectangle(img, (180, 150), (330, 215), (90, 90, 90), -1)      # dark grey bar
    noise = np.random.default_rng(0).integers(-12, 13, img.shape)
    return np.clip(img.astype(int) + noise, 0, 255).astype(np.uint8)
```

An image is a NumPy array: **height × width × channels**, each value 0–255 (uint8). OpenCV stores colour as **BGR** (blue, green, red). **Grayscale** has one channel. **HSV** separates Hue (colour, 0–179 in OpenCV), Saturation (0–255, grey → vivid) and Value (0–255, brightness), which makes colour detection robust to lighting.

PIXELS.PY

```
import cv2, numpy as np
from scene import make_scene
img = make_scene()
print("shape:", img.shape, "dtype:", img.dtype, "pixels:", img.shape[0] * img.shape[1])
b, g, r = img[70, 65]                      # row (y) first, then column (x)
print("pixel at x=65, y=70 (red square): B,G,R =", (int(b), int(g), int(r)))
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
for name, (x, y) in {"red square": (65, 70), "green circle": (180, 75), "orange ball": (90,
185)}.items():
    h, s, v = hsv[y, x]
    print(f"{name:13} H={int(h):3d} S={int(s):3d} V={int(v):3d}")
```

OUTPUT

```
shape: (240, 360, 3) dtype: uint8 pixels: 86400
pixel at x=65, y=70 (red square): B,G,R = (67, 56, 203)
red square    H=178 S=185 V=203
green circle  H= 62 S=194 V=160
orange ball   H= 17 S=255 V=255
```

Colour	Approximate OpenCV hue (0–179)
red	0–10 and 170–179 (wraps round)
orange	10–25
yellow	25–35
green	35–85
blue	100–130
purple	130–160

SHORTCUT TRICK

Remember **image[y, x]**: rows (y) come first. And OpenCV hue is degrees ÷ 2, so it runs 0–179 instead of 0–359.

22.2.2 Reading, showing and saving images

READING AND SAVING

```
img = cv2.imread("photo.jpg")            # returns None if the file is missing: check it!
if img is None: raise FileNotFoundError("photo.jpg")
cv2.imwrite("out.png", img)              # format chosen by extension
cv2.imwrite("small.jpg", img, [cv2.IMWRITE_JPEG_QUALITY, 80])
cv2.imshow("view", img); cv2.waitKey(0)  # only with a desktop
```

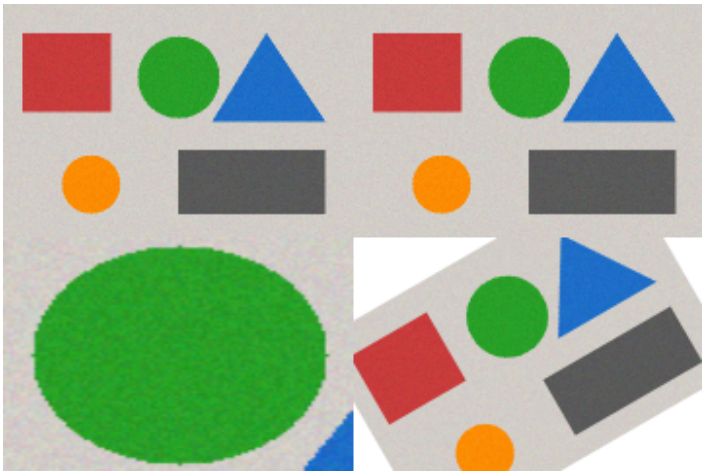
22.2.3 Resizing, cropping and rotating

GEOMETRY.PY

```
import cv2, numpy as np
from scene import make_scene
img = make_scene()
small = cv2.resize(img, (180, 120), interpolation=cv2.INTER_AREA)       # half size
crop = img[30:120, 130:230]                                             # rows 30-119, cols 130-229
M = cv2.getRotationMatrix2D((180, 120), 30, 1.0)                        # 30 degrees about the
centre
rot = cv2.warpAffine(img, M, (360, 240), borderValue=(255, 255, 255))
flip = cv2.flip(img, 1)                                                 # mirror left-right
def fit(x): return cv2.resize(x, (180, 120))
top = cv2.hconcat([fit(img), fit(small)]); bottom = cv2.hconcat([fit(crop), fit(rot)])
cv2.imwrite("geometry.png", cv2.vconcat([top, bottom]))
print("crop", crop.shape, "| small", small.shape, "| rotated", rot.shape)
```

OUTPUT

```
crop (90, 100, 3) | small (120, 180, 3) | rotated (240, 360, 3)
```



ORIGINAL, RESIZED, CROPPED (THE GREEN CIRCLE) AND ROTATED 30°

Cropping is just NumPy slicing: `img[y1:y2, x1:x2]` (it is a view, so changing the crop changes the original; use `.copy()` to separate them). Use INTER_AREA to shrink and INTER_LINEAR/CUBIC to enlarge.

22.3 Image processing

22.3.1 Thresholding

Thresholding turns a grayscale image into black and white: pixels above the threshold become 255, others 0. **Otsu's method** picks the threshold automatically from the histogram; **adaptive** thresholding uses a different threshold for each neighbourhood, coping with uneven lighting.

22.3.2 Blurring and noise removal

Blurring averages neighbouring pixels to remove noise before thresholding or edge detection. **GaussianBlur** (smooth weighting) is the usual choice; **medianBlur** removes salt-and-pepper speckles while keeping edges sharp; kernel sizes must be odd (3, 5, 7…).

22.3.3 Edge detection (Canny)

Canny finds edges where brightness changes sharply, using two thresholds: strong edges above the high one are kept, weak edges between the two are kept only if connected to strong ones. A ratio of about 1:2 or 1:3 works well.

THRESHOLD_EDGES.PY

```
import cv2, numpy as np
from scene import make_scene
img = make_scene()
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5, 5), 0)
t, bw = cv2.threshold(blur, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
edges = cv2.Canny(blur, 50, 150)
print("Otsu threshold chosen:", int(t))
print("white pixels after threshold:", int((bw > 0).sum()), "| edge pixels:", int((edges >
0).sum()))
row = cv2.hconcat([gray, bw, edges])
cv2.imwrite("process.png", cv2.resize(row, (720, 160)))
```

OUTPUT

```
Otsu threshold chosen: 151
white pixels after threshold: 28151 | edge pixels: 1699
```

GRAYSCALE → OTSU THRESHOLD (INVERTED) → CANNY EDGES

22.3.4 Morphological operations

Operation	Effect	Use
erode	shrinks white regions	remove small white specks, separate touching objects
dilate	grows white regions	fill small holes, join broken parts
opening (erode then dilate)	removes small noise, keeps shape size	clean a colour mask
closing (dilate then erode)	fills small holes	solidify objects

22.3.5 Contours and shape detection

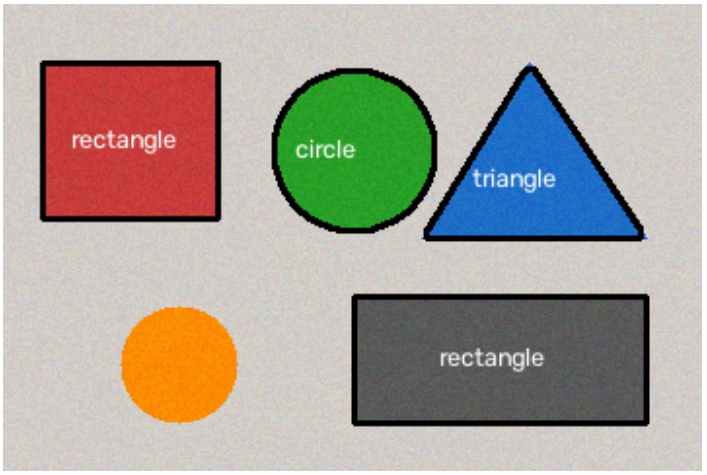
Contours are the outlines of white regions in a binary image. For each contour OpenCV gives its **area**, **perimeter**, **bounding box** and **moments** (for the centre). `approxPolyDP` simplifies the outline to a polygon: 3 corners = triangle, 4 = rectangle, many = circle.

SHAPES.PY

```
import cv2, numpy as np
from scene import make_scene
img = make_scene()
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, bw = cv2.threshold(cv2.GaussianBlur(gray, (5, 5), 0), 0, 255, cv2.THRESH_BINARY_INV +
cv2.THRESH_OTSU)
bw = cv2.morphologyEx(bw, cv2.MORPH_OPEN, np.ones((5, 5), np.uint8))
contours, _ = cv2.findContours(bw, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
out = img.copy()
for c in sorted(contours, key=lambda c: cv2.boundingRect(c)[0]):
    area = cv2.contourArea(c)
    if area < 300:
        continue                                        # ignore specks
    approx = cv2.approxPolyDP(c, 0.03 * cv2.arcLength(c, True), True)
    n = len(approx)
    shape = {3: "triangle", 4: "rectangle"}.get(n, "circle" if n > 6 else f"{n}-gon")
    m = cv2.moments(c); cx, cy = int(m["m10"] / m["m00"]), int(m["m01"] / m["m00"])
    cv2.drawContours(out, [c], -1, (0, 0, 0), 2)
    cv2.putText(out, shape, (cx - 30, cy + 4), cv2.FONT_HERSHEY_SIMPLEX, 0.45, (255, 255, 255), 1)
    print(f"{shape:9} centre ({cx:3d},{cy:3d})  area {area:6.0f}  corners {n}")
cv2.imwrite("shapes.png", out)
```

OUTPUT

```
rectangle centre ( 65, 70)  area   7198  corners 4
circle    centre (180, 75)  area   5377  corners 8
rectangle centre (254,182)  area   9750  corners 4
triangle  centre (271, 90)  area   5162  corners 3
```



CONTOURS DRAWN AND LABELLED BY THE PROGRAM

Notice the orange ball was **not** found: in grayscale it is almost as bright as the background, so the threshold left it out. Brightness alone cannot separate it, which is exactly why the next section uses **colour**.

22.4 Colour tracking

22.4.1 HSV masks

`cv2.inRange(hsv, lower, upper)` returns a **mask**: white where the pixel's H, S and V all lie in range. Pick ranges by sampling the object's HSV values under your real lighting; require a minimum saturation so grey and white areas are ignored.

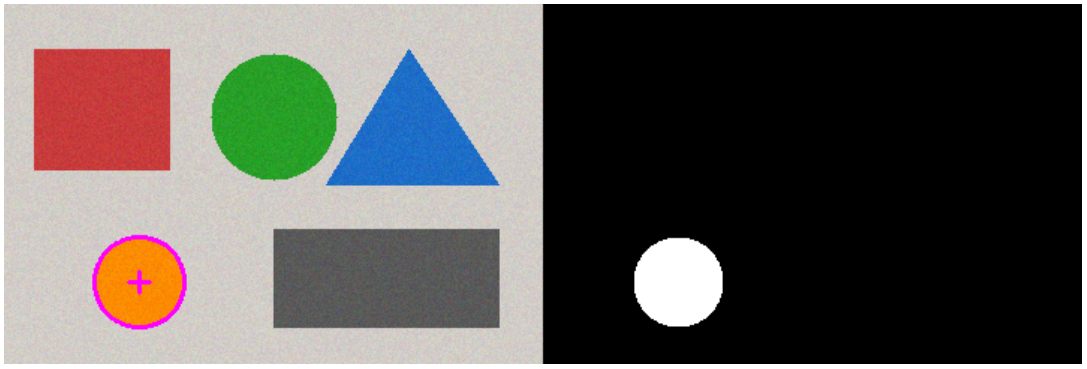
22.4.2 Object tracking by colour

TRACK_BALL.PY

```
import cv2, numpy as np
from scene import make_scene
img = make_scene()
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
lower, upper = np.array([10, 120, 120]), np.array([25, 255, 255])        # orange
mask = cv2.inRange(hsv, lower, upper)
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, np.ones((5, 5), np.uint8))
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
ball = max(contours, key=cv2.contourArea)
(x, y), r = cv2.minEnclosingCircle(ball)
out = img.copy()
cv2.circle(out, (int(x), int(y)), int(r), (255, 0, 255), 2)
cv2.drawMarker(out, (int(x), int(y)), (255, 0, 255), cv2.MARKER_CROSS, 14, 2)
err_x = x - img.shape[1] / 2                    # how far from the image centre
print(f"ball at ({x:.0f}, {y:.0f}), radius {r:.0f} px, horizontal error {err_x:+.0f} px")
print("steer", "left" if err_x < -20 else "right" if err_x > 20 else "straight")
cv2.imwrite("track.png", cv2.hconcat([out, cv2.cvtColor(mask, cv2.COLOR_GRAY2BGR)]))
```

OUTPUT

```
ball at (90, 185), radius 30 px, horizontal error -90 px
steer left
```



DETECTED BALL (LEFT) AND THE HSV MASK (RIGHT)

The **horizontal error** (object x - image centre) is exactly what a pan servo or a robot's steering needs: turn until the error is near zero, ideally with a proportional controller (module 25).

22.5 Detection

22.5.1 Face detection with Haar cascades

A **Haar cascade** is a classic, fast face detector: a chain of simple rectangle-feature tests trained on thousands of faces. It runs in real time on a Pi at 320 × 240, but misses tilted faces and gives false positives. For better accuracy use a small neural network (module 23).

HAAR FACE DETECTION (THE XML FILES SHIP WITH PYTHON3-OPENCV)

```
face_cascade =
cv2.CascadeClassifier("/usr/share/opencv4/haarcascades/haarcascade_frontalface_default.xml")
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5, minSize=(40, 40))
for (x, y, w, h) in faces:
    cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
print(len(faces), "face(s)")
```

22.5.2 QR codes and ArUco markers

QR_ARUCO.PY

```
import cv2, numpy as np
qr = cv2.QRCodeEncoder.create().encode("LBF-ROOM-204")              # make a QR code image
qr = cv2.resize(qr, None, fx=8, fy=8, interpolation=cv2.INTER_NEAREST)
qr = cv2.copyMakeBorder(qr, 40, 40, 40, 40, cv2.BORDER_CONSTANT, value=255)
text, pts, _ = cv2.QRCodeDetector().detectAndDecode(qr)
print("QR says:", text, "| corners found:", None if pts is None else len(pts[0]))

d = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_4X4_50)
canvas = np.full((300, 520), 255, np.uint8)
for i, (mid, x) in enumerate([(7, 30), (23, 290)]):
    canvas[50:250, x:x + 200] = cv2.aruco.generateImageMarker(d, mid, 200)
corners, ids, _ = cv2.aruco.ArucoDetector(d).detectMarkers(canvas)
for c, i in zip(corners, ids.flatten()):
    cx, cy = c[0].mean(axis=0)
    print(f"ArUco id {i} centre ({cx:.0f}, {cy:.0f})")
```

OUTPUT

```
QR says: LBF-ROOM-204 | corners found: 4
ArUco id 23 centre (390, 150)
ArUco id 7 centre (130, 150)
```

ArUco markers are square black-and-white codes designed for robots: each carries an ID and its four corners let the camera compute the marker's exact position and angle (pose), given the camera calibration. They are used for docking stations, navigation landmarks and AR.

22.5.3 Motion detection by frame differencing

MOTION_DIFF.PY

```
import cv2, numpy as np

def frame(t):                                       # synthetic camera: a box moves after t = 3
    f = np.full((120, 160), 90, np.uint8)
    if 3 <= t < 6:
        x = 20 + (t - 3) * 30
        f[40:80, x:x + 30] = 200
    return cv2.GaussianBlur(f, (5, 5), 0)

prev = frame(0)
for t in range(1, 8):
    cur = frame(t)
    diff = cv2.absdiff(prev, cur)
    _, th = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
    th = cv2.dilate(th, None, iterations=2)
    changed = cv2.countNonZero(th) / th.size * 100
    contours, _ = cv2.findContours(th, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    print(f"t={t}  changed {changed:5.1f} %  regions {len(contours)}  {'MOTION' if changed > 1 else
''}")
    prev = cur
```

OUTPUT

```
t=1  changed   0.0 %  regions 0
t=2  changed   0.0 %  regions 0
t=3  changed   8.6 %  regions 1  MOTION
t=4  changed  15.8 %  regions 1  MOTION
t=5  changed  15.8 %  regions 1  MOTION
t=6  changed   8.6 %  regions 1  MOTION
t=7  changed   0.0 %  regions 0
```

For a camera with slow lighting changes, compare against a **running average background** (`cv2.accumulateWeighted`) or use `cv2.createBackgroundSubtractorMOG2()`, which also handles waving trees and shadows.

22.6 Performance on the Pi

22.6.1 Lower resolution and frame skipping

WORKED EXAMPLE

Pixels to process

1920 × 1080 = 2 073 600 pixels; 640 × 480 = 307 200; 320 × 240 = 76 800. Dropping from 1080p to 320 × 240 means **27 times** fewer pixels for every filter. Use the camera's lores stream for analysis and the main stream only for saving.

Also process every 2nd or 3rd frame when the scene changes slowly, restrict work to a **region of interest** (crop), and convert to grayscale early when colour is not needed.

22.6.2 Threading the capture loop

Capturing waits for the camera; processing waits for the CPU. Doing them in sequence wastes time. A capture **thread** keeps grabbing the latest frame into a shared variable while the main loop processes whatever is newest, so the processing rate is not limited by capture latency (and stale frames are dropped instead of queued). Picamera2 already captures in the background, so its capture_array() returns quickly; threading helps most with USB webcams via cv2.VideoCapture.

A CAPTURE THREAD FOR A USB WEBCAM

```
import threading, cv2
class Grabber(threading.Thread):
    def __init__(self, src=0):
        super().__init__(daemon=True)
        self.cap = cv2.VideoCapture(src); self.frame = None; self.lock = threading.Lock()
    def run(self):
        while True:
            ok, f = self.cap.read()
            if ok:
                with self.lock: self.frame = f
    def latest(self):
        with self.lock: return None if self.frame is None else self.frame.copy()
```

22.6.3 Measuring FPS

FPS.PY

```
from picamera2 import Picamera2
import cv2, time

picam2 = Picamera2()
picam2.configure(picam2.create_video_configuration(main={"size": (320, 240), "format": "RGB888"}))
picam2.start()
frames, t0 = 0, time.perf_counter()
while frames < 90:
    f = picam2.capture_array()
    gray = cv2.cvtColor(f, cv2.COLOR_BGR2GRAY)
    edges = cv2.Canny(gray, 50, 150)
    frames += 1
fps = frames / (time.perf_counter() - t0)
cv2.putText(f, f"{fps:.1f} FPS", (5, 20), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)
print(f"{frames} frames, {fps:.1f} FPS (simulated camera at 30 fps)")
```

OUTPUT

```
90 frames, 30.0 FPS (simulated camera at 30 fps)
```

Here the simulated camera delivers frames at its 30 fps limit, so the program reports the capture rate; on a real Pi the processing time is added. Always print FPS during development: it shows immediately whether a change helped.

PRACTICE CHECKPOINT

Build a vision system that tracks a coloured ball and moves a pan-tilt camera to keep it centred, displaying FPS on screen.

TRACK_PAN.PY (CHECKPOINT SOLUTION CORE)

```
from picamera2 import Picamera2
from gpiozero import AngularServo
import cv2, numpy as np, time

pan = AngularServo(12, min_angle=-90, max_angle=90)
cam = Picamera2(); cam.configure(cam.create_video_configuration(main={"size": (320, 240), "format":
"RGB888"})); cam.start()
KP = 0.05                                    # degrees per pixel of error
angle, t0 = 0.0, time.perf_counter()
for n in range(1, 31):
    f = cam.capture_array()
    hsv = cv2.cvtColor(f, cv2.COLOR_BGR2HSV)
    mask = cv2.inRange(hsv, (100, 120, 80), (130, 255, 255))          # blue ball
    m = cv2.moments(mask)
    if m["m00"] > 500:
        err = m["m10"] / m["m00"] - 160
        angle = max(-90, min(90, angle - KP * err))
        pan.angle = round(angle)
        if n % 6 == 0:
            print(f"frame {n:2d}: error {err:+5.0f} px -> pan {pan.angle:+.0f} deg  ({n /
(time.perf_counter() - t0):.0f} FPS)")
```

OUTPUT

```
frame  6: error   +96 px -> pan -29 deg  (30 FPS)
frame 12: error   +96 px -> pan -57 deg  (30 FPS)
frame 18: error   +96 px -> pan -86 deg  (30 FPS)
frame 24: error   +96 px -> pan -90 deg  (30 FPS)
frame 30: error   +96 px -> pan -90 deg  (30 FPS)
```

The simulated ball stays at 80 % of the frame width, so the error stays at about +96 px and the proportional controller keeps turning the pan servo (here in the negative direction) a little more on every frame. With a real camera on the servo, the image would shift as it turns and the error would shrink towards zero.

22.7 Practice questions

22.7.1 Basic level

Q1 In which colour order does OpenCV store images?

(a) RGB (b) HSV (c) BGR (d) CMYK

Q2 What is the shape of a 640 × 480 colour image array?

(a) (640, 480, 3) (b) (3, 640, 480) (c) (480, 640) (d) (480, 640, 3)

Q3 Which function saves an image file?

(a) cv2.imshow (b) cv2.imwrite (c) cv2.imread (d) cv2.save

Q4 Which colour space separates colour from brightness?

(a) HSV (b) BGR (c) grayscale (d) binary

Q5 Which function finds edges?

(a) cv2.threshold (b) cv2.Canny (c) cv2.resize (d) cv2.inRange

Q6 What does cv2.inRange produce?

(a) a binary mask of pixels inside the range
(b) a colour image
(c) a list of contours
(d) a histogram

Q7 What does cv2.imread return when the file is missing?

(a) an empty image (b) an exception always
(c) None (d) a black image

Q8 What range does OpenCV use for hue?

(a) 0–255 (b) 0–359 (c) 0–100 (d) 0–179

Q9 Which OpenCV package suits a headless Pi installed with pip?

(a) opencv-gui (b) cv2-desktop
(c) opencv-python-headless (d) opencv-windows

22.7.2 Intermediate level

Q10 Why blur before thresholding or edge detection?

(a) to make colours brighter
(b) to change BGR to RGB
(c) to enlarge the image
(d) to remove noise that would create false edges or specks

Q11 What does Otsu's method choose?

(a) the blur kernel
(b) the camera exposure
(c) the threshold value automatically from the histogram

(d) the number of contours

Q12 What does morphological opening do?

(a) removes small white noise while keeping object size
(b) fills holes
(c) finds edges
(d) rotates the image

Q13 How many corners does approxPolyDP return for a triangle?

(a) 4 (b) 0 (c) many (d) 3

Q14 How do you crop rows 50–149 and columns 100–299 of img?

(a) img[100:300, 50:150] (b) img[50:150, 100:300]
(c) img.crop(50,150,100,300) (d) cv2.crop(img)

Q15 Why is red tricky in HSV?

(a) it has no saturation
(b) its hue wraps round, so you need two ranges (near 0 and near 179)
(c) it is not in BGR
(d) OpenCV cannot see red

Q16 What do ArUco markers give a robot besides an ID?

(a) the temperature
(b) a Wi-Fi password
(c) audio
(d) their position and orientation (pose) relative to the camera

Q17 Which method detects motion by comparing frames?

(a) absdiff then threshold (b) Canny on one frame
(c) cv2.flip (d) QRCodeDetector

Q18 How many times fewer pixels does 320 × 240 have than 1920 × 1080?

(a) 6 (b) 9 (c) 27 (d) 81

22.7.3 Advanced level

Q19 A tracker sees the ball centre at x = 250 in a 320-pixel-wide image. What is the horizontal error?

(a) -90 px (b) +250 px (c) +70 px (d) +90 px

Q20 Create a 100 × 100 image, draw a white filled circle of radius 30 in the middle on black, and print its area in pixels compared with πr^2.

Q21 Make a mask for "red" using two HSV ranges and count red pixels in a test image containing a red and a blue square.

Q22 Count coins in a synthetic image of 5 white circles of different sizes using thresholding and contours, ignoring specks below 50 px.

Q23 Compute the steering command for a line-following robot: in a binary row of pixels (1 = black line), find the line centre and print the error from the middle and "left", "right" or "straight".

Q24 A Pi 4 runs colour tracking at 6 FPS at 1280 × 720. Which change helps most?

(a) use a bigger blur kernel
(b) process the 320 × 240 lores stream instead
(c) save every frame as PNG
(d) convert to HSV twice

Q25 Which pipeline best tracks an orange ball reliably?

(a) BGR → HSV → inRange → opening → largest contour → centre → servo correction
(b) BGR → Canny → QR detector
(c) grayscale → Otsu → face cascade
(d) resize → rotate → save

22.8 Solutions

Q1 **Answer: (c) BGR**

BGR.

Q2 **Answer: (d) (480, 640, 3)**

height, width, channels.

Q3 **Answer: (b) cv2.imwrite**

`imwrite(filename, img)`.

Q4 **Answer: (a) HSV**

Hue, Saturation, Value.

Q5 **Answer: (b) cv2.Canny**

Canny.

Q6 **Answer: (a) a binary mask of pixels inside the range**

White = in range.

Q7 **Answer: (c) None**

Always check for **None**.

Q8 **Answer: (d) 0–179**

Degrees ÷ 2.

Q9 **Answer: (c) opencv-python-headless**

No GUI dependencies.

Q10 **Answer: (d) to remove noise that would create false edges or specks**

Smoothing suppresses noise.

Q11 **Answer: (c) the threshold value automatically from the histogram**

Automatic global threshold.

Q12 **Answer: (a) removes small white noise while keeping object size**

Erode then dilate.

Q13 **Answer: (d) 3**

Polygon simplification.

Q14 **Answer: (b) img[50:150, 100:300]**

Slicing: rows first.

Q15 **Answer: (b) its hue wraps round, so you need two ranges (near 0 and near 179)**

Combine two masks.

Q16 **Answer: (d) their position and orientation (pose) relative to the camera**

Four known corners → pose.

Q17 **Answer: (a) absdiff then threshold**

Frame differencing.

Q18 **Answer: (c) 27**

2 073 600 ÷ 76 800 = **27**.

Q19 **Answer: (d) +90 px**

250 - 160 = **+90**.

Q20 **Answer: Solution program**

countNonZero counts the white pixels; the drawn circle closely matches πr² ≈ 2827.

```
import cv2, numpy as np, math
img = np.zeros((100, 100), np.uint8)
cv2.circle(img, (50, 50), 30, 255, -1)
print("white pixels:", cv2.countNonZero(img), "| pi r^2 =", round(math.pi * 30 ** 2))
```

OUTPUT

```
white pixels: 2821 | pi r^2 = 2827
```

Q21 **Answer: Solution program**

Red sits at both ends of the hue range, so OR two masks together.

```
import cv2, numpy as np
img = np.zeros((100, 200, 3), np.uint8)
img[20:80, 20:80] = (30, 30, 220)        # red (BGR)
img[20:80, 120:180] = (220, 60, 30)      # blue
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
m1 = cv2.inRange(hsv, (0, 100, 80), (10, 255, 255))
m2 = cv2.inRange(hsv, (170, 100, 80), (179, 255, 255))
red = cv2.bitwise_or(m1, m2)
print("red pixels:", cv2.countNonZero(red), "(expected 60 x 60 =", 60 * 60, ")")
```

OUTPUT

```
red pixels: 3600 (expected 60 x 60 = 3600 )
```

Q22 **Answer: Solution program**

An area filter separates real objects from noise; minEnclosingCircle measures each coin.

```
import cv2, numpy as np
img = np.zeros((150, 300), np.uint8)
for x, r in ((30, 15), (80, 20), (140, 18), (200, 25), (265, 12)):
    cv2.circle(img, (x, 75), r, 255, -1)
img[10, 10] = 255                                  # a speck of noise
contours, _ = cv2.findContours(img, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
coins = [c for c in contours if cv2.contourArea(c) >= 50]
print("contours:", len(contours), "| coins:", len(coins))
print("radii:", sorted(round(cv2.minEnclosingCircle(c)[1]) for c in coins))
```

OUTPUT

```
contours: 6 | coins: 5
radii: [12, 15, 18, 20, 25]
```

Q23 **Answer: Solution program**

The mean x of the line pixels minus the image centre is the error, the same idea as colour tracking.

```
import numpy as np
def steer(row):
    xs = np.flatnonzero(row)
    if xs.size == 0:
        return "line lost"
    err = xs.mean() - (len(row) - 1) / 2
    word = "left" if err < -3 else "right" if err > 3 else "straight"
    return f"centre {xs.mean():.1f}, error {err:+.1f} -> {word}"
row = np.zeros(41, np.uint8)
for start in (5, 18, 30):
    r = row.copy(); r[start:start + 5] = 1
    print(steer(r))
print(steer(row))
```

OUTPUT

```
centre 7.0, error -13.0 -> left
centre 20.0, error +0.0 -> straight
centre 32.0, error +12.0 -> right
line lost
```

Q24 **Answer: (b) process the 320 × 240 lores stream instead**

12× fewer pixels.

Q25 **Answer: (a) BGR → HSV → inRange → opening → largest contour → centre → servo correction**

Colour mask, clean, find, act.

Images are NumPy arrays [y, x, BGR]. Convert to grayscale or HSV (hue 0–179). Pipeline: blur → threshold (Otsu/adaptive) or Canny → morphology → contours (area, centre, approxPolyDP). Colour tracking: inRange mask → largest contour → error from centre → control. Detect faces (Haar), QR/ArUco, motion (absdiff). Speed: low resolution, ROI, frame skipping, threads; always measure FPS.

MODULE 23 · ADVANCED

AI and machine learning at the edge

Minimum duration **3 days** | Level **Advanced**

Running AI models on the Pi itself enables private, low-latency smart devices. This module explains cloud versus edge AI, training versus inference, and the trade-offs of model size and quantisation; applies NumPy and scikit-learn to sensor data for classification and anomaly detection; runs TensorFlow Lite / LiteRT image classification, object detection and pose estimation; trains custom models with Teachable Machine and Edge Impulse; uses accelerators (the AI Kit and AI HAT+ with Hailo, the AI Camera, USB accelerators) for YOLO; runs small language models locally and calls cloud AI APIs; and considers privacy, followed by 25 solved questions and programs. The machine-learning examples were run with scikit-learn and NumPy.

IN THIS MODULE

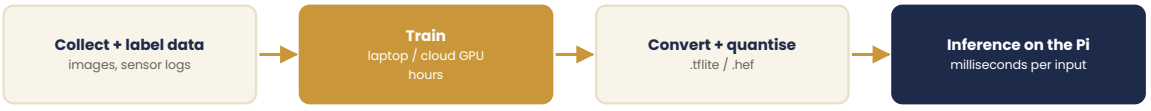
23.1 Edge AI concepts
23.2 Machine learning basics on the Pi
23.3 TensorFlow Lite / LiteRT
23.4 Training custom models
23.5 AI accelerators
23.6 Generative AI on the Pi
23.7 Responsible AI
23.8 Practice questions
23.9 Solutions

23.1 Edge AI concepts

23.1.1 Cloud AI vs edge AI

	Cloud AI	Edge AI (on the Pi)
Where the model runs	remote data centre	on the device
Latency	100 ms – seconds (network round trip)	milliseconds
Works offline	no	yes
Privacy	images/audio leave the home	raw data never leaves the device
Cost	pay per request	one-off hardware
Model size	huge models possible	limited by RAM and compute

23.1.2 Training vs inference



TRAIN ONCE ON POWERFUL HARDWARE; RUN (INFER) MANY TIMES ON THE EDGE DEVICE

Training adjusts millions of model parameters using labelled examples and needs lots of compute, usually a GPU. **Inference** uses the finished model to make predictions and is much lighter: that is what the Pi does.

23.1.3 Model size, quantisation and accuracy trade-offs

Models store weights as 32-bit floats. **Quantisation** converts them to 8-bit integers: the model becomes **4× smaller** and runs 2–4× faster on CPUs and NPUs, usually losing under 1–2 % accuracy. Each float range is mapped onto -128…127 with a **scale** and **zero point**:

QUANTISE.PY

```
import numpy as np
rng = np.random.default_rng(0)
w = rng.normal(0, 0.4, 10_000).astype(np.float32)          # pretend layer weights

scale = (w.max() - w.min()) / 255
zero = np.round(-128 - w.min() / scale)
q = np.clip(np.round(w / scale + zero), -128, 127).astype(np.int8)    # quantise
w_back = (q.astype(np.float32) - zero) * scale                         # dequantise

print(f"float32 size {w.nbytes} B -> int8 size {q.nbytes} B ({w.nbytes // q.nbytes}x smaller)")
print(f"scale {scale:.5f}, zero point {int(zero)}")
print(f"mean absolute error {np.abs(w - w_back).mean():.5f} (max {np.abs(w - w_back).max():.5f})")
```

OUTPUT

```
float32 size 40000 B -> int8 size 10000 B (4x smaller)
scale 0.01158, zero point 7
mean absolute error 0.00288 (max 0.00579)
```

Model (image classification)	Size	Typical Pi 5 CPU speed	Accuracy (ImageNet top-1)
MobileNet V2 int8	3.4 MB	≈ 20–30 ms	≈ 70 %
EfficientNet-Lite0 int8	5 MB	≈ 30–40 ms	≈ 75 %
ResNet-50 float	98 MB	≈ 300+ ms	≈ 76 %

NOTE

Timings vary with model version, image size, threads and cooling; treat them as orders of magnitude, and always measure on your own Pi.

23.2 Machine learning basics on the Pi

23.2.1 NumPy and scikit-learn for sensor data

Many useful "AI" tasks need no neural network: recognising activities from accelerometer data, predicting when a room will be too hot, spotting a failing motor from vibration. **scikit-learn** (`sudo apt install python3-sklearn`) trains classic models in seconds on the Pi itself. The steps: build **features** (numbers that describe each sample), split data into **training** and **test** sets, fit, and measure accuracy on the test set only.

23.2.2 Simple classifiers and anomaly detection

ACTIVITY_CLASSIFIER.PY

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

rng = np.random.default_rng(1)
def window(activity, n=50):                      # 50 accelerometer magnitudes (g) = 1 s
    base = {"still": (1.0, 0.02, 0), "walking": (1.0, 0.25, 2), "running": (1.0, 0.6, 3)}[activity]
    t = np.arange(n) / 50
    return base[0] + base[1] * np.sin(2 * np.pi * base[2] * t) + rng.normal(0, 0.05, n)

def features(x):                                 # describe each window with 4 numbers
    return [x.mean(), x.std(), x.max() - x.min(), np.abs(np.diff(x)).mean()]

X, y = [], []
for act in ("still", "walking", "running"):
    for _ in range(120):
        X.append(features(window(act))); y.append(act)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=0, stratify=y)
model = RandomForestClassifier(n_estimators=50, random_state=0).fit(Xtr, ytr)
pred = model.predict(Xte)
print("test accuracy:", round(accuracy_score(yte, pred), 3))
print("labels:", [str(c) for c in model.classes_])
print(confusion_matrix(yte, pred, labels=model.classes_))
print("new 1-s window classified as:", model.predict([features(window("walking"))])[0])
```

OUTPUT

```
test accuracy: 1.0
labels: ['running', 'still', 'walking']
[[30  0  0]
 [ 0 30  0]
 [ 0  0 30]]
new 1-s window classified as: walking
```

The **confusion matrix** shows, for each true class (row), how many test samples were predicted as each class (column): the diagonal holds the correct answers.

ANOMALY.PY

```
import numpy as np
from sklearn.ensemble import IsolationForest

rng = np.random.default_rng(2)
normal = np.column_stack([rng.normal(0.20, 0.03, 500), rng.normal(45, 2, 500)])   # vibration g,
motor temp C
model = IsolationForest(contamination=0.01, random_state=0).fit(normal)

tests = {"normal": [0.21, 46], "slightly warm": [0.22, 50], "bearing wear": [0.45, 47],
"overheating": [0.20, 62]}
for name, x in tests.items():
    verdict = "ANOMALY" if model.predict([x])[0] == -1 else "ok"
    print(f"{name:14} vib {x[0]:.2f} g, {x[1]} C -> {verdict}")

z = (62 - normal[:, 1].mean()) / normal[:, 1].std()          # simplest method: z-score
print(f"z-score of 62 C: {z:.1f} (|z| > 3 is unusual)")
```

OUTPUT

```
normal         vib 0.21 g, 46 C -> ok
slightly warm  vib 0.22 g, 50 C -> ok
bearing wear   vib 0.45 g, 47 C -> ANOMALY
overheating    vib 0.20 g, 62 C -> ANOMALY
z-score of 62 C: 8.4 (|z| > 3 is unusual)
```

SHORTCUT TRICK

Start simple: a **z-score** (how many standard deviations from the mean) or a moving-average threshold often catches faults. Use IsolationForest when several sensors must be judged together.

23.3 TensorFlow Lite / LiteRT

TensorFlow Lite, now called **LiteRT**, runs compact `.tflite` models on phones and SBCs. On the Pi install the small runtime (`pip install ai-edge-litert`, or the older `tflite-runtime`) in a venv. ONNX Runtime and PyTorch (ExecuTorch) are alternatives.

23.3.1 Image classification

CLASSIFY.PY (REFERENCE: NEEDS THE MODEL FILE)

```
from ai_edge_litert.interpreter import Interpreter
import numpy as np, cv2

interp = Interpreter("mobilenet_v2_1.0_224_quant.tflite", num_threads=4)
interp.allocate_tensors()
inp, out = interp.get_input_details()[0], interp.get_output_details()[0]
labels = open("labels.txt").read().splitlines()

img = cv2.resize(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB), (224, 224))   # model expects 224x224 RGB
interp.set_tensor(inp["index"], np.expand_dims(img, 0).astype(inp["dtype"]))
interp.invoke()
scores = interp.get_tensor(out["index"])[0]
top = scores.argsort()[-3:][::-1]
for i in top:
    print(labels[i], scores[i])
```

The pattern is always the same: resize and convert the image to the model's input size and type, `set_tensor`, `invoke`, read the outputs. Turning raw scores into probabilities and picking the top results is ordinary NumPy:

TOP3.PY

```
import numpy as np
labels = ["cat", "dog", "person", "bicycle", "car"]
logits = np.array([1.2, 3.1, 0.4, -0.5, 2.2])              # raw model outputs
probs = np.exp(logits - logits.max()); probs /= probs.sum()  # softmax
for i in probs.argsort()[::-1][:3]:
    print(f"{labels[i]:8} {probs[i]:.1%}")
```

OUTPUT

```
dog      60.6%
car      24.6%
cat      9.1%
```

23.3.2 Object detection

A detector (SSD-MobileNet, EfficientDet-Lite, YOLO) returns **boxes**, **classes** and **scores** for every object found. Keep detections above a confidence threshold (for example 0.5) and remove duplicates with **non-maximum suppression** (NMS), which uses the **IoU** (intersection over union) of overlapping boxes:

NMS.PY

```
def iou(a, b):                          # boxes as (x1, y1, x2, y2)
    ix = max(0, min(a[2], b[2]) - max(a[0], b[0]))
    iy = max(0, min(a[3], b[3]) - max(a[1], b[1]))
    inter = ix * iy
    union = (a[2] - a[0]) * (a[3] - a[1]) + (b[2] - b[0]) * (b[3] - b[1]) - inter
    return inter / union

def nms(dets, iou_thr=0.5, score_thr=0.5):
    dets = sorted([d for d in dets if d[1] >= score_thr], key=lambda d: -d[1])
    keep = []
    for box, score, label in dets:
        if all(iou(box, k[0]) < iou_thr or label != k[2] for k in keep):
            keep.append((box, score, label))
    return keep

raw = [((50, 40, 150, 240), 0.91, "person"), ((55, 45, 152, 238), 0.84, "person"),
       ((300, 120, 380, 200), 0.77, "dog"), ((10, 10, 40, 40), 0.31, "cat")]
print("IoU of the two person boxes:", round(iou(raw[0][0], raw[1][0]), 2))
for box, s, lab in nms(raw):
    print(f"{lab:7} {s:.2f} {box}")
```

OUTPUT

```
IoU of the two person boxes: 0.9
person  0.91 (50, 40, 150, 240)
dog     0.77 (300, 120, 380, 200)
```

23.3.3 Pose estimation

Pose models (MoveNet, PoseNet, YOLO-pose) output **keypoints**: 17 body joints (nose, shoulders, elbows, wrists, hips, knees, ankles) with coordinates and confidence. Angles between keypoints count exercise repetitions, detect falls or control games by gesture.

SQUAT_COUNTER.PY

```
import math
def angle(a, b, c):                      # angle at b, in degrees
    ab = (a[0] - b[0], a[1] - b[1]); cb = (c[0] - b[0], c[1] - b[1])
    cos = (ab[0] * cb[0] + ab[1] * cb[1]) / (math.hypot(*ab) * math.hypot(*cb))
    return math.degrees(math.acos(max(-1, min(1, cos))))

reps, down = 0, False
frames = [(160, 0.9), (120, 0.9), (75, 0.8), (70, 0.9), (110, 0.9), (165, 0.95), (80, 0.9), (168,
0.9)]
for knee_angle, conf in frames:                # knee angle per frame from hip-knee-ankle keypoints
    if conf < 0.5: continue
    if knee_angle < 90: down = True
    if down and knee_angle > 150: reps += 1; down = False
print("squats counted:", reps)
print("example angle:", round(angle((100, 50), (100, 150), (180, 150)), 1), "degrees")
```

OUTPUT

```
squats counted: 2
example angle: 90.0 degrees
```

23.4 Training custom models

23.4.1 Collecting and labelling data

A custom model is only as good as its data. Collect images (or sensor windows) in the **real conditions** where the device will work: same camera, angles, lighting and backgrounds. Aim for at least 100–300 examples per class to start, balanced between classes, plus a "nothing/background" class. Label boxes with tools such as Label Studio or Roboflow. Keep a separate **test set** that is never used for training.

23.4.2 Training on a laptop or cloud (Teachable Machine, Edge Impulse)

Tool	Strength
Teachable Machine (Google)	train image, sound or pose classifiers in a browser in minutes; export TensorFlow Lite
Edge Impulse	complete pipeline for sensor, audio and vision data: collect from the device, train, test, quantise and deploy (runs on Pi, Pico, ESP32)
Ultralytics YOLO	train custom object detectors in Python on a GPU (Colab works); export to TFLite, ONNX or Hailo
Transfer learning	start from a pre-trained network and retrain only the last layers: needs far less data and time

23.4.3 Converting and deploying to the Pi

Export the trained model to the runtime format (`.tflite` int8 for CPU, `.hef` for Hailo, `.rpk` for the AI Camera, ONNX), copy it to the Pi, check its expected input size and normalisation, and compare accuracy on your test set **after** quantisation. Version your models like code.

23.5 AI accelerators

23.5.1 Raspberry Pi AI Kit and AI HAT+ (Hailo)

The **AI HAT+** (Hailo-8L at 13 TOPS or Hailo-8 at 26 TOPS) and the earlier **AI Kit** connect a neural processing unit (NPU) to the Pi 5's PCIe port. Object detection that takes hundreds of milliseconds on the CPU runs at 30+ frames per second, leaving the CPU free. `sudo apt install hailo-all`, then the rpicam-apps post-processing stages and the hailo-rpi5-examples run YOLO detection, pose and segmentation on the camera stream.

WORKED EXAMPLE

TOPS and frame rate

A YOLOv8s detector needs about 28.6 GOPs (billion operations) per 640 × 640 frame. At 13 TOPS with perhaps 30 % real utilisation: 13 000 × 0.3 ÷ 28.6 ≈ **136 frames/s** of compute headroom, so the camera, not the NPU, becomes the limit.

23.5.2 Raspberry Pi AI Camera (on-sensor AI)

The **AI Camera** (Sony IMX500) runs the neural network inside the image sensor and sends only the results (with or without the image) to the Pi. It works with any Pi, including a Zero 2 W, uses little power and keeps

images private if you discard them. Models (MobileNet SSD, PoseNet…) are packaged as .rpk files and loaded through Picamera2 or rpicam-apps.

23.5.3 USB accelerators overview

USB sticks such as the Coral Edge TPU (4 TOPS, TFLite models compiled for it) and Intel Movidius-based devices add AI acceleration to Pi 4s and older boards, limited by USB bandwidth. For a Pi 5, the PCIe-based Hailo boards are faster and officially supported.

23.5.4 Running YOLO object detection

YOLO ON THE CPU (REFERENCE)

```
from ultralytics import YOLO                # pip install ultralytics (in a venv)
model = YOLO("yolo11n.pt")                   # nano model, downloads on first use
model.export(format="ncnn")                  # NCNN is fast on Raspberry Pi CPUs
ncnn = YOLO("yolo11n_ncnn_model")
results = ncnn("street.jpg", conf=0.5)
for b in results[0].boxes:
    print(ncnn.names[int(b.cls)], float(b.conf), b.xyxy.tolist())
```

On a Pi 5 CPU the nano model at 640 × 640 manages a few frames per second; with a Hailo NPU it reaches real time. Lower the input size (320) for more speed.

23.6 Generative AI on the Pi

23.6.1 Running small language models locally (Ollama, llama.cpp)

Small language models (about 1–4 billion parameters, quantised to 4 bits) run on a Pi 5 with 8–16 GB RAM: **Ollama** (`curl -fsSL https://ollama.com/install.sh | sh`, then `ollama run llama3.2:1b`) or **llama.cpp**. Expect a few to around ten words per second: fine for offline assistants, summaries and turning voice commands into actions.

LLM_MEMORY.PY

```
for params_b, bits in [(1, 4), (3, 4), (3, 8), (8, 4)]:
    gb = params_b * 1e9 * bits / 8 / 1e9 * 1.2          # weights + ~20 % runtime overhead
    fits = "fits in 8 GB" if gb < 6 else "needs 16 GB" if gb < 14 else "too big"
    print(f"{params_b} B params at {bits}-bit: about {gb:.1f} GB  -> {fits}")
```

OUTPUT

```
1 B params at 4-bit: about 0.6 GB  -> fits in 8 GB
3 B params at 4-bit: about 1.8 GB  -> fits in 8 GB
3 B params at 8-bit: about 3.6 GB  -> fits in 8 GB
8 B params at 4-bit: about 4.8 GB  -> fits in 8 GB
```

23.6.2 Calling cloud AI APIs from Python

CLOUD_AI.PY

```
import requests, os, json

API_URL = "https://api.example-ai.com/v1/chat"         # your provider's endpoint
KEY = os.environ.get("AI_API_KEY", "demo-key")          # from a 600-permission .env file
reading = {"room": "server rack", "temp_c": 41.5, "fan_rpm": 0}
prompt = f"Sensor data: {json.dumps(reading)}. In one sentence, what is wrong and what should I do?"
r = requests.post(API_URL, headers={"Authorization": f"Bearer {KEY}"},
                  json={"model": "small-model", "messages": [{"role": "user", "content": prompt}]},
timeout=30)
print("status", r.status_code)
print("reply:", r.json()["reply"])
```

SIMULATED RUN (hardware events and printed output)

```
[  0.15 s] HTTP POST https://api.example-ai.com/v1/chat {'model': 'small-model', 'messages':
[{'role': 'user', 'content': 'Sensor data: {"room": "server rack", "temp_c": 41.5, "fan_rpm": 0}.
In one sentence, what is wrong and what should I do?'}]}
status 200
reply: The fan has stopped and the rack is overheating; check the fan power and reduce load until
it is fixed.
```

Send only the data the task needs (here a few numbers, not camera images), handle network errors and rate limits, and keep a local fallback rule (if temp > 40 and fan = 0 → alarm) so safety never depends on the cloud.

23.7 Responsible AI

23.7.1 Privacy when using cameras and microphones

Principle	In practice
Data minimisation	process on the device; store counts or events, not video, unless necessary
Transparency	visible notices where cameras and microphones operate; tell household members and visitors
Consent and law	do not record neighbours' property or public streets beyond what the law allows; follow India's Digital Personal Data Protection Act, 2023 for personal data
Security	encrypt stored footage, secure remote access (module 21), delete old data automatically
Fairness	test models on varied people and conditions: accuracy can differ across skin tones, ages and lighting

PRACTICE CHECKPOINT

Build a smart camera that detects people or chosen objects in real time, counts them and publishes counts over MQTT to a dashboard.

PEOPLE_COUNTER.PY (CHECKPOINT SOLUTION CORE)

```
import paho.mqtt.client as mqtt, json, time

def detections_for_frame(t):          # stand-in for the detector: (label, score) per frame
    people = [0, 0, 1, 2, 2, 3, 3, 2, 1, 1, 0, 0][t % 12]
    return [("person", 0.8)] * people + [("dog", 0.7)] * (t % 5 == 0)

client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2, client_id="smartcam")
client.connect("localhost"); client.loop_start()
last = None
for t in range(12):
    count = sum(1 for label, score in detections_for_frame(t) if label == "person" and score >= 0.5)
    if count != last:                                    # publish only on change
        client.publish("cam/front/people", json.dumps({"count": count, "ts": int(time.time())}),
retain=True)
        last = count
    time.sleep(1)
```

SIMULATED RUN (hardware events and printed output)

```
[  0.00 s] MQTT smartcam connect localhost:1883 -> accepted
[  0.00 s] MQTT publish cam/front/people {"count": 0, "ts": 1773565200} [retained]
[  2.00 s] MQTT publish cam/front/people {"count": 1, "ts": 1773565202} [retained]
[  3.00 s] MQTT publish cam/front/people {"count": 2, "ts": 1773565203} [retained]
[  5.00 s] MQTT publish cam/front/people {"count": 3, "ts": 1773565205} [retained]
[  7.00 s] MQTT publish cam/front/people {"count": 2, "ts": 1773565207} [retained]
[  8.00 s] MQTT publish cam/front/people {"count": 1, "ts": 1773565208} [retained]
[ 10.00 s] MQTT publish cam/front/people {"count": 0, "ts": 1773565210} [retained]
```

23.8 Practice questions

23.8.1 Basic level

Q1 Where does an edge AI model run?

(a) on the device itself
(b) only in a data centre
(c) in the router
(d) on the phone app only

Q2 Which step needs the most computing power?

(a) inference
(b) training
(c) labelling
(d) saving the model

Q3 What is the file extension of TensorFlow Lite models?

(a) .csv (b) .hef (c) .pt (d) .tflite

Q4 What does int8 quantisation do to model size compared with float32?

(a) makes it 2× bigger
(b) no change
(c) makes it about 4× smaller
(d) makes it 32× smaller

Q5 Which Pi add-on contains a Hailo NPU?

(a) Sense HAT
(b) PoE HAT
(c) AI HAT+ / AI Kit
(d) DAC HAT

Q6 Where does the AI Camera run its neural network?

(a) on the Pi CPU only
(b) in the cloud
(c) on the SD card
(d) inside the image sensor (IMX500)

Q7 Which tool trains image or sound classifiers in a browser?

(a) Mosquitto
(b) Teachable Machine
(c) Grafana
(d) Samba

Q8 Which tool runs small language models locally?

(a) Ollama (b) Pi-hole (c) Portainer (d) Kodi

Q9 What does TOPS measure for an NPU?

(a) temperature of processing silicon
(b) total output pixels per second
(c) trillions of operations per second
(d) tokens per second

23.8.2 Intermediate level

Q10 Why evaluate accuracy on a separate test set?

(a) to train faster
(b) to measure how the model performs on data it has never seen
(c) test sets are smaller to store
(d) scikit-learn requires it to start

Q11 What does the diagonal of a confusion matrix contain?

(a) correct predictions for each class
(b) all errors

(c) the probabilities
(d) the training time

Q12 What does IsolationForest detect?

(a) faces
(b) QR codes
(c) unusual samples (anomalies)
(d) network ports

Q13 What does non-maximum suppression remove?

(a) low-light pixels
(b) small models
(c) labels
(d) duplicate overlapping boxes for the same object

Q14 What do pose models output?

(a) audio
(b) temperature
(c) QR codes
(d) keypoints (body joints) with coordinates and confidence

Q15 What is transfer learning?

(a) copying a model to another Pi
(b) converting to .tflite
(c) retraining only the last layers of a pre-trained model on new data
(d) sending data to the cloud

Q16 Why keep a local rule even when a cloud AI gives advice?

(a) safety must not depend on the network or a remote service
(b) cloud AI is always wrong
(c) it is cheaper to run both
(d) local rules are more accurate

Q17 Which principle says store counts rather than video when counts are enough?

(a) transfer learning
(b) quantisation
(c) overfitting
(d) data minimisation

Q18 A float32 model is 24 MB. About how big is its int8 version?

(a) 3 MB
(b) 6 MB
(c) 12 MB
(d) 96 MB

23.8.3 Advanced level

Q19 Two boxes each have area 100 and overlap by 50. What is their IoU?

(a) 0.5
(b) 0.333
(c) 0.25
(d) 1.0

Q20 Compute softmax probabilities for the logits [2.0, 1.0, 0.1] and print the predicted class index and its probability.

Q21 Train a k-nearest-neighbours classifier that says "comfortable" or "uncomfortable" from (temperature, humidity) examples, and classify (27, 55) and (33, 80).

Q22 Detect anomalies in a vibration series with a rolling z-score: flag readings more than 3 standard

deviations from the mean of the previous 20 readings.

Q23 Estimate inference throughput: a model takes 38 ms per frame on 4 CPU threads; preprocessing takes 6 ms and drawing 4 ms. Print the FPS, and the FPS if an NPU cuts inference to 5 ms.

Q24 A shop wants a privacy-friendly visitor counter. Which design is best?

(a) stream all video to a cloud service for counting
(b) record 24/7 video and count later by hand
(c) post camera images to a public website
(d) AI Camera or Pi + NPU counts people on the device and publishes only counts

Q25 Your custom model scores 98 % in training but 70 % on new photos. What is the most likely cause?

(a) overfitting or training data that does not match real conditions
(b) the Pi is too slow
(c) quantisation always does this
(d) the test set is too accurate

23.9 Solutions

Q1 **Answer: (a) on the device itself**
Edge = local.

Q2 **Answer: (b) training**
Training adjusts millions of weights.

Q3 **Answer: (d) .tflite**
.tflite; .hef is Hailo, .pt PyTorch.

Q4 **Answer: (c) makes it about 4× smaller**
32 bits → 8 bits.

Q5 **Answer: (c) AI HAT+ / AI Kit**
PCIe NPU for the Pi 5.

Q6 **Answer: (d) inside the image sensor (IMX500)**
On-sensor AI.

Q7 **Answer: (b) Teachable Machine**
Google Teachable Machine.

Q8 **Answer: (a) Ollama**
Ollama (or llama.cpp).

Q9 **Answer: (c) trillions of operations per second**

A raw compute rating.

Q10 **Answer: (b) to measure how the model performs on data it has never seen**

Training accuracy is over-optimistic.

Q11 **Answer: (a) correct predictions for each class**

True class = predicted class.

Q12 **Answer: (c) unusual samples (anomalies)**

Anomaly detection.

Q13 **Answer: (d) duplicate overlapping boxes for the same object**

Keeps the best box.

Q14 **Answer: (d) keypoints (body joints) with coordinates and confidence**

E.g. 17 COCO keypoints.

Q15 **Answer: (c) retraining only the last layers of a pre-trained model on new data**

Less data, faster.

Q16 **Answer: (a) safety must not depend on the network or a remote service**

Fail-safe design.

Q17 **Answer: (d) data minimisation**

Collect only what is needed.

Q18 **Answer: (b) 6 MB**

24 ÷ 4 = **6 MB**.

Q19 **Answer: (b) 0.333**

50 ÷ (100 + 100 − 50) = 50/150 = **0.333**.

Q20 **Answer: Solution program**

Subtracting the maximum keeps exp() numerically stable without changing the result.

```
import numpy as np
z = np.array([2.0, 1.0, 0.1])
p = np.exp(z - z.max()); p /= p.sum()
print(np.round(p, 3), "-> class", int(p.argmax()), f"({p.max():.1%})")
```

OUTPUT

```
[0.659 0.242 0.099] -> class 0 (65.9%)
```

Q21 **Answer: Solution program**

KNN labels a new point by majority vote of its 3 closest training examples. (For real data, scale features so temperature and humidity count equally.)

```
from sklearn.neighbors import KNeighborsClassifier
X = [[22, 45], [24, 50], [25, 55], [23, 60], [26, 50], [31, 75], [34, 70], [30, 85], [33, 65], [35,
80]]
y = ["comfortable"] * 5 + ["uncomfortable"] * 5
knn = KNeighborsClassifier(n_neighbors=3).fit(X, y)
for s in ([27, 55], [33, 80]):
    print(s, "->", knn.predict([s])[0])
```

OUTPUT

```
[27, 55] -> comfortable
[33, 80] -> uncomfortable
```

Q22 **Answer: Solution program**

Each reading is compared with the recent past, so slow drifts are tolerated but sudden spikes are caught.

```
import numpy as np
rng = np.random.default_rng(5)
x = rng.normal(0.2, 0.02, 60); x[45] = 0.33          # one spike
for i in range(20, len(x)):
    win = x[i - 20:i]
    z = (x[i] - win.mean()) / win.std()
    if abs(z) > 3:
        print(f"reading {i}: {x[i]:.3f} g, z = {z:.1f} -> ANOMALY")
```

OUTPUT

```
reading 45: 0.330 g, z = 8.2 -> ANOMALY
```

Q23 **Answer: Solution program**

Frame time is the sum of all stages; once inference is fast, pre- and post-processing dominate (Amdahl's law).

```
pre, draw = 6, 4
for name, infer in (("CPU", 38), ("NPU", 5)):
    total = pre + infer + draw
    print(f"{name}: {total} ms per frame -> {1000 / total:.1f} FPS")
```

OUTPUT

```
CPU: 48 ms per frame -> 20.8 FPS
NPU: 15 ms per frame -> 66.7 FPS
```

Q24 **Answer: (d) AI Camera or Pi + NPU counts people on the device and publishes only counts**

On-device inference, minimal data.

Q25 **Answer: (a) overfitting or training data that does not match real conditions**

Collect more varied, realistic data.

Edge AI = inference on the device: fast, offline, private. Train elsewhere, convert and quantise (int8: 4× smaller, faster, small accuracy loss). Classic ML with scikit-learn handles sensor data (features → train/test split → classifier; IsolationForest or z-scores for anomalies). LiteRT runs .tflite models; detectors need thresholds + NMS (IoU). Hailo AI HAT+ or the AI Camera give real-time vision. Small LLMs run with Ollama. Minimise and protect personal data.

MODULE 24 · ADVANCED

Raspberry Pi Pico and MicroPython

Minimum duration **2 days** | Level **Advanced**

The Pico adds real-time microcontroller control to the Pi ecosystem and pairs well with a Pi as the main computer. This module compares the Pico, Pico W, Pico 2 and Pico 2 W and their RP2040/RP2350 chips, pinout and power; flashes MicroPython and uses Thonny and the REPL; programs the machine module (Pin, PWM, ADC, I2C, SPI, UART), timers and interrupts; connects the Pico W to Wi-Fi with a web server and an MQTT client; introduces PIO state machines and NeoPixels, the C/C++ SDK, the Pi + Pico architecture with serial links, and low-power operation, followed by 25 solved questions and programs. MicroPython programs were run on a simulated machine module.

IN THIS MODULE

24.1 Pico family
24.2 MicroPython
24.3 Pico W networking
24.4 PIO (programmable I/O)
24.5 C/C++ SDK overview
24.6 Pi + Pico architecture
24.7 Low-power operation
24.8 Practice questions
24.9 Solutions

24.1 Pico family

24.1.1 Pico, Pico W, Pico 2 and Pico 2 W

Board	Chip	Wireless	Notes
Pico (2021)	RP2040	—	the original; about ₹350–400
Pico W (2022)	RP2040	2.4 GHz Wi-Fi + Bluetooth (Infineon CYW43439)	IoT nodes
Pico 2 (2024)	RP2350	—	faster, more memory, security features
Pico 2 W (2024)	RP2350	Wi-Fi + Bluetooth	the best all-rounder for new IoT projects

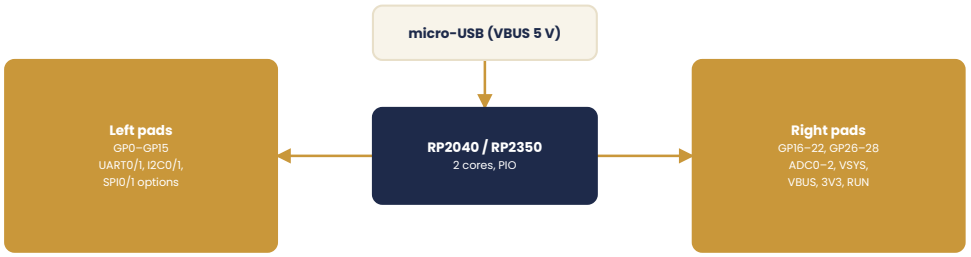
24.1.2 RP2040 and RP2350 chips

	RP2040	RP2350
CPU	2 × Arm Cortex-M0+ at 133 MHz	2 × Arm Cortex-M33 (or 2 × Hazard3 RISC-V) at 150 MHz
SRAM	264 KB	520 KB

	RP2040	RP2350
Flash on the Pico board	2 MB	4 MB
PIO state machines	8 (2 blocks)	12 (3 blocks)
Extras	—	floating-point unit, secure boot, TrustZone, OTP

24.1.3 Pinout and power

The Pico has 26 usable GPIO pins (GP0–GP22, GP26–GP28) on two rows of 20 castellated pads, all **3.3 V**. GP26–28 double as the 12-bit ADC inputs ADC0–ADC2; ADC4 is an internal temperature sensor. Every GPIO can be PWM, and the chip has 2 × I2C, 2 × SPI and 2 × UART that can be routed to several pin choices. Power: micro-USB (5 V on VBUS), or 1.8–5.5 V on **VSYS** (batteries) via the on-board buck-boost regulator; **3V3(OUT)** can supply about 300 mA to sensors. The on-board LED is GP25 on the Pico ("LED" on the W versions, where it is wired to the wireless chip).



PICO LAYOUT (SIMPLIFIED): CHECK THE OFFICIAL PINOUT BEFORE WIRING

> **WATCH OUT**
> Pico GPIO pins are 3.3 V only, exactly like the Pi. Never connect 5 V signals directly, and do not draw more than about 12 mA per pin or 50 mA in total.

24.2 MicroPython

24.2.1 Flashing MicroPython firmware

1. Download the MicroPython `.uf2` file for your exact board (Pico, Pico W, Pico 2…) from micropython.org or raspberrypi.com.
2. Hold the **BOOTSEL** button while plugging in the USB cable: the Pico appears as a USB drive called RPI-RP2 (or RP2350).
3. Drag the .uf2 file onto it. The Pico reboots running MicroPython. (Thonny can also do this: Tools → Options → Interpreter → Install MicroPython.)

24.2.2 Thonny and the REPL

In Thonny choose the interpreter "MicroPython (Raspberry Pi Pico)". The shell pane is the **REPL** (Read-Eval-Print Loop): type Python and the Pico runs it immediately. Save a file as **main.py** on the Pico and it runs automatically at every power-up (boot.py runs first if present). From a Pi or PC terminal, `mpremote` does the same: `mpremote run blink.py`, `mpremote cp main.py :`.

BLINK.PY (MICROPYTHON)

```
from machine import Pin
import time

led = Pin(25, Pin.OUT)            # Pin("LED", Pin.OUT) on a Pico W / Pico 2 W
for _ in range(3):
    led.toggle()
    time.sleep_ms(500)
    led.toggle()
    time.sleep_ms(500)
print("done")
```

SIMULATED RUN (hardware events and printed output)

```
[  0.00 s] Pin(25) HIGH
[  0.50 s] Pin(25) LOW
[  1.00 s] Pin(25) HIGH
[  1.50 s] Pin(25) LOW
[  2.00 s] Pin(25) HIGH
[  2.50 s] Pin(25) LOW
done
```

24.2.3 machine module: Pin, PWM, ADC, I2C, SPI, UART

Class	Example
Pin	Pin(15, Pin.IN, Pin.PULL_UP).value()
PWM	p = PWM(Pin(16)); p.freq(1000); p.duty_u16(32768) # 50 %
ADC	ADC(26).read_u16() # 0–65535 (12-bit value scaled to 16 bits)
I2C	I2C(0, scl=Pin(1), sda=Pin(0), freq=400_000).scan()
SPI	SPI(0, baudrate=1_000_000, sck=Pin(18), mosi=Pin(19), miso=Pin(16))
UART	UART(0, baudrate=9600, tx=Pin(0), rx=Pin(1))

POT_PWM.PY (MICROPYTHON)

```
from machine import Pin, PWM, ADC
import time

pot = ADC(26)                         # potentiometer wiper on GP26 (ADC0)
led = PWM(Pin(15)); led.freq(1000)
sensor = ADC(4)                       # internal temperature sensor
for _ in range(3):
    raw = pot.read_u16()
    led.duty_u16(raw)                 # knob position = brightness
    volts = raw * 3.3 / 65535
    t = 27 - (sensor.read_u16() * 3.3 / 65535 - 0.706) / 0.001721
    print(f"pot {raw:5d} = {volts:.2f} V   chip {t:.1f} C")
    time.sleep(1)
```

OUTPUT

```
pot 16368 = 0.82 V   chip 31.3 C
pot 39312 = 1.98 V   chip 31.3 C
pot 62256 = 3.13 V   chip 31.3 C
```

WORKED EXAMPLE

Internal temperature formula

The sensor gives about 0.706 V at 27 °C and falls 1.721 mV per °C: T = 27 - (V - 0.706) ÷ 0.001721. A reading of 0.6991 V → 27 - (-0.0069 ÷ 0.001721) = 27 + 4.0 = **31.0 °C**. It measures the chip, not the room, and is only accurate to a few degrees.

24.2.4 Timers and interrupts

A **Timer** calls a function at a fixed rate in the background; a pin **IRQ** calls a handler the instant an edge occurs. Keep handlers very short (set a flag or counter) and do the work in the main loop.

TIMER_IRQ.PY (MICROPYTHON)

```
from machine import Pin, Timer
import time

led = Pin(25, Pin.OUT)
presses = 0
def on_press(pin):                          # interrupt handler: keep it tiny
    global presses
    presses += 1

btn = Pin(14, Pin.IN, Pin.PULL_UP)
btn.irq(trigger=Pin.IRQ_FALLING, handler=on_press)
tim = Timer(period=250, mode=Timer.PERIODIC, callback=lambda t: led.toggle())   # 2 Hz blink

last = 0
start = time.ticks_ms()
while time.ticks_diff(time.ticks_ms(), start) < 3000:
    if presses != last:
        print(f"{time.ticks_diff(time.ticks_ms(), start)} ms: presses = {presses}")
        last = presses
    time.sleep_ms(10)
tim.deinit()
print("LED toggled by the timer while the main loop watched the button")
```

OUTPUT

```
800 ms: presses = 1
2099 ms: presses = 2
LED toggled by the timer while the main loop watched the button
```

SHORTCUT TRICK

Use **time.ticks_ms()** and **time.ticks_diff()** for timing on MicroPython: the tick counter wraps round, and ticks_diff handles that correctly where plain subtraction would fail.

24.3 Pico W networking

24.3.1 Wi-Fi connection

WIFI.PY (MICROPYTHON)

```
import network, time

wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect("HomeNet", "s3cret-pass")
for i in range(20):                              # wait up to 10 s
    if wlan.isconnected():
        break
    time.sleep(0.5)
if wlan.isconnected():
    print("connected after", (i * 0.5), "s, IP", wlan.ifconfig()[0])
else:
    print("could not connect")
```

OUTPUT

```
connected after 2.5 s, IP 192.168.1.77
```

Keep Wi-Fi credentials in a separate `secrets.py` on the Pico, not in shared code. The Pico W supports 2.4 GHz networks only.

24.3.2 Simple web server

MAIN.PY: A TINY WEB SERVER (REFERENCE). LIBRARIES SUCH AS MICRODOT GIVE FLASK-LIKE ROUTES.

```
import socket, network
from machine import Pin
led = Pin("LED", Pin.OUT)
# (connect to Wi-Fi first, as above)
s = socket.socket(); s.bind(("0.0.0.0", 80)); s.listen(1)
while True:
    cl, addr = s.accept()
    req = cl.recv(1024).decode()
    if "GET /on" in req: led.on()
    if "GET /off" in req: led.off()
    cl.send("HTTP/1.0 200 OK\r\nContent-Type: text/html\r\n\r\n")
    cl.send(f"<h1>Pico W</h1><p>LED is {'ON' if led.value() else 'off'}</p>"
            "<a href='/on'>On</a> | <a href='/off'>Off</a>")
    cl.close()
```

24.3.3 MQTT client

MQTT_NODE.PY (MICROPYTHON)

```
from umqtt.simple import MQTTClient
from machine import ADC
import time, json

c = MQTTClient("pico-w-1", "192.168.1.42", user="sensor", password="s3cret")
c.connect()
temp = ADC(4)
for _ in range(3):
    t = 27 - (temp.read_u16() * 3.3 / 65535 - 0.706) / 0.001721
    c.publish(b"home/pico1/temp", json.dumps({"t": round(t, 1)}))
    time.sleep(30)
c.disconnect()
```

SIMULATED RUN (hardware events and printed output)

```
[  0.00 s] MQTT pico-w-1 connect 192.168.1.42:1883 -> accepted
[  0.00 s] MQTT publish home/pico1/temp {"t": 28.1}
[ 30.00 s] MQTT publish home/pico1/temp {"t": 29.0}
[ 60.00 s] MQTT publish home/pico1/temp {"t": 29.5}
[ 90.00 s] MQTT pico-w-1 disconnected
```

24.4 PIO (programmable I/O)

24.4.1 State machines overview

PIO blocks are tiny independent processors inside the chip, each with 4 (RP2040) state machines that run short assembly programs (up to 32 instructions per block) with **cycle-exact timing**, without the CPU. They can implement protocols the chip lacks: WS2812 NeoPixels, extra UARTs, VGA/DVI video, quadrature encoders, stepper pulses, even an SD card interface. Instructions include `set`, `out`, `in_`, `jmp`, `wait`, with delays in square brackets.

24.4.2 Driving NeoPixels with PIO

MicroPython's built-in `neopixel` module uses PIO-quality timing internally; this is all you need:

NEOPIXEL_BAR.PY (MICROPYTHON)

```
import machine, neopixel, time

np = neopixel.NeoPixel(machine.Pin(22), 8)
for i in range(8):
    np[i] = (0, 0, 0)
for i in range(8):                          # light up one by one in rising brightness
    np[i] = (0, 0, 20 + i * 30)
    np.write()
    time.sleep_ms(100)
print("last pixel:", np[7])
print(open("strip.txt").read())
```

OUTPUT

```
last pixel: (0, 0, 230)
0,0,20;0,0,50;0,0,80;0,0,110;0,0,140;0,0,170;0,0,200;0,0,230
```

THE SAME THING WRITTEN AS A PIO PROGRAM (REFERENCE)

```
import rp2
@rp2.asm_pio(sideset_init=rp2.PIO.OUT_LOW, out_shiftdir=rp2.PIO.SHIFT_LEFT, autopull=True,
pull_thresh=24)
def ws2812():
    T1, T2, T3 = 2, 5, 3
    wrap_target()
    label("bitloop")
    out(x, 1)               .side(0)    [T3 - 1]
    jmp(not_x, "do_zero")   .side(1)    [T1 - 1]
    jmp("bitloop")          .side(1)    [T2 - 1]
    label("do_zero")
    nop()                   .side(0)    [T2 - 1]
    wrap()
sm = rp2.StateMachine(0, ws2812, freq=8_000_000, sideset_base=machine.Pin(22))
sm.active(1)
sm.put(0x00FF00 << 8)       # one pixel, GRB order
```

24.5 C/C++ SDK overview

The official **Pico SDK** (C/C++, CMake) gives full speed and access to every feature: both cores, DMA, PIO, low-power modes and USB device classes. Set it up with the VS Code Raspberry Pi Pico extension, build to a .uf2 and drag it onto the board. C code typically runs 10–100× faster than MicroPython; the Arduino-Pico core is another C++ option. Use MicroPython to prototype, then C/C++ where timing or speed demand it.

BLINK.C WITH THE PICO SDK

```
#include "pico/stdlib.h"
int main() {
    const uint LED = 25;
    gpio_init(LED); gpio_set_dir(LED, GPIO_OUT);
    while (true) { gpio_put(LED, 1); sleep_ms(250); gpio_put(LED, 0); sleep_ms(250); }
}
```

24.6 Pi + Pico architecture

24.6.1 Pico for real-time control, Pi for processing and networking

DIVISION OF LABOUR

Linux is not real-time: a Python loop on the Pi can be delayed by tens of milliseconds at any moment. The Pico runs one program with microsecond-precise timing and starts instantly, so it handles motor PWM, encoder counting and watchdog safety, while the Pi does vision, planning and networking.

24.6.2 USB serial and UART communication between them

Over USB the Pico's MicroPython REPL appears on the Pi as `/dev/ttyACM0`; `print()` on the Pico is read by pyserial on the Pi, and `sys.stdin.readline()` on the Pico reads what the Pi writes. Alternatively connect UART pins (Pico GP0/GP1 to Pi GPIO15/GPIO14, crossed, plus GND): both sides are 3.3 V, so no level shifting.

MOTOR_CONTROLLER.PY (ON THE PICO)

```
from machine import UART, Pin, PWM
import time

uart = UART(0, baudrate=115200, tx=Pin(0), rx=Pin(1))
left, right = PWM(Pin(16)), PWM(Pin(17))
for p in (left, right):
    p.freq(20_000)
last_cmd = time.ticks_ms()
while time.ticks_diff(time.ticks_ms(), last_cmd) < 1000:      # watchdog: stop if silent for 1 s
    if uart.any():
        parts = uart.readline().decode().split()
        if len(parts) == 3 and parts[0] == "M":
            l, r = (max(0.0, min(1.0, float(v))) for v in parts[1:])
            left.duty_u16(int(l * 65535)); right.duty_u16(int(r * 65535))
            uart.write(f"OK {l:.2f} {r:.2f}\n")
            last_cmd = time.ticks_ms()
    time.sleep_ms(5)
left.duty_u16(0); right.duty_u16(0)
print("no command for 1 s: motors stopped (fail-safe)")
```

SIMULATED RUN (hardware events and printed output)

```
[  0.00 s] PWM(GPIO16) freq 20000 Hz
[  0.00 s] PWM(GPIO17) freq 20000 Hz
[  0.33 s] PWM(GPIO16) duty 32767/65535 = 50.0%
[  0.33 s] PWM(GPIO17) duty 32767/65535 = 50.0%
[  0.33 s] UART TX -> 'OK 0.50 0.50'
[  0.67 s] PWM(GPIO16) duty 52428/65535 = 80.0%
[  0.67 s] PWM(GPIO17) duty 19660/65535 = 30.0%
[  0.67 s] UART TX -> 'OK 0.80 0.30'
[  1.00 s] PWM(GPIO16) duty 65535/65535 = 100.0%
[  1.00 s] PWM(GPIO17) duty 0/65535 = 0.0%
[  1.00 s] UART TX -> 'OK 1.00 0.00'
[  2.02 s] PWM(GPIO16) duty 0/65535 = 0.0%
no command for 1 s: motors stopped (fail-safe)
```

Note the **fail-safe**: if the Pi crashes or the cable is pulled, the Pico stops the motors after one second without commands. Values are clamped, so the invalid speed 1.5 became 1.0.

24.7 Low-power operation

Mode (Pico, approximate)	Current
running at 125 MHz, busy loop	≈ 20–25 mA
Pico W with Wi-Fi active	≈ 40–70 mA (peaks higher when transmitting)
machine.lightsleep()	≈ 1–2 mA (RAM kept, wakes on timer or pin)
machine.deepsleep()	lowest, but the Pico restarts from main.py on wake

For battery nodes: wake, measure, connect, send, turn Wi-Fi off (`wlan.active(False)`), sleep for minutes. Remove the power LED on sensor modules, lower the clock (`machine.freq(48_000_000)`), and use an external RTC or a TPL5110 timer chip that switches power off completely between readings.

WORKED EXAMPLE

Battery life of a sensor node

Every 10 minutes: awake 5 s at 60 mA, then 595 s at 1.5 mA. Average = (5 × 60 + 595 × 1.5) ÷ 600 = (300 + 892.5) ÷ 600 = **1.99 mA**. Three AA cells (≈ 2500 mAh): 2500 ÷ 1.99 ≈ 1257 h ≈ **52 days**. Halving awake time or using a power-switch timer extends this a lot.

PRACTICE CHECKPOINT

Build a Pico W sensor node that publishes readings over MQTT to the Pi, and a Pico motor controller that takes commands from the Pi over USB serial.

24.8 Practice questions

24.8.1 Basic level

Q1 Which chip is on the original Raspberry Pi Pico?

(a) RP2350 (b) BCM2712 (c) ESP32 (d) RP2040

Q2 Which Pico boards have Wi-Fi?

(a) Pico and Pico 2
(b) Pico W and Pico 2 W
(c) all Picos
(d) none

Q3 How do you put a Pico into USB drive mode to flash firmware?

(a) hold BOOTSEL while plugging in USB
(b) press RUN twice
(c) short GP0 to GND
(d) type sudo flash

Q4 What file type is dragged onto the Pico to flash it?

(a) .hex (b) .img (c) .uf2 (d) .tflite

Q5 What name must a MicroPython file have to run automatically at power-up?

(a) main.py (b) start.py (c) boot.txt (d) run.py

Q6 Which pin is the on-board LED of the original Pico?

(a) GP0 (b) GP25 (c) GP15 (d) GP28

Q7 What range does ADC.read_u16() return?

(a) 0–1023 (b) 0–4095 (c) 0–255 (d) 0–65535

Q8 What is the REPL?

(a) a power regulator
(b) a file system
(c) an interactive prompt that runs code line by line on the board
(d) a PIO program

Q9 What voltage are the Pico's GPIO pins?

(a) 5 V (b) 3.3 V (c) 1.8 V (d) 12 V

24.8.2 Intermediate level

Q10 Which pins are the Pico's ADC inputs?

(a) GP0, GP1, GP2
(b) GP25 only
(c) GP26, GP27, GP28
(d) all pins

Q11 Why keep interrupt handlers short?

(a) MicroPython forbids more than one line
(b) to save flash
(c) they run on the Pi
(d) they interrupt the main program; long work blocks other interrupts and timing

Q12 Why use ticks_diff() instead of subtracting ticks_ms() values?

(a) it is faster
(b) the tick counter wraps round and ticks_diff handles it
(c) subtraction is not allowed in MicroPython
(d) it returns seconds

Q13 What is PIO?

(a) small programmable state machines that do cycle-exact I/O without the CPU
(b) a Wi-Fi chip
(c) a Python package
(d) a power mode

Q14 What does PWM.duty_u16(32768) give?

(a) 100 %
(b) about 50 % duty cycle
(c) 3.3 %
(d) 0 %

Q15 Why combine a Pi with a Pico in a robot?

(a) the Pico gives real-time control while the Pi does vision and networking
(b) the Pico is faster at OpenCV
(c) the Pi cannot use USB
(d) the Pico runs Linux

Q16 How does a Pico usually appear on a Pi when connected by USB with MicroPython?

(a) /dev/i2c-1 (b) /dev/spidev0.0 (c) /dev/ttyACM0 (d) /dev/video0

Q17 What happens after machine.deepsleep() ends on a Pico?

(a) it resumes on the next line
(b) it stays off
(c) it reflashes firmware
(d) the board restarts and runs main.py again

Q18 ADC(26).read_u16() returns 49 152. What is the voltage?

(a) about 1.65 V (b) about 3.3 V (c) about 0.75 V (d) about 2.48 V

24.8.3 Advanced level

Q19 Awake 10 s at 50 mA, asleep 290 s at 1 mA, repeating. What is the average current?

(a) about 25.5 mA (b) about 1 mA (c) about 2.63 mA (d) about 5 mA

Q20 MicroPython: read a button on GP14 (pull-up) and toggle the LED on GP25 on each press, using an IRQ with simple 200 ms debouncing.

Q21 MicroPython: dim an LED on GP15 smoothly up and down once using PWM at 1 kHz in 5 steps each way.

Q22 MicroPython: connect a Pico W to Wi-Fi with a 10-second timeout and print the IP address or an error; Wi-Fi takes 3.2 s in the simulation.

Q23 Calculate the battery life of a Pico W node on a 2000 mAh battery: awake 4 s at 55 mA every 5 minutes, asleep otherwise at 1.3 mA.

Q24 Which design gives a robot a safe stop if the Pi crashes?

(a) the Pico stops the motors when no command has arrived for about 1 s
(b) the Pi sends a stop command when it crashes
(c) MQTT retained messages
(d) a bigger battery

Q25 When should you move Pico code from MicroPython to the C/C++ SDK?

(a) always, before testing
(b) never, MicroPython is always as fast
(c) only for blinking LEDs
(d) when timing or speed requirements are beyond MicroPython

24.9 Solutions

Q1 **Answer: (d) RP2040**

RP2040.

Q2 **Answer: (b) Pico W and Pico 2 W**

The W versions.

Q3 **Answer: (a) hold BOOTSEL while plugging in USB**

It appears as RPI-RP2.

Q4 **Answer: (c) .uf2**

.uf2.

Q5 **Answer: (a) main.py**

main.py (after boot.py).

Q6 **Answer: (b) GP25**

GP25 ("LED" on W boards).

Q7 **Answer: (d) 0–65535**

16-bit scaled (12-bit real resolution).

Q8 **Answer: (c) an interactive prompt that runs code line by line on the board**

Read-Eval-Print Loop.

Q9 **Answer: (b) 3.3 V**

Same as the Pi: 3.3 V only.

Q10 **Answer: (c) GP26, GP27, GP28**

ADC0–2 (ADC4 = internal temperature).

Q11 **Answer: (d) they interrupt the main program; long work blocks other interrupts and timing**

Set a flag, work in the loop.

Q12 **Answer: (b) the tick counter wraps round and ticks_diff handles it**

Wrap-safe timing.

Q13 **Answer: (a) small programmable state machines that do cycle-exact I/O without the CPU**

Programmable I/O.

Q14 **Answer: (b) about 50 % duty cycle**

32768 ÷ 65535 ≈ 50 %.

Q15 **Answer: (a) the Pico gives real-time control while the Pi does vision and networking**

Division of labour.

Q16 **Answer: (c) /dev/ttyACM0**

USB CDC serial.

Q17 **Answer: (d) the board restarts and runs main.py again**

Deep sleep loses RAM.

Q18 **Answer: (d) about 2.48 V**

49 152 × 3.3 ÷ 65 535 = **2.475 V**.

Q19 **Answer: (c) about 2.63 mA**

(500 + 290) ÷ 300 = **2.63 mA**.

Q20 **Answer: Solution program**

Edges within 200 ms of the last accepted press are ignored. The second press bounced (two falling edges 5 ms apart) but toggled only once.

```
from machine import Pin
import time
led = Pin(25, Pin.OUT)
last = 0
def pressed(p):
    global last
    now = time.ticks_ms()
    if time.ticks_diff(now, last) > 200:
        led.toggle()
        last = now
btn = Pin(14, Pin.IN, Pin.PULL_UP)
btn.irq(trigger=Pin.IRQ_FALLING, handler=pressed)
time.sleep(3)
```

SIMULATED RUN

```
[  0.50 s] Pin(25) HIGH
[  1.50 s] Pin(25) LOW
[  2.40 s] Pin(25) HIGH
```

Q21 **Answer: Solution program**

Duty 0 → 100 % → 0 % in quarter steps; deinit() stops the PWM output.

```
from machine import Pin, PWM
import time
led = PWM(Pin(15)); led.freq(1000)
levels = [0, 16384, 32768, 49152, 65535]
for d in levels + levels[::-1][1:]:
    led.duty_u16(d)
    time.sleep_ms(100)
led.deinit()
```

SIMULATED RUN

```
[  0.00 s] PWM(GPIO15) freq 1000 Hz
[  0.10 s] PWM(GPIO15) duty 16384/65535 = 25.0%
[  0.20 s] PWM(GPIO15) duty 32768/65535 = 50.0%
[  0.30 s] PWM(GPIO15) duty 49152/65535 = 75.0%
[  0.40 s] PWM(GPIO15) duty 65535/65535 = 100.0%
[  0.50 s] PWM(GPIO15) duty 49152/65535 = 75.0%
[  0.60 s] PWM(GPIO15) duty 32768/65535 = 50.0%
[  0.70 s] PWM(GPIO15) duty 16384/65535 = 25.0%
[  0.80 s] PWM(GPIO15) duty 0/65535 = 0.0%
[  0.90 s] PWM(GPIO15) off
```

Q22 **Answer: Solution program**

Polling isconnected() with a timeout prevents the node hanging forever.

```
import network, time
wlan = network.WLAN(network.STA_IF); wlan.active(True)
wlan.connect("HomeNet", "pass")
t0 = time.ticks_ms()
while not wlan.isconnected() and time.ticks_diff(time.ticks_ms(), t0) < 10_000:
    time.sleep_ms(200)
if wlan.isconnected():
    print("IP", wlan.ifconfig()[0], "after", time.ticks_diff(time.ticks_ms(), t0), "ms")
else:
    print("Wi-Fi failed: check SSID, password and 2.4 GHz")
```

OUTPUT

```
IP 192.168.1.77 after 3200 ms
```

Q23 **Answer: Solution program**

Weighted average current, then capacity ÷ current. Sleep current dominates over long periods, so reduce it first.

```
awake_s, awake_ma, period_s, sleep_ma, cap = 4, 55, 300, 1.3, 2000
avg = (awake_s * awake_ma + (period_s - awake_s) * sleep_ma) / period_s
hours = cap / avg
print(f"average {avg:.2f} mA -> {hours:.0f} h = {hours / 24:.0f} days")
```

OUTPUT

```
average 2.02 mA -> 992 h = 41 days
```

Q24 **Answer: (a) the Pico stops the motors when no command has arrived for about 1 s**

A watchdog on the controller.

Q25 **Answer: (d) when timing or speed requirements are beyond MicroPython**

Prototype in MicroPython, optimise in C.

Pico = RP2040/RP2350 microcontroller, 3.3 V GPIO, ADC on GP26–28, flashed by BOOTSEL + .uf2. MicroPython: main.py runs at boot; machine.Pin/PWM/ADC/I2C/SPI/UART; Timer and short IRQ handlers; ticks_ms/ticks_diff. Pico W: network.WLAN + umqtt.simple. PIO for exact protocols (NeoPixels). C SDK for speed. Pair with a Pi over USB serial/UART with a fail-safe watchdog; sleep between readings for battery life.

MODULE 25 · ADVANCED

Robotics with Raspberry Pi and ROS 2

Minimum duration **3 days** | Level **Advanced**

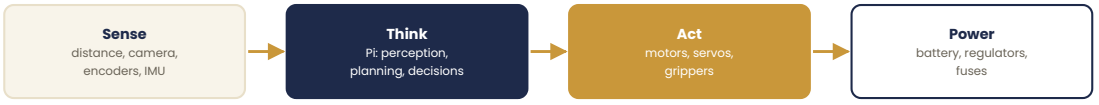
The Pi is the brain of many robots, and ROS 2 is the industry framework for building them. This module covers robot architecture (sensors, controller, actuators, power; the Pi for high-level thinking and a microcontroller for low-level control), building a mobile robot (chassis, motors, drivers, batteries and regulation, web and gamepad control), autonomous behaviours (obstacle avoidance, camera line following, wheel encoders and odometry, PID speed and heading control), ROS 2 fundamentals (installation, nodes, topics, services, actions, rclpy publishers and subscribers, launch files, parameters, rviz2 and rqt), mapping and navigation with LiDAR, SLAM and Nav2, and simulation in Gazebo, followed by 25 solved questions and programs.

IN THIS MODULE

25.1 Robot architecture
25.2 Building a mobile robot
25.3 Autonomous behaviours
25.4 ROS 2 fundamentals
25.5 Mapping and navigation
25.6 Simulation with Gazebo
25.7 Practice questions
25.8 Solutions

25.1 Robot architecture

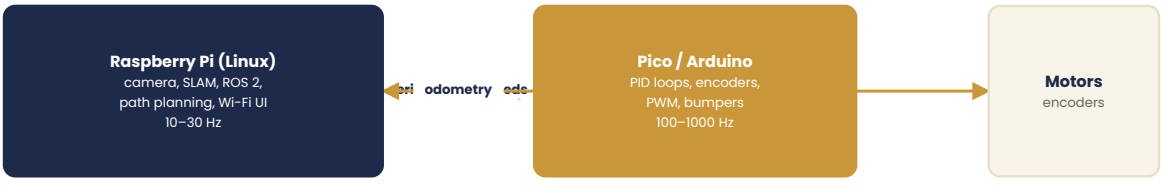
25.1.1 Sensors, controller, actuators and power



EVERY ROBOT: SENSE → THINK → ACT, ALL RUNNING FROM A POWER SYSTEM

A robot repeats the **sense–think–act** loop many times per second. Its four subsystems: **sensors** (ultrasonic, IR, camera, LiDAR, IMU, wheel encoders, bump switches), the **controller** (computers running the software), **actuators** (DC motors, servos, steppers) and **power** (batteries, regulators, protection). Most robot problems are power or mechanical problems, so design those carefully.

25.1.2 Pi as the high-level brain, microcontroller for low-level control



TWO-LEVEL CONTROL

Fast loops (motor speed control every 1–10 ms, encoder counting at kHz rates, emergency stop) belong on the microcontroller, where timing is guaranteed. The Pi sends target speeds ("left 0.3 m/s, right 0.25 m/s") and receives wheel counts. This is exactly how ROS 2 robots such as TurtleBot are built.

25.2 Building a mobile robot

25.2.1 Chassis, motors and drivers

Part	Typical choice	Notes
Chassis	2-wheel differential drive + caster, or 4WD	differential drive steers by wheel speed difference; simplest to control
Motors	N20 or JGA25-370 gear motors with Hall encoders, 6–12 V	encoders are essential for odometry and straight driving
Driver	TB6612FNG, DRV8833 (small); L298N or BTS7960 (bigger)	match the stall current, not just the running current
Sensors	HC-SR04 / VL53L1X ToF, line sensors, Camera Module 3, IMU (MPU6050/BNO055), LiDAR (RPLIDAR, LD19)	start simple: distance + camera

WORKED EXAMPLE

Choosing motor speed

A 65 mm wheel has circumference π × 0.065 = 0.204 m. For a top speed of 0.5 m/s it must turn 0.5 ÷ 0.204 = 2.45 rev/s = **147 rpm**: choose a gear motor rated about 200 rpm at your battery voltage to keep a margin.

25.2.2 Battery and power regulation

Use a **2S Li-ion/LiPo pack (7.4 V nominal, 8.4 V full)** or 6 × AA NiMH for the motors, and a **5 V 5 A buck converter** (or a UPS/battery HAT) for the Pi. Motor current spikes make the voltage dip, so the Pi's regulator must have headroom; add a large capacitor (≥ 470 µF) across the motor supply, a fuse and a main power switch, and share GND. Never discharge Li-ion cells below about 3.0 V per cell; use a BMS or protected cells, and charge them with the correct charger.

WORKED EXAMPLE

Robot runtime

Pi 5 average 6 W + motors average 4 W = 10 W. A 2S 3000 mAh pack holds 7.4 × 3 = 22.2 Wh; with ~85 % converter efficiency and 80 % usable capacity: 22.2 × 0.8 × 0.85 ÷ 10 ≈ **1.5 hours**.

25.2.3 Remote control from a web page or gamepad

A Flask page with arrow buttons that POST to /drive (module 15), or a gamepad read with evdev (module 11), sends speed commands. Use a **dead-man timeout**: if no command arrives for 0.5 s, stop. Mixing joystick axes into wheel speeds for differential drive:

ARCADE_MIX.PY

```
def mix(throttle, turn, max_speed=1.0):
    """joystick throttle/turn in -1..1 -> left/right wheel speeds in -1..1"""
    left, right = throttle + turn, throttle - turn
    scale = max(1.0, abs(left), abs(right))            # keep within limits, preserve ratio
    return round(left / scale * max_speed, 2), round(right / scale * max_speed, 2)

for t, r in [(1, 0), (0.5, 0.5), (0, 1), (1, 1), (-0.6, 0.2)]:
    print(f"throttle {t:+.1f} turn {r:+.1f} -> left/right {mix(t, r)}")
```

OUTPUT

```
throttle +1.0 turn +0.0 -> left/right (1.0, 1.0)
throttle +0.5 turn +0.5 -> left/right (1.0, 0.0)
throttle +0.0 turn +1.0 -> left/right (1.0, -1.0)
throttle +1.0 turn +1.0 -> left/right (1.0, 0.0)
throttle -0.6 turn +0.2 -> left/right (-0.4, -0.8)
```

25.3 Autonomous behaviours

25.3.1 Obstacle avoidance

AVOID.PY

```
from gpiozero import Robot, DistanceSensor
from time import sleep

bot = Robot(left=(7, 8), right=(9, 10))
eyes = DistanceSensor(echo=24, trigger=23, max_distance=2)
SAFE, SLOW = 0.30, 0.60                        # metres
for _ in range(10):
    d = eyes.distance
    if d < SAFE:
        bot.backward(0.4); sleep(0.4)
        bot.right(0.6);    sleep(0.5)          # turn away, then re-check
        action = "obstacle: back up and turn"
    elif d < SLOW:
        bot.forward(0.35); action = "slow down"
        sleep(0.3)
    else:
        bot.forward(0.7);  action = "cruise"
        sleep(0.3)
    print(f"{d:4.2f} m  {action}")
bot.stop()
```

OUTPUT

```
1.50 m  cruise
1.50 m  cruise
0.90 m  cruise
0.90 m  cruise
0.50 m  slow down
0.50 m  slow down
0.22 m  obstacle: back up and turn
0.22 m  obstacle: back up and turn
```

```
1.20 m  cruise
1.20 m  cruise
```

One forward sensor misses obstacles at the sides; add left and right sensors (or a servo that sweeps one sensor), and a bump switch as a last-resort stop.

25.3.2 Line following with a camera

Crop a strip near the bottom of each frame, threshold it to find the dark line, compute the line's x position (moments) and steer with the error from the image centre (module 22). A camera sees the line ahead, so it can anticipate curves better than IR sensors under the chassis.

CAMERA_LINE.PY

```
import numpy as np
KP, BASE = 0.004, 0.45
def follow(strip):                        # strip: 2-D grayscale image near the bottom of the frame
    mask = strip < 60                     # dark line
    cols = np.nonzero(mask)[1]
    if cols.size < 20:
        return None
    err = cols.mean() - strip.shape[1] / 2
    turn = KP * err
    return round(BASE + turn, 2), round(BASE - turn, 2), round(err)

for line_x in (160, 120, 230):            # synthetic 320-pixel-wide strips with a line at line_x
    strip = np.full((30, 320), 200, np.uint8); strip[:, line_x - 8:line_x + 8] = 20
    left, right, err = follow(strip)
    print(f"line at x={line_x}: error {err:+4d} px -> left {left}, right {right}")
```

OUTPUT

```
line at x=160: error   +0 px -> left 0.45, right 0.45
line at x=120: error  -40 px -> left 0.29, right 0.61
line at x=230: error  +70 px -> left 0.73, right 0.17
```

25.3.3 Wheel encoders and odometry

An **encoder** gives a fixed number of pulses (ticks) per wheel revolution. From the ticks of both wheels, **odometry** estimates the robot's position (x, y) and heading θ. For a differential-drive robot with wheel separation (track) L:

distance per tick = π × D ÷ ticks_per_rev; d = (d_L + d_R) ÷ 2; Δθ = (d_R − d_L) ÷ L; x += d·cos(θ + Δθ/2), y += d·sin(θ + Δθ/2)

ODOMETRY.PY

```
import math
D, TICKS, L = 0.065, 1440, 0.15                 # wheel diameter m, ticks/rev, track m
per_tick = math.pi * D / TICKS
x = y = th = 0.0
moves = [(4000, 4000), (4000, 4000), (-500, 500), (-207, 207), (3000, 3000)]    # (left, right)
ticks
for tl, tr in moves:
    dl, dr = tl * per_tick, tr * per_tick
    d, dth = (dl + dr) / 2, (dr - dl) / L
    x += d * math.cos(th + dth / 2); y += d * math.sin(th + dth / 2); th += dth
    print(f"ticks {tl:5d},{tr:5d} -> x {x:5.3f} m  y {y:5.3f} m  heading {math.degrees(th):6.1f}
deg")
```

OUTPUT

```
ticks  4000, 4000 -> x 0.567 m  y 0.000 m  heading    0.0 deg
ticks  4000, 4000 -> x 1.134 m  y 0.000 m  heading    0.0 deg
ticks  -500,  500 -> x 1.134 m  y 0.000 m  heading   54.2 deg
ticks  -207,  207 -> x 1.134 m  y 0.000 m  heading   76.6 deg
ticks  3000, 3000 -> x 1.233 m  y 0.414 m  heading   76.6 deg
```

Odometry drifts because wheels slip and dimensions are never exact; an IMU (gyro) improves heading, and SLAM (25.5) corrects position against a map.

25.3.4 PID speed and heading control

A **PID controller** computes a correction from the error e = target - measured: **P** (proportional: kp·e) reacts to the present error, **I** (integral: ki·∑e·dt) removes steady offset, **D** (derivative: kd·de/dt) damps overshoot. Tune P first until it responds briskly, add D to stop oscillation, then a little I to remove the remaining error.

PID.PY

```
class PID:
    def __init__(self, kp, ki, kd, out_min=-1.0, out_max=1.0):
        self.kp, self.ki, self.kd = kp, ki, kd
        self.lo, self.hi = out_min, out_max
        self.i = 0.0; self.prev = None
    def update(self, target, measured, dt):
        e = target - measured
        self.i = max(self.lo, min(self.hi, self.i + e * dt * self.ki))     # anti-windup clamp
        d = 0.0 if self.prev is None else (e - self.prev) / dt
        self.prev = e
        return max(self.lo, min(self.hi, self.kp * e + self.i + self.kd * d))

def simulate(pid, steps=80, dt=0.05):              # simple motor: speed lags behind the drive
signal
    speed, log = 0.0, []
    for k in range(steps):
        u = pid.update(1.0, speed, dt)             # target speed 1.0 m/s
        speed += (1.6 * u - speed) * 0.25          # first-order motor + friction
        log.append(speed)
    over = max(log) - 1.0
    return round(log[-1], 3), round(max(over, 0), 3)

for name, gains in [("P only", (0.9, 0, 0)), ("PI, ki=3", (0.9, 3, 0)), ("PI, ki=12", (0.9, 12,
0))]:
    final, over = simulate(PID(*gains))
    print(f"{name:10} final speed {final}  overshoot {over}")
```

OUTPUT

```
P only     final speed 0.59  overshoot 0
PI, ki=3   final speed 1.0  overshoot 0
PI, ki=12  final speed 1.0  overshoot 0.164
```

The P-only controller settles well below the target (steady-state error). A moderate integral term reaches the target exactly; too much integral overshoots by 16 %. On real motors a small D term then reduces overshoot, but D also amplifies sensor noise, so filter the measurement first. The same class can hold a heading (target angle from the IMU, output = turn rate).

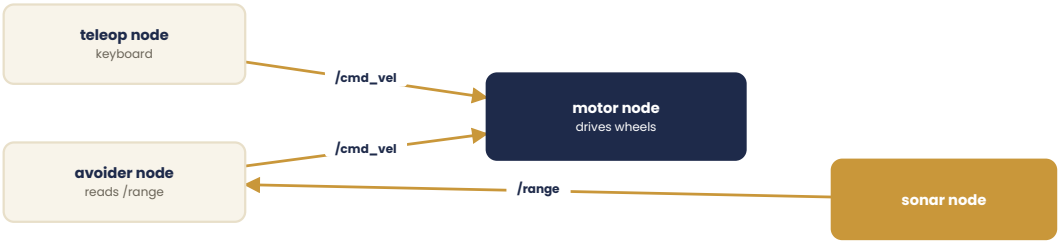
25.4 ROS 2 fundamentals

25.4.1 Installing ROS 2 (Ubuntu on Pi or Docker)

ROS 2 (Robot Operating System) is not an OS but a framework of libraries and tools: a communication layer (DDS), drivers, navigation, visualisation and simulation, used across industry and research. Each release targets an Ubuntu version: **Jazzy Jalisco** (LTS, 2024) runs on Ubuntu 24.04. Options for a Pi: install **Ubuntu Server 24.04** and the ROS 2 apt packages, or run ROS 2 in **Docker** on Raspberry Pi OS (`docker run -it --net=host ros:jazzy`).

25.4.2 Nodes, topics, services and actions

Concept	What it is	Example
Node	one process with one job	camera driver, motor controller, planner
Topic	named stream of typed messages, publish/subscribe (many-to-many)	/cmd_vel (geometry_msgs/Twist), /scan, /odom
Service	request → response call	/reset_odometry
Action	long-running goal with feedback and cancel	navigate_to_pose
Parameter	a node setting that can be changed	max_speed = 0.4
Message type	the data structure	Twist: linear.x (m/s), angular.z (rad/s)



NODES COMMUNICATING OVER TOPICS

25.4.3 Publishers and subscribers in Python (rclpy)

AVOIDER_NODE.PY

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Range
from geometry_msgs.msg import Twist

class Avoider(Node):
    def __init__(self):
        super().__init__("avoider")
        self.declare_parameter("safe_distance", 0.30)
        self.pub = self.create_publisher(Twist, "/cmd_vel", 10)
        self.create_subscription(Range, "/range", self.on_range, 10)

    def on_range(self, msg):
        safe = self.get_parameter("safe_distance").value
        cmd = Twist()
        if msg.range < safe:
            cmd.angular.z = 1.0                          # rotate on the spot (rad/s)
            self.get_logger().info(f"obstacle at {msg.range:.2f} m -> turning")
        else:
            cmd.linear.x = 0.2                           # forward 0.2 m/s
        self.pub.publish(cmd)

class FakeSonar(Node):                                   # stands in for the real sensor driver
    def __init__(self):
        super().__init__("sonar")
        self.pub = self.create_publisher(Range, "/range", 10)
        self.readings = iter([0.9, 0.6, 0.25, 0.18, 0.7])
        self.create_timer(0.5, self.tick)
    def tick(self):
        r = next(self.readings, None)
        if r is not None:
            self.pub.publish(Range(range=r))

class Motors(Node):
    def __init__(self):
        super().__init__("motors")
        self.create_subscription(Twist, "/cmd_vel", lambda m: self.get_logger().info(f"drive {m}"),
10)

rclpy.init()
nodes = [Avoider(), FakeSonar(), Motors()]
rclpy.spin(nodes[0])
```

OUTPUT

```
[INFO] [1773565200.502] [motors]: drive Twist(linear.x=0.2, angular.z=0.0)
[INFO] [1773565201.002] [motors]: drive Twist(linear.x=0.2, angular.z=0.0)
[INFO] [1773565201.501] [avoider]: obstacle at 0.25 m -> turning
[INFO] [1773565201.502] [motors]: drive Twist(linear.x=0.0, angular.z=1.0)
[INFO] [1773565202.001] [avoider]: obstacle at 0.18 m -> turning
[INFO] [1773565202.002] [motors]: drive Twist(linear.x=0.0, angular.z=1.0)
[INFO] [1773565202.502] [motors]: drive Twist(linear.x=0.2, angular.z=0.0)
```

On a real robot each class is its own node (often its own process or package), and you would start them with `ros2 run` or a launch file. The command-line tools inspect a running system:

ESSENTIAL ROS2 COMMANDS

```
ros2 node list
ros2 topic list
ros2 topic echo /cmd_vel
ros2 topic hz /scan                      # message rate
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.1}}"
ros2 param set /avoider safe_distance 0.4
ros2 interface show sensor_msgs/msg/Range
```

25.4.4 Launch files and parameters

ROBOT.LAUNCH.PY: START THE WHOLE ROBOT WITH "ROS2 LAUNCH MY_ROBOT ROBOT.LAUNCH.PY"

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(package="my_robot", executable="sonar"),
        Node(package="my_robot", executable="avoider", parameters=[{"safe_distance": 0.35}]),
        Node(package="my_robot", executable="motors"),
        Node(package="v4l2_camera", executable="v4l2_camera_node"),
    ])
```

Packages are built with **colcon** in a workspace (`~/ros2_ws/src/my_robot`, `colcon build`, `source install/setup.bash`). Parameters can also live in YAML files, so the same code runs on different robots.

25.4.5 rviz2 and rqt tools

rviz2 visualises the robot in 3-D: its model (URDF), coordinate frames (TF), laser scans, camera images, maps and planned paths. **rqt** offers plug-ins such as rqt_graph (the node/topic graph), rqt_plot (live graphs of values) and rqt_image_view. Run them on a laptop on the same network; ROS 2 nodes find each other automatically (same ROS_DOMAIN_ID).

25.5 Mapping and navigation

25.5.1 LiDAR sensors

A 2-D **LiDAR** (RPLIDAR A1/C1, LDROBOT LD19) spins a laser range-finder to measure distances all around, typically 360 points 5–10 times per second up to 8–12 m, published as a **sensor_msgs/LaserScan** on /scan. It is the key sensor for mapping indoors.

SCAN_SECTORS.PY

```
import math
ranges = [2.0] * 360                                    # one reading per degree
for a in range(350, 360): ranges[a] = 0.35              # obstacle slightly right of front
for a in range(0, 8): ranges[a] = 0.40                   # and straight ahead
for a in range(80, 100): ranges[a] = 0.8                 # wall on the left (90 deg)

def sector_min(lo, hi):
    return min(ranges[a % 360] for a in range(lo, hi))
front, left, right = sector_min(-20, 21), sector_min(60, 120), sector_min(240, 300)
print(f"front {front} m, left {left} m, right {right} m")
best = max(range(0, 360, 10), key=lambda a: sector_min(a - 10, a + 11))
print("most open direction:", best, "degrees")
x, y = ranges[90] * math.cos(math.radians(90)), ranges[90] * math.sin(math.radians(90))
print(f"wall point at 90 deg -> x={x:.2f}, y={y:.2f} m (robot frame)")
```

OUTPUT

```
front 0.35 m, left 0.8 m, right 2.0 m
most open direction: 20 degrees
wall point at 90 deg -> x=0.00, y=0.80 m (robot frame)
```

25.5.2 SLAM overview

SLAM (Simultaneous Localisation and Mapping) builds a map while working out where the robot is in it: it matches each new LiDAR scan against the map built so far, corrects odometry drift, and recognises places it has seen before ("loop closure"). In ROS 2, **slam_toolbox** produces an occupancy-grid map (free, occupied, unknown cells) that you save and reuse.

25.5.3 Nav2 overview

Nav2 is ROS 2's navigation stack: given a map and a goal pose (clicked in rviz2), it localises the robot (AMCL), plans a global path around known obstacles, follows it with a local controller that avoids new obstacles, and recovers when stuck (back up, spin). Inputs: map, /scan, /odom, TF; output: /cmd_vel. A Pi 4/5 can run Nav2 for a small robot, often with heavier tasks on a laptop.

25.6 Simulation with Gazebo

Gazebo simulates robots and worlds with physics, sensors (LiDAR, cameras, IMU) and the same ROS 2 interfaces as the real robot. Develop and test navigation and behaviours in simulation on a PC, then run the identical nodes on the Pi. A robot is described in **URDF**/SDF (links, joints, sensors, inertia). TurtleBot3/TurtleBot4 simulations are a good starting point.

PRACTICE CHECKPOINT

Build a Pi robot that avoids obstacles autonomously and streams its camera to a web page, then publish its sensor data as ROS 2 topics.

25.7 Practice questions

25.7.1 Basic level

Q1 What are the three steps every robot repeats?

(a) plan, build, test
(b) sense, think, act
(c) input, store, print
(d) charge, drive, stop

Q2 In a two-level design, what does the microcontroller usually do?

(a) SLAM mapping
(b) web dashboards
(c) object detection
(d) fast low-level control such as PID motor loops and encoder counting

Q3 What does a wheel encoder measure?

(a) wheel rotation, as ticks per revolution
(b) distance to walls
(c) battery voltage
(d) temperature

Q4 What is the standard ROS 2 topic for velocity commands?

(a) /scan **(b)** /odom **(c)** /cmd_vel **(d)** /map

Q5 In ROS 2, what is a node?

(a) a cable
(b) a type of LiDAR
(c) a file system
(d) a process that performs one job and communicates over topics, services and actions

Q6 Which tool visualises maps, scans and robot models in ROS 2?

(a) colcon **(b)** rviz2 **(c)** Gazebo only **(d)** nano

Q7 What does SLAM stand for?

(a) Simultaneous Localisation and Mapping
(b) Sensor Logging And Monitoring
(c) Smart LiDAR Auto Motion
(d) Serial Link And Messaging

Q8 Which simulator is commonly used with ROS 2?

(a) Thonny **(b)** Kodi **(c)** Gazebo **(d)** Grafana

Q9 Which message type does /cmd_vel use?

(a) sensor_msgs/Range
(b) std_msgs/String
(c) nav_msgs/Map
(d) geometry_msgs/Twist

25.7.2 Intermediate level

Q10 How does a differential-drive robot turn?

(a) by running its left and right wheels at different speeds

(b) by steering the front wheels
(c) by tilting the body
(d) with a rudder

Q11 What does the I term of a PID controller fix?

(a) oscillation only
(b) steady-state error that P alone leaves
(c) sensor noise
(d) battery drain

Q12 What does the D term of a PID controller do?

(a) removes all errors
(b) increases speed
(c) counts encoder ticks
(d) damps overshoot by reacting to the rate of change of error

Q13 What is a ROS 2 action used for?

(a) single numbers
(b) logging
(c) long-running goals with feedback and the option to cancel
(d) compiling code

Q14 Which command shows messages arriving on a topic?

(a) ros2 node kill
(b) ros2 build
(c) ros2 topic echo /topic
(d) ros2 cat

Q15 Why does odometry drift over time?

(a) the Pi clock is slow
(b) encoders forget ticks at night
(c) ROS 2 rounds numbers
(d) wheel slip and small dimension errors accumulate

Q16 Which ROS 2 stack plans paths and drives a robot to a goal on a map?

(a) slam_toolbox (b) Nav2 (c) rqt_plot (d) colcon

Q17 What is a dead-man timeout in remote control?

(a) the robot stops if no command arrives within a short time
(b) a battery alarm
(c) a LiDAR setting
(d) a ROS parameter type

Q18 A 70 mm wheel must reach 0.6 m/s. About what rpm is needed?

(a) 82 rpm (b) 164 rpm (c) 36 rpm (d) 600 rpm

25.7.3 Advanced level

Q19 An encoder gives 1440 ticks per revolution on a 65 mm wheel. What distance is one tick?

(a) about 0.142 mm (b) about 1.42 mm (c) about 0.045 mm (d) about 14.2 mm

Q20 A robot's left wheel moves 0.50 m and right wheel 0.60 m; the track is 0.15 m. Compute the

distance travelled by the centre and the heading change in degrees.

Q21 Implement a P-controller for heading: target 90°, current heading list [0, 30, 60, 80, 88, 95], kp = 0.02, output limited to ±1. Print the turn command for each reading.

Q22 Write a gpiozero obstacle avoider that drives forward and, when the distance drops below 0.25 m, stops, turns right for 0.5 s and continues; print each decision.

Q23 Write a ROS 2 node that publishes the CPU temperature as std_msgs/Float32 on /pi/temperature every second, and a second node that logs a warning above 60 °C.

Q24 Which architecture is typical for a Pi + ROS 2 mobile robot?

(a) the Pico runs SLAM; the Pi only blinks LEDs

(b) everything runs in the cloud

(c) Pico runs motor PID and encoders; the Pi runs ROS 2 nodes (driver, LiDAR, SLAM, Nav2); a laptop runs rviz2

(d) the laptop drives the motors directly over Wi-Fi

Q25 A robot's Pi resets whenever the motors start. What is the most likely cause?

(a) ROS 2 is misconfigured

(b) the LiDAR is too fast

(c) the encoder has too many ticks

(d) the motor current makes the shared supply voltage dip below what the Pi needs

25.8 Solutions

Q1 **Answer: (b) sense, think, act**

The robot control loop.

Q2 **Answer: (d) fast low-level control such as PID motor loops and encoder counting**

Real-time tasks.

Q3 **Answer: (a) wheel rotation, as ticks per revolution**

For odometry and speed control.

Q4 **Answer: (c) /cmd_vel**

geometry_msgs/Twist.

Q5 **Answer: (d) a process that performs one job and communicates over topics, services and actions**

Building block.

Q6 **Answer: (b) rviz2**

rviz2.

Q7 **Answer: (a) Simultaneous Localisation and Mapping**

Map + position together.

Q8 **Answer: (c) Gazebo**

Gazebo.

Q9 **Answer: (d) geometry_msgs/Twist**

linear.x and angular.z.

Q10 **Answer: (a) by running its left and right wheels at different speeds**

Speed difference.

Q11 **Answer: (b) steady-state error that P alone leaves**

Integral accumulates past error.

Q12 **Answer: (d) damps overshoot by reacting to the rate of change of error**

Derivative damping.

Q13 **Answer: (c) long-running goals with feedback and the option to cancel**

E.g. navigate_to_pose.

Q14 **Answer: (c) ros2 topic echo /topic**

echo prints each message.

Q15 **Answer: (d) wheel slip and small dimension errors accumulate**

Correct with IMU and SLAM.

Q16 **Answer: (b) Nav2**

Nav2.

Q17 **Answer: (a) the robot stops if no command arrives within a short time**

Safety on signal loss.

Q18 **Answer: (b) 164 rpm**

0.6 ÷ (π × 0.07) = 2.73 rev/s × 60 = **164 rpm**.

Q19 **Answer: (a) about 0.142 mm**

π × 65 ÷ 1440 = **0.1418 mm**.

Q20 **Answer: Solution program**

d = average of the wheels; $\Delta\theta = (d_R - d_L) \div L = 0.10 \div 0.15 = 0.667$ rad = 38.2°.

```
import math
dl, dr, L = 0.50, 0.60, 0.15
d = (dl + dr) / 2
dth = (dr - dl) / L
print(f"centre moved {d:.2f} m, heading change {math.degrees(dth):.1f} degrees (turned left)")
```

OUTPUT

```
centre moved 0.55 m, heading change 38.2 degrees (turned left)
```

Q21 **Answer: Solution program**

The modulo trick keeps the error between -180° and +180°, so the robot always turns the short way. The command saturates at +1 for large errors.

```
def turn_cmd(target, heading, kp=0.02):
    err = (target - heading + 180) % 360 - 180          # shortest way round
    return max(-1.0, min(1.0, kp * err)), err
for h in [0, 30, 60, 80, 88, 95]:
    u, e = turn_cmd(90, h)
    print(f"heading {h:3d} -> error {e:+4.0f}, turn {u:+.2f}")
```

OUTPUT

```
heading   0 -> error  +90, turn +1.00
heading  30 -> error  +60, turn +1.00
heading  60 -> error  +30, turn +0.60
heading  80 -> error  +10, turn +0.20
heading  88 -> error   +2, turn +0.04
heading  95 -> error   -5, turn -0.10
```

Q22 **Answer: Solution program**

A simple reactive behaviour: check distance every loop and choose an action.

```
from gpiozero import Robot, DistanceSensor
from time import sleep
bot = Robot(left=(7, 8), right=(9, 10))
sensor = DistanceSensor(echo=24, trigger=23)
for _ in range(6):
    d = sensor.distance
    if d < 0.25:
        bot.stop(); bot.right(0.6); sleep(0.5); print(f"{d:.2f} m: turning")
    else:
        bot.forward(0.5); sleep(0.4); print(f"{d:.2f} m: forward")
bot.stop()
```

OUTPUT

```
0.80 m: forward
0.80 m: forward
0.40 m: forward
0.20 m: turning
```

```
0.20 m: turning
0.90 m: forward
```

Q23 **Answer: Solution program**

A timer publishes; a subscription callback checks each message. The simulated CPU warmed past 60 °C.

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import Float32
from gpiozero import CPUTemperature

class TempPub(Node):
    def __init__(self):
        super().__init__("temp_pub")
        self.pub = self.create_publisher(Float32, "/pi/temperature", 10)
        self.cpu = CPUTemperature()
        self.create_timer(1.0, lambda: self.pub.publish(Float32(data=self.cpu.temperature)))

class TempWatch(Node):
    def __init__(self):
        super().__init__("temp_watch")
        self.create_subscription(Float32, "/pi/temperature", self.check, 10)
    def check(self, msg):
        if msg.data > 60:
            self.get_logger().warn(f"CPU hot: {msg.data:.1f} C")
        else:
            self.get_logger().info(f"CPU {msg.data:.1f} C")

rclpy.init()
pub, watch = TempPub(), TempWatch()
rclpy.spin(pub)
```

OUTPUT

```
[INFO] [1773565201.001] [temp_watch]: CPU 55.0 C
[INFO] [1773565202.001] [temp_watch]: CPU 58.5 C
[WARN] [1773565203.001] [temp_watch]: CPU hot: 61.2 C
[WARN] [1773565204.001] [temp_watch]: CPU hot: 63.0 C
```

Q24 **Answer: (c) Pico runs motor PID and encoders; the Pi runs ROS 2 nodes (driver, LiDAR, SLAM, Nav2); a laptop runs rviz2**

Each layer does what it is best at.

Q25 **Answer: (d) the motor current makes the shared supply voltage dip below what the Pi needs**

Separate/regulated supplies, capacitors, a stronger battery.

Sense → think → act, with a solid power system. Pi for perception and planning, microcontroller for real-time motor control (with a dead-man stop). Differential drive: mix throttle/turn; encoders → odometry ($d = (dL+dR)/2$, $\Delta\theta = (dR-dL)/L$); PID for speed and heading. ROS 2: nodes, topics (/cmd_vel, /scan, /odom), services, actions, parameters, launch files, rviz2/rqt; LiDAR + slam_toolbox for maps, Nav2 for navigation, Gazebo for simulation.

MODULE 26 · ADVANCED

Performance, power and product design

Minimum duration **2 days** | Level **Advanced**

Turning a prototype into a reliable product needs attention to speed, heat, power, storage and production hardware. This module profiles CPU, memory and I/O, uses threads, processes, asyncio and compiled code for speed, manages throttling with heatsinks and fans, builds power budgets, uses battery/UPS HATs and solar power with safe shutdown, reduces consumption, prevents SD card corruption, and moves from prototype to product with Compute Modules, custom HATs (KiCad, HAT+ specification and EEPROM), enclosures, mass deployment, remote updates and certification, followed by 25 solved questions and programs.

IN THIS MODULE

26.1 Performance
26.2 Thermal management
26.3 Power
26.4 Storage reliability
26.5 From prototype to product
26.6 Practice questions
26.7 Solutions

26.1 Performance

26.1.1 Profiling CPU, memory and I/O

Never guess where a program is slow: **measure**. `htop` shows CPU per core and memory per process; `iotop` and `vmstat 1` show disk activity; `time python3 app.py` gives total run time. Inside Python, **cProfile** shows which functions take the time:

PROFILE_DEMO.PY

```
import cProfile, pstats, io, math

def slow_filter(data):
    out = []
    for i in range(len(data)):                      # moving average, recomputed every time
        window = data[max(0, i - 500):i + 1]
        out.append(sum(window) / len(window))
    return out

def fast_filter(data):
    out, total = [], 0.0
    for i, v in enumerate(data):                     # running sum: O(1) per sample
        total += v
        if i > 500:
            total -= data[i - 501]
        out.append(total / min(i + 1, 501))
    return out

data = [math.sin(i / 10) for i in range(20000)]
pr = cProfile.Profile(); pr.enable()
a = slow_filter(data); b = fast_filter(data)
pr.disable()
s = io.StringIO(); pstats.Stats(pr, stream=s).sort_stats("cumulative").print_stats("filter")
for l in s.getvalue().splitlines():
    if "_filter)" in l:
        cumtime, func = l.split()[3], l.split("(")[-1].rstrip(")")
        print(f"{func:12} cumulative {float(cumtime):.3f} s")
print("same results:", max(abs(x - y) for x, y in zip(a, b)) < 1e-9)
```

OUTPUT

```
slow_filter  cumulative 0.080 s
fast_filter  cumulative 0.011 s
same results: True
```

The times come from the machine that built this book; on a Pi they are larger, but the **ratio** is what matters: the running-sum algorithm does a constant amount of work per sample instead of adding up 500 values, a bigger gain than any hardware upgrade. For memory, `tracemalloc` finds which lines allocate the most.

26.1.2 Multithreading and multiprocessing in Python

Python's **GIL** (global interpreter lock) lets only one thread run Python code at a time. **Threads** therefore help when the program *waits* (network, sensors, disk), but not for heavy computation. **Processes** (multiprocessing, concurrent.futures.ProcessPoolExecutor) each have their own interpreter and use all four cores for CPU-bound work, at the cost of start-up time and copying data between them.

THREADS_VS_PROCESSES.PY

```
import time
from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor

def cpu_task(n):                          # pure computation
    return sum(i * i for i in range(n))
def io_task(x):                           # mostly waiting (like a network request)
    time.sleep(0.2); return x

if __name__ == "__main__":
    for label, Pool, task, args in [("CPU work, 4 threads  ", ThreadPoolExecutor, cpu_task,
[3_000_000] * 4),
                                    ("CPU work, 4 processes", ProcessPoolExecutor, cpu_task,
[3_000_000] * 4),
                                    ("I/O wait, sequential ", None, io_task, range(8)),
                                    ("I/O wait, 8 threads  ", ThreadPoolExecutor, io_task,
range(8))]:
        t0 = time.perf_counter()
        if Pool is None:
            [task(a) for a in args]
        else:
            with Pool(max_workers=len(args)) as p:
                list(p.map(task, args))
        print(f"{label} {time.perf_counter() - t0:5.2f} s")
```

OUTPUT

```
CPU work, 4 threads    0.53 s
CPU work, 4 processes  0.30 s
I/O wait, sequential   1.60 s
I/O wait, 8 threads    0.20 s
```

Threads did not speed up the computation (the GIL), while processes ran it in parallel. The machine that built this book has 2 cores, so processes were up to about twice as fast; on a Pi 4 or 5 with 4 cores the gain approaches 4×. For waiting tasks, threads cut 8 × 0.2 s to about 0.2 s.

26.1.3 asyncio for I/O-heavy programs

asyncio runs many waiting tasks in one thread: each `await` hands control back to the event loop while that task waits. It suits gateways that poll many sensors, MQTT clients and web servers (FastAPI), with less overhead than one thread per task.

ASYNC_SENSORS.PY

```
import asyncio, time

async def read_sensor(name, delay):
    await asyncio.sleep(delay)                # stands in for a network or bus wait
    return name, round(20 + delay * 3, 1)

async def main():
    t0 = time.perf_counter()
    results = await asyncio.gather(read_sensor("kitchen", 0.3), read_sensor("garage", 0.5),
                                   read_sensor("garden", 0.4), read_sensor("attic", 0.2))
    for name, value in results:
        print(f"{name:8} {value} C")
    print(f"4 sensors in {time.perf_counter() - t0:.2f} s (sequential would take 1.4 s)")

asyncio.run(main())
```

OUTPUT

```
kitchen  20.9 C
garage   21.5 C
garden   21.2 C
attic    20.6 C
4 sensors in 0.50 s (sequential would take 1.4 s)
```

26.1.4 Using C extensions or compiled code for speed

When a hot loop stays slow after algorithm fixes, use **NumPy** (vectorised C), **Numba** (JIT-compiles Python functions), **Cython**, or write the loop in C and call it with **ctypes** or **cffi**. Here a C function is compiled with gcc and called from Python:

C_SPEED.PY

```
import ctypes, subprocess, time, numpy as np

open("sq.c", "w").write("""
double sum_sq(const double *x, long n) {
    double s = 0; for (long i = 0; i < n; i++) s += x[i] * x[i]; return s;
}""")
subprocess.run(["gcc", "-O2", "-shared", "-fPIC", "sq.c", "-o", "libsq.so"], check=True)
lib = ctypes.CDLL("./libsq.so")
lib.sum_sq.restype = ctypes.c_double
lib.sum_sq.argtypes = [ctypes.POINTER(ctypes.c_double), ctypes.c_long]

x = np.random.default_rng(0).random(2_000_000)
t = time.perf_counter(); py = sum(v * v for v in x.tolist()); t_py = time.perf_counter() - t
t = time.perf_counter(); c = lib.sum_sq(x.ctypes.data_as(ctypes.POINTER(ctypes.c_double)), x.size);
t_c = time.perf_counter() - t
t = time.perf_counter(); n = float(np.dot(x, x)); t_np = time.perf_counter() - t
print(f"pure Python {t_py * 1000:7.1f} ms")
print(f"C (ctypes)  {t_c * 1000:7.1f} ms   {t_py / t_c:5.0f}x faster")
print(f"NumPy dot   {t_np * 1000:7.1f} ms   {t_py / t_np:5.0f}x faster")
print("results agree:", abs(py - c) < 1e-6 * py and abs(c - n) < 1e-6 * c)
```

OUTPUT

```
pure Python   112.0 ms
C (ctypes)      6.1 ms      18x faster
NumPy dot       1.2 ms      93x faster
results agree: True
```

26.2 Thermal management

26.2.1 Throttling and temperature monitoring

From module 2: soft throttling starts at about 80 °C and hard throttling at 85 °C, and `vcgencmd get_throttled` records whether it happened. Log temperature and frequency under your real workload for at least 30 minutes inside the final enclosure, which can run 10–20 °C hotter than open air.

SOAK_TEST.PY

```
from gpiozero import CPUTemperature
import time
cpu = CPUTemperature()
peak, samples = 0, []
for minute in range(0, 35, 5):
    t = cpu.temperature
    samples.append(t); peak = max(peak, t)
    state = "THROTTLING" if t >= 80 else "warning" if t >= 75 else "ok"
    print(f"{minute:2d} min  {t:4.1f} C  {state}")
    time.sleep(300)
print(f"peak {peak} C, rise {samples[-1] - samples[0]:.1f} C over the test")
```

OUTPUT

```
 0 min  46.0 C  ok
 5 min  61.0 C  ok
10 min  70.0 C  ok
15 min  76.0 C  warning
20 min  79.0 C  warning
25 min  81.0 C  THROTTLING
30 min  81.5 C  THROTTLING
peak 81.5 C, rise 35.5 C over the test
```

26.2.2 Heatsinks, fans and cases

Cooling	Typical Pi 5 full-load result	Notes
none, open air	reaches 85 °C and throttles	fine for light loads only
passive heatsink case	about 65–75 °C	silent; the case must have metal contact and ventilation
official Active Cooler	about 55–65 °C	fan curve set by firmware via the fan connector
case fan + vents	depends on airflow path	pull cool air across the SoC, exhaust warm air out

Fan control on the Pi 5 can be tuned in config.txt (`dtparam=fan_temp0=50000` etc.). In dusty places use filters or a sealed case with a large external heatsink.

26.3 Power

26.3.1 Power budgets for Pi and peripherals

POWER_BUDGET.PY

```
loads = {                       # name: (volts, amps average, amps peak)
    "Raspberry Pi 5 (busy)":     (5.0, 1.20, 2.0),
    "NVMe SSD":                  (3.3, 0.60, 1.5),
    "Camera Module 3":           (3.3, 0.25, 0.3),
    "Servo x2 (moving)":         (5.0, 0.50, 1.4),
    "NeoPixel ring 12 (30 %)":   (5.0, 0.22, 0.72),
}
avg = sum(v * a for v, a, p in loads.values())
peak = sum(v * p for v, a, p in loads.values())
for name, (v, a, p) in loads.items():
    print(f"{name:26} {v * a:5.2f} W avg   {v * p:5.2f} W peak")
print(f"TOTAL                      {avg:5.2f} W avg   {peak:5.2f} W peak")
print(f"5 V supply for the peak plus 20 % margin: {peak * 1.2 / 5:.1f} A "
      "(3.3 V loads are fed from 5 V through regulators, which adds a little more)")
```

OUTPUT

```
Raspberry Pi 5 (busy)       6.00 W avg   10.00 W peak
NVMe SSD                    1.98 W avg    4.95 W peak
Camera Module 3             0.82 W avg    0.99 W peak
Servo x2 (moving)           2.50 W avg    7.00 W peak
NeoPixel ring 12 (30 %)     1.10 W avg    3.60 W peak
TOTAL                      12.40 W avg   26.54 W peak
5 V supply for the peak plus 20 % margin: 6.4 A (3.3 V loads are fed from 5 V through regulators,
which adds a little more)
```

26.3.2 Battery and UPS HATs

A **UPS HAT** keeps the Pi running from Li-ion cells when mains power fails and charges them when it returns. Good boards report battery voltage and power status over I2C or a GPIO pin, so software can shut down cleanly before the battery is empty. The Pi 5's RTC battery connector keeps time only; it does not power the Pi.

26.3.3 Solar-powered projects

WORKED EXAMPLE

Sizing a solar Pi

A Pi Zero 2 W node uses 1.2 W average = 28.8 Wh per day. With 4 peak sun hours (typical for much of India) and 70 % system efficiency, the panel needs 28.8 ÷ (4 × 0.7) = **10.3 W** → use 20 W for cloudy days. For 2 days without sun the battery must store 57.6 Wh; at 80 % usable capacity that is 72 Wh ≈ a **12 V 6 Ah** battery, with an MPPT charge controller and a 5 V buck converter.

26.3.4 Safe shutdown on power loss

SAFE_SHUTDOWN.PY

```
from gpiozero import Button
from signal import pause
import subprocess, time

power_ok = Button(6, pull_up=True)          # UPS HAT pulls GPIO6 low when mains fails (example
wiring)
GRACE = 20                                   # seconds to wait for power to return

def mains_lost():
    print(time.strftime("%H:%M:%S"), "mains power lost, running on battery")
    for s in range(GRACE):
        time.sleep(1)
        if not power_ok.is_pressed:
            print(time.strftime("%H:%M:%S"), f"power restored after {s + 1} s: cancelled")
            return
    print(time.strftime("%H:%M:%S"), "still on battery: saving data and shutting down")
    # subprocess.run(["sudo", "shutdown", "-h", "now"])     # real action on the Pi

power_ok.when_pressed = mains_lost
pause()
```

OUTPUT

```
09:00:05 mains power lost, running on battery
09:00:12 power restored after 7 s: cancelled
09:00:40 mains power lost, running on battery
09:01:00 still on battery: saving data and shutting down
```

The first outage lasted 7 s and was ridden out; the second lasted longer than the grace period, so the Pi would shut down cleanly. A one-line alternative for simple cases: `dtoverlay=gpio-shutdown,gpio_pin=6,active_low=1` in config.txt makes the kernel shut down when the pin goes low.

26.3.5 Reducing power consumption

Measure	Saving (approximate)
Use a Pi Zero 2 W or Pico where possible	Zero 2 W ≈ 0.6–1.5 W vs Pi 5 ≈ 3–8 W
Disable HDMI / unused USB devices; headless Lite OS	0.2–0.5 W
Turn off Wi-Fi/Bluetooth if wired (dtoverlay=disable-wifi, disable-bt)	0.2–0.5 W
Disable LEDs (dtparam=act_led_trigger=none)	small
Lower CPU frequency or use the powersave governor	up to 1 W at load
Pi 5: POWER_OFF_ON_HALT=1 in the EEPROM	halted draw from ≈ 1.4 W to ≈ 0.01 W
Sleep with a timer chip that cuts power between readings	the biggest saving for battery nodes

26.4 Storage reliability

26.4.1 SD card wear and corruption

Corruption happens when power is cut while the card is writing (metadata half-written), and flash cells wear out after many writes. Prevent both: proper shutdown (26.3.4), fewer writes (module 16.6), high-endurance cards, the ext4 journal (on by default) and `fsck.repair=yes` in cmdline.txt.

26.4.2 SSD boot and read-only systems

For products, boot from an SSD or the Compute Module's **eMMC** (soldered flash, far more robust than a card), and run the root file system **read-only** with the overlay file system (module 19.6.3), keeping writable data on a separate partition. Then a power cut can at worst lose the last few seconds of data, never the operating system.

26.5 From prototype to product

26.5.1 Compute Modules and carrier boards

A **Compute Module** (CM4, CM5) is the Pi's processor, RAM, optional eMMC and wireless on a small board with high-density connectors. You design a **carrier board** with only the connectors your product needs (power input, Ethernet, USB, camera, custom I/O), in the shape of your enclosure. Raspberry Pi guarantees long availability for industrial customers, and the CM IO boards are reference designs to start from.

26.5.2 Designing a custom HAT (KiCad, HAT+ specification and EEPROM)

Design the board in **KiCad** (free): schematic, PCB layout, then Gerber files for a PCB maker. The **HAT+** specification defines the board size, mounting holes, the 40-pin connector position, power rules and an **ID EEPROM** on the ID_SD/ID_SC pins (GPIO0/1) that tells the Pi the vendor, product and which device-tree overlay to load, so the HAT works automatically.

HAT+ EEPROM DESCRIPTION (COMPILED WITH EEPMAKE, WRITTEN WITH EEPFLASH.SH)

```
# eeprom_settings.txt (HAT+ format, tools in raspberrypi/utils: eeptools)
vendor "Learn Being Forward"
product "Garden Sensor HAT"
product_id 0x0001
product_ver 0x0001
dt_blob "lbf-garden"           # loads overlay lbf-garden.dtbo automatically
current_supply 0
```

26.5.3 Enclosures and 3D printing

Design enclosures in Fusion, Onshape or FreeCAD; print prototypes in PLA (indoors) or PETG/ASA (heat and outdoor use). Plan for ventilation or a heatsink path, cable glands and strain relief, mounting bosses for the board, access to connectors and the status LED, and an IP rating for dust and water outdoors (IP65 = dust-tight and protected against water jets). Leave 0.2–0.4 mm clearance for printed parts that fit together.

26.5.4 Mass-deploying images and remote updates

Build one **golden image** with everything installed (rpi-image-gen or pi-gen produce reproducible custom images; Imager can write customisation). Give each device a unique hostname and credentials on first boot. Update fleets with A/B partition systems that roll back on failure (Mender, RAUC, balena), containers pulled from a registry, or a careful apt + configuration-management approach (Ansible). Raspberry Pi Connect and VPNs give remote access for support.

UPDATE_MANIFEST.PY

```
import hashlib, json

def make_manifest(version, files):           # files: {name: bytes}
    return {"version": version,
            "files": {n: hashlib.sha256(b).hexdigest() for n, b in files.items()}}

def verify(manifest, files):
    bad = [n for n, h in manifest["files"].items() if hashlib.sha256(files.get(n, b"")).hexdigest()
!= h]
    return "OK: install update" if not bad else f"REJECT: corrupted {bad}"

release = {"app.py": b"print('v2')", "config.json": b'{"interval": 60}'}
m = make_manifest("2.0.0", release)
print(json.dumps(m, indent=1)[:120] + " ...")
print(verify(m, release))
damaged = dict(release, **{"app.py": b"print('v2'"})          # one byte missing in transfer
print(verify(m, damaged))
```

OUTPUT

```
{
 "version": "2.0.0",
 "files": {
  "app.py": "254deec6f0b299f05865fcf49ef207625e7dbf7e0b5ef8249eae173a4a4ba1c1",
  "co ...
OK: install update
REJECT: corrupted ['app.py']
```

Real update systems also **sign** the manifest with a private key so devices install only genuine updates, and keep the previous version to roll back if the new one fails its health check.

26.5.5 Compliance and certification overview

Requirement	What it covers
Radio approval	products with Wi-Fi/Bluetooth need radio certification per country (in India WPC/ETA approval); using Raspberry Pi's pre-certified modules simplifies this
EMC	the product must not emit interference and must tolerate it (CE in Europe, FCC in the USA)
Electrical safety	especially for mains-powered products (BIS registration for many electronics in India, IEC 62368-1)
Environmental	RoHS (no hazardous substances), WEEE (recycling)

Requirement	What it covers
Data protection	products handling personal data must follow privacy law (DPDP Act 2023 in India, GDPR in the EU)

Raspberry Pi publishes compliance documents for its boards and modules, which helps, but the finished product as a whole must be tested. Budget time and money for it early.

PRACTICE CHECKPOINT

Take one earlier project, measure its power and temperature, add safe shutdown and a battery or UPS, and design a 3D-printable enclosure or simple custom HAT for it.

26.6 Practice questions

26.6.1 Basic level

Q1 Which Python tool shows which functions take the most time?

(a) pip **(b)** venv **(c)** cProfile **(d)** pytest-cov

Q2 Which helps a CPU-bound Python task use all four cores?

(a) multiprocessing (processes) **(b)** threads
(c) asyncio **(d)** time.sleep

Q3 Which suits a program waiting on many network requests?

(a) only multiprocessing **(b)** a bigger heatsink
(c) overclocking **(d)** asyncio or threads

Q4 At about what temperature does a Pi 5 begin soft throttling?

(a) 50 °C **(b)** 80 °C **(c)** 65 °C **(d)** 100 °C

Q5 What does a UPS HAT do?

(a) speeds up the CPU
(b) keeps the Pi running on battery during power cuts and signals for safe shutdown
(c) adds Wi-Fi
(d) cools the board

Q6 What is a Compute Module?

(a) a calculator app
(b) a type of SD card
(c) a cooling fan
(d) the Pi's core electronics on a small board for custom carrier boards

Q7 Which free software designs PCBs such as custom HATs?

(a) KiCad **(b)** Thonny **(c)** Grafana **(d)** Kodi

Q8 What does a HAT's ID EEPROM tell the Pi?

(a) the Wi-Fi password
(b) the CPU speed
(c) who made it and which device-tree overlay to load
(d) the user's files

Q9 What does the ext4 option fsck.repair=yes in cmdline.txt do?

(a) formats the card
(b) repairs file-system errors automatically at boot
(c) encrypts the disk
(d) mounts it read-only

26.6.2 Intermediate level

Q10 What is the GIL?

(a) a GPIO library
(b) a power standard
(c) a file system
(d) Python's global interpreter lock that lets one thread run Python code at a time

Q11 Why test temperature inside the final enclosure?

(a) enclosures cool the Pi
(b) enclosures can run 10–20 °C hotter than open air
(c) vcgencmd only works in cases
(d) it is required by Python

Q12 Why is eMMC better than an SD card in a product?

(a) it is soldered and more robust and durable
(b) it is removable
(c) it is always bigger
(d) it needs no power

Q13 What does POWER_OFF_ON_HALT=1 do on a Pi 5?

(a) overclocks the CPU
(b) disables Wi-Fi permanently
(c) cuts the halted power draw to almost zero
(d) enables the fan

Q14 What is a golden image?

(a) a fully prepared OS image copied to every device
(b) a gold-plated connector
(c) a backup of photos
(d) a Docker registry

Q15 Why sign software updates?

(a) to compress them
(b) so devices install only genuine updates from you
(c) to make them faster
(d) to bypass checksums

Q16 Which certification concerns radio emissions for Wi-Fi products in India?

(a) RoHS
(b) IP65
(c) HAT+
(d) WPC/ETA approval

Q17 What does IP65 mean for an enclosure?

(a) waterproof to 1 m
(b) no protection
(c) dust-tight and protected against water jets
(d) heat-proof to 65 °C

Q18 A Pi uses 5 W on average. What is its daily energy?

(a) 5 Wh (b) 50 Wh (c) 120 Wh (d) 1.2 kWh

26.6.3 Advanced level

Q19 Pi 4 W, SSD 2 W, camera 1 W, with 20 % margin. What 5 V supply current is needed?

(a) 1.4 A (b) 7 A (c) 0.84 A (d) 1.68 A

Q20 Size a solar system for a 2 W device with 5 sun hours, 70 % efficiency and 3 days of autonomy on a 12 V battery with 80 % usable capacity.

Q21 Time a simple sum of 1–5 000 000 three ways: a Python loop, the built-in sum(range()), and the formula n(n+1)/2.

Q22 Write a safe-shutdown decision: given battery voltage readings every 10 s from a 1-cell Li-ion UPS, shut down when two readings in a row are below 3.4 V.

Q23 Build a SHA-256 checksum for an update file and verify a downloaded copy, printing whether to install.

Q24 A product must survive sudden power loss without breaking the OS. Which combination works?

(a) a bigger SD card
(b) eMMC or SSD + read-only overlay root + UPS signal for clean shutdown
(c) overclocking
(d) writing logs every second to the root file system

Q25 Your Python image pipeline uses 100 % of one core and 0 % of the others. What should you try first after profiling?

(a) vectorise with NumPy/OpenCV or split frames across processes
(b) add more threads doing the same Python loop
(c) buy a bigger SD card
(d) reduce the fan speed

26.7 Solutions

Q1 **Answer: (c) cProfile**

cProfile.

Q2 **Answer: (a) multiprocessing (processes)**

Processes avoid the GIL.

Q3 **Answer: (d) asyncio or threads**

I/O-bound work.

Q4 **Answer: (b) 80 °C**

Soft at ~80 °C, hard at 85 °C.

Q5 **Answer: (b) keeps the Pi running on battery during power cuts and signals for safe shutdown**

Uninterruptible power supply.

Q6 **Answer: (d) the Pi's core electronics on a small board for custom carrier boards**

For products.

Q7 **Answer: (a) KiCad**

KiCad.

Q8 **Answer: (c) who made it and which device-tree overlay to load**

Automatic configuration.

Q9 **Answer: (b) repairs file-system errors automatically at boot**

Recovery after unclean shutdowns.

Q10 **Answer: (d) Python's global interpreter lock that lets one thread run Python code at a time**

Limits threads for CPU work.

Q11 **Answer: (b) enclosures can run 10–20 °C hotter than open air**

Real conditions.

Q12 **Answer: (a) it is soldered and more robust and durable**

Compute Modules offer eMMC.

Q13 **Answer: (c) cuts the halted power draw to almost zero**

EEPROM setting.

Q14 **Answer: (a) a fully prepared OS image copied to every device**

Mass deployment.

Q15 **Answer: (b) so devices install only genuine updates from you**

Authenticity.

Q16 **Answer: (d) WPC/ETA approval**

Wireless Planning & Coordination.

Q17 **Answer: (c) dust-tight and protected against water jets**

6 = dust, 5 = jets.

Q18 **Answer: (c) 120 Wh**

5 × 24 = **120 Wh**.

Q19 **Answer: (d) 1.68 A**

7 W × 1.2 = 8.4 W ÷ 5 V = **1.68 A**.

Q20 **Answer: Solution program**

48 Wh/day; panel 48 ÷ 3.5 = 13.7 W (choose 20 W+); battery 144 Wh ÷ 0.8 = 180 Wh ÷ 12 V = 15 Ah.

```
watts, sun_h, eff, days, volts, usable = 2.0, 5, 0.70, 3, 12, 0.8
daily_wh = watts * 24
panel_w = daily_wh / (sun_h * eff)
battery_ah = daily_wh * days / usable / volts
print(f"daily energy {daily_wh} Wh, panel at least {panel_w:.1f} W, battery {battery_ah:.1f} Ah at
{volts} V")
```

OUTPUT

```
daily energy 48.0 Wh, panel at least 13.7 W, battery 15.0 Ah at 12 V
```

Q21 **Answer: Solution program**

Built-ins run in C, and a better algorithm (the formula) needs no loop at all. Times are from the build machine.

```
import time
n = 5_000_000
t = time.perf_counter(); s1 = 0
for i in range(1, n + 1): s1 += i
t1 = time.perf_counter() - t
t = time.perf_counter(); s2 = sum(range(1, n + 1)); t2 = time.perf_counter() - t
t = time.perf_counter(); s3 = n * (n + 1) // 2; t3 = time.perf_counter() - t
print(f"loop {t1:.3f} s | sum() {t2:.3f} s | formula {t3 * 1e6:.1f} us | equal: {s1 == s2 == s3}")
```

OUTPUT

```
loop 0.290 s | sum() 0.069 s | formula 3.1 us | equal: True
```

Q22 **Answer: Solution program**

Requiring two consecutive low readings avoids shutting down because of a single dip under load.

```
readings = [3.92, 3.80, 3.61, 3.45, 3.38, 3.41, 3.37, 3.33]
low = 0
for i, v in enumerate(readings):
    low = low + 1 if v < 3.4 else 0
    if low == 2:
        print(f"t={i * 10} s: {v} V, second low reading in a row -> shutdown"); break
    print(f"t={i * 10} s: {v} V")
```

OUTPUT

```
t=0 s: 3.92 V
t=10 s: 3.8 V
t=20 s: 3.61 V
```

```
t=30 s: 3.45 V
t=40 s: 3.38 V
t=50 s: 3.41 V
t=60 s: 3.37 V
t=70 s: 3.33 V, second low reading in a row -> shutdown
```

Q23 **Answer: Solution program**

Any changed byte produces a completely different hash, so corrupted or tampered files are rejected.

```
import hashlib
original = b"firmware v3.1" * 1000
published = hashlib.sha256(original).hexdigest()
for name, data in (("good download", original), ("truncated download", original[:-5])):
    ok = hashlib.sha256(data).hexdigest() == published
    print(f"{name:19} {'install' if ok else 'reject'}")
print("checksum starts", published[:16])
```

OUTPUT

```
good download       install
truncated download  reject
checksum starts 2e11c35d50ec6b26
```

Q24 **Answer: (b) eMMC or SSD + read-only overlay root + UPS signal for clean shutdown**

Robust storage + protected root + clean shutdown.

Q25 **Answer: (a) vectorise with NumPy/OpenCV or split frames across processes**

Compiled code or processes use the CPU better.

Measure before optimising (cProfile, htop): better algorithms first, then NumPy/compiled code; processes for CPU-bound work, threads/asyncio for waiting. Keep under 80 °C in the real enclosure with heatsinks or fans. Budget power at peak with margin; UPS + safe shutdown; size solar by daily Wh and autonomy days. Protect storage (eMMC/SSD, read-only root). Products: Compute Module + carrier, HAT+ EEPROM, designed enclosure, golden images, signed updates, certification.

MODULE 27 · ADVANCED

Capstone project and career path

Minimum duration **4 days** | Level **Advanced**

The final module turns everything learned into a portfolio-ready project and a clear career plan. It presents six capstone ideas (smart agriculture, an AI security camera, an autonomous ROS 2 robot, a smart home hub, an industrial monitoring gateway and a self-hosted home server), walks through the project process (requirements and system design, bill of materials and budget, building, testing and iterating, documentation and demo video, presenting), builds a portfolio on GitHub, blogs and competitions, maps careers in embedded/IoT, robotics, edge AI, Linux/DevOps, prototyping and education, and lists further learning, followed by 25 solved questions and programs.

IN THIS MODULE

27.1 Capstone project ideas
27.2 Project process
27.3 Portfolio
27.4 Careers
27.5 Further learning
27.6 Practice questions
27.7 Solutions

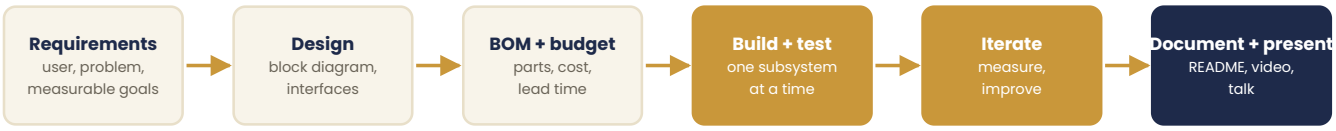
27.1 Capstone project ideas

A strong capstone combines **hardware**, **software**, **networking** and either **AI or automation**, solves a real problem for a real user, and is documented well enough that someone else could rebuild it. Choose one idea and scope it so a first working version is possible in about two weeks, then improve it.

Project	Hardware	Software and skills (modules)
27.1.1 Smart agriculture system	soil moisture + DHT22/BME280 nodes (ESP32/Pico W), camera, pump via relay/MOSFET, solar power	MQTT, SQLite/InfluxDB, Grafana or Flask dashboard, irrigation rules, plant-health images (8, 12, 16, 17, 18, 26)
27.1.2 AI-powered security camera with alerts	Camera Module 3 (NoIR + IR LEDs) or AI Camera, PIR, Pi 5 + optional AI HAT+	person detection, motion clips with CircularOutput, Telegram alerts, VPN access, privacy controls (13, 21, 22, 23)
27.1.3 Autonomous robot with vision and ROS 2	differential-drive chassis, encoders, Pico motor controller, LiDAR or ToF, camera	ROS 2 nodes, odometry, PID, obstacle avoidance or Nav2, web teleop (12, 24, 25)
27.1.4 Smart home hub with custom devices	Pi 5 + SSD, Zigbee coordinator, ESPHome nodes, relay boards	Home Assistant, Zigbee2MQTT, MQTT discovery for your own devices, automations, voice (17, 18, 19)
27.1.5 Industrial machine monitoring gateway	vibration (MPU6050) and current sensors, Modbus/RS-485 adapter, UPS HAT	anomaly detection, time-series DB, alerts, read-only root, remote updates (10, 16, 23, 26)

Project	Hardware	Software and skills (modules)
27.1.6 Self-hosted home server with backups and VPN	Pi 5, NVMe/USB SSDs, UPS	Docker Compose: Pi-hole, Nextcloud, Grafana, Uptime Kuma; WireGuard; 3-2-1 backups; hardening (19, 20, 21)

27.2 Project process



PROJECT LIFE CYCLE

27.2.1 Requirements and system design

Write **functional requirements** (what it does) and **non-functional requirements** (how well: response time, battery life, accuracy, cost, reliability). Make each one testable and give it an ID so tests can refer to it.

ID	Requirement (smart agriculture example)	Test
F1	measure soil moisture in 3 beds every 10 minutes	dashboard shows 3 series with 10-minute spacing for 24 h
F2	water a bed when moisture < 30 % and no rain is forecast	dry sensor in a test pot triggers the pump within 10 min
F3	send a Telegram alert if the tank is empty	float switch open → alert within 1 min
N1	run 3 days without sun	battery test with panel disconnected
N2	total cost under ₹6000	BOM total
N3	recover automatically after a power cut	pull the power, restore, check service and data

Then draw a **system block diagram**: every device, every connection and protocol (I2C, MQTT, HTTP), where data is stored and who sees it. Decide interfaces early: topic names, JSON message formats, API endpoints.

27.2.2 Bill of materials and budget

BOM.PY

```
BOM = [  # part, qty, unit price (Rs), supplier lead time (days)
    ("Raspberry Pi 4 (2 GB)", 1, 3800, 3),
    ("32 GB A2 microSD", 1, 450, 2),
    ("ESP32 dev board", 3, 380, 4),
    ("Capacitive soil sensor v1.2", 3, 120, 7),
    ("BME280 module", 1, 290, 4),
    ("12 V mini pump + MOSFET module", 1, 420, 5),
    ("20 W solar panel + MPPT controller", 1, 2100, 10),
    ("12 V 7 Ah battery", 1, 1250, 3),
]
total = sum(q * p for _, q, p, _ in BOM)
for name, q, p, d in BOM:
    print(f"{name:34} {q} x {p:5} = {q * p:6}")
contingency = round(total * 0.15)
print(f"{'subtotal':34} {total:15}")
print(f"{'15 % contingency (spares, wires)':34} {contingency:15}")
print(f"{'TOTAL':34} {total + contingency:15}")
print("longest lead time:", max(d for *_, d in BOM), "days -> order that part first")
```

OUTPUT

```
Raspberry Pi 4 (2 GB)              1 x  3800 =   3800
32 GB A2 microSD                   1 x   450 =    450
ESP32 dev board                    3 x   380 =   1140
Capacitive soil sensor v1.2        3 x   120 =    360
BME280 module                      1 x   290 =    290
12 V mini pump + MOSFET module     1 x   420 =    420
20 W solar panel + MPPT controller 1 x  2100 =   2100
12 V 7 Ah battery                  1 x  1250 =   1250
subtotal                                        9810
15 % contingency (spares, wires)                1472
TOTAL                                          11282
longest lead time: 10 days -> order that part first
```

The total exceeds requirement N2, so a decision is needed: drop solar for a mains adapter, or use a Pi Zero 2 W as the hub. Doing this sum **before** buying avoids surprises. Add 10–20 % for spares, wires, connectors and mistakes, and order long-lead parts first.

27.2.3 Building, testing and iterating

Practice	Why
Build and test one subsystem at a time	when something fails you know where
Version control from day one (Git)	undo mistakes, see what changed
Automated tests for logic (pytest)	rules such as the irrigation decision are checked on every change
Log everything with timestamps	diagnose problems that happen at 3 a.m.
Measure against the requirements	progress is objective, not a feeling
Keep an engineering diary	decisions, problems and fixes become your report

TEST_RULES.PY

```
def should_water(moisture, rain_mm, tank_ok, limit=30):
    return tank_ok and rain_mm < 5 and moisture < limit

TESTS = [  # (moisture, rain, tank_ok, expected, requirement)
    (25, 0, True, True, "F2 dry soil, no rain"),
    (25, 8, True, False, "F2 rain forecast"),
    (45, 0, True, False, "F2 soil wet enough"),
    (25, 0, False, False, "F3 tank empty: never run the pump dry"),
    (30, 0, True, False, "boundary: exactly at the limit"),
]
passed = 0
for m, r, t, exp, req in TESTS:
    ok = should_water(m, r, t) == exp
    passed += ok
    print(f"{'PASS' if ok else 'FAIL'}  {req}")
print(f"{passed}/{len(TESTS)} tests passed")
```

OUTPUT

```
PASS  F2 dry soil, no rain
PASS  F2 rain forecast
PASS  F2 soil wet enough
PASS  F3 tank empty: never run the pump dry
PASS  boundary: exactly at the limit
5/5 tests passed
```

27.2.4 Documentation, diagrams and demo video

Documentation is part of the product. Include: a README (what, why, how to build and run, troubleshooting), the block diagram and circuit diagram (Fritzing/KiCad), the BOM, photos of the build, the requirements with test results, and known limitations. A **2–3 minute demo video**: the problem (15 s), the system diagram (20 s), the live demo of each key feature (90 s), results and what you learned (30 s). Show it working, not just slides.

27.2.5 Presenting the project

1. Open with the problem and who it helps, in one sentence.
2. Show the architecture diagram and explain the key design decisions (why MQTT, why a Pico for the motors).
3. Live demo, with a recorded video as back-up.
4. Results against requirements, with numbers (battery life measured, detection accuracy, cost).
5. Challenges, what you would do differently, next steps.
6. Leave time for questions; know your trade-offs and limitations.

27.3 Portfolio

27.3.1 GitHub repositories with clean READMEs

A TIDY PROJECT REPOSITORY

```
smart-garden/
├── README.md              # overview, photo, diagram, quick start, results
├── docs/
│   ├── architecture.png
│   ├── circuit.png
│   ├── bom.csv
│   └── test-report.md
├── hub/                   # Raspberry Pi code
│   ├── app.py
│   ├── rules.py
│   └── tests/test_rules.py
├── nodes/esp32-soil/      # firmware for the sensor nodes
├── deploy/                # systemd units, docker-compose.yml, install.sh
├── .gitignore             # env/, *.log, .env
└── LICENSE
```

Recruiters and teachers spend about a minute on a repository: put a photo or GIF, a one-line summary and the architecture diagram at the top of the README. Use meaningful commit messages, no secrets, and a licence (MIT or Apache-2.0 for code; CERN-OHL for hardware).

27.3.2 Blog posts and project write-ups

Write one post per project on a blog, Hackster.io, Instructables or LinkedIn: the problem, the design, the hardest bug and how you solved it, and results. Teaching what you built shows understanding and communication skills, which employers value as much as the technology.

27.3.3 Hackathons, maker fairs and competitions

Hackathons teach working fast in a team; maker faires and college tech fests give feedback from real users; competitions (Smart India Hackathon, robotics contests, Raspberry Pi Foundation challenges such as Astro Pi for school students) add credibility. Contribute to open-source projects you use (documentation and bug reports count) to learn professional workflows.

27.4 Careers

Role	What you do	Skills from this course	Grow next
27.4.1 Embedded and IoT developer	firmware and Linux software for connected devices	GPIO, buses, MQTT, Pico, power, security	embedded C, RTOS, PCB basics, cloud IoT
27.4.2 Robotics engineer	robot software, control, perception	motors, PID, odometry, ROS 2, OpenCV	C++, kinematics, SLAM, simulation, control theory
27.4.3 Edge AI engineer	deploy and optimise ML models on devices	OpenCV, TFLite, quantisation, accelerators	deep learning, model optimisation, MLOps

Role	What you do	Skills from this course	Grow next
27.4.4 Linux system administrator and DevOps	run and automate servers and fleets	Linux, Bash, systemd, Docker, networking, security	cloud platforms, Kubernetes, CI/CD, Ansible, monitoring
27.4.5 Hardware prototyping and product development	turn ideas into working prototypes and products	sensors, actuators, power, enclosures, HATs	KiCad PCB design, DFM, certification, project management
27.4.6 STEM educator	teach coding, electronics and robotics	all of it, plus documenting projects clearly	curriculum design, classroom practice, Raspberry Pi Certified Educator

27.5 Further learning

Area	Options
27.5.1 Linux certifications	LPI Linux Essentials (entry level), LPIC-1, CompTIA Linux+, Linux Foundation LFCS (hands-on administration)
27.5.2 Cloud and IoT certifications	cloud fundamentals and associate certifications from AWS, Microsoft Azure or Google Cloud; check each provider's current IoT learning paths, as specialist IoT exams change
27.5.3 Embedded C, RTOS and electronics design	C on microcontrollers (Pico SDK, STM32), FreeRTOS or Zephyr, KiCad PCB design, basic analogue electronics
27.5.4 Computer vision and deep learning	deep learning courses (for example fast.ai, DeepLearning.AI), PyTorch, OpenCV specialisations, Ultralytics YOLO, Hailo and Edge Impulse documentation

SHORTCUT TRICK

Learn in projects, not just courses: each new skill should end in something you can show. A good rhythm is one small project per month and one larger, documented project per term.

PRACTICE CHECKPOINT

Complete and present a capstone project that combines hardware, software, networking and either AI or automation, with full documentation, a demo video and a public GitHub repository.

27.6 Practice questions

27.6.1 Basic level

Q1 Which mix should a strong capstone project contain?

(a) hardware, software, networking and AI or automation
(b) only hardware
(c) only a slide presentation
(d) only a web page

Q2 What does BOM stand for?

(a) board operating mode
(b) basic output module
(c) bill of materials
(d) bus over MQTT

Q3 What is a non-functional requirement?

(a) a feature that does not work
(b) a missing part
(c) a software bug
(d) a measure of how well the system works, such as battery life or response time

Q4 What should a good requirement be?

(a) vague
(b) secret
(c) as long as possible
(d) specific and testable

Q5 Which file is the front page of a GitHub project?

(a) LICENSE
(b) .gitignore
(c) README.md
(d) setup.cfg

Q6 Which should never be committed to a public repository?

(a) passwords, API keys and .env files
(b) circuit diagrams
(c) the BOM
(d) test code

Q7 Which role focuses on running and automating servers and fleets?

(a) STEM educator
(b) robotics engineer
(c) graphic designer
(d) Linux system administrator / DevOps engineer

Q8 Which certification is an entry-level Linux credential?

(a) Cisco CCIE
(b) LPI Linux Essentials
(c) PMP
(d) CFA

Q9 Which document records decisions, problems and fixes during a project?

(a) the LICENSE file
(b) the .gitignore file
(c) an engineering diary (lab notebook)
(d) the crontab

27.6.2 Intermediate level

Q10 Why build and test one subsystem at a time?

(a) it is cheaper to buy parts one by one
(b) when something fails, you know where to look
(c) Python requires it
(d) it avoids writing documentation

Q11 What does a contingency line in a budget cover?

(a) the teacher's fee
(b) the final grade
(c) nothing
(d) spares, wires, connectors, mistakes and price changes

Q12 Why order the part with the longest lead time first?

(a) it determines when the full system can be built
(b) it is always the cheapest
(c) suppliers require it
(d) it needs no testing

Q13 Why write automated tests for decision rules?

(a) tests make the code run faster
(b) tests replace documentation
(c) every change is checked, including boundary cases, without manual effort
(d) they are needed for Wi-Fi

Q14 What should a project demo video focus on?

(a) ten minutes of slides
(b) only the code
(c) unboxing the parts
(d) the system working live, after a short problem statement and diagram

Q15 Which skills does a robotics engineer grow next after this course?

(a) accounting
(b) C++, kinematics, SLAM and control theory
(c) video editing only
(d) HTML only

Q16 Which licence suits open hardware designs?

(a) CERN Open Hardware Licence
(b) GPL for fonts
(c) no licence at all
(d) a software EULA

Q17 What is a good first step when the BOM total exceeds the budget requirement?

(a) buy anyway and hide the cost
(b) remove the testing
(c) revisit the design or requirement, for example a cheaper board or mains power instead of solar
(d) ignore the requirement

Q18 A BOM subtotal is ₹8 400. With 15 % contingency, what is the total?

(a) ₹9 660 (b) ₹8 415 (c) ₹9 400 (d) ₹12 600

27.6.3 Advanced level

Q19 A project has 12 requirements; tests pass for 9. What percentage is met?

(a) 90 % (b) 75 % (c) 66 % (d) 9 %

Q20 Write a BOM cost calculator that reads a CSV (part,qty,price), prints the total, the most expensive line and each line's share of the cost.

Q21 Track requirements and test results: given a dict of requirement IDs → pass/fail, print the coverage and list failing requirements.

Q22 Estimate a project schedule: tasks with durations (days) and dependencies; compute the earliest finish day of each task and the total project length.

Q23 Generate a README skeleton for a project from a few fields and print it.

Q24 Which capstone plan is best scoped?

(a) a humanoid robot that does all housework, finished in one week

(b) a project with no requirements so it cannot fail

(c) only buying parts and writing a report

(d) a smart-garden system with 3 sensor nodes, a dashboard and an automatic pump, with a working version in two weeks and improvements after

Q25 In an interview you are asked about your capstone. Which answer is strongest?

(a) list every part you bought

(b) say it worked perfectly with no problems

(c) explain the problem, your architecture and trade-offs, measured results, and the hardest bug you fixed

(d) say a friend did the hardware

27.7 Solutions

Q1 **Answer: (a) hardware, software, networking and AI or automation**

Integration shows real skill.

Q2 **Answer: (c) bill of materials**

Parts list with quantities and costs.

Q3 **Answer: (d) a measure of how well the system works, such as battery life or response time**

Quality attributes.

Q4 **Answer: (d) specific and testable**

You must be able to prove it is met.

Q5 **Answer: (c) README.md**

The README.

Q6 **Answer: (a) passwords, API keys and .env files**

Secrets stay local.

Q7 **Answer: (d) Linux system administrator / DevOps engineer**

Linux, containers, automation.

Q8 **Answer: (b) LPI Linux Essentials**

Entry level from the Linux Professional Institute.

Q9 **Answer: (c) an engineering diary (lab notebook)**

It becomes your report.

Q10 **Answer: (b) when something fails, you know where to look**

Incremental integration.

Q11 **Answer: (d) spares, wires, connectors, mistakes and price changes**

Usually 10–20 %.

Q12 **Answer: (a) it determines when the full system can be built**

Avoid waiting at the end.

Q13 **Answer: (c) every change is checked, including boundary cases, without manual effort**

Confidence when changing code.

Q14 **Answer: (d) the system working live, after a short problem statement and diagram**

Show, don't tell.

Q15 **Answer: (b) C++, kinematics, SLAM and control theory**

Deeper robotics.

Q16 **Answer: (a) CERN Open Hardware Licence**

Made for hardware.

Q17 **Answer: (c) revisit the design or requirement, for example a cheaper board or mains power instead of solar**

Make an explicit trade-off.

Q18 **Answer: (a) ₹9 660**

8400 × 1.15 = **₹9 660**.

Q19 **Answer: (b) 75 %**

9 ÷ 12 = **75 %**.

Q20 **Answer: Solution program**

DictReader parses the CSV; sorting by cost shows where the money goes (the Pi is over half).

```
import csv, io
data = """part,qty,price
Pi 5 4GB,1,6200
Camera Module 3,1,2500
PIR sensor,2,90
Case + cooler,1,950
SD card 64GB,1,700
"""
rows = [(r["part"], int(r["qty"]) * float(r["price"])) for r in csv.DictReader(io.StringIO(data))]
total = sum(c for _, c in rows)
for part, cost in sorted(rows, key=lambda x: -x[1]):
    print(f"{part:16} Rs {cost:7.0f}  {cost / total:5.1%}")
print(f"TOTAL            Rs {total:7.0f}")
```

OUTPUT

```
Pi 5 4GB         Rs    6200  58.9%
Camera Module 3  Rs    2500  23.7%
Case + cooler    Rs     950   9.0%
SD card 64GB     Rs     700   6.6%
PIR sensor       Rs     180   1.7%
TOTAL            Rs   10530
```

Q21 **Answer: Solution program**

A simple requirements-to-test map keeps progress honest and tells you what to fix next.

```
results = {"F1": True, "F2": True, "F3": False, "F4": True, "N1": False, "N2": True, "N3": True}
passed = [r for r, ok in results.items() if ok]
failed = [r for r, ok in results.items() if not ok]
print(f"{len(passed)}/{len(results)} requirements met ({len(passed) / len(results):.0%})")
print("failing:", ", ".join(failed))
```

OUTPUT

```
5/7 requirements met (71%)
failing: F3, N1
```

Q22 **Answer: Solution program**

Each task can start only when all its dependencies are done; the longest chain (the critical path: order parts → build → integrate → test → docs) sets the project length.

```
tasks = {"order parts": (10, []), "write hub code": (6, []), "build nodes": (4, ["order parts"]),
         "integrate": (3, ["build nodes", "write hub code"]), "test + fix": (4, ["integrate"]),
         "docs + video": (3, ["test + fix"])}
finish = {}
def earliest(t):
    if t not in finish:
        dur, deps = tasks[t]
        finish[t] = dur + max((earliest(d) for d in deps), default=0)
    return finish[t]
for t in tasks:
    print(f"{t:15} finishes on day {earliest(t)}")
print("project length:", max(finish.values()), "days")
```

OUTPUT

```
order parts     finishes on day 10
write hub code  finishes on day 6
build nodes     finishes on day 14
integrate       finishes on day 17
test + fix      finishes on day 21
docs + video    finishes on day 24
project length: 24 days
```

Q23 **Answer: Solution program**

Generating the skeleton keeps every project's documentation consistent; you then fill in results and photos.

```
proj = {"name": "Smart Garden", "pitch": "Waters three garden beds only when they need it.",
        "hardware": ["Raspberry Pi 4", "3 x ESP32 soil nodes", "12 V pump"],
        "run": "docker compose up -d"}
lines = [f"# {proj['name']}", "", proj["pitch"], "", "![architecture](docs/architecture.png)", "",
         "## Hardware"] + [f"- {h}" for h in proj["hardware"]] + ["", "## Quick start", "",
         f"    {proj['run']}", "", "## Results", "", "## Licence", "", "MIT"]
print("\n".join(lines))
```

OUTPUT

```
# Smart Garden

Waters three garden beds only when they need it.

![architecture](docs/architecture.png)

## Hardware
- Raspberry Pi 4
- 3 x ESP32 soil nodes
- 12 V pump

## Quick start

    docker compose up -d

## Results

## Licence
```

```
MIT
```

Q24 **Answer: (d) a smart-garden system with 3 sensor nodes, a dashboard and an automatic pump, with a working version in two weeks and improvements after**

Achievable, testable, extendable.

Q25 **Answer: (c) explain the problem, your architecture and trade-offs, measured results, and the hardest bug you fixed**

Show understanding and honesty.

Pick a capstone that solves a real problem and integrates hardware, software, networking and AI or automation. Follow the process: testable requirements → block diagram and interfaces → BOM with contingency → build and test subsystems with Git and automated tests → measure against requirements → document (README, diagrams, BOM, results) → 2–3 minute demo → confident presentation. Build a portfolio, choose a career direction, and keep learning through projects.